Chapter 3. Installer-provisioned infrastructure

docs.redhat.com/en/documentation/openshift_container_platform/4.22/html/installing_on_aws/installer-provisioned-infrastructure

3.1. Preparing to install a cluster on AWS

To install an OpenShift Container Platform cluster on Amazon Web Services (AWS), you must verify your internet connectivity, download the installation program, install the OpenShift CLI (`oc`), and generate an SSH key pair.

If required, you also need to manually create long-term credentials for AWS or configure an AWS cluster to use short-term credentials with Amazon Web Services Security Token Service (AWS STS).

The following list outlines in detail the steps to prepare to install an OpenShift Container Platform cluster on AWS:

- Verifying internet connectivity for your cluster.
- Configuring an AWS account. For more information, see "Configuring an AWS account".
- Downloading the installation program.

  If you are installing in a disconnected environment, you extract the installation program from the mirrored content. For more information, see "Mirroring images for a disconnected installation".

- Installing the OpenShift CLI (`oc`).

  If you are installing in a disconnected environment, install `oc` to the mirror host.

- Generating an SSH key pair. You can use this key pair to authenticate into the OpenShift Container Platform cluster's nodes after it is deployed.
- If the cloud identity and access management (IAM) APIs are not accessible in your environment, or if you do not want to store an administrator-level credential secret in the `kube-system` namespace, manually creating long-term credentials for AWS or configuring an AWS cluster to use short-term credentials with (AWS STS). For more information, see "Manually creating long-term credentials" and "Configuring an AWS cluster to use short-term credentials".

3.1.1. Internet access for OpenShift Container Platform

In OpenShift Container Platform 4.22, you require access to the internet to install your cluster.

You must have internet access to perform the following actions:

- Access Red Hat Hybrid Cloud Console to download the installation program and perform subscription management. If the cluster has internet access and you do not disable Telemetry, that service automatically entitles your cluster.
- Access Quay.io to obtain the packages that are required to install your cluster.
- Obtain the packages that are required to perform cluster updates.

If your cluster cannot have direct internet access, you can perform a restricted network installation on some types of infrastructure that you provision. During that process, you download the required content and use it to populate a mirror registry with the installation packages. With some installation types, the environment that you install your cluster in will not require internet access. Before you update the cluster, you update the content of the mirror registry.

3.1.2. Obtaining the installation program

Before you install OpenShift Container Platform, download the installation file on the host you are using for installation, so that installation assets exist for deployment in your environment.

Prerequisites

You have a computer that runs Linux or macOS, with 500 MB of local disk space.

Procedure

1. Go to the Cluster Type page on the Red Hat Hybrid Cloud Console. If you have a Red Hat account, log in with your credentials. If you do not, create an account.
2. Select your infrastructure provider from the **Run it yourself** section of the page.
3. Select your host operating system and architecture from the dropdown menus under **OpenShift Installer** and click **Download Installer**.
4. Place the downloaded file in the directory where you want to store the installation configuration files.
   - The installation program creates several files on the computer that you use to install your cluster. You must keep the installation program and the files that the installation program creates after you finish installing the cluster. Both of the files are required to delete the cluster.
   - Deleting the files created by the installation program does not remove your cluster, even if the cluster failed during installation. To remove your cluster, complete the OpenShift Container Platform uninstallation procedures for your specific cloud provider.
5. Extract the installation program. For example, on a computer that uses a Linux operating system, run the following command:

   ```
   $ tar -xvf openshift-install-linux.tar.gz
   ```

6. Download your installation pull secret from Red Hat OpenShift Cluster Manager. This pull secret allows you to authenticate with the services that are provided by the included authorities, including Quay.io, which serves the container images for OpenShift Container Platform components.

   Alternatively, you can retrieve the installation program from the Red Hat Customer Portal, where you can specify a version of the installation program to download. However, you must have an active subscription to access this page.

3.1.3. Install the OpenShift CLI on Linux

To manage your cluster and deploy applications from the command line on Linux, install the OpenShift CLI (`oc`) binary. You can download the OpenShift CLI (`oc`) from the Red Customer Portal.

If you installed an earlier version of `oc`, you cannot use it to complete all of the commands in OpenShift Container Platform.

Download and install the new version of `oc`.

Procedure

1. Navigate to the Download OpenShift Container Platform page on the Red Hat Customer Portal.
2. Select the architecture from the **Product Variant** list.
3. Select the appropriate version from the **Version** list.
4. Click **Download Now** next to the **OpenShift v4.22 Linux Clients** entry and save the file.

5. Unpack the archive:

```
$ tar xvf <file>
```

6. Place the `oc` binary in a directory that is on your `PATH`.

   To check your `PATH`, run the following command:

```
$ echo $PATH
```

Verification

After you install the OpenShift CLI, it is available using the `oc` command:

```
$ oc <command>
```

3.1.4. Install the OpenShift CLI on Windows

To manage your cluster and deploy applications from the command line on Windows, install the OpenShift CLI (`oc`) binary. You can download the OpenShift CLI (`oc`) from the Red Customer Portal.

If you installed an earlier version of `oc`, you cannot use it to complete all of the commands in OpenShift Container Platform.

Download and install the new version of `oc`.

Procedure

1. Navigate to the Download OpenShift Container Platform page on the Red Hat Customer Portal.
2. Select the appropriate version from the **Version** list.
3. Click **Download Now** next to the **OpenShift v4.22 Windows Client** entry and save the file.
4. Extract the archive with a ZIP program.
5. Move the `oc` binary to a directory that is on your `PATH` variable.

   To check your `PATH` variable, open the Command Prompt and run the following command:

```
C:\> path
```

Verification

After you install the OpenShift CLI, it is available using the `oc` command:

```
C:\> oc <command>
```

3.1.5. Install the OpenShift CLI on macOS

To manage your cluster and deploy applications from the command line on macOS, install the OpenShift CLI (`oc`) binary. You can download the OpenShift CLI (`oc`) from the Red Customer Portal.

If you installed an earlier version of `oc`, you cannot use it to complete all of the commands in OpenShift Container Platform.

Download and install the new version of `oc`.

Procedure

1. Navigate to the Download OpenShift Container Platform page on the Red Hat Customer Portal.
2. Select the architecture from the **Product Variant** list.

3. Select the appropriate version from the **Version** list.
4. Click **Download Now** next to the **OpenShift v4.22 macOS Clients** entry and save the file.

   For macOS arm64, choose the **OpenShift v4.22 macOS arm64 Client** entry.

5. Extract the archive.
6. Move the `oc` binary to a directory on your `PATH` variable.

   To check your `PATH` variable, open a terminal and run the following command:

   ```
   $ echo $PATH
   ```

Verification

Verify your installation by using an `oc` command:

```
$ oc <command>
```

3.1.6. Generating a key pair for cluster node SSH access

During an OpenShift Container Platform installation, you can provide an SSH public key to the installation program. The key is passed to Red Hat Enterprise Linux CoreOS (RHCOS) nodes through their Ignition config files and is added to `~/.ssh/authorized_keys` for the `core` user on each node.

The key is added to the `~/.ssh/authorized_keys` list for the `core` user on each node, which enables password-less authentication. After the key is passed to the nodes, you can use the key pair to SSH in to the RHCOS nodes as the user `core`. To access the nodes through SSH, the private key identity must be managed by SSH for your local user.

If you want to SSH in to your cluster nodes to perform installation debugging or disaster recovery, you must provide the SSH public key during the installation process. The `./openshift-install gather` command also requires the SSH public key to be in place on the cluster nodes.

Do not skip this procedure in production environments, where disaster recovery and debugging is required.

You must use a local key, not one that you configured with platform-specific approaches.

Procedure

1. If you do not have an existing SSH key pair on your local machine to use for authentication onto your cluster nodes, create one. For example, on a computer that uses a Linux operating system, run the following command:

   ```
   $ ssh-keygen -t ed25519 -N '' -f <path>/<file_name>
   ```

   Specifies the path and file name, such as `~/.ssh/id_ed25519`, of the new SSH key. If you have an existing key pair, ensure your public key is in the your `~/.ssh` directory.

   If you plan to install an OpenShift Container Platform cluster that uses the RHEL cryptographic libraries that have been submitted to NIST for FIPS 140-2/140-3 Validation on only the `x86_64`, `ppc64le`, and `s390x` architectures, do not create a key that uses the `ed25519` algorithm. Instead, create a key that uses the `rsa` or `ecdsa` algorithm.

2. View the public SSH key:

```
$ cat <path>/<file_name>.pub
```

For example, run the following to view the `~/.ssh/id_ed25519.pub` public key:

```
$ cat ~/.ssh/id_ed25519.pub
```

3. Add the SSH private key identity to the SSH agent for your local user, if it has not already been added. SSH agent management of the key is required for password-less SSH authentication onto your cluster nodes, or if you want to use the `./openshift-install gather` command.

On some distributions, default SSH private key identities such as `~/.ssh/id_rsa` and `~/.ssh/id_dsa` are managed automatically.

a. If the `ssh-agent` process is not already running for your local user, start it as a background task:

```
$ eval "$(ssh-agent -s)"
```

Example output

```
Agent pid 31874
```

If your cluster is in FIPS mode, only use FIPS-compliant algorithms to generate the SSH key. The key must be either RSA or ECDSA.

4. Add your SSH private key to the `ssh-agent`:

```
$ ssh-add <path>/<file_name>
```

Specify the path and file name for your SSH private key, such as `~/.ssh/id_ed25519`.

Example output

```
Identity added: /home/<you>/<path>/<file_name> (<computer_name>)
```

Next steps

When you install OpenShift Container Platform, provide the SSH public key to the installation program.

3.1.7. Telemetry access for OpenShift Container Platform

To provide metrics about cluster health and the success of updates, the Telemetry service requires internet access. When connected, this service runs automatically by default and registers your cluster to OpenShift Cluster Manager.

After you confirm that your OpenShift Cluster Manager inventory is correct, either maintained automatically by Telemetry or manually by using OpenShift Cluster Manager,use subscription watch to track your OpenShift Container Platform subscriptions at the account or multi-cluster level. For more information about subscription watch, see "Data Gathered and Used by Red Hat's subscription services" in the *Additional resources* section.

3.2. Installing a cluster on AWS

In OpenShift Container Platform version 4.22, you can install a cluster on Amazon Web Services (AWS) that uses the default configuration options.

3.2.1. Prerequisites

Before you install a cluster on Amazon Web Services (AWS) that uses the default configuration options, you must meet several prerequisites.

The following prerequisites must be met:

- You reviewed details about the OpenShift Container Platform installation and update processes.
- You read the documentation on selecting a cluster installation method and preparing it for users.
- You configured an AWS account to host the cluster.

  If you have an AWS profile stored on your computer, it must not use a temporary session token that you generated while using a multi-factor authentication device. The cluster continues to use your current AWS credentials to create AWS resources for the entire life of the cluster, so you must use key-based, long-term credentials. To generate appropriate keys, see Managing Access Keys for IAM Users in the AWS documentation. You can supply the keys when you run the installation program.

- If you use a firewall, you configured it to allow the sites that your cluster requires access to.

3.2.2. Deploying the cluster

To deploy your OpenShift Container Platform cluster, you initialize installation by running the `openshift-install create cluster` command from the directory that contains the installation program. The installation program provisions the required infrastructure and completes the cluster setup.

You can run the `create cluster` command of the installation program only once, during initial installation.

Prerequisites

- You have configured an account with the cloud platform that hosts your cluster.
- You have the OpenShift Container Platform installation program and the pull secret for your cluster.
- You have verified that the cloud provider account on your host has the correct permissions to deploy the cluster. An account with incorrect permissions causes the installation process to fail with an error message that displays the missing permissions.

Procedure

1. In the directory that contains the installation program, initialize the cluster deployment by running the following command:

   ```
   $ ./openshift-install create cluster --dir <installation_directory> \
       --log-level=info
   ```

   where:

   - `<installation_directory>`: Specifies the directory name to store the files that the installation program creates.
   - `--log-level`: Specifies the log level. To view different installation details, specify `warn`, `debug`, or `error` instead of `info`.

   When specifying the directory: * Verify that the directory has the `execute` permission. This permission is required to run Terraform binaries under the installation directory. * Use an empty directory. Some installation assets, such as bootstrap X.509 certificates, have short expiration intervals, therefore you must not reuse an installation directory. If you want to reuse individual files from another cluster installation, you can copy them into your directory. However, the file names for the installation assets might change between releases. Use caution when copying installation files from an earlier OpenShift Container Platform version.

2. Provide values at the prompts:

   a. Optional: Select an SSH key to use to access your cluster machines.

      For production OpenShift Container Platform clusters on which you want to perform installation debugging or disaster recovery, specify an SSH key that your `ssh-agent` process uses.

   b. Select **aws** as the platform to target.
   c. If you do not have an Amazon Web Services (AWS) profile stored on your computer, enter the AWS access key ID and secret access key for the user that you configured to run the installation program.

      The AWS access key ID and secret access key are stored in `~/.aws/credentials` in the home directory of the current user on the installation host. You are prompted for the credentials by the installation program if the credentials for the exported profile are not present in the file. Any credentials that you provide to the installation program are stored in the file.

   d. Select the AWS region to deploy the cluster to.
   e. Select the base domain for the Route 53 service that you configured for your cluster.
   f. Enter a descriptive name for your cluster.
   g. Paste the pull secret from Red Hat OpenShift Cluster Manager.

3. Optional: Remove or disable the `AdministratorAccess` policy from the IAM account that you used to install the cluster.

   The elevated permissions provided by the `AdministratorAccess` policy are required only during installation.

Verification

When the cluster deployment completes successfully:

- The terminal displays directions for accessing your cluster, including a link to the web console and credentials for the `kubeadmin` user.

- Credential information also outputs to `<installation_directory>/.openshift_install.log`.

  Do not delete the installation program or the files that the installation program creates. Both are required to delete the cluster.

  The following example shows the expected output:

```
...
INFO Install complete!
INFO To access the cluster as the system:admin user when using 'oc', run 'export
KUBECONFIG=/home/myuser/install_dir/auth/kubeconfig'
INFO Access the OpenShift web-console here: https://console-openshift-
console.apps.mycluster.example.com
INFO Login to the console with user: "kubeadmin", and password: "password"
INFO Time elapsed: 36m22s
```

  - The Ignition config files that the installation program generates contain certificates that expire after 24 hours, which are then renewed at that time. If the cluster is shut down before renewing the certificates and the cluster is later restarted after the 24 hours have elapsed, the cluster automatically recovers the expired certificates. The exception is that you must manually approve the pending `node-bootstrapper` certificate signing requests (CSRs) to recover kubelet certificates. See the documentation for *Recovering from expired control plane certificates* for more information.
  - It is recommended that you use Ignition config files within 12 hours after they are generated because the 24-hour certificate rotates from 16 to 22 hours after the cluster is installed. By using the Ignition config files within 12 hours, you can avoid installation failure if the certificate update runs during installation.

3.2.3. Logging in to the cluster by using the CLI

To log in to your cluster as the default system user, export the `kubeconfig` file. This configuration enables the CLI to authenticate and connect to the specific API server created during OpenShift Container Platform installation.

The `kubeconfig` file is specific to a cluster and OpenShift Container Platform generates it during installation.

Prerequisites

- You deployed an OpenShift Container Platform cluster.
- You installed the OpenShift CLI (`oc`).

Procedure

1. Export the `kubeadmin` credentials by running the following command:

   ```
   $ export KUBECONFIG=<installation_directory>/auth/kubeconfig
   ```

   where:

   `<installation_directory>`
   Specifies the path to the directory that stores the installation files.

2. Verify you can run `oc` commands successfully using the exported configuration by running the following command:

   ```
   $ oc whoami
   ```

   Example output

   ```
   system:admin
   ```

Next steps

- "Customize your cluster"
- "Remote health reporting"

3.2.4. Logging in to the cluster by using the web console

To verify that your cluster deployed successfully and access its features, log in to the OpenShift Container Platform web console as the `kubeadmin` user.

Prerequisites

- You have access to the installation host.
- You completed a cluster installation and all cluster Operators are available.

Procedure

1. Obtain the password for the `kubeadmin` user from the `kubeadmin-password` file on the installation host:

   ```
   $ cat <installation_directory>/auth/kubeadmin-password
   ```

   Or, you can obtain the `kubeadmin` password from the `<installation_directory>/.openshift_install.log` log file on the installation host.

2. List the OpenShift Container Platform web console route:

   ```
   $ oc get routes -n openshift-console | grep 'console-openshift'
   ```

   Or, you can obtain the OpenShift Container Platform route from the `<installation_directory>/.openshift_install.log` log file on the installation host.

   Example output

   ```
   console     console-openshift-console.apps.<cluster_name>.<base_domain>            console
   https   reencrypt/Redirect   None
   ```

3. Navigate to the route detailed in the output of the preceding command in a web browser and log in as the `kubeadmin` user.

3.3. Installing a cluster on AWS with customizations

In OpenShift Container Platform version 4.22, you can install a cluster on Amazon Web Services (AWS) by using installer-provisioned infrastructure with customizations, including network configuration options.

In each, you modify parameters in the `install-config.yaml` file before you install the cluster. By customizing your network configuration, your cluster can coexist with existing IP address allocations in your environment and integrate with existing MTU and VXLAN configurations.

You must set most of the network configuration parameters during installation, and you can modify only `kubeProxy` configuration parameters in a running cluster.

The scope of the OpenShift Container Platform installation configurations is intentionally narrow. It is designed for simplicity and ensured success. You can complete many more OpenShift Container Platform configuration tasks after an installation completes.

3.3.1. Prerequisites

Before you install a cluster on Amazon Web Services (AWS) by using installer-provisioned infrastructure with customizations, you must meet several prerequisites.

The following prerequisites must be met:

- You reviewed details about the OpenShift Container Platform installation and update processes.
- You read the documentation on selecting a cluster installation method and preparing it for users.
- You configured an AWS account to host the cluster.

  If you have an AWS profile stored on your computer, it must not use a temporary session token that you generated while using a multi-factor authentication device. The cluster continues to use your current AWS credentials to create AWS resources for the entire life of the cluster, so you must use long-term credentials. To generate appropriate keys, see Managing Access Keys for IAM Users in the AWS documentation. You can supply the keys when you run the installation program.

- If you use a firewall, you configured it to allow the sites that your cluster requires access to.

3.3.2. Obtaining an AWS Marketplace image

If you are deploying an OpenShift Container Platform cluster by using an Amazon Web Services (AWS) Marketplace image, you must first subscribe through AWS. Subscribing to the offer provides you with the Amazon Machine Image (AMI) ID that the installation program uses to deploy compute nodes.

You should only modify the RHCOS image for compute machines to use an AWS Marketplace image. Control plane machines and infrastructure nodes do not require an OpenShift Container Platform subscription and use the public RHCOS default image by default, which does not incur subscription costs on your AWS bill. Therefore, you should not modify the cluster default boot image or the control plane boot images. Applying the AWS Marketplace image to them will incur additional licensing costs that cannot be recovered.

Prerequisites

You have an AWS account to buy the offer. This account does not have to be the same account that you use to install the cluster.

Procedure

1. Complete the OpenShift Container Platform subscription from the AWS Marketplace.

2. Record the AMI ID for your specific AWS region. As part of the installation process, you must update the `install-config.yaml` file with this value before deploying the cluster.

   Sample `install-config.yaml` file with AWS Marketplace compute nodes

   ```
   apiVersion: v1
   baseDomain: example.com
   compute:
   - hyperthreading: Enabled
     name: worker
     platform:
       aws:
         amiID: ami-06c4d345f7c207239
         type: m5.4xlarge
     replicas: 3
   metadata:
     name: test-cluster
   platform:
     aws:
       region: us-east-2
   sshKey: ssh-ed25519 AAAA...
   pullSecret: '{"auths": ...}'
   ```

   where:

   `compute.platform.aws.amiID`
   Specifies the AMI ID from your AWS Marketplace subscription.

   `platform.aws.region`
   Specifies the `platform.aws.region` parameter. Your AMI ID is associated with a specific AWS region. When creating the installation configuration file, ensure that you select the same AWS region that you specified when configuring your subscription.

3.3.3. Network configuration phases

You can customize your OpenShift Container Platform network plugin configuration, such as cluster network CIDR and service network ranges, during two phases before installation to integrate with your existing network environment.

Phase 1
You can customize the following network-related fields in the `install-config.yaml` file before you create the manifest files:

- `networking.networkType`
- `networking.clusterNetwork`
- `networking.serviceNetwork`
- `networking.machineNetwork`
- `nodeNetworking`

  For more information, see "Installation configuration parameters".

  Set the `networking.machineNetwork` to match the Classless Inter-Domain Routing (CIDR) where the preferred subnet is located.

  The CIDR range `172.17.0.0/16` is reserved by `libVirt`. You cannot use any other CIDR range that overlaps with the `172.17.0.0/16` CIDR range for networks in your cluster.

Phase 2

After creating the manifest files by running `openshift-install create manifests`, you can define a customized Cluster Network Operator manifest with only the fields you want to modify. You can use the manifest to specify an advanced network configuration.

During phase 2, you cannot override the values that you specified in phase 1 in the `install-config.yaml` file. However, you can customize the network plugin during phase 2.

3.3.4. Creating the installation configuration file

You can customize the OpenShift Container Platform cluster you install on Amazon Web Services (AWS).

Prerequisites

You have the OpenShift Container Platform installation program and the pull secret for your cluster.

Procedure

1. Create the `install-config.yaml` file.

   a. Change to the directory that contains the installation program and run the following command:

   ```
   $ ./openshift-install create install-config --dir <installation_directory>
   ```

   - `<installation_directory>`: For `<installation_directory>`, specify the directory name to store the files that the installation program creates.

     When specifying the directory:

   - Verify that the directory has the `execute` permission. This permission is required to run Terraform binaries under the installation directory.
   - Use an empty directory. Some installation assets, such as bootstrap X.509 certificates, have short expiration intervals, therefore you must not reuse an installation directory. If you want to reuse individual files from another cluster installation, you can copy them into your directory. However, the file names for the installation assets might change between releases. Use caution when copying installation files from an earlier OpenShift Container Platform version.

   b. At the prompts, provide the configuration details for your cloud:

      i. Optional: Select an SSH key to use to access your cluster machines.

      For production OpenShift Container Platform clusters on which you want to perform installation debugging or disaster recovery, specify an SSH key that your `ssh-agent` process uses.

      ii. Select **AWS** as the platform to target.
      iii. If you do not have an Amazon Web Services (AWS) profile stored on your computer, enter the AWS access key ID and secret access key for the user that you configured to run the installation program.
      iv. Select the AWS region to deploy the cluster to.
      v. Select the base domain for the Route 53 service that you configured for your cluster.
      vi. Enter a descriptive name for your cluster.

2. Modify the `install-config.yaml` file. You can find more information about the available parameters in the "Installation configuration parameters" section.

   If you are installing a three-node cluster, be sure to set the `compute.replicas` parameter to `0`. This ensures that the cluster's control planes are schedulable. For more information, see "Installing a three-node cluster on AWS".

3. Back up the `install-config.yaml` file so that you can use it to install multiple clusters.

   The `install-config.yaml` file is consumed during the installation process. If you want to reuse the file, you must back it up now.

3.3.4.1. Minimum resource requirements for cluster installation

To ensure that your OpenShift Container Platform cluster runs as expected, each cluster machine must meet minimum CPU, memory, and storage requirements.

Machine	Operating system	vCPU	Virtual RAM	Storage	Input/Output Per Second (IOPS)
Bootstrap	RHCOS	4	16 GB	100 GB	300
Control plane	RHCOS	4	16 GB	100 GB	300
Compute	RHCOS	2	8 GB	100 GB	300

Table 3.1. Minimum resource requirements

- One vCPU is equal to one physical core when simultaneous multithreading (SMT), or Hyper-Threading, is not enabled. When enabled, use the following formula to calculate the corresponding ratio: (threads per core × cores) × sockets = vCPUs.
- OpenShift Container Platform and Kubernetes are sensitive to disk performance, and Red Hat recommends faster storage, particularly for etcd on the control plane nodes which require a 10 ms p99 fsync duration. On many cloud platforms, storage size and IOPS scale together, so you might need to provision more storage to get enough performance.
- As with all user-provisioned installations, if you choose to use RHEL compute machines in your cluster, you take responsibility for all operating system life cycle management and maintenance, including performing system updates, applying patches, and completing all other required tasks. OpenShift Container Platform 4.10 and later do not support RHEL 7 compute machines.

In OpenShift Container Platform version 4.22, RHCOS uses RHEL version 9.8, which updates the micro-architecture requirements. Each architecture requires the following minimum instruction set architectures (ISA):

- x86-64 architecture requires x86-64-v2 ISA
- ARM64 architecture requires ARMv8.0-A ISA
- ppc64le architecture requires IBM® Power9 ISA
- s390x architecture requires IBM® z14 ISA

For more information, see Architectures in the RHEL documentation.

If an instance type for your platform meets the minimum requirements for cluster machines, it is supported to use in OpenShift Container Platform.

3.3.4.2. Tested instance types for AWS

To ensure cluster stability and performance, use one of the tested Amazon Web Services (AWS) instance types for your OpenShift Container Platform machines.

The following AWS instance types have been tested with OpenShift Container Platform.

Use the machine types included in the following charts for your AWS instances. If you use an instance type that is not listed in the chart, ensure that the instance size you use matches the minimum resource requirements in "Minimum resource requirements for cluster installation".

See the following machine types based on 64-bit x86 architecture:

- `c4.*`
- `c5.*`
- `c5a.*`
- `i3.*`
- `m4.*`
- `m5.*`
- `m5a.*`
- `m6a.*`
- `m6i.*`
- `m7a.*`
- `m7i.*`
- `m7i-flex.*`
- `r4.*`
- `r5.*`
- `r5a.*`
- `r6i.*`
- `t3.*`
- `t3a.*`

3.3.4.3. Tested instance types for AWS on 64-bit ARM infrastructures

To ensure cluster stability and performance, use one of the tested Amazon Web Services (AWS) 64-bit ARM instance types for your OpenShift Container Platform machines.

The following AWS 64-bit ARM instance types have been tested with OpenShift Container Platform.

Use the machine types included in the following charts for your AWS ARM instances. If you use an instance type that is not listed in the chart, ensure that the instance size you use matches the minimum resource requirements listed in "Minimum resource requirements for cluster installation".

See the following machine types based on 64-bit ARM architecture:

- `c6g.*`
- `c7g.*`
- `c8g.*`
- `m6g.*`
- `m7g.*`
- `m7gd.*`
- `r8g.*`

3.3.4.4. Enabling a user-managed DNS

You can install a cluster with a domain name server (DNS) solution that you manage instead of the default cluster-provisioned DNS solution that uses the Route 53 service for Amazon Web Services (AWS).

For example, your organization's security policies might not allow the use of public DNS services such as Amazon Web Services DNS. In such scenarios, you can use your own DNS service to bypass the public DNS service and manage your own DNS for the IP addresses of the API and Ingress services.

If you enable user-managed DNS during installation, the installation program provisions DNS records for the API and Ingress services only within the cluster. To ensure access from outside the cluster, you must provision the DNS records in an external DNS service of your choice for the API and Ingress services after installation.

User-provisioned DNS is a Technology Preview feature only. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see Technology Preview Features Support Scope.

Procedure

Before you deploy your cluster, use a text editor to open the `install-config.yaml` file and add the following stanza:

To enable user-managed DNS:

```
featureSet: CustomNoUpgrade
featureGates: ["AWSClusterHostedDNSInstall=true"]
```

```
platform:
  aws:
    userProvisionedDNS: Enabled
```

where:

`userProvisionedDNS`
Enables user-provisioned DNS management.

Next steps

For information about provisioning your DNS records for the API server and the Ingress services, see "Provisioning your own DNS records".

3.3.4.5. Sample customized install-config.yaml file for AWS

You can customize the installation configuration file (`install-config.yaml`) to specify more details about your OpenShift Container Platform cluster's platform or modify the values of the required parameters.

This sample YAML file is provided for reference only. You must obtain your `install-config.yaml` file by using the installation program and modify it. For a full list and description of all installation configuration parameters, see *Installation configuration parameters for AWS*.

Sample `install-config.yaml` file for AWS

```
apiVersion: v1
baseDomain: example.com
sshKey: ssh-ed25519 AAAA...
```

```
pullSecret: '{"auths": ...}'
metadata:
  name: example-cluster
controlPlane:
  name: master
  platform:
    aws:
      type: m6i.xlarge
  replicas: 3
compute:
-  name: worker
  platform:
    aws:
      type: c5.4xlarge
  replicas: 3
networking:
  clusterNetwork:
  - cidr: 10.128.0.0/14
    hostPrefix: 23
platform:
  aws:
    region: us-west-2
```

- Parameters at the first level of indentation apply to the cluster globally.
- The `controlPlane` stanza applies to control plane machines.
- The `compute` stanza applies to compute machines.
- The `networking` stanza applies to the cluster networking configuration. If you do not provide networking values, the installation program provides default values.
- The `platform` stanza applies to the infrastructure platform that hosts the cluster.

3.3.4.6. Configuring the cluster-wide proxy during installation

Production environments can deny direct access to the internet and instead have an HTTP or HTTPS proxy available. You can configure a new OpenShift Container Platform cluster to use a proxy by configuring the proxy settings in the `install-config.yaml` file.

Prerequisites

- You have an existing `install-config.yaml` file.
- You have reviewed the sites that your cluster requires access to and determined whether any of them need to bypass the proxy. By default, the proxy handles all cluster egress traffic, including calls to hosting cloud provider APIs. You added sites to the `Proxy` object's `spec.noProxy` field to bypass the proxy if necessary.

  The `Proxy` object `status.noProxy` field includes the values of the `networking.machineNetwork[].cidr`, `networking.clusterNetwork[].cidr`, and `networking.serviceNetwork[]` fields from your installation configuration.

  For installations on Amazon Web Services (AWS), Google Cloud, Microsoft Azure, and Red Hat OpenStack Platform (RHOSP), the `Proxy` object `status.noProxy` field also includes the instance metadata endpoint (`169.254.169.254`).

Procedure

1. Edit your `install-config.yaml` file and add the proxy settings. For example:

```
apiVersion: v1
baseDomain: my.domain.com
proxy:
  httpProxy: http://<username>:<pswd>@<ip>:<port>
  httpsProxy: https://<username>:<pswd>@<ip>:<port>
  noProxy: ec2.<aws_region>.amazonaws.com,elasticloadbalancing.<aws_region>.amazonaws.com,s3.
<aws_region>.amazonaws.com
additionalTrustBundle: |
    -----BEGIN CERTIFICATE-----
    <MY_TRUSTED_CA_CERT>
    -----END CERTIFICATE-----
additionalTrustBundlePolicy: <policy_to_add_additionalTrustBundle>
```

   where:

   `proxy.httpProxy`
   : Specifies a proxy URL to use for creating HTTP connections outside the cluster. The URL scheme must be `http`.

   `proxy.httpsProxy`
   : Specifies a proxy URL to use for creating HTTPS connections outside the cluster.

   `proxy.noProxy`
   : Specifies a comma-separated list of destination domain names, IP addresses, or other network CIDRs to exclude from proxying. Preface a domain with `.` to match subdomains only. For example, `.y.com` matches `x.y.com`, but not `y.com`. Use `*` to bypass the proxy for all destinations. If you have added the Amazon `EC2`, `Elastic Load Balancing`, and `S3` VPC endpoints to your VPC, you must add these endpoints to the `noProxy` field.

   `additionalTrustBundle`
   : If you specify this value, the installation program generates a config map named `user-ca-bundle` in the `openshift-config` namespace to hold the additional CA certificates. If you specify `additionalTrustBundle` and at least one proxy setting, the `Proxy` object references the `user-ca-bundle` config map in the `trustedCA` field. The Cluster Network Operator then creates a `trusted-ca-bundle` config map that merges the contents specified for the `trustedCA` parameter with the RHCOS trust bundle. You must set the `additionalTrustBundle` field unless an authority from the RHCOS trust bundle signs the proxy's identity certificate.

   `additionalTrustBundlePolicy`
   : Specifies the policy that determines the configuration of the `Proxy` object to reference the `user-ca-bundle` config map in the `trustedCA` field. The allowed values are `Proxyonly` and `Always`. Use `Proxyonly` to reference the `user-ca-bundle` config map only when you configure an `http/https` proxy. Use `Always` to always reference the `user-ca-bundle` config map. The default value is `Proxyonly`. Optional parameter.

   The installation program does not support the proxy `readinessEndpoints` field.

   If the installation program times out, restart and then complete the deployment by using the `wait-for` command of the installation program. For example:

   ```
   $ ./openshift-install wait-for install-complete --log-level debug
   ```

2. Save the file and reference it when installing OpenShift Container Platform.

   The installation program creates a cluster-wide proxy named `cluster` that uses the proxy settings in the `install-config.yaml` file. If you do not give proxy settings, the installation program still creates a `cluster` `Proxy` object, but it has a nil `spec`.

   Only the `Proxy` object named `cluster` is supported, and you cannot create additional proxies.

3.3.5. Alternatives to storing administrator-level secrets in the kube-system project

By default, administrator secrets are stored in the `kube-system` project.

If you configured the `credentialsMode` parameter in the `install-config.yaml` file to `Manual`, you must use one of the following alternatives:

- To manage long-term cloud credentials manually, follow the procedure in "Manually creating long-term credentials".
- To implement short-term credentials that are managed outside the cluster for individual components, follow the procedures in "Configuring an AWS cluster to use short-term credentials".

3.3.5.1. Manually creating long-term credentials

You can put the Cloud Credential Operator (CCO) into manual mode before OpenShift Container Platform installation if the cloud identity and access management (IAM) APIs are not reachable, or if you prefer not to store an administrator-level credential secret in the cluster `kube-system` namespace.

Procedure

1. If you did not set the `credentialsMode` parameter in the `install-config.yaml` configuration file to `Manual`, modify the value as shown:

   Sample configuration file snippet

   ```
   apiVersion: v1
   baseDomain: example.com
   credentialsMode: Manual
   ```

2. If you have not previously created installation manifest files, do so by running the following command:

   ```
   $ openshift-install create manifests --dir <installation_directory>
   ```

   where `<installation_directory>` is the directory in which the installation program creates files.

3. Set a `$RELEASE_IMAGE` variable with the release image from your installation file by running the following command:

   ```
   $ RELEASE_IMAGE=$(./openshift-install version | awk '/release image/ {print $3}')
   ```

4. Extract the list of `CredentialsRequest` custom resources (CRs) from the OpenShift Container Platform release image by running the following command:

```
$ oc adm release extract \
  --from=$RELEASE_IMAGE \
  --credentials-requests \
  --included \
  --install-config=<path_to_directory_with_installation_configuration>/install-config.yaml \
  --to=<path_to_directory_for_credentials_requests>
```

where:

`--included`
: Specifies only the manifests that your specific cluster configuration requires.

`<path_to_directory_with_installation_configuration>`
: Specifies the location of the `install-config.yaml` file.

`<path_to_directory_for_credentials_requests>`
: Specifies the path to the directory where you want to store the `CredentialsRequest` objects. If the specified directory does not exist, this command creates it.

  This command creates a YAML file for each `CredentialsRequest` object.

  Sample `CredentialsRequest` object

```
apiVersion: cloudcredential.openshift.io/v1
kind: CredentialsRequest
metadata:
  name: <component_credentials_request>
  namespace: openshift-cloud-credential-operator
  ...
spec:
  providerSpec:
    apiVersion: cloudcredential.openshift.io/v1
    kind: AWSProviderSpec
    statementEntries:
    - effect: Allow
      action:
      - iam:GetUser
      - iam:GetUserPolicy
      - iam:ListAccessKeys
      resource: "*"
  ...
```

5. Create YAML files for secrets in the `openshift-install` manifests directory that you generated previously. The secrets must be stored using the namespace and secret name defined in the `spec.secretRef` for each `CredentialsRequest` object.

   Sample `CredentialsRequest` object with secrets

```
apiVersion: cloudcredential.openshift.io/v1
kind: CredentialsRequest
metadata:
  name: <component_credentials_request>
  namespace: openshift-cloud-credential-operator
  ...
spec:
  providerSpec:
    apiVersion: cloudcredential.openshift.io/v1
    kind: AWSProviderSpec
    statementEntries:
    - effect: Allow
      action:
      - s3:CreateBucket
      - s3:DeleteBucket
      resource: "*"
      ...
  secretRef:
    name: <component_secret>
    namespace: <component_namespace>
  ...
```

   Sample `Secret` object

```
apiVersion: v1
kind: Secret
metadata:
  name: <component_secret>
  namespace: <component_namespace>
data:
  aws_access_key_id: <base64_encoded_aws_access_key_id>
  aws_secret_access_key: <base64_encoded_aws_secret_access_key>
```

   Before upgrading a cluster that uses manually maintained credentials, you must ensure that the CCO is in an upgradeable state.

3.3.5.2. Configuring an AWS cluster to use short-term credentials

To install a cluster that is configured to use the AWS Security Token Service (STS), you must configure the CCO utility and create the required AWS resources for your cluster.

3.3.5.2.1. Configuring the Cloud Credential Operator utility

To create and manage cloud credentials from outside of the cluster when the Cloud Credential Operator (CCO) is operating in manual mode, extract and prepare the CCO utility (`ccoctl`) binary.

The `ccoctl` utility is a Linux binary that must run in a Linux environment.

Prerequisites

- You have access to an OpenShift Container Platform account with cluster administrator access.
- You have installed the OpenShift CLI (`oc`).

- You have created an AWS account for the `ccoctl` utility to use with the following permissions:

  Required `iam` permissions

  - `iam:CreateOpenIDConnectProvider`
  - `iam:CreateRole`
  - `iam:DeleteOpenIDConnectProvider`
  - `iam:DeleteRole`
  - `iam:DeleteRolePolicy`
  - `iam:GetOpenIDConnectProvider`
  - `iam:GetRole`
  - `iam:GetUser`
  - `iam:ListOpenIDConnectProviders`
  - `iam:ListRolePolicies`
  - `iam:ListRoles`
  - `iam:PutRolePolicy`
  - `iam:TagOpenIDConnectProvider`
  - `iam:TagRole`

  Required `s3` permissions

  - `s3:CreateBucket`
  - `s3:DeleteBucket`
  - `s3:DeleteObject`
  - `s3:GetBucketAcl`
  - `s3:GetBucketTagging`
  - `s3:GetObject`
  - `s3:GetObjectAcl`
  - `s3:GetObjectTagging`
  - `s3:ListBucket`
  - `s3:PutBucketAcl`
  - `s3:PutBucketPolicy`
  - `s3:PutBucketPublicAccessBlock`
  - `s3:PutBucketTagging`
  - `s3:PutObject`
  - `s3:PutObjectAcl`
  - `s3:PutObjectTagging`

  Required `cloudfront` permissions

  - `cloudfront:ListCloudFrontOriginAccessIdentities`
  - `cloudfront:ListDistributions`
  - `cloudfront:ListTagsForResource`

- If you plan to store the OIDC configuration in a private S3 bucket that is accessed by the IAM identity provider through a public CloudFront distribution URL, the AWS account that runs the `ccoctl` utility requires the following additional permissions:
  - `cloudfront:CreateCloudFrontOriginAccessIdentity`
  - `cloudfront:CreateDistribution`
  - `cloudfront:DeleteCloudFrontOriginAccessIdentity`
  - `cloudfront:DeleteDistribution`
  - `cloudfront:GetCloudFrontOriginAccessIdentity`
  - `cloudfront:GetCloudFrontOriginAccessIdentityConfig`
  - `cloudfront:GetDistribution`
  - `cloudfront:TagResource`
  - `cloudfront:UpdateDistribution`

  These additional permissions support the use of the `--create-private-s3-bucket` option when processing credentials requests with the `ccoctl aws create-all` command.

Procedure

1. Set a variable for the OpenShift Container Platform release image by running the following command:

   ```
   $ RELEASE_IMAGE=$(./openshift-install version | awk '/release image/ {print $3}')
   ```

2. Obtain the CCO container image from the OpenShift Container Platform release image by running the following command:

   ```
   $ CCO_IMAGE=$(oc adm release info --image-for='cloud-credential-operator' $RELEASE_IMAGE -a
   ~/.pull-secret)
   ```

   Ensure that the architecture of the `$RELEASE_IMAGE` matches the architecture of the environment in which you will use the `ccoctl` tool.

3. Extract the `ccoctl` binary from the CCO container image within the OpenShift Container Platform release image by running the following command:

   ```
   $ oc image extract $CCO_IMAGE \
     --file="/usr/bin/ccoctl.<rhel_version>" \
     -a ~/.pull-secret
   ```

   For `<rhel_version>`, specify the value that corresponds to the version of Red Hat Enterprise Linux (RHEL) that the host uses. If no value is specified, `ccoctl.rhel8` is used by default. The following values are valid:

   - `rhel8`: Specify this value for hosts that use RHEL 8.
   - `rhel9`: Specify this value for hosts that use RHEL 9.

   The `ccoctl` binary is created in the directory from where you executed the command and not in `/usr/bin/`. You must rename the directory or move the `ccoctl.<rhel_version>` binary to `ccoctl`.

4. Change the permissions to make `ccoctl` executable by running the following command:

   ```
   $ chmod 775 ccoctl
   ```

Verification

To verify that `ccoctl` is ready to use, display the help file. Use a relative file name when you run the command, for example:

```
$ ./ccoctl
```

Example output

```
OpenShift credentials provisioning tool

Usage:
  ccoctl [command]

Available Commands:
  aws          Manage credentials objects for AWS cloud
  azure        Manage credentials objects for Azure
  gcp          Manage credentials objects for Google cloud
  help         Help about any command
  ibmcloud     Manage credentials objects for IBM Cloud
  nutanix      Manage credentials objects for Nutanix

Flags:
  -h, --help   help for ccoctl

Use "ccoctl [command] --help" for more information about a command.
```

3.3.5.2.2. Creating AWS resources with a single command

If the process the `ccoctl` tool uses to create AWS resources automatically meets the requirements of your organization, you can use the `ccoctl aws create-all` command to automate the creation of AWS resources.

Otherwise, you can create the AWS resources individually. For more information, see "Creating AWS resources individually".

By default, `ccoctl` creates objects in the directory in which the commands are run. To create the objects in a different directory, use the `--output-dir` flag. This procedure uses `<path_to_ccoctl_output_dir>` to refer to this directory.

Prerequisites

You must have:

- Extracted and prepared the `ccoctl` binary.

Procedure

1. Set a `$RELEASE_IMAGE` variable with the release image from your installation file by running the following command:

```
$ RELEASE_IMAGE=$(./openshift-install version | awk '/release image/ {print $3}')
```

2. Extract the list of `CredentialsRequest` objects from the OpenShift Container Platform release image by running the following command:

```
$ oc adm release extract \
  --from=$RELEASE_IMAGE \
  --credentials-requests \
  --included \
  --install-config=<path_to_directory_with_installation_configuration>/install-config.yaml \
  --to=<path_to_directory_for_credentials_requests>
```

where:

`--included`
: Specifies to include only the manifests that your specific cluster configuration requires.

`<path_to_directory_with_installation_configuration>`
: Specifies the location of the `install-config.yaml` file.

`<path_to_directory_for_credentials_requests>`
: Specifies the path to the directory where you want to store the `CredentialsRequest` objects. If the specified directory does not exist, this command creates it.

  This command might take a few moments to run.

3. Use the `ccoctl` tool to process all `CredentialsRequest` objects by running the following command:

```
$ ccoctl aws create-all \
  --name=<name> \
  --region=<aws_region> \
  --credentials-requests-dir=<path_to_credentials_requests_directory> \
  --output-dir=<path_to_ccoctl_output_dir> \
  --create-private-s3-bucket \
  --permissions-boundary-arn=<policy_arn>
```

where:

`<name>`
: Specifies the name used to tag any cloud resources that are created for tracking.

`<aws_region>`
: Specifies the AWS region in which cloud resources will be created.

`<path_to_credentials_requests_directory>`
: Specifies the directory containing the files for the component `CredentialsRequest` objects.

`<path_to_ccoctl_output_dir>`
: Specifies the directory in which you want the `ccoctl` utility to create objects. By default, the utility creates objects in the directory in which the commands are run. This parameter is optional.

`--create-private-s3-bucket`
: Specifies that the OpenID Connect (OIDC) configuration files should be stored in a private S3 bucket that is accessed by the IAM identity provider through a public CloudFront distribution URL. Note that by default, the `ccoctl` utility stores the OIDC configuration files in a public S3 bucket and uses the S3 URL as the public OIDC endpoint. This parameter is optional.

`<policy_arn>`
: Specifies the Amazon Resource Name (ARN) of the AWS IAM policy to use as the permissions boundary for the IAM roles created by the `ccoctl` utility. This parameter is optional.

  If your cluster uses Technology Preview features that are enabled by the `TechPreviewNoUpgrade` feature set, you must include the `--enable-tech-preview` parameter.

Verification

To verify that the OpenShift Container Platform secrets are created, list the files in the `<path_to_ccoctl_output_dir>/manifests` directory:

```
$ ls <path_to_ccoctl_output_dir>/manifests
```

Example output

```
cluster-authentication-02-config.yaml
openshift-cloud-credential-operator-cloud-credential-operator-iam-ro-creds-credentials.yaml
openshift-cloud-network-config-controller-cloud-credentials-credentials.yaml
openshift-cluster-api-capa-manager-bootstrap-credentials-credentials.yaml
openshift-cluster-csi-drivers-ebs-cloud-credentials-credentials.yaml
openshift-image-registry-installer-cloud-credentials-credentials.yaml
openshift-ingress-operator-cloud-credentials-credentials.yaml
openshift-machine-api-aws-cloud-credentials-credentials.yaml
```

You can verify that the IAM roles are created by querying AWS. For more information, refer to AWS documentation on listing IAM roles.

3.3.5.2.3. Creating AWS resources individually

If you need to review the JSON files that the `ccoctl` tool creates before modifying AWS resources, or if the process the `ccoctl` tool uses to create AWS resources automatically does not meet the requirements of your organization, you can create the AWS resources individually.

This option might be useful for an organization that shares the responsibility for creating these resources among different users or departments.

Otherwise, you can use the `ccoctl aws create-all` command to create the AWS resources automatically. For more information, see "Creating AWS resources with a single command".

By default, `ccoctl` creates objects in the directory in which the commands are run. To create the objects in a different directory, use the `--output-dir` flag. This procedure uses `<path_to_ccoctl_output_dir>` to refer to this directory.

Some `ccoctl` commands make AWS API calls to create or modify AWS resources. You can use the `--dry-run` flag to avoid making API calls. Using this flag creates JSON files on the local file system instead. You can review and modify the JSON files and then apply them with the AWS CLI tool using the `--cli-input-json` parameters.

Prerequisites

Extract and prepare the `ccoctl` binary.

Procedure

1. Generate the public and private RSA key files that are used to set up the OpenID Connect provider for the cluster by running the following command:

```
$ ccoctl aws create-key-pair
```

Example output

```
2021/04/13 11:01:02 Generating RSA keypair
2021/04/13 11:01:03 Writing private key to /<path_to_ccoctl_output_dir>/serviceaccount-signer.private
2021/04/13 11:01:03 Writing public key to /<path_to_ccoctl_output_dir>/serviceaccount-signer.public
2021/04/13 11:01:03 Copying signing key for use by installer
```

where `serviceaccount-signer.private` and `serviceaccount-signer.public` are the generated key files.

This command also creates a private key that the cluster requires during installation in `/<path_to_ccoctl_output_dir>/tls/bound-service-account-signing-key.key`.

2. Create an OpenID Connect identity provider and S3 bucket on AWS by running the following command:

```
$ ccoctl aws create-identity-provider \
  --name=<name> \
  --region=<aws_region> \
  --public-key-file=<path_to_ccoctl_output_dir>/serviceaccount-signer.public
```

where:

`<name>`
: Specifies the name used to tag any cloud resources that are created for tracking.

`<aws_region>`
: Specifies the AWS region in which cloud resources will be created.

`<path_to_ccoctl_output_dir>`
: Specifies the path to the public key file that the `ccoctl aws create-key-pair` command generated.

Example output

```
2021/04/13 11:16:09 Bucket <name>-oidc created
2021/04/13 11:16:10 OpenID Connect discovery document in the S3 bucket <name>-oidc at .well-known/openid-configuration updated
2021/04/13 11:16:10 Reading public key
2021/04/13 11:16:10 JSON web key set (JWKS) in the S3 bucket <name>-oidc at keys.json updated
2021/04/13 11:16:18 Identity Provider created with ARN: arn:aws:iam::<aws_account_id>:oidc-provider/<name>-oidc.s3.<aws_region>.amazonaws.com
```

where `openid-configuration` is a discovery document and `keys.json` is a JSON web key set file.

This command also creates a YAML configuration file in `/<path_to_ccoctl_output_dir>/manifests/cluster-authentication-02-config.yaml`. This file sets the issuer URL field for the service account tokens that the cluster generates, so that the AWS IAM identity provider trusts the tokens.

3. Create IAM roles for each component in the cluster:

   a. Set a `$RELEASE_IMAGE` variable with the release image from your installation file by running the following command:

   ```
   $ RELEASE_IMAGE=$(./openshift-install version | awk '/release image/ {print $3}')
   ```

   b. Extract the list of `CredentialsRequest` objects from the OpenShift Container Platform release image:

   ```
   $ oc adm release extract \
     --from=$RELEASE_IMAGE \
     --credentials-requests \
     --included \
     --install-config=<path_to_directory_with_installation_configuration>/install-config.yaml
   \
     --to=<path_to_directory_for_credentials_requests>
   ```

   where:

   `--included`
   Specifies the `--included` parameter, which includes only the manifests that your specific cluster configuration requires.

   `--install-config`
   Specifies the location of the `install-config.yaml` file.

   `-to`
   Specifies the path to the directory where you want to store the `CredentialsRequest` objects. If the specified directory does not exist, this command creates it.

   c. Use the `ccoctl` tool to process all `CredentialsRequest` objects by running the following command:

   ```
   $ ccoctl aws create-iam-roles \
     --name=<name> \
     --region=<aws_region> \
     --credentials-requests-dir=<path_to_credentials_requests_directory> \
     --identity-provider-arn=arn:aws:iam::<aws_account_id>:oidc-provider/<name>-oidc.s3.
   <aws_region>.amazonaws.com
   ```

   For AWS environments that use alternative IAM API endpoints, such as GovCloud, you must also specify your region with the `--region` parameter.

   If your cluster uses Technology Preview features that are enabled by the `TechPreviewNoUpgrade` feature set, you must include the `--enable-tech-preview` parameter.

   For each `CredentialsRequest` object, `ccoctl` creates an IAM role with a trust policy that is tied to the specified OIDC identity provider, and a permissions policy as defined in each `CredentialsRequest` object from the OpenShift Container Platform release image.

Verification

To verify that the OpenShift Container Platform secrets are created, list the files in the `<path_to_ccoctl_output_dir>/manifests` directory:

```
$ ls <path_to_ccoctl_output_dir>/manifests
```

Example output

```
cluster-authentication-02-config.yaml
openshift-cloud-credential-operator-cloud-credential-operator-iam-ro-creds-credentials.yaml
openshift-cloud-network-config-controller-cloud-credentials-credentials.yaml
openshift-cluster-api-capa-manager-bootstrap-credentials-credentials.yaml
openshift-cluster-csi-drivers-ebs-cloud-credentials-credentials.yaml
openshift-image-registry-installer-cloud-credentials-credentials.yaml
openshift-ingress-operator-cloud-credentials-credentials.yaml
openshift-machine-api-aws-cloud-credentials-credentials.yaml
```

You can verify that the IAM roles are created by querying AWS. For more information, refer to AWS documentation on listing IAM roles.

3.3.5.2.4. Incorporating the Cloud Credential Operator utility manifests

To implement short-term security credentials managed outside the cluster for individual OpenShift Container Platform components, you must move the manifest files that the Cloud Credential Operator utility (`ccoctl`) created to the correct directories for the installation program.

Prerequisites

- You have configured an account with the cloud platform that hosts your cluster.
- You have configured the Cloud Credential Operator utility (`ccoctl`).
- You have created the cloud provider resources that are required for your cluster with the `ccoctl` utility.

Procedure

1. If you did not set the `credentialsMode` parameter in the `install-config.yaml` configuration file to `Manual`, modify the value as shown:

   Sample configuration file snippet

   ```
   apiVersion: v1
   baseDomain: example.com
   credentialsMode: Manual
   ```

2. If you have not previously created installation manifest files, do so by running the following command:

   ```
   $ openshift-install create manifests --dir <installation_directory>
   ```

   where `<installation_directory>` is the directory in which the installation program creates files.

3. Copy the manifests that the `ccoctl` utility generated to the `manifests` directory that the installation program created by running the following command:

   ```
   $ cp /<path_to_ccoctl_output_dir>/manifests/* ./manifests/
   ```

4. Copy the `tls` directory that contains the private key to the installation directory:

   ```
   $ cp -a /<path_to_ccoctl_output_dir>/tls .
   ```

3.3.6. Cluster Network Operator configuration

To manage cluster networking, configure the Cluster Network Operator (CNO) `Network` custom resource (CR) named `cluster` so the cluster uses the correct IP ranges and network plugin settings for reliable pod

and service connectivity.

Some settings and fields are inherited at the time of install or by the `default.Network.type` plugin, OVN-Kubernetes.

The CNO configuration inherits the following fields during cluster installation from the `Network` API in the `Network.config.openshift.io` API group:

`clusterNetwork`
IP address pools from which pod IP addresses are allocated.

`serviceNetwork`
IP address pool for services.

`defaultNetwork.type`
Cluster network plugin. `OVNKubernetes` is the only supported plugin during installation.

You can specify the cluster network plugin configuration for your cluster by setting the fields for the `defaultNetwork` object in the CNO object named `cluster`.

3.3.6.1. Cluster Network Operator configuration object

The fields for the Cluster Network Operator (CNO) are described in the following table:

Field	Type	Description
`metadata.name`	`string`	The name of the CNO object. This name is always `cluster`.
`spec.clusterNetwork`	`array`	A list specifying the blocks of IP addresses from which pod IP addresses are allocated and the subnet prefix length assigned to each individual node in the cluster. If you use dual-stack networking, specify IPv4 and IPv6 address families. For example: `spec: clusterNetwork: - cidr: 10.128.0.0/19 hostPrefix: 23 - cidr: fd01::/48 hostPrefix: 64` If you install a cluster on AWS with dual-stack networking, the order of addresses must match the dual-stack configuration you selected. For example, if you specified the `DualStackIPv4Primary`, list the IPv4 address first.

Field	Type	Description
`spec.serviceNetwork`	`array`	A block of IP addresses for services. If you use dual-stack networking, specify IPv4 and IPv6 address families. For example: `spec: serviceNetwork: - 172.30.0.0/14 - fd02::/112` If you install a cluster on AWS with dual-stack networking, the order of addresses must match the dual-stack configuration you selected. For example, if you specified the `DualStackIPv4Primary`, list the IPv4 address first. You can customize this field only in the `install-config.yaml` file before you create the manifests. The value is read-only in the manifest file.
`spec.defaultNetwork`	`object`	Configures the network plugin for the cluster network.
`spec.additionalRoutingCapabilities.providers`	`array`	This setting enables a dynamic routing provider. The FRR routing capability provider is required for the route advertisement feature. The only supported value is `FRR`. `FRR`: The FRR routing provider `spec: additionalRoutingCapabilities: providers: - FRR`

Table 3.2. Cluster Network Operator configuration object

For a cluster that needs to deploy objects across multiple networks, ensure that you specify the same value for the `clusterNetwork.hostPrefix` parameter for each network type that is defined in the `install-config.yaml` file. Setting a different value for each `clusterNetwork.hostPrefix` parameter can impact the OVN-Kubernetes network plugin, where the plugin cannot effectively route object traffic among different nodes.

3.3.6.2. defaultNetwork object configuration

The values for the `defaultNetwork` object are defined in the following table:

Field	Type	Description
`type`	`string`	`OVNKubernetes`. The Red Hat OpenShift Networking network plugin is selected during installation. This value cannot be changed after cluster installation. Note OpenShift Container Platform uses the OVN-Kubernetes network plugin by default.
`ovnKubernetesConfig`	`object`	This object is only valid for the OVN-Kubernetes network plugin.

Table 3.3. defaultNetwork object

3.3.6.3. Configuration for the OVN-Kubernetes network plugin

The following table describes the configuration fields for the OVN-Kubernetes network plugin:

Field	Type	Description
`mtu`	`integer`	The maximum transmission unit (MTU) for the Geneve (Generic Network Virtualization Encapsulation) overlay network. This is detected automatically based on the MTU of the primary network interface. You do not normally need to override the detected MTU. If the auto-detected value is not what you expect it to be, confirm that the MTU on the primary network interface on your nodes is correct. You cannot use this option to change the MTU value of the primary network interface on the nodes. If your cluster requires different MTU values for different nodes, you must set this value to `100` less than the lowest MTU value in your cluster. For example, if some nodes in your cluster have an MTU of `9001`, and some have an MTU of `1500`, you must set this value to `1400`.
`genevePort`	`integer`	The port to use for all Geneve packets. The default value is `6081`. This value cannot be changed after cluster installation.
`ipsecConfig`	`object`	Specify a configuration object for customizing the IPsec configuration.
`ipv4`	`object`	Specifies a configuration object for IPv4 settings.
`ipv6`	`object`	Specifies a configuration object for IPv6 settings.
`policyAuditConfig`	`object`	Specify a configuration object for customizing network policy audit logging. If unset, the defaults audit log settings are used.

Field	Type	Description
routeAdvertisements	string	Specifies whether to advertise cluster network routes. The default value is Disabled. • Enabled: Import routes to the cluster network and advertise cluster network routes as configured in RouteAdvertisements objects. • Disabled: Do not import routes to the cluster network or advertise cluster network routes.
gatewayConfig	object	Optional: Specify a configuration object for customizing how egress traffic is sent to the node gateway. Valid values are Shared and Local. The default value is Shared. In the default setting, the Open vSwitch (OVS) outputs traffic directly to the node IP interface. If you are using hardware offloading, Red Hat recommends to use the default Shared gateway mode to bypass the host routing plane. In the Local setting, it traverses the host network; consequently, it gets applied to the routing table of the host. Note While migrating egress traffic, you can expect some disruption to workloads and service traffic until the Cluster Network Operator (CNO) successfully rolls out the changes.

Table 3.4. ovnKubernetesConfig object

Field	Type	Description
internalTransitSwitchSubnet	string	If your existing network infrastructure overlaps with the 100.88.0.0/16 IPv4 subnet, you can specify a different IP address range for internal use by OVN-Kubernetes. The subnet for the distributed transit switch that enables east-west traffic. This subnet cannot overlap with any other subnets used by OVN-Kubernetes or on the host itself. It must be large enough to accommodate one IP address per node in your cluster. The default value is 100.88.0.0/16.
internalJoinSubnet	string	If your existing network infrastructure overlaps with the 100.64.0.0/16 IPv4 subnet, you can specify a different IP address range for internal use by OVN-Kubernetes. You must ensure that the IP address range does not overlap with any other subnet used by your OpenShift Container Platform installation. The IP address range must be larger than the maximum number of nodes that can be added to the cluster. For example, if the clusterNetwork.cidr value is 10.128.0.0/14 and the clusterNetwork.hostPrefix value is /23, then the maximum number of nodes is 2^(23-14)=512. The default value is 100.64.0.0/16.

Table 3.5. ovnKubernetesConfig.ipv4 object

Field	Type	Description
`internalTransitSwitchSubnet`	string	If your existing network infrastructure overlaps with the `fd97::/64` IPv6 subnet, you can specify a different IP address range for internal use by OVN-Kubernetes. The subnet for the distributed transit switch that enables east-west traffic. This subnet cannot overlap with any other subnets used by OVN-Kubernetes or on the host itself. It must be large enough to accommodate one IP address per node in your cluster. The default value is `fd97::/64`.
`internalJoinSubnet`	string	If your existing network infrastructure overlaps with the `fd98::/64` IPv6 subnet, you can specify a different IP address range for internal use by OVN-Kubernetes. You must ensure that the IP address range does not overlap with any other subnet used by your OpenShift Container Platform installation. The IP address range must be larger than the maximum number of nodes that can be added to the cluster. The default value is `fd98::/64`.

Table 3.6. ovnKubernetesConfig.ipv6 object

Field	Type	Description
`rateLimit`	integer	The maximum number of messages to generate every second per node. The default value is `20` messages per second.
`maxFileSize`	integer	The maximum size for the audit log in bytes. The default value is `50000000` or 50 MB.
`maxLogFiles`	integer	The maximum number of log files that are retained.
`destination`	string	One of the following additional audit log targets: **`libc`** The libc `syslog()` function of the journald process on the host. **`udp:<host>:<port>`** A syslog server. Replace `<host>:<port>` with the host and port of the syslog server. **`unix:<file>`** A UNIX Domain Socket file specified by `<file>`. **`null`** Do not send the audit logs to any additional target.
`syslogFacility`	string	The syslog facility, such as `kern`, as defined by RFC5424. The default value is `local0`.

Table 3.7. policyAuditConfig object

Field	Type	Description
routingViaHost	boolean	Set this field to `true` to send egress traffic from pods to the host networking stack. For highly-specialized installations and applications that rely on manually configured routes in the kernel routing table, you might want to route egress traffic to the host networking stack. By default, egress traffic is processed in OVN to exit the cluster and is not affected by specialized routes in the kernel routing table. The default value is `false`. This field has an interaction with the Open vSwitch hardware offloading feature. If you set this field to `true`, you do not receive the performance benefits of the offloading because egress traffic is processed by the host networking stack.
ipForwarding	object	You can control IP forwarding for all traffic on OVN-Kubernetes managed interfaces by using the `ipForwarding` specification in the `Network` resource. Specify `Restricted` to only allow IP forwarding for Kubernetes related traffic. Specify `Global` to allow forwarding of all IP traffic. For new installations, the default is `Restricted`. For updates to OpenShift Container Platform 4.14 or later, the default is `Global`. Note The default value of `Restricted` sets the IP forwarding to drop.
ipv4	object	Optional: Specify an object to configure the internal OVN-Kubernetes masquerade address for host to service traffic for IPv4 addresses.
ipv6	object	Optional: Specify an object to configure the internal OVN-Kubernetes masquerade address for host to service traffic for IPv6 addresses.

Table 3.8. gatewayConfig object

Field	Type	Description
internalMasqueradeSubnet	string	The masquerade IPv4 addresses that are used internally to enable host to service traffic. The host is configured with these IP addresses and the shared gateway bridge interface. The default value is `169.254.169.0/29`. Important For OpenShift Container Platform 4.17 and later versions, clusters use `169.254.0.0/17` as the default masquerade subnet. For upgraded clusters, there is no change to the default masquerade subnet.

Table 3.9. gatewayConfig.ipv4 object

Field	Type	Description
`internalMasqueradeSubnet`	`string`	The masquerade IPv6 addresses that are used internally to enable host to service traffic. The host is configured with these IP addresses and the shared gateway bridge interface. The default value is `fd69::/125`. Important For OpenShift Container Platform 4.17 and later versions, clusters use `fd69::/112` as the default masquerade subnet. For upgraded clusters, there is no change to the default masquerade subnet.

Table 3.10. gatewayConfig.ipv6 object

Field	Type	Description
`mode`	`string`	Specifies the behavior of the IPsec implementation. Must be one of the following values: • `Disabled`: IPsec is not enabled on cluster nodes. • `External`: IPsec is enabled for network traffic with external hosts. • `Full`: IPsec is enabled for pod traffic and network traffic with external hosts.

Table 3.11. ipsecConfig object

Example OVN-Kubernetes configuration with IPsec enabled

```
defaultNetwork:
  type: OVNKubernetes
  ovnKubernetesConfig:
    mtu: 1400
    genevePort: 6081
    ipsecConfig:
      mode: Full
```

3.3.7. Specifying advanced network configuration

You can use advanced network configuration for your OpenShift Container Platform network plugin to integrate your cluster into your existing network environment.

You can specify advanced network configuration only before you install the cluster.

Customizing your network configuration by modifying the OpenShift Container Platform manifest files created by the installation program is not supported. Applying a manifest file that you create, as in the following procedure, is supported.

Prerequisites

- You have created the `install-config.yaml` file and completed any modifications to it.

Procedure

1. Change to the directory that contains the installation program and create the manifests:

   ```
   $ ./openshift-install create manifests --dir <installation_directory>
   ```

   where `<installation_directory>` specifies the name of the directory that contains the `install-config.yaml` file for your cluster.

2. Create a stub manifest file for the advanced network configuration that is named `cluster-network-03-config.yml` in the `<installation_directory>/manifests/` directory:

   ```
   apiVersion: operator.openshift.io/v1
   kind: Network
   metadata:
     name: cluster
   spec:
   ```

3. Specify the advanced network configuration for your cluster in the `cluster-network-03-config.yml` file, such as in the following example:

   The following example enables IPsec for the OVN-Kubernetes network provider:

   ```
   apiVersion: operator.openshift.io/v1
   kind: Network
   metadata:
     name: cluster
   spec:
     defaultNetwork:
       ovnKubernetesConfig:
         ipsecConfig:
           mode: Full
   ```

4. Optional: Back up the `manifests/cluster-network-03-config.yml` file. The installation program consumes the `manifests/` directory when you create the Ignition config files.
5. Remove the Kubernetes manifest files that define the control plane machines and compute `MachineSets`:

   ```
   $ rm -f openshift/99_openshift-cluster-api_master-machines-*.yaml openshift/99_openshift-cluster-api_worker-machineset-*.yaml
   ```

   Because you create and manage these resources yourself, you do not have to initialize them.

   > You can preserve the `MachineSet` files to create compute machines by using the machine API, but you must update references to them to match your environment.

3.3.8. Configuring an Ingress Controller Network Load Balancer on a new AWS cluster

You can create an Ingress Controller backed by an Amazon Web Services Network Load Balancer (NLB) on a new cluster in situations where you need more transparent networking capabilities.

Prerequisites

> Create and edit the `install-config.yaml` file. For instructions, see "Creating the installation configuration file" in the *Additonal resources* section.

Procedure

1. Change to the directory that contains the installation program and create the manifests:

```
$ ./openshift-install create manifests --dir <installation_directory>
```

   For `<installation_directory>`, specify the name of the directory that contains the `install-config.yaml` file for your cluster.

2. Create a file that is named `cluster-ingress-default-ingresscontroller.yaml` in the `<installation_directory>/manifests/` directory:

```
$ touch <installation_directory>/manifests/cluster-ingress-default-ingresscontroller.yaml
```

   `<installation_directory>`
   Specifies the directory name that contains the `manifests/` directory for your cluster.

3. Check the several network configuration files that exist in the `manifests/` directory by entering the following command:

```
$ ls <installation_directory>/manifests/cluster-ingress-default-ingresscontroller.yaml
```

   Example output

```
cluster-ingress-default-ingresscontroller.yaml
```

4. Open the `cluster-ingress-default-ingresscontroller.yaml` file in an editor and enter a custom resource (CR) that describes the Operator configuration you want:

```
apiVersion: operator.openshift.io/v1
kind: IngressController
metadata:
  creationTimestamp: null
  name: default
  namespace: openshift-ingress-operator
spec:
  endpointPublishingStrategy:
    loadBalancer:
      scope: External
      providerParameters:
        type: AWS
        aws:
          type: NLB
    type: LoadBalancerService
```

5. Save the `cluster-ingress-default-ingresscontroller.yaml` file and quit the text editor.
6. Optional: Back up the `manifests/cluster-ingress-default-ingresscontroller.yaml` file because the installation program deletes the `manifests/` directory during cluster creation.

3.3.9. Configure hybrid networking with OVN-Kubernetes

To configure hybrid networking with OVN-Kubernetes, you can set `hybridOverlayConfig` during installation or patch the Cluster Network Operator (CNO) after installation.

This configuration is necessary to run both Linux and Windows nodes in the same cluster.

Prerequisites

You defined `OVNKubernetes` for the `networking.networkType` parameter in the `install-config.yaml` file. See the installation documentation for configuring OpenShift Container Platform network customizations on your chosen cloud provider for more information.

Procedure

1. Change to the directory that contains the installation program and create the manifests:

   ```
   $ ./openshift-install create manifests --dir <installation_directory>
   ```

   For the `<installation_directory>`, specify the name of the directory that contains the `install-config.yaml` file for your cluster.

2. Create a stub manifest file for the advanced network configuration that is named `cluster-network-03-config.yml` in the `<installation_directory>/manifests/` directory:

   ```
   $ cat <<EOF > <installation_directory>/manifests/cluster-network-03-config.yml
   apiVersion: operator.openshift.io/v1
   kind: Network
   metadata:
     name: cluster
   spec:
   EOF
   ```

   For `<installation_directory>`, specify the directory name that contains the `manifests/` directory for your cluster.

3. Open the `cluster-network-03-config.yml` file in an editor and specify a hybrid networking configuration similar to the following example:

   ```
   apiVersion: operator.openshift.io/v1
   kind: Network
   metadata:
     name: cluster
   spec:
     defaultNetwork:
       ovnKubernetesConfig:
         hybridOverlayConfig:
           hybridClusterNetwork:
           - cidr: 10.132.0.0/14
             hostPrefix: 23
           hybridOverlayVXLANPort: 9898
   ```

   where:

   `spec.defaultNetwork.ovnKubernetesConfig.hybridOverlayConfig.hybridClusterNetwork`
   Specifies the CIDR configuration used for nodes on the additional overlay network. The `hybridClusterNetwork` CIDR must not overlap with the `clusterNetwork` CIDR.

   `spec.defaultNetwork.ovnKubernetesConfig.hybridOverlayConfig.hybridOverlayVXLANPort`
   Specifies a custom VXLAN port for the additional overlay network. This is required for running Windows nodes in a cluster installed on vSphere, and must not be configured for any other cloud provider. The custom port can be any open port excluding the default `6081` port. For more information on this requirement, see Pod-to-pod connectivity between hosts is broken in the Microsoft documentation.

   Windows Server Long-Term Servicing Channel (LTSC): Windows Server 2019 is not supported on clusters with a custom `hybridOverlayVXLANPort` value because this Windows server version does not support selecting a custom VXLAN port.

4. Save the `cluster-network-03-config.yml` file and quit the text editor.
5. Optional: Back up the `manifests/cluster-network-03-config.yml` file. The installation program deletes the `manifests/` directory when creating the cluster.

3.3.10. Deploying the cluster

To deploy your OpenShift Container Platform cluster, you initialize installation by running the `openshift-install create cluster` command from the directory that contains the installation program. The installation program provisions the required infrastructure and completes the cluster setup.

You can run the `create cluster` command of the installation program only once, during initial installation.

Prerequisites

- You have configured an account with the cloud platform that hosts your cluster.
- You have the OpenShift Container Platform installation program and the pull secret for your cluster.
- You have verified that the cloud provider account on your host has the correct permissions to deploy the cluster. An account with incorrect permissions causes the installation process to fail with an error message that displays the missing permissions.

Procedure

1. In the directory that contains the installation program, initialize the cluster deployment by running the following command:

   ```
   $ ./openshift-install create cluster --dir <installation_directory> \
       --log-level=info
   ```

   where:

   - `<installation_directory>`: Specifies the location of your customized `./install-config.yaml` file.
   - `--log-level`: Specifies the log level. To view different installation details, specify `warn`, `debug`, or `error` instead of `info`.
2. Optional: Remove or disable the `AdministratorAccess` policy from the IAM account that you used to install the cluster.

   The elevated permissions provided by the `AdministratorAccess` policy are required only during installation.

Verification

When the cluster deployment completes successfully:

- The terminal displays directions for accessing your cluster, including a link to the web console and credentials for the `kubeadmin` user.

- Credential information also outputs to `<installation_directory>/.openshift_install.log`.

  Do not delete the installation program or the files that the installation program creates. Both are required to delete the cluster.

  The following example shows the expected output:

```
...
INFO Install complete!
INFO To access the cluster as the system:admin user when using 'oc', run 'export
KUBECONFIG=/home/myuser/install_dir/auth/kubeconfig'
INFO Access the OpenShift web-console here: https://console-openshift-
console.apps.mycluster.example.com
INFO Login to the console with user: "kubeadmin", and password: "password"
INFO Time elapsed: 36m22s
```

  - The Ignition config files that the installation program generates contain certificates that expire after 24 hours, which are then renewed at that time. If the cluster is shut down before renewing the certificates and the cluster is later restarted after the 24 hours have elapsed, the cluster automatically recovers the expired certificates. The exception is that you must manually approve the pending `node-bootstrapper` certificate signing requests (CSRs) to recover kubelet certificates. See the documentation for *Recovering from expired control plane certificates* for more information.
  - It is recommended that you use Ignition config files within 12 hours after they are generated because the 24-hour certificate rotates from 16 to 22 hours after the cluster is installed. By using the Ignition config files within 12 hours, you can avoid installation failure if the certificate update runs during installation.

3.3.11. Provisioning your own DNS records

Use your cluster name and base cluster domain to configure a CNAME record for the API service `api.<cluster_name>.<base_domain>.` with the API load balancer DNS name, and provision a CNAME record for the Ingress hostname `*.apps.<cluster_name>.<base_domain>.` with the Ingress load balancer DNS name.

User-provisioned DNS is a Technology Preview feature only. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see Technology Preview Features Support Scope.

Prerequisites

You have installed the AWS CLI.

Procedure

1. Add the `userProvisionedDNS` parameter to the `install-config.yaml` file and enable the parameter. For more information, see "Enabling a user-managed DNS".
2. Install your cluster.
3. If you are installing a private cluster, set the `api_lb_name` variable by running the following command:

```
$ api_lb_name="${INFRA_ID}-int"
```

4. If you are installing a public cluster, set the `api_lb_name` variable by running the following command:

```
$ api_lb_name="${INFRA_ID}-ext"
```

5. To retrieve the DNS name of the API service, run the following command:

```
$ aws --region ${REGION} elbv2 describe-load-balancers --names ${api_lb_name} --query
'LoadBalancers[*].DNSName' --output text
```

6. Use the DNS name and your cluster name and base cluster domain to configure your own DNS record with the `api.<cluster_name>.<base_domain>.` hostname.
7. To retrieve the DNS name of the Ingress service, run the following command:

```
$ ingress_lb_name=$(aws --region ${REGION} resourcegroupstaggingapi get-resources --resource-
type-filters elasticloadbalancing:loadbalancer --tag-filters
Key=kubernetes.io/cluster/${INFRA_ID},Values=owned Key=kubernetes.io/service-
name,Values=openshift-ingress/router-default --query 'ResourceTagMappingList[*].ResourceARN |
[0]' --output text | awk -F'/' '{print $2}')
```

8. Run the following command, which uses the variable `ingress_lb_name` generated from the previous command:

```
$ aws --region ${REGION} elb describe-load-balancers --load-balancer-names ${ingress_lb_name}
--query 'LoadBalancerDescriptions[].DNSName' --output text
```

9. Use the DNS name and your cluster name and base cluster domain to configure your own DNS record with the `*.apps.<cluster_name>.<base_domain>.` hostname.

3.3.12. Logging in to the cluster by using the CLI

To log in to your cluster as the default system user, export the `kubeconfig` file. This configuration enables the CLI to authenticate and connect to the specific API server created during OpenShift Container Platform installation.

The `kubeconfig` file is specific to a cluster and OpenShift Container Platform generates it during installation.

Prerequisites

- You deployed an OpenShift Container Platform cluster.
- You installed the OpenShift CLI (`oc`).

Procedure

1. Export the `kubeadmin` credentials by running the following command:

```
$ export KUBECONFIG=<installation_directory>/auth/kubeconfig
```

   where:

   `<installation_directory>`
   Specifies the path to the directory that stores the installation files.

2. Verify you can run `oc` commands successfully using the exported configuration by running the following command:

```
$ oc whoami
```

   Example output

```
system:admin
```

Next steps

- "Customize your cluster"
- "Remote health reporting"

3.3.13. Logging in to the cluster by using the web console

To verify that your cluster deployed successfully and access its features, log in to the OpenShift Container Platform web console as the `kubeadmin` user.

Prerequisites

- You have access to the installation host.
- You completed a cluster installation and all cluster Operators are available.

Procedure

1. Obtain the password for the `kubeadmin` user from the `kubeadmin-password` file on the installation host:

   ```
   $ cat <installation_directory>/auth/kubeadmin-password
   ```

   Or, you can obtain the `kubeadmin` password from the `<installation_directory>/.openshift_install.log` log file on the installation host.

2. List the OpenShift Container Platform web console route:

   ```
   $ oc get routes -n openshift-console | grep 'console-openshift'
   ```

   Or, you can obtain the OpenShift Container Platform route from the `<installation_directory>/.openshift_install.log` log file on the installation host.

   Example output

   ```
   console     console-openshift-console.apps.<cluster_name>.<base_domain>            console
   https   reencrypt/Redirect   None
   ```

3. Navigate to the route detailed in the output of the preceding command in a web browser and log in as the `kubeadmin` user.

3.4. Installing a cluster on AWS in a disconnected environment

You can install a cluster on Amazon Web Services (AWS) in a restricted network by creating an internal mirror of the installation release content on an existing AWS Virtual Private Cloud (VPC). By using this configuration, you can deploy a cluster in an environment with limited internet connectivity to help ensure compliance with security policies.

3.4.1. Prerequisites

Before you install a cluster on Amazon Web Services (AWS) in a restricted network by using installer-provisioned infrastructure, you must meet several prerequisites.

The following prerequisites must be met:

- You reviewed details about the OpenShift Container Platform installation and update processes.
- You read the documentation on selecting a cluster installation method and preparing it for users.

- You mirrored the images for a disconnected installation to your registry and obtained the `imageContentSources` data for your version of OpenShift Container Platform.

  Because the installation media is on the mirror host, you can use that computer to complete all installation steps.

- You have an existing VPC in AWS. When installing to a restricted network using installer-provisioned infrastructure, you cannot use the installer-provisioned VPC. You must use a user-provisioned VPC that satisfies one of the following requirements:

  - Contains the mirror registry
  - Has firewall rules or a peering connection to access the mirror registry hosted elsewhere
- You configured an AWS account to host the cluster.

  If you have an AWS profile stored on your computer, it must not use a temporary session token that you generated while using a multi-factor authentication device. The cluster continues to use your current AWS credentials to create AWS resources for the entire life of the cluster, so you must use key-based, long-term credentials. To generate appropriate keys, see Managing Access Keys for IAM Users in the AWS documentation. You can supply the keys when you run the installation program.

- You downloaded the AWS CLI and installed it on your computer. See Install the AWS CLI Using the Bundled Installer (Linux, macOS, or UNIX) in the AWS documentation.
- If you use a firewall and plan to use the Telemetry service, you configured the firewall to allow the sites that your cluster requires access to.

  If you are configuring a proxy, be sure to also review this site list.

3.4.2. About installations in restricted networks

You can install OpenShift Container Platform 4.22 in a restricted network that lacks an active internet connection. These restricted installations use installer-provisioned or user-provisioned infrastructure, depending on your cloud platform.

If you choose to perform a restricted network installation on a cloud platform, you still require access to its cloud APIs. Some cloud functions, such as Amazon Web Service's Route 53 DNS and IAM services, require internet access. Depending on your network, you might require less internet access for an installation on bare-metal hardware, Nutanix, or on VMware vSphere.

To complete a restricted network installation, you must create a registry that mirrors the contents of the OpenShift image registry and contains the installation media. You can create this registry on a mirror host, which can access both the internet and your closed network, or by using other methods that meet your restrictions.

3.4.2.1. Additional limits

Clusters in restricted networks have the following additional limitations and restrictions:

- The `ClusterVersion` status includes an `Unable to retrieve available updates` error.
- By default, you cannot use the contents of the Developer Catalog because you cannot access the required image stream tags.

3.4.3. About using a custom VPC

You can deploy a cluster into existing subnets in an existing Amazon Virtual Private Cloud (VPC) in Amazon Web Services (AWS).

By deploying OpenShift Container Platform into an existing AWS VPC, you might be able to avoid limit constraints in new accounts or more easily abide by the operational constraints that your company's guidelines set. If you cannot obtain the infrastructure creation permissions that are required to create the VPC yourself, use this installation option.

Because the installation program cannot know what other components are also in your existing subnets, it cannot choose subnet CIDRs and so forth on your behalf. You must configure networking for the subnets that you install your cluster to yourself.

The installation program no longer creates the following components:

- Internet gateways
- NAT gateways
- Subnets
- Route tables
- VPCs
- VPC DHCP options
- VPC endpoints

The installation program requires that you use the cloud-provided DNS server. Using a custom DNS server is not supported and causes the installation to fail.

If you use a custom VPC, you must correctly configure it and its subnets for the installation program and the cluster to use. See "Create a VPC" in the Amazon Web Services documentation for more information about AWS VPC console wizard configurations and creating and managing an AWS VPC.

The installation program cannot:

- Subdivide network ranges for the cluster to use.
- Set route tables for the subnets.
- Set VPC options like DHCP.

You must complete these tasks before you install the cluster. See "VPC networking components" and "Route tables for your VPC" for more information on configuring networking in an AWS VPC.

Your VPC must meet the following characteristics:

- The VPC must not use the `kubernetes.io/cluster/.*: owned`, `Name`, and `openshift.io/cluster` tags.

  The installation program modifies your subnets to add the `kubernetes.io/cluster/.*: shared` tag, so your subnets must have at least one free tag slot available for it. See "Tag Restrictions" in the AWS documentation to confirm that the installation program can add a tag to each subnet that you specify. You cannot use a `Name` tag, because it overlaps with the EC2 `Name` field and the installation fails.

- If you want to extend your OpenShift Container Platform cluster into an AWS Outpost and have an existing Outpost subnet, the existing subnet must use the `kubernetes.io/cluster/unmanaged: true` tag. If you do not apply this tag, the installation might fail due to the Cloud Controller Manager creating a service load balancer in the Outpost subnet, which is an unsupported configuration.

- You must enable the `enableDnsSupport` and `enableDnsHostnames` attributes in your VPC, so that the cluster can use the Route 53 zones that are attached to the VPC to resolve cluster's internal DNS records. See "DNS Support in Your VPC" in the AWS documentation.

  If you prefer to use your own Route 53 hosted private zone, you must associate the existing hosted zone with your VPC prior to installing a cluster. You can define your hosted zone using the `platform.aws.hostedZone` and `platform.aws.hostedZoneRole` fields in the `install-config.yaml` file. You can use a private hosted zone from another account by sharing it with the account where you install the cluster. If you use a private hosted zone from another account, you must use the `Passthrough` or `Manual` credentials mode.

If you are working in a disconnected environment, you are unable to reach the public IP addresses for EC2, ELB, and S3 endpoints. Depending on the level to which you want to restrict internet traffic during the installation, the following configuration options are available:

3.4.3.1. Option 1: Create VPC endpoints

Create a VPC endpoint and attach it to the subnets that the clusters are using. Name the endpoints as follows:

- `ec2.<aws_region>.amazonaws.com`
- `elasticloadbalancing.<aws_region>.amazonaws.com`
- `s3.<aws_region>.amazonaws.com`

With this option, network traffic remains private between your VPC and the required AWS services.

3.4.3.2. Option 2: Create a proxy without VPC endpoints

As part of the installation process, you can configure an HTTP or HTTPS proxy. With this option, internet traffic goes through the proxy to reach the required AWS services.

3.4.3.3. Option 3: Create a proxy with VPC endpoints

As part of the installation process, you can configure an HTTP or HTTPS proxy with VPC endpoints. Create a VPC endpoint and attach it to the subnets that the clusters are using. Name the endpoints as follows:

- `ec2.<aws_region>.amazonaws.com`
- `elasticloadbalancing.<aws_region>.amazonaws.com`
- `s3.<aws_region>.amazonaws.com`

When configuring the proxy in the `install-config.yaml` file, add these endpoints to the `noProxy` field. With this option, the proxy prevents the cluster from accessing the internet directly. However, network traffic remains private between your VPC and the required AWS services.

You must provide a suitable VPC and subnets that allow communication to your machines.

<table>
<tr><th>Component</th><th>AWS type</th><th>Description</th></tr>
<tr><td>VPC</td><td><code>AWS::EC2::VPC</code>
<code>AWS::EC2::VPCEndpoint</code></td><td>You must provide a public VPC for the cluster to use. The VPC uses an endpoint that references the route tables for each subnet to improve communication with the registry that is hosted in S3.</td></tr>
<tr><td>Public subnets</td><td><code>AWS::EC2::Subnet</code>
<code>AWS::EC2::SubnetNetworkAclAssociation</code></td><td>Your VPC must have public subnets for between 1 and 3 availability zones and associate them with appropriate Ingress rules.</td></tr>
<tr><td>Internet gateway</td><td><code>AWS::EC2::InternetGateway</code>
<code>AWS::EC2::VPCGatewayAttachment</code>
<code>AWS::EC2::RouteTable</code>
<code>AWS::EC2::Route</code>
<code>AWS::EC2::SubnetRouteTableAssociation</code>
<code>AWS::EC2::NatGateway</code>
<code>AWS::EC2::EIP</code></td><td>You must have a public internet gateway, with public routes, attached to the VPC. In the provided templates, each public subnet has a NAT gateway with an EIP address. These NAT gateways allow cluster resources, like private subnet instances, to reach the internet and are not required for some restricted network or proxy scenarios.</td></tr>
<tr><td>Network access control</td><td><code>AWS::EC2::NetworkAcl</code>
<code>AWS::EC2::NetworkAclEntry</code></td><td>You must allow the VPC to access the following ports:<table><tr><th>Port</th><th>Reason</th></tr><tr><td><code>80</code></td><td>Inbound HTTP traffic</td></tr><tr><td><code>443</code></td><td>Inbound HTTPS traffic</td></tr><tr><td><code>22</code></td><td>Inbound SSH traffic</td></tr><tr><td><code>1024 - 65535</code></td><td>Inbound ephemeral traffic</td></tr><tr><td><code>0 - 65535</code></td><td>Outbound ephemeral traffic</td></tr></table></td></tr>
</table>

Component	AWS type	Description
Private subnets	• `AWS::EC2::Subnet` • `AWS::EC2::RouteTable` • `AWS::EC2::SubnetRouteTableAssociation`	Your VPC can have private subnets. The provided CloudFormation templates can create private subnets for between 1 and 3 availability zones. If you use private subnets, you must provide appropriate routes and tables for them.

Table 3.12. Required VPC components

3.4.3.4. VPC validation

To ensure that the subnets that you provide are suitable, the installation program confirms the following data:

- All the subnets that you specify exist.
- You provide private subnets.
- The subnet CIDRs belong to the machine CIDR that you specified.
- You provide subnets for each availability zone. Each availability zone contains no more than one public and one private subnet. If you use a private cluster, provide only a private subnet for each availability zone. Otherwise, provide exactly one public and private subnet for each availability zone.
- You provide a public subnet for each private subnet availability zone. Machines are not provisioned in availability zones that you do not provide private subnets for.

If you destroy a cluster that uses an existing VPC, the VPC is not deleted. When you remove the OpenShift Container Platform cluster from a VPC, the `kubernetes.io/cluster/.*: shared` tag is removed from the subnets that it used.

3.4.3.5. Division of permissions

Starting with OpenShift Container Platform 4.3, you do not need all of the permissions that are required for an installation program-provisioned infrastructure cluster to deploy a cluster. This change mimics the division of permissions that you might have at your company: some individuals can create different resource in your clouds than others. For example, you might be able to create application-specific items, like instances, buckets, and load balancers, but not networking-related components such as VPCs, subnets, or ingress rules.

The AWS credentials that you use when you create your cluster do not need the networking permissions that are required to make VPCs and core networking components within the VPC, such as subnets, routing tables, internet gateways, NAT, and VPN. You still need permission to make the application resources that the machines within the cluster require, such as ELBs, security groups, S3 buckets, and nodes.

3.4.3.6. Isolation between clusters

If you deploy OpenShift Container Platform to an existing network, the isolation of cluster services is reduced in the following ways:

- You can install multiple OpenShift Container Platform clusters in the same VPC.
- ICMP ingress is allowed from the entire network.
- TCP 22 ingress (SSH) is allowed to the entire network.
- Control plane TCP 6443 ingress (Kubernetes API) is allowed to the entire network.

- Control plane TCP 22623 ingress (MCS) is allowed to the entire network.

3.4.4. Creating the installation configuration file

You can customize the OpenShift Container Platform cluster you install on Amazon Web Services (AWS).

Prerequisites

- You have the OpenShift Container Platform installation program and the pull secret for your cluster. For a restricted network installation, these files are on your mirror host.
- You have the `imageContentSources` values that were generated during mirror registry creation.
- You have obtained the contents of the certificate for your mirror registry.

Procedure

1. Create the `install-config.yaml` file.

   a. Change to the directory that contains the installation program and run the following command:

   ```
   $ ./openshift-install create install-config --dir <installation_directory>
   ```

   - `<installation_directory>`: For `<installation_directory>`, specify the directory name to store the files that the installation program creates.

     When specifying the directory:

   - Verify that the directory has the `execute` permission. This permission is required to run Terraform binaries under the installation directory.
   - Use an empty directory. Some installation assets, such as bootstrap X.509 certificates, have short expiration intervals, therefore you must not reuse an installation directory. If you want to reuse individual files from another cluster installation, you can copy them into your directory. However, the file names for the installation assets might change between releases. Use caution when copying installation files from an earlier OpenShift Container Platform version.

   b. At the prompts, provide the configuration details for your cloud:

   i. Optional: Select an SSH key to use to access your cluster machines.

      For production OpenShift Container Platform clusters on which you want to perform installation debugging or disaster recovery, specify an SSH key that your `ssh-agent` process uses.

   ii. Select **AWS** as the platform to target.
   iii. If you do not have an Amazon Web Services (AWS) profile stored on your computer, enter the AWS access key ID and secret access key for the user that you configured to run the installation program.
   iv. Select the AWS region to deploy the cluster to.
   v. Select the base domain for the Route 53 service that you configured for your cluster.
   vi. Enter a descriptive name for your cluster.

2. Edit the `install-config.yaml` file to give the additional information that is required for an installation in a restricted network.

    a. Update the `pullSecret` value to contain the authentication information for your registry:

    ```
    pullSecret: '{"auths":{"<mirror_host_name>:5000": {"auth": "<credentials>","email":
    "you@example.com"}}}'
    ```

    For `<mirror_host_name>`, specify the registry domain name that you specified in the certificate for your mirror registry, and for `<credentials>`, specify the base64-encoded user name and password for your mirror registry.

    b. Add the `additionalTrustBundle` parameter and value.

    ```
    additionalTrustBundle: |
      -----BEGIN CERTIFICATE-----
      ZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZ
      -----END CERTIFICATE-----
    ```

    The value must be the contents of the certificate file that you used for your mirror registry. The certificate file can be an existing, trusted certificate authority, or the self-signed certificate that you generated for the mirror registry.

    c. Define the subnets for the VPC to install the cluster in, as in the following example:

    ```
    platform:
      aws:
        vpc:
          subnets:
            - id: subnet-<id_1>
            - id: subnet-<id_2>
            - id: subnet-<id_3>
    ```

    d. Add the image content resources, which resemble the following YAML excerpt:

    ```
    imageContentSources:
    - mirrors:
      - <mirror_host_name>:5000/<repo_name>/release
      source: quay.io/openshift-release-dev/ocp-release
    - mirrors:
      - <mirror_host_name>:5000/<repo_name>/release
      source: registry.redhat.io/ocp/release
    ```

    For these values, use the `imageContentSources` that you recorded during mirror registry creation.

    e. Optionally, set the publishing strategy to `Internal`:

    ```
    publish: Internal
    ```

    By setting this option, you create an internal Ingress Controller and a private load balancer.

3. Make any other modifications to the `install-config.yaml` file that you require.

    For more information about the parameters, see "Installation configuration parameters".

4. Back up the `install-config.yaml` file so that you can use it to install multiple clusters.

    The `install-config.yaml` file is consumed during the installation process. If you want to reuse the file, you must back it up now.

3.4.4.1. Minimum resource requirements for cluster installation

To ensure that your OpenShift Container Platform cluster runs as expected, each cluster machine must meet minimum CPU, memory, and storage requirements.

Machine	Operating system	vCPU	Virtual RAM	Storage	Input/Output Per Second (IOPS)
Bootstrap	RHCOS	4	16 GB	100 GB	300
Control plane	RHCOS	4	16 GB	100 GB	300
Compute	RHCOS	2	8 GB	100 GB	300

Table 3.13. Minimum resource requirements

- One vCPU is equal to one physical core when simultaneous multithreading (SMT), or Hyper-Threading, is not enabled. When enabled, use the following formula to calculate the corresponding ratio: (threads per core × cores) × sockets = vCPUs.
- OpenShift Container Platform and Kubernetes are sensitive to disk performance, and Red Hat recommends faster storage, particularly for etcd on the control plane nodes which require a 10 ms p99 fsync duration. On many cloud platforms, storage size and IOPS scale together, so you might need to provision more storage to get enough performance.
- As with all user-provisioned installations, if you choose to use RHEL compute machines in your cluster, you take responsibility for all operating system life cycle management and maintenance, including performing system updates, applying patches, and completing all other required tasks. OpenShift Container Platform 4.10 and later do not support RHEL 7 compute machines.

In OpenShift Container Platform version 4.22, RHCOS uses RHEL version 9.8, which updates the micro-architecture requirements. Each architecture requires the following minimum instruction set architectures (ISA):

- x86-64 architecture requires x86-64-v2 ISA
- ARM64 architecture requires ARMv8.0-A ISA
- ppc64le architecture requires IBM® Power9 ISA
- s390x architecture requires IBM® z14 ISA

For more information, see Architectures in the RHEL documentation.

If an instance type for your platform meets the minimum requirements for cluster machines, it is supported to use in OpenShift Container Platform.

3.4.4.2. Enabling a user-managed DNS

You can install a cluster with a domain name server (DNS) solution that you manage instead of the default cluster-provisioned DNS solution that uses the Route 53 service for Amazon Web Services (AWS).

For example, your organization's security policies might not allow the use of public DNS services such as Amazon Web Services DNS. In such scenarios, you can use your own DNS service to bypass the public DNS service and manage your own DNS for the IP addresses of the API and Ingress services.

If you enable user-managed DNS during installation, the installation program provisions DNS records for the API and Ingress services only within the cluster. To ensure access from outside the cluster, you must provision the DNS records in an external DNS service of your choice for the API and Ingress services after installation.

User-provisioned DNS is a Technology Preview feature only. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see Technology Preview Features Support Scope.

Procedure

Before you deploy your cluster, use a text editor to open the `install-config.yaml` file and add the following stanza:

To enable user-managed DNS:

```
featureSet: CustomNoUpgrade
featureGates: ["AWSClusterHostedDNSInstall=true"]
```

```
platform:
  aws:
    userProvisionedDNS: Enabled
```

where:

`userProvisionedDNS`
Enables user-provisioned DNS management.

Next steps

For information about provisioning your DNS records for the API server and the Ingress services, see "Provisioning your own DNS records".

3.4.4.3. Sample customized install-config.yaml file for AWS

You can customize the installation configuration file (`install-config.yaml`) to specify more details about your OpenShift Container Platform cluster's platform or modify the values of the required parameters.

This sample YAML file is provided for reference only. You must obtain your `install-config.yaml` file by using the installation program and modify it. For a full list and description of all installation configuration parameters, see *Installation configuration parameters for AWS*.

Sample `install-config.yaml` file for AWS

```
apiVersion: v1
baseDomain: example.com
sshKey: ssh-ed25519 AAAA...
pullSecret: '{"auths": ...}'
metadata:
  name: example-cluster
controlPlane:
  name: master
```

```
  platform:
    aws:
      type: m6i.xlarge
  replicas: 3
compute:
-  name: worker
  platform:
    aws:
      type: c5.4xlarge
  replicas: 3
networking:
  clusterNetwork:
  - cidr: 10.128.0.0/14
    hostPrefix: 23
platform:
  aws:
    region: us-west-2
```

- Parameters at the first level of indentation apply to the cluster globally.
- The `controlPlane` stanza applies to control plane machines.
- The `compute` stanza applies to compute machines.
- The `networking` stanza applies to the cluster networking configuration. If you do not provide networking values, the installation program provides default values.
- The `platform` stanza applies to the infrastructure platform that hosts the cluster.

3.4.4.4. Configuring the cluster-wide proxy during installation

Production environments can deny direct access to the internet and instead have an HTTP or HTTPS proxy available. You can configure a new OpenShift Container Platform cluster to use a proxy by configuring the proxy settings in the `install-config.yaml` file.

Prerequisites

- You have an existing `install-config.yaml` file.
- You have reviewed the sites that your cluster requires access to and determined whether any of them need to bypass the proxy. By default, the proxy handles all cluster egress traffic, including calls to hosting cloud provider APIs. You added sites to the `Proxy` object's `spec.noProxy` field to bypass the proxy if necessary.

  The `Proxy` object `status.noProxy` field includes the values of the `networking.machineNetwork[].cidr`, `networking.clusterNetwork[].cidr`, and `networking.serviceNetwork[]` fields from your installation configuration.

  For installations on Amazon Web Services (AWS), Google Cloud, Microsoft Azure, and Red Hat OpenStack Platform (RHOSP), the `Proxy` object `status.noProxy` field also includes the instance metadata endpoint (`169.254.169.254`).

Procedure

1. Edit your `install-config.yaml` file and add the proxy settings. For example:

```
apiVersion: v1
baseDomain: my.domain.com
proxy:
  httpProxy: http://<username>:<pswd>@<ip>:<port>
  httpsProxy: https://<username>:<pswd>@<ip>:<port>
  noProxy: ec2.<aws_region>.amazonaws.com,elasticloadbalancing.<aws_region>.amazonaws.com,s3.
<aws_region>.amazonaws.com
additionalTrustBundle: |
    -----BEGIN CERTIFICATE-----
    <MY_TRUSTED_CA_CERT>
    -----END CERTIFICATE-----
additionalTrustBundlePolicy: <policy_to_add_additionalTrustBundle>
```

   where:

   `proxy.httpProxy`
   : Specifies a proxy URL to use for creating HTTP connections outside the cluster. The URL scheme must be `http`.

   `proxy.httpsProxy`
   : Specifies a proxy URL to use for creating HTTPS connections outside the cluster.

   `proxy.noProxy`
   : Specifies a comma-separated list of destination domain names, IP addresses, or other network CIDRs to exclude from proxying. Preface a domain with `.` to match subdomains only. For example, `.y.com` matches `x.y.com`, but not `y.com`. Use `*` to bypass the proxy for all destinations. If you have added the Amazon `EC2`, `Elastic Load Balancing`, and `S3` VPC endpoints to your VPC, you must add these endpoints to the `noProxy` field.

   `additionalTrustBundle`
   : If you specify this value, the installation program generates a config map named `user-ca-bundle` in the `openshift-config` namespace to hold the additional CA certificates. If you specify `additionalTrustBundle` and at least one proxy setting, the `Proxy` object references the `user-ca-bundle` config map in the `trustedCA` field. The Cluster Network Operator then creates a `trusted-ca-bundle` config map that merges the contents specified for the `trustedCA` parameter with the RHCOS trust bundle. You must set the `additionalTrustBundle` field unless an authority from the RHCOS trust bundle signs the proxy's identity certificate.

   `additionalTrustBundlePolicy`
   : Specifies the policy that determines the configuration of the `Proxy` object to reference the `user-ca-bundle` config map in the `trustedCA` field. The allowed values are `Proxyonly` and `Always`. Use `Proxyonly` to reference the `user-ca-bundle` config map only when you configure an `http/https` proxy. Use `Always` to always reference the `user-ca-bundle` config map. The default value is `Proxyonly`. Optional parameter.

   The installation program does not support the proxy `readinessEndpoints` field.

   If the installation program times out, restart and then complete the deployment by using the `wait-for` command of the installation program. For example:

   ```
   $ ./openshift-install wait-for install-complete --log-level debug
   ```

2. Save the file and reference it when installing OpenShift Container Platform.

   The installation program creates a cluster-wide proxy named `cluster` that uses the proxy settings in the `install-config.yaml` file. If you do not give proxy settings, the installation program still creates a `cluster` `Proxy` object, but it has a nil `spec`.

   Only the `Proxy` object named `cluster` is supported, and you cannot create additional proxies.

3.4.5. Alternatives to storing administrator-level secrets in the kube-system project

By default, administrator secrets are stored in the `kube-system` project.

If you configured the `credentialsMode` parameter in the `install-config.yaml` file to `Manual`, you must use one of the following alternatives:

- To manage long-term cloud credentials manually, follow the procedure in "Manually creating long-term credentials".
- To implement short-term credentials that are managed outside the cluster for individual components, follow the procedures in "Configuring an AWS cluster to use short-term credentials".

3.4.5.1. Manually creating long-term credentials

You can put the Cloud Credential Operator (CCO) into manual mode before OpenShift Container Platform installation if the cloud identity and access management (IAM) APIs are not reachable, or if you prefer not to store an administrator-level credential secret in the cluster `kube-system` namespace.

Procedure

1. If you did not set the `credentialsMode` parameter in the `install-config.yaml` configuration file to `Manual`, modify the value as shown:

   Sample configuration file snippet

   ```
   apiVersion: v1
   baseDomain: example.com
   credentialsMode: Manual
   ```

2. If you have not previously created installation manifest files, do so by running the following command:

   ```
   $ openshift-install create manifests --dir <installation_directory>
   ```

   where `<installation_directory>` is the directory in which the installation program creates files.

3. Set a `$RELEASE_IMAGE` variable with the release image from your installation file by running the following command:

   ```
   $ RELEASE_IMAGE=$(./openshift-install version | awk '/release image/ {print $3}')
   ```

4. Extract the list of `CredentialsRequest` custom resources (CRs) from the OpenShift Container Platform release image by running the following command:

```
$ oc adm release extract \
  --from=$RELEASE_IMAGE \
  --credentials-requests \
  --included \
  --install-config=<path_to_directory_with_installation_configuration>/install-config.yaml \
  --to=<path_to_directory_for_credentials_requests>
```

where:

`--included`
: Specifies only the manifests that your specific cluster configuration requires.

`<path_to_directory_with_installation_configuration>`
: Specifies the location of the `install-config.yaml` file.

`<path_to_directory_for_credentials_requests>`
: Specifies the path to the directory where you want to store the `CredentialsRequest` objects. If the specified directory does not exist, this command creates it.

  This command creates a YAML file for each `CredentialsRequest` object.

  Sample `CredentialsRequest` object

```
apiVersion: cloudcredential.openshift.io/v1
kind: CredentialsRequest
metadata:
  name: <component_credentials_request>
  namespace: openshift-cloud-credential-operator
  ...
spec:
  providerSpec:
    apiVersion: cloudcredential.openshift.io/v1
    kind: AWSProviderSpec
    statementEntries:
    - effect: Allow
      action:
      - iam:GetUser
      - iam:GetUserPolicy
      - iam:ListAccessKeys
      resource: "*"
  ...
```

5. Create YAML files for secrets in the `openshift-install` manifests directory that you generated previously. The secrets must be stored using the namespace and secret name defined in the `spec.secretRef` for each `CredentialsRequest` object.

   Sample `CredentialsRequest` object with secrets

```
apiVersion: cloudcredential.openshift.io/v1
kind: CredentialsRequest
metadata:
  name: <component_credentials_request>
  namespace: openshift-cloud-credential-operator
  ...
spec:
  providerSpec:
    apiVersion: cloudcredential.openshift.io/v1
    kind: AWSProviderSpec
    statementEntries:
    - effect: Allow
      action:
      - s3:CreateBucket
      - s3:DeleteBucket
      resource: "*"
      ...
  secretRef:
    name: <component_secret>
    namespace: <component_namespace>
  ...
```

   Sample `Secret` object

```
apiVersion: v1
kind: Secret
metadata:
  name: <component_secret>
  namespace: <component_namespace>
data:
  aws_access_key_id: <base64_encoded_aws_access_key_id>
  aws_secret_access_key: <base64_encoded_aws_secret_access_key>
```

   Before upgrading a cluster that uses manually maintained credentials, you must ensure that the CCO is in an upgradeable state.

3.4.5.2. Configuring an AWS cluster to use short-term credentials

To install a cluster that is configured to use the AWS Security Token Service (STS), you must configure the CCO utility and create the required AWS resources for your cluster.

3.4.5.2.1. Configuring the Cloud Credential Operator utility

To create and manage cloud credentials from outside of the cluster when the Cloud Credential Operator (CCO) is operating in manual mode, extract and prepare the CCO utility (`ccoctl`) binary.

The `ccoctl` utility is a Linux binary that must run in a Linux environment.

Prerequisites

- You have access to an OpenShift Container Platform account with cluster administrator access.
- You have installed the OpenShift CLI (`oc`).

- You have created an AWS account for the `ccoctl` utility to use with the following permissions:

  Required `iam` permissions

  - `iam:CreateOpenIDConnectProvider`
  - `iam:CreateRole`
  - `iam:DeleteOpenIDConnectProvider`
  - `iam:DeleteRole`
  - `iam:DeleteRolePolicy`
  - `iam:GetOpenIDConnectProvider`
  - `iam:GetRole`
  - `iam:GetUser`
  - `iam:ListOpenIDConnectProviders`
  - `iam:ListRolePolicies`
  - `iam:ListRoles`
  - `iam:PutRolePolicy`
  - `iam:TagOpenIDConnectProvider`
  - `iam:TagRole`

  Required `s3` permissions

  - `s3:CreateBucket`
  - `s3:DeleteBucket`
  - `s3:DeleteObject`
  - `s3:GetBucketAcl`
  - `s3:GetBucketTagging`
  - `s3:GetObject`
  - `s3:GetObjectAcl`
  - `s3:GetObjectTagging`
  - `s3:ListBucket`
  - `s3:PutBucketAcl`
  - `s3:PutBucketPolicy`
  - `s3:PutBucketPublicAccessBlock`
  - `s3:PutBucketTagging`
  - `s3:PutObject`
  - `s3:PutObjectAcl`
  - `s3:PutObjectTagging`

  Required `cloudfront` permissions

  - `cloudfront:ListCloudFrontOriginAccessIdentities`
  - `cloudfront:ListDistributions`
  - `cloudfront:ListTagsForResource`

- If you plan to store the OIDC configuration in a private S3 bucket that is accessed by the IAM identity provider through a public CloudFront distribution URL, the AWS account that runs the `ccoctl` utility requires the following additional permissions:
  - `cloudfront:CreateCloudFrontOriginAccessIdentity`
  - `cloudfront:CreateDistribution`
  - `cloudfront:DeleteCloudFrontOriginAccessIdentity`
  - `cloudfront:DeleteDistribution`
  - `cloudfront:GetCloudFrontOriginAccessIdentity`
  - `cloudfront:GetCloudFrontOriginAccessIdentityConfig`
  - `cloudfront:GetDistribution`
  - `cloudfront:TagResource`
  - `cloudfront:UpdateDistribution`

  These additional permissions support the use of the `--create-private-s3-bucket` option when processing credentials requests with the `ccoctl aws create-all` command.

Procedure

1. Set a variable for the OpenShift Container Platform release image by running the following command:

   ```
   $ RELEASE_IMAGE=$(./openshift-install version | awk '/release image/ {print $3}')
   ```

2. Obtain the CCO container image from the OpenShift Container Platform release image by running the following command:

   ```
   $ CCO_IMAGE=$(oc adm release info --image-for='cloud-credential-operator' $RELEASE_IMAGE -a
   ~/.pull-secret)
   ```

   Ensure that the architecture of the `$RELEASE_IMAGE` matches the architecture of the environment in which you will use the `ccoctl` tool.

3. Extract the `ccoctl` binary from the CCO container image within the OpenShift Container Platform release image by running the following command:

   ```
   $ oc image extract $CCO_IMAGE \
     --file="/usr/bin/ccoctl.<rhel_version>" \
     -a ~/.pull-secret
   ```

   For `<rhel_version>`, specify the value that corresponds to the version of Red Hat Enterprise Linux (RHEL) that the host uses. If no value is specified, `ccoctl.rhel8` is used by default. The following values are valid:

   - `rhel8`: Specify this value for hosts that use RHEL 8.
   - `rhel9`: Specify this value for hosts that use RHEL 9.

   The `ccoctl` binary is created in the directory from where you executed the command and not in `/usr/bin/`. You must rename the directory or move the `ccoctl.<rhel_version>` binary to `ccoctl`.

4. Change the permissions to make `ccoctl` executable by running the following command:

   ```
   $ chmod 775 ccoctl
   ```

Verification

To verify that `ccoctl` is ready to use, display the help file. Use a relative file name when you run the command, for example:

```
$ ./ccoctl
```

Example output

```
OpenShift credentials provisioning tool

Usage:
  ccoctl [command]

Available Commands:
  aws          Manage credentials objects for AWS cloud
  azure        Manage credentials objects for Azure
  gcp          Manage credentials objects for Google cloud
  help         Help about any command
  ibmcloud     Manage credentials objects for IBM Cloud
  nutanix      Manage credentials objects for Nutanix

Flags:
  -h, --help   help for ccoctl

Use "ccoctl [command] --help" for more information about a command.
```

3.4.5.2.2. Creating AWS resources with a single command

If the process the `ccoctl` tool uses to create AWS resources automatically meets the requirements of your organization, you can use the `ccoctl aws create-all` command to automate the creation of AWS resources.

Otherwise, you can create the AWS resources individually. For more information, see "Creating AWS resources individually".

By default, `ccoctl` creates objects in the directory in which the commands are run. To create the objects in a different directory, use the `--output-dir` flag. This procedure uses `<path_to_ccoctl_output_dir>` to refer to this directory.

Prerequisites

You must have:

- Extracted and prepared the `ccoctl` binary.

Procedure

1. Set a `$RELEASE_IMAGE` variable with the release image from your installation file by running the following command:

```
$ RELEASE_IMAGE=$(./openshift-install version | awk '/release image/ {print $3}')
```

2. Extract the list of `CredentialsRequest` objects from the OpenShift Container Platform release image by running the following command:

```
$ oc adm release extract \
  --from=$RELEASE_IMAGE \
  --credentials-requests \
  --included \
  --install-config=<path_to_directory_with_installation_configuration>/install-config.yaml \
  --to=<path_to_directory_for_credentials_requests>
```

where:

`--included`
: Specifies to include only the manifests that your specific cluster configuration requires.

`<path_to_directory_with_installation_configuration>`
: Specifies the location of the `install-config.yaml` file.

`<path_to_directory_for_credentials_requests>`
: Specifies the path to the directory where you want to store the `CredentialsRequest` objects. If the specified directory does not exist, this command creates it.

  This command might take a few moments to run.

3. Use the `ccoctl` tool to process all `CredentialsRequest` objects by running the following command:

```
$ ccoctl aws create-all \
  --name=<name> \
  --region=<aws_region> \
  --credentials-requests-dir=<path_to_credentials_requests_directory> \
  --output-dir=<path_to_ccoctl_output_dir> \
  --create-private-s3-bucket \
  --permissions-boundary-arn=<policy_arn>
```

where:

`<name>`
: Specifies the name used to tag any cloud resources that are created for tracking.

`<aws_region>`
: Specifies the AWS region in which cloud resources will be created.

`<path_to_credentials_requests_directory>`
: Specifies the directory containing the files for the component `CredentialsRequest` objects.

`<path_to_ccoctl_output_dir>`
: Specifies the directory in which you want the `ccoctl` utility to create objects. By default, the utility creates objects in the directory in which the commands are run. This parameter is optional.

`--create-private-s3-bucket`
: Specifies that the OpenID Connect (OIDC) configuration files should be stored in a private S3 bucket that is accessed by the IAM identity provider through a public CloudFront distribution URL. Note that by default, the `ccoctl` utility stores the OIDC configuration files in a public S3 bucket and uses the S3 URL as the public OIDC endpoint. This parameter is optional.

`<policy_arn>`
: Specifies the Amazon Resource Name (ARN) of the AWS IAM policy to use as the permissions boundary for the IAM roles created by the `ccoctl` utility. This parameter is optional.

  If your cluster uses Technology Preview features that are enabled by the `TechPreviewNoUpgrade` feature set, you must include the `--enable-tech-preview` parameter.

Verification

To verify that the OpenShift Container Platform secrets are created, list the files in the `<path_to_ccoctl_output_dir>/manifests` directory:

```
$ ls <path_to_ccoctl_output_dir>/manifests
```

Example output

```
cluster-authentication-02-config.yaml
openshift-cloud-credential-operator-cloud-credential-operator-iam-ro-creds-credentials.yaml
openshift-cloud-network-config-controller-cloud-credentials-credentials.yaml
openshift-cluster-api-capa-manager-bootstrap-credentials-credentials.yaml
openshift-cluster-csi-drivers-ebs-cloud-credentials-credentials.yaml
openshift-image-registry-installer-cloud-credentials-credentials.yaml
openshift-ingress-operator-cloud-credentials-credentials.yaml
openshift-machine-api-aws-cloud-credentials-credentials.yaml
```

You can verify that the IAM roles are created by querying AWS. For more information, refer to AWS documentation on listing IAM roles.

3.4.5.2.3. Creating AWS resources individually

If you need to review the JSON files that the `ccoctl` tool creates before modifying AWS resources, or if the process the `ccoctl` tool uses to create AWS resources automatically does not meet the requirements of your organization, you can create the AWS resources individually.

This option might be useful for an organization that shares the responsibility for creating these resources among different users or departments.

Otherwise, you can use the `ccoctl aws create-all` command to create the AWS resources automatically. For more information, see "Creating AWS resources with a single command".

By default, `ccoctl` creates objects in the directory in which the commands are run. To create the objects in a different directory, use the `--output-dir` flag. This procedure uses `<path_to_ccoctl_output_dir>` to refer to this directory.

Some `ccoctl` commands make AWS API calls to create or modify AWS resources. You can use the `--dry-run` flag to avoid making API calls. Using this flag creates JSON files on the local file system instead. You can review and modify the JSON files and then apply them with the AWS CLI tool using the `--cli-input-json` parameters.

Prerequisites

Extract and prepare the `ccoctl` binary.

Procedure

1. Generate the public and private RSA key files that are used to set up the OpenID Connect provider for the cluster by running the following command:

```
$ ccoctl aws create-key-pair
```

Example output

```
2021/04/13 11:01:02 Generating RSA keypair
2021/04/13 11:01:03 Writing private key to /<path_to_ccoctl_output_dir>/serviceaccount-signer.private
2021/04/13 11:01:03 Writing public key to /<path_to_ccoctl_output_dir>/serviceaccount-signer.public
2021/04/13 11:01:03 Copying signing key for use by installer
```

where `serviceaccount-signer.private` and `serviceaccount-signer.public` are the generated key files.

This command also creates a private key that the cluster requires during installation in `/<path_to_ccoctl_output_dir>/tls/bound-service-account-signing-key.key`.

2. Create an OpenID Connect identity provider and S3 bucket on AWS by running the following command:

```
$ ccoctl aws create-identity-provider \
  --name=<name> \
  --region=<aws_region> \
  --public-key-file=<path_to_ccoctl_output_dir>/serviceaccount-signer.public
```

where:

`<name>`
: Specifies the name used to tag any cloud resources that are created for tracking.

`<aws_region>`
: Specifies the AWS region in which cloud resources will be created.

`<path_to_ccoctl_output_dir>`
: Specifies the path to the public key file that the `ccoctl aws create-key-pair` command generated.

Example output

```
2021/04/13 11:16:09 Bucket <name>-oidc created
2021/04/13 11:16:10 OpenID Connect discovery document in the S3 bucket <name>-oidc at .well-known/openid-configuration updated
2021/04/13 11:16:10 Reading public key
2021/04/13 11:16:10 JSON web key set (JWKS) in the S3 bucket <name>-oidc at keys.json updated
2021/04/13 11:16:18 Identity Provider created with ARN: arn:aws:iam::<aws_account_id>:oidc-provider/<name>-oidc.s3.<aws_region>.amazonaws.com
```

where `openid-configuration` is a discovery document and `keys.json` is a JSON web key set file.

This command also creates a YAML configuration file in `/<path_to_ccoctl_output_dir>/manifests/cluster-authentication-02-config.yaml`. This file sets the issuer URL field for the service account tokens that the cluster generates, so that the AWS IAM identity provider trusts the tokens.

3. Create IAM roles for each component in the cluster:

   a. Set a `$RELEASE_IMAGE` variable with the release image from your installation file by running the following command:

   ```
   $ RELEASE_IMAGE=$(./openshift-install version | awk '/release image/ {print $3}')
   ```

   b. Extract the list of `CredentialsRequest` objects from the OpenShift Container Platform release image:

   ```
   $ oc adm release extract \
     --from=$RELEASE_IMAGE \
     --credentials-requests \
     --included \
     --install-config=<path_to_directory_with_installation_configuration>/install-config.yaml
   \
     --to=<path_to_directory_for_credentials_requests>
   ```

   where:

   `--included`
   Specifies the `--included` parameter, which includes only the manifests that your specific cluster configuration requires.

   `--install-config`
   Specifies the location of the `install-config.yaml` file.

   `-to`
   Specifies the path to the directory where you want to store the `CredentialsRequest` objects. If the specified directory does not exist, this command creates it.

   c. Use the `ccoctl` tool to process all `CredentialsRequest` objects by running the following command:

   ```
   $ ccoctl aws create-iam-roles \
     --name=<name> \
     --region=<aws_region> \
     --credentials-requests-dir=<path_to_credentials_requests_directory> \
     --identity-provider-arn=arn:aws:iam::<aws_account_id>:oidc-provider/<name>-oidc.s3.
   <aws_region>.amazonaws.com
   ```

   For AWS environments that use alternative IAM API endpoints, such as GovCloud, you must also specify your region with the `--region` parameter.

   If your cluster uses Technology Preview features that are enabled by the `TechPreviewNoUpgrade` feature set, you must include the `--enable-tech-preview` parameter.

   For each `CredentialsRequest` object, `ccoctl` creates an IAM role with a trust policy that is tied to the specified OIDC identity provider, and a permissions policy as defined in each `CredentialsRequest` object from the OpenShift Container Platform release image.

Verification

To verify that the OpenShift Container Platform secrets are created, list the files in the `<path_to_ccoctl_output_dir>/manifests` directory:

```
$ ls <path_to_ccoctl_output_dir>/manifests
```

Example output

```
cluster-authentication-02-config.yaml
openshift-cloud-credential-operator-cloud-credential-operator-iam-ro-creds-credentials.yaml
openshift-cloud-network-config-controller-cloud-credentials-credentials.yaml
openshift-cluster-api-capa-manager-bootstrap-credentials-credentials.yaml
openshift-cluster-csi-drivers-ebs-cloud-credentials-credentials.yaml
openshift-image-registry-installer-cloud-credentials-credentials.yaml
openshift-ingress-operator-cloud-credentials-credentials.yaml
openshift-machine-api-aws-cloud-credentials-credentials.yaml
```

You can verify that the IAM roles are created by querying AWS. For more information, refer to AWS documentation on listing IAM roles.

3.4.5.2.4. Incorporating the Cloud Credential Operator utility manifests

To implement short-term security credentials managed outside the cluster for individual OpenShift Container Platform components, you must move the manifest files that the Cloud Credential Operator utility (`ccoctl`) created to the correct directories for the installation program.

Prerequisites

- You have configured an account with the cloud platform that hosts your cluster.
- You have configured the Cloud Credential Operator utility (`ccoctl`).
- You have created the cloud provider resources that are required for your cluster with the `ccoctl` utility.

Procedure

1. If you did not set the `credentialsMode` parameter in the `install-config.yaml` configuration file to `Manual`, modify the value as shown:

   Sample configuration file snippet

   ```
   apiVersion: v1
   baseDomain: example.com
   credentialsMode: Manual
   ```

2. If you have not previously created installation manifest files, do so by running the following command:

   ```
   $ openshift-install create manifests --dir <installation_directory>
   ```

   where `<installation_directory>` is the directory in which the installation program creates files.

3. Copy the manifests that the `ccoctl` utility generated to the `manifests` directory that the installation program created by running the following command:

   ```
   $ cp /<path_to_ccoctl_output_dir>/manifests/* ./manifests/
   ```

4. Copy the `tls` directory that contains the private key to the installation directory:

   ```
   $ cp -a /<path_to_ccoctl_output_dir>/tls .
   ```

3.4.6. Deploying the cluster

To deploy your OpenShift Container Platform cluster, you initialize installation by running the `openshift-install create cluster` command from the directory that contains the installation program. The

installation program provisions the required infrastructure and completes the cluster setup.

You can run the `create cluster` command of the installation program only once, during initial installation.

Prerequisites

- You have configured an account with the cloud platform that hosts your cluster.
- You have the OpenShift Container Platform installation program and the pull secret for your cluster.
- You have verified that the cloud provider account on your host has the correct permissions to deploy the cluster. An account with incorrect permissions causes the installation process to fail with an error message that displays the missing permissions.

Procedure

1. In the directory that contains the installation program, initialize the cluster deployment by running the following command:

   ```
   $ ./openshift-install create cluster --dir <installation_directory> \
       --log-level=info
   ```

   where:

   - `<installation_directory>`: Specifies the location of your customized `./install-config.yaml` file.
   - `--log-level`: Specifies the log level. To view different installation details, specify `warn`, `debug`, or `error` instead of `info`.
2. Optional: Remove or disable the `AdministratorAccess` policy from the IAM account that you used to install the cluster.

   The elevated permissions provided by the `AdministratorAccess` policy are required only during installation.

Verification

When the cluster deployment completes successfully:

- The terminal displays directions for accessing your cluster, including a link to the web console and credentials for the `kubeadmin` user.

- Credential information also outputs to `<installation_directory>/.openshift_install.log`.

  Do not delete the installation program or the files that the installation program creates. Both are required to delete the cluster.

  The following example shows the expected output:

```
...
INFO Install complete!
INFO To access the cluster as the system:admin user when using 'oc', run 'export
KUBECONFIG=/home/myuser/install_dir/auth/kubeconfig'
INFO Access the OpenShift web-console here: https://console-openshift-
console.apps.mycluster.example.com
INFO Login to the console with user: "kubeadmin", and password: "password"
INFO Time elapsed: 36m22s
```

  - The Ignition config files that the installation program generates contain certificates that expire after 24 hours, which are then renewed at that time. If the cluster is shut down before renewing the certificates and the cluster is later restarted after the 24 hours have elapsed, the cluster automatically recovers the expired certificates. The exception is that you must manually approve the pending `node-bootstrapper` certificate signing requests (CSRs) to recover kubelet certificates. See the documentation for *Recovering from expired control plane certificates* for more information.
  - It is recommended that you use Ignition config files within 12 hours after they are generated because the 24-hour certificate rotates from 16 to 22 hours after the cluster is installed. By using the Ignition config files within 12 hours, you can avoid installation failure if the certificate update runs during installation.

3.4.7. Provisioning your own DNS records

Use your cluster name and base cluster domain to configure a CNAME record for the API service `api.<cluster_name>.<base_domain>.` with the API load balancer DNS name, and provision a CNAME record for the Ingress hostname `*.apps.<cluster_name>.<base_domain>.` with the Ingress load balancer DNS name.

User-provisioned DNS is a Technology Preview feature only. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see Technology Preview Features Support Scope.

Prerequisites

You have installed the AWS CLI.

Procedure

1. Add the `userProvisionedDNS` parameter to the `install-config.yaml` file and enable the parameter. For more information, see "Enabling a user-managed DNS".
2. Install your cluster.
3. If you are installing a private cluster, set the `api_lb_name` variable by running the following command:

```
$ api_lb_name="${INFRA_ID}-int"
```

4. If you are installing a public cluster, set the `api_lb_name` variable by running the following command:

```
$ api_lb_name="${INFRA_ID}-ext"
```

5. To retrieve the DNS name of the API service, run the following command:

```
$ aws --region ${REGION} elbv2 describe-load-balancers --names ${api_lb_name} --query
'LoadBalancers[*].DNSName' --output text
```

6. Use the DNS name and your cluster name and base cluster domain to configure your own DNS record with the `api.<cluster_name>.<base_domain>.` hostname.
7. To retrieve the DNS name of the Ingress service, run the following command:

```
$ ingress_lb_name=$(aws --region ${REGION} resourcegroupstaggingapi get-resources --resource-
type-filters elasticloadbalancing:loadbalancer --tag-filters
Key=kubernetes.io/cluster/${INFRA_ID},Values=owned Key=kubernetes.io/service-
name,Values=openshift-ingress/router-default --query 'ResourceTagMappingList[*].ResourceARN |
[0]' --output text | awk -F'/' '{print $2}')
```

8. Run the following command, which uses the variable `ingress_lb_name` generated from the previous command:

```
$ aws --region ${REGION} elb describe-load-balancers --load-balancer-names ${ingress_lb_name}
--query 'LoadBalancerDescriptions[].DNSName' --output text
```

9. Use the DNS name and your cluster name and base cluster domain to configure your own DNS record with the `*.apps.<cluster_name>.<base_domain>.` hostname.

3.4.8. Logging in to the cluster by using the CLI

To log in to your cluster as the default system user, export the `kubeconfig` file. This configuration enables the CLI to authenticate and connect to the specific API server created during OpenShift Container Platform installation.

The `kubeconfig` file is specific to a cluster and OpenShift Container Platform generates it during installation.

Prerequisites

- You deployed an OpenShift Container Platform cluster.
- You installed the OpenShift CLI (`oc`).

Procedure

1. Export the `kubeadmin` credentials by running the following command:

```
$ export KUBECONFIG=<installation_directory>/auth/kubeconfig
```

   where:

   `<installation_directory>`
   Specifies the path to the directory that stores the installation files.

2. Verify you can run `oc` commands successfully using the exported configuration by running the following command:

```
$ oc whoami
```

   Example output

```
system:admin
```

Next steps

- "Customize your cluster"
- "Remote health reporting"

3.4.9. Disabling the default software catalog sources

To use only trusted or locally available Operator catalogs, disable the default software catalog sources that OpenShift Container Platform configures during installation. In a restricted network environment, you must disable the default catalogs as a cluster administrator.

Procedure

Disable the sources for the default catalogs by adding `disableAllDefaultSources: true` to the `OperatorHub` object:

```
$ oc patch OperatorHub cluster --type json \
    -p '[{"op": "add", "path": "/spec/disableAllDefaultSources", "value": true}]'
```

Or, you can use the web console to manage catalog sources. From the **Administration Cluster Settings Configuration OperatorHub** page, click the **Sources** tab, where you can create, update, delete, disable, and enable individual sources.

3.5. Installing a cluster on AWS into an existing VPC

In OpenShift Container Platform version 4.22, you can install a cluster into an existing AWS Virtual Private Cloud (VPC) on Amazon Web Services (AWS). The installation program provisions the rest of the required infrastructure, which you can further customize. To customize the installation, you modify parameters in the `install-config.yaml` file before you install the cluster.

3.5.1. Prerequisites

Before you install a cluster into an existing AWS Virtual Private Cloud (VPC) on Amazon Web Services (AWS), you must meet several prerequisites.

The following prerequisites must be met:

- You reviewed details about the OpenShift Container Platform installation and update processes.
- You read the documentation on selecting a cluster installation method and preparing it for users.
- You configured an AWS account to host the cluster.
- If the existing VPC is owned by a different account than the cluster, you shared the VPC between accounts.

  If you have an AWS profile stored on your computer, it must not use a temporary session token that you generated while using a multi-factor authentication device. The cluster continues to use your current AWS credentials to create AWS resources for the entire life of the cluster, so you must use long-term credentials. To generate appropriate keys, see Managing Access Keys for IAM Users in the AWS documentation. You can supply the keys when you run the installation program.

- If you use a firewall, you configured it to allow the sites that your cluster requires access to.

3.5.2. About using a custom VPC

You can deploy a cluster into existing subnets in an existing Amazon Virtual Private Cloud (VPC) in Amazon Web Services (AWS).

By deploying OpenShift Container Platform into an existing AWS VPC, you might be able to avoid limit constraints in new accounts or more easily abide by the operational constraints that your company's guidelines set. If you cannot obtain the infrastructure creation permissions that are required to create the VPC yourself, use this installation option.

Because the installation program cannot know what other components are also in your existing subnets, it cannot choose subnet CIDRs and so forth on your behalf. You must configure networking for the subnets that you install your cluster to yourself.

The installation program no longer creates the following components:

- Internet gateways
- NAT gateways
- Subnets
- Route tables
- VPCs
- VPC DHCP options
- VPC endpoints

The installation program requires that you use the cloud-provided DNS server. Using a custom DNS server is not supported and causes the installation to fail.

If you use a custom VPC, you must correctly configure it and its subnets for the installation program and the cluster to use. See "Create a VPC" in the Amazon Web Services documentation for more information about AWS VPC console wizard configurations and creating and managing an AWS VPC.

The installation program cannot:

- Subdivide network ranges for the cluster to use.
- Set route tables for the subnets.
- Set VPC options like DHCP.

You must complete these tasks before you install the cluster. See "VPC networking components" and "Route tables for your VPC" for more information on configuring networking in an AWS VPC.

Your VPC must meet the following characteristics:

- Create a public and private subnet for each availability zone that your cluster uses. Each availability zone can contain no more than one public and one private subnet. For an example of this type of configuration, see "VPC with public and private subnets (NAT)" in the AWS documentation.

  Record each subnet ID. Completing the installation requires that you enter these values in the `platform` section of the `install-config.yaml` file. See "Finding a subnet ID" in the AWS documentation.

- The VPC's CIDR block must contain the `Networking.MachineCIDR` range, which is the IP address pool for cluster machines. The subnet CIDR blocks must belong to the machine CIDR that you specify.

- The VPC must have a public internet gateway attached to it. For each availability zone:
    - The public subnet requires a route to the internet gateway.
    - The public subnet requires a NAT gateway with an EIP address.
    - The private subnet requires a route to the NAT gateway in public subnet.
- The VPC must not use the `kubernetes.io/cluster/.*: owned`, `Name`, and `openshift.io/cluster` tags.

  The installation program modifies your subnets to add the `kubernetes.io/cluster/.*: shared` tag, so your subnets must have at least one free tag slot available for it. See "Tag Restrictions" in the AWS documentation to confirm that the installation program can add a tag to each subnet that you specify. You cannot use a `Name` tag, because it overlaps with the EC2 `Name` field and the installation fails.

- If you want to extend your OpenShift Container Platform cluster into an AWS Outpost and have an existing Outpost subnet, the existing subnet must use the `kubernetes.io/cluster/unmanaged: true` tag. If you do not apply this tag, the installation might fail due to the Cloud Controller Manager creating a service load balancer in the Outpost subnet, which is an unsupported configuration.
- You must enable the `enableDnsSupport` and `enableDnsHostnames` attributes in your VPC, so that the cluster can use the Route 53 zones that are attached to the VPC to resolve cluster's internal DNS records. See "DNS Support in Your VPC" in the AWS documentation.

  If you prefer to use your own Route 53 hosted private zone, you must associate the existing hosted zone with your VPC prior to installing a cluster. You can define your hosted zone using the `platform.aws.hostedZone` and `platform.aws.hostedZoneRole` fields in the `install-config.yaml` file. You can use a private hosted zone from another account by sharing it with the account where you install the cluster. If you use a private hosted zone from another account, you must use the `Passthrough` or `Manual` credentials mode.

If you are working in a disconnected environment, you are unable to reach the public IP addresses for EC2, ELB, and S3 endpoints. Depending on the level to which you want to restrict internet traffic during the installation, the following configuration options are available:

3.5.2.1. Option 1: Create VPC endpoints

Create a VPC endpoint and attach it to the subnets that the clusters are using. Name the endpoints as follows:

- `ec2.<aws_region>.amazonaws.com`
- `elasticloadbalancing.<aws_region>.amazonaws.com`
- `s3.<aws_region>.amazonaws.com`

With this option, network traffic remains private between your VPC and the required AWS services.

3.5.2.2. Option 2: Create a proxy without VPC endpoints

As part of the installation process, you can configure an HTTP or HTTPS proxy. With this option, internet traffic goes through the proxy to reach the required AWS services.

3.5.2.3. Option 3: Create a proxy with VPC endpoints

As part of the installation process, you can configure an HTTP or HTTPS proxy with VPC endpoints. Create a VPC endpoint and attach it to the subnets that the clusters are using. Name the endpoints as follows:

- `ec2.<aws_region>.amazonaws.com`
- `elasticloadbalancing.<aws_region>.amazonaws.com`
- `s3.<aws_region>.amazonaws.com`

When configuring the proxy in the `install-config.yaml` file, add these endpoints to the `noProxy` field. With this option, the proxy prevents the cluster from accessing the internet directly. However, network traffic remains private between your VPC and the required AWS services.

You must provide a suitable VPC and subnets that allow communication to your machines.

Component	AWS type	Description
VPC	• `AWS::EC2::VPC` • `AWS::EC2::VPCEndpoint`	You must provide a public VPC for the cluster to use. The VPC uses an endpoint that references the route tables for each subnet to improve communication with the registry that is hosted in S3.
Public subnets	• `AWS::EC2::Subnet` • `AWS::EC2::SubnetNetworkAclAssociation`	Your VPC must have public subnets for between 1 and 3 availability zones and associate them with appropriate Ingress rules.
Internet gateway	• `AWS::EC2::InternetGateway` • `AWS::EC2::VPCGatewayAttachment` • `AWS::EC2::RouteTable` • `AWS::EC2::Route` • `AWS::EC2::SubnetRouteTableAssociation` • `AWS::EC2::NatGateway` • `AWS::EC2::EIP`	You must have a public internet gateway, with public routes, attached to the VPC. In the provided templates, each public subnet has a NAT gateway with an EIP address. These NAT gateways allow cluster resources, like private subnet instances, to reach the internet and are not required for some restricted network or proxy scenarios.

<table>
<tr><th>Component</th><th>AWS type</th><th>Description</th></tr>
<tr><td>Network access control</td><td><code>AWS::EC2::NetworkAcl</code>
<code>AWS::EC2::NetworkAclEntry</code></td><td>You must allow the VPC to access the following ports:<table><tr><th>Port</th><th>Reason</th></tr><tr><td><code>80</code></td><td>Inbound HTTP traffic</td></tr><tr><td><code>443</code></td><td>Inbound HTTPS traffic</td></tr><tr><td><code>22</code></td><td>Inbound SSH traffic</td></tr><tr><td><code>1024 - 65535</code></td><td>Inbound ephemeral traffic</td></tr><tr><td><code>0 - 65535</code></td><td>Outbound ephemeral traffic</td></tr></table></td></tr>
<tr><td>Private subnets</td><td><code>AWS::EC2::Subnet</code>
<code>AWS::EC2::RouteTable</code>
<code>AWS::EC2::SubnetRouteTableAssociation</code></td><td>Your VPC can have private subnets. The provided CloudFormation templates can create private subnets for between 1 and 3 availability zones. If you use private subnets, you must provide appropriate routes and tables for them.</td></tr>
</table>

Table 3.14. Required VPC components

3.5.2.4. VPC validation

To ensure that the subnets that you provide are suitable, the installation program confirms the following data:

- All the subnets that you specify exist.
- You provide private subnets.
- The subnet CIDRs belong to the machine CIDR that you specified.
- You provide subnets for each availability zone. Each availability zone contains no more than one public and one private subnet. If you use a private cluster, provide only a private subnet for each availability zone. Otherwise, provide exactly one public and private subnet for each availability zone.
- You provide a public subnet for each private subnet availability zone. Machines are not provisioned in availability zones that you do not provide private subnets for.

If you destroy a cluster that uses an existing VPC, the VPC is not deleted. When you remove the OpenShift Container Platform cluster from a VPC, the `kubernetes.io/cluster/.*: shared` tag is removed from the subnets that it used.

3.5.2.5. Division of permissions

Starting with OpenShift Container Platform 4.3, you do not need all of the permissions that are required for an installation program-provisioned infrastructure cluster to deploy a cluster. This change mimics the division of permissions that you might have at your company: some individuals can create different resource in your clouds than others. For example, you might be able to create application-specific items, like instances, buckets, and load balancers, but not networking-related components such as VPCs, subnets, or ingress rules.

The AWS credentials that you use when you create your cluster do not need the networking permissions that are required to make VPCs and core networking components within the VPC, such as subnets, routing tables, internet gateways, NAT, and VPN. You still need permission to make the application resources that the machines within the cluster require, such as ELBs, security groups, S3 buckets, and nodes.

3.5.2.6. Isolation between clusters

If you deploy OpenShift Container Platform to an existing network, the isolation of cluster services is reduced in the following ways:

- You can install multiple OpenShift Container Platform clusters in the same VPC.
- ICMP ingress is allowed from the entire network.
- TCP 22 ingress (SSH) is allowed to the entire network.
- Control plane TCP 6443 ingress (Kubernetes API) is allowed to the entire network.
- Control plane TCP 22623 ingress (MCS) is allowed to the entire network.

3.5.2.7. Optional: AWS security groups

By default, the installation program creates and attaches security groups to control plane and compute machines. The rules associated with the default security groups cannot be modified.

However, you can apply additional existing AWS security groups, which are associated with your existing VPC, to control plane and compute machines. Applying custom security groups can help you meet the security needs of your organization, in such cases where you need to control the incoming or outgoing traffic of these machines.

As part of the installation process, you apply custom security groups by modifying the `install-config.yaml` file before deploying the cluster.

For more information, see "Applying existing AWS security groups to the cluster".

By default, AWS allows 5 security groups per network interface. The installation program creates 2 security groups for compute machines and 3 security groups for control plane machines. If you are installing a cluster into a shared VPC, there are three scenarios in which you must increase this quota:

- You specified 4 or more custom security groups for compute machines using the `compute.platform.aws.additionalSecurityGroupIDs` parameter in the `install-config.yaml` file.
- You specified 3 or more custom security groups for control plane machines using the `controlPlane.platform.aws.additionalSecurityGroupIDs` parameter in the `install-config.yaml` file.
- You specified 3 or more custom security groups for all machines using the `platform.aws.defaultMachinePlatform` parameter in the `install-config.yaml` file.

You must increase the quota of security groups per network interface to a number greater than or equal to `3 + (number of control plane custom security groups OR number of default machine platform`

`custom security groups)`, or `2 + (number of compute custom security groups OR number of default machine platform custom security groups)`, whichever is higher. If you do not specify a sufficient quota, the installation will succeed, but it will generate `SecurityGroupsPerInterfaceLimitExceeded` errors in the installation log, and the additional security groups will not be applied. The maximum allowed quota is 16 and the maximum number of user-specified security groups is 10.

3.5.2.8. Modifying trust policy when installing into a shared VPC

If you install your cluster using a shared VPC, you can use the `Passthrough` or `Manual` credentials mode. You must add the IAM role used to install the cluster as a principal in the trust policy of the account that owns the VPC.

If you use `Passthrough` mode, add the Amazon Resource Name (ARN) of the account that creates the cluster, such as `arn:aws:iam::123456789012:user/clustercreator`, to the trust policy as a principal.

If you use `Manual` mode, add the ARN of the account that creates the cluster as well as the ARN of the ingress operator role in the cluster owner account, such as `arn:aws:iam::123456789012:role/<cluster-name>-openshift-ingress-operator-cloud-credentials`, to the trust policy as principals.

You must add the following actions to the policy:

Example 3.1. Required actions for shared VPC installation

- `route53:ChangeResourceRecordSets`
- `route53:ListHostedZones`
- `route53:ListHostedZonesByName`
- `route53:ListResourceRecordSets`
- `route53:ChangeTagsForResource`
- `route53:GetAccountLimit`
- `route53:GetChange`
- `route53:GetHostedZone`
- `route53:ListTagsForResource`
- `route53:UpdateHostedZoneComment`
- `tag:GetResources`
- `tag:UntagResources`

3.5.3. Creating the installation configuration file

You can customize the OpenShift Container Platform cluster you install on Amazon Web Services (AWS).

Prerequisites

You have the OpenShift Container Platform installation program and the pull secret for your cluster.

Procedure

1. Create the `install-config.yaml` file.

   a. Change to the directory that contains the installation program and run the following command:

   ```
   $ ./openshift-install create install-config --dir <installation_directory>
   ```

   - `<installation_directory>`: For `<installation_directory>`, specify the directory name to store the files that the installation program creates.

     When specifying the directory:

   - Verify that the directory has the `execute` permission. This permission is required to run Terraform binaries under the installation directory.
   - Use an empty directory. Some installation assets, such as bootstrap X.509 certificates, have short expiration intervals, therefore you must not reuse an installation directory. If you want to reuse individual files from another cluster installation, you can copy them into your directory. However, the file names for the installation assets might change between releases. Use caution when copying installation files from an earlier OpenShift Container Platform version.

   b. At the prompts, provide the configuration details for your cloud:

      i. Optional: Select an SSH key to use to access your cluster machines.

         For production OpenShift Container Platform clusters on which you want to perform installation debugging or disaster recovery, specify an SSH key that your `ssh-agent` process uses.

      ii. Select **AWS** as the platform to target.
      iii. If you do not have an Amazon Web Services (AWS) profile stored on your computer, enter the AWS access key ID and secret access key for the user that you configured to run the installation program.
      iv. Select the AWS region to deploy the cluster to.
      v. Select the base domain for the Route 53 service that you configured for your cluster.
      vi. Enter a descriptive name for your cluster.

2. Modify the `install-config.yaml` file. You can find more information about the available parameters in the "Installation configuration parameters" section.
3. Back up the `install-config.yaml` file so that you can use it to install multiple clusters.

   The `install-config.yaml` file is consumed during the installation process. If you want to reuse the file, you must back it up now.

3.5.3.1. Minimum resource requirements for cluster installation

To ensure that your OpenShift Container Platform cluster runs as expected, each cluster machine must meet minimum CPU, memory, and storage requirements.

Machine	Operating system	vCPU	Virtual RAM	Storage	Input/Output Per Second (IOPS)
Bootstrap	RHCOS	4	16 GB	100 GB	300
Control plane	RHCOS	4	16 GB	100 GB	300

Machine	Operating system	vCPU	Virtual RAM	Storage	Input/Output Per Second (IOPS)
Compute	RHCOS	2	8 GB	100 GB	300

Table 3.15. Minimum resource requirements

- One vCPU is equal to one physical core when simultaneous multithreading (SMT), or Hyper-Threading, is not enabled. When enabled, use the following formula to calculate the corresponding ratio: (threads per core × cores) × sockets = vCPUs.
- OpenShift Container Platform and Kubernetes are sensitive to disk performance, and Red Hat recommends faster storage, particularly for etcd on the control plane nodes which require a 10 ms p99 fsync duration. On many cloud platforms, storage size and IOPS scale together, so you might need to provision more storage to get enough performance.
- As with all user-provisioned installations, if you choose to use RHEL compute machines in your cluster, you take responsibility for all operating system life cycle management and maintenance, including performing system updates, applying patches, and completing all other required tasks. OpenShift Container Platform 4.10 and later do not support RHEL 7 compute machines.

In OpenShift Container Platform version 4.22, RHCOS uses RHEL version 9.8, which updates the micro-architecture requirements. Each architecture requires the following minimum instruction set architectures (ISA):

- x86-64 architecture requires x86-64-v2 ISA
- ARM64 architecture requires ARMv8.0-A ISA
- ppc64le architecture requires IBM® Power9 ISA
- s390x architecture requires IBM® z14 ISA

For more information, see Architectures in the RHEL documentation.

If an instance type for your platform meets the minimum requirements for cluster machines, it is supported to use in OpenShift Container Platform.

3.5.3.2. Tested instance types for AWS

To ensure cluster stability and performance, use one of the tested Amazon Web Services (AWS) instance types for your OpenShift Container Platform machines.

The following AWS instance types have been tested with OpenShift Container Platform.

Use the machine types included in the following charts for your AWS instances. If you use an instance type that is not listed in the chart, ensure that the instance size you use matches the minimum resource requirements in "Minimum resource requirements for cluster installation".

See the following machine types based on 64-bit x86 architecture:

- `c4.*`
- `c5.*`
- `c5a.*`
- `i3.*`
- `m4.*`
- `m5.*`
- `m5a.*`
- `m6a.*`

- m6i.*
- m7a.*
- m7i.*
- m7i-flex.*
- r4.*
- r5.*
- r5a.*
- r6i.*
- t3.*
- t3a.*

3.5.3.3. Tested instance types for AWS on 64-bit ARM infrastructures

To ensure cluster stability and performance, use one of the tested Amazon Web Services (AWS) 64-bit ARM instance types for your OpenShift Container Platform machines.

The following AWS 64-bit ARM instance types have been tested with OpenShift Container Platform.

Use the machine types included in the following charts for your AWS ARM instances. If you use an instance type that is not listed in the chart, ensure that the instance size you use matches the minimum resource requirements listed in "Minimum resource requirements for cluster installation".

See the following machine types based on 64-bit ARM architecture:

- c6g.*
- c7g.*
- c8g.*
- m6g.*
- m7g.*
- m7gd.*
- r8g.*

3.5.3.4. Enabling a user-managed DNS

You can install a cluster with a domain name server (DNS) solution that you manage instead of the default cluster-provisioned DNS solution that uses the Route 53 service for Amazon Web Services (AWS).

For example, your organization’s security policies might not allow the use of public DNS services such as Amazon Web Services DNS. In such scenarios, you can use your own DNS service to bypass the public DNS service and manage your own DNS for the IP addresses of the API and Ingress services.

If you enable user-managed DNS during installation, the installation program provisions DNS records for the API and Ingress services only within the cluster. To ensure access from outside the cluster, you must provision the DNS records in an external DNS service of your choice for the API and Ingress services after installation.

User-provisioned DNS is a Technology Preview feature only. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see Technology Preview Features Support Scope.

Procedure

Before you deploy your cluster, use a text editor to open the `install-config.yaml` file and add the following stanza:

To enable user-managed DNS:

```
featureSet: CustomNoUpgrade
featureGates: ["AWSClusterHostedDNSInstall=true"]
```

```
platform:
  aws:
    userProvisionedDNS: Enabled
```

where:

`userProvisionedDNS`
Enables user-provisioned DNS management.

Next steps

For information about provisioning your DNS records for the API server and the Ingress services, see "Provisioning your own DNS records".

3.5.3.5. Sample customized install-config.yaml file for AWS

You can customize the installation configuration file (`install-config.yaml`) to specify more details about your OpenShift Container Platform cluster's platform or modify the values of the required parameters.

This sample YAML file is provided for reference only. You must obtain your `install-config.yaml` file by using the installation program and modify it. For a full list and description of all installation configuration parameters, see *Installation configuration parameters for AWS*.

Sample `install-config.yaml` file for AWS

```
apiVersion: v1
baseDomain: example.com
sshKey: ssh-ed25519 AAAA...
pullSecret: '{"auths": ...}'
metadata:
  name: example-cluster
controlPlane:
  name: master
  platform:
    aws:
      type: m6i.xlarge
  replicas: 3
compute:
-  name: worker
  platform:
    aws:
      type: c5.4xlarge
  replicas: 3
networking:
  clusterNetwork:
  - cidr: 10.128.0.0/14
    hostPrefix: 23
platform:
```

```
aws:
  region: us-west-2
```

- Parameters at the first level of indentation apply to the cluster globally.
- The `controlPlane` stanza applies to control plane machines.
- The `compute` stanza applies to compute machines.
- The `networking` stanza applies to the cluster networking configuration. If you do not provide networking values, the installation program provides default values.
- The `platform` stanza applies to the infrastructure platform that hosts the cluster.

3.5.3.6. Configuring the cluster-wide proxy during installation

Production environments can deny direct access to the internet and instead have an HTTP or HTTPS proxy available. You can configure a new OpenShift Container Platform cluster to use a proxy by configuring the proxy settings in the `install-config.yaml` file.

Prerequisites

- You have an existing `install-config.yaml` file.
- You have reviewed the sites that your cluster requires access to and determined whether any of them need to bypass the proxy. By default, the proxy handles all cluster egress traffic, including calls to hosting cloud provider APIs. You added sites to the `Proxy` object's `spec.noProxy` field to bypass the proxy if necessary.

  The `Proxy` object `status.noProxy` field includes the values of the `networking.machineNetwork[].cidr`, `networking.clusterNetwork[].cidr`, and `networking.serviceNetwork[]` fields from your installation configuration.

  For installations on Amazon Web Services (AWS), Google Cloud, Microsoft Azure, and Red Hat OpenStack Platform (RHOSP), the `Proxy` object `status.noProxy` field also includes the instance metadata endpoint (`169.254.169.254`).

Procedure

1. Edit your `install-config.yaml` file and add the proxy settings. For example:

```
apiVersion: v1
baseDomain: my.domain.com
proxy:
  httpProxy: http://<username>:<pswd>@<ip>:<port>
  httpsProxy: https://<username>:<pswd>@<ip>:<port>
  noProxy: ec2.<aws_region>.amazonaws.com,elasticloadbalancing.<aws_region>.amazonaws.com,s3.
<aws_region>.amazonaws.com
additionalTrustBundle: |
    -----BEGIN CERTIFICATE-----
    <MY_TRUSTED_CA_CERT>
    -----END CERTIFICATE-----
additionalTrustBundlePolicy: <policy_to_add_additionalTrustBundle>
```

   where:

   `proxy.httpProxy`
   : Specifies a proxy URL to use for creating HTTP connections outside the cluster. The URL scheme must be `http`.

   `proxy.httpsProxy`
   : Specifies a proxy URL to use for creating HTTPS connections outside the cluster.

   `proxy.noProxy`
   : Specifies a comma-separated list of destination domain names, IP addresses, or other network CIDRs to exclude from proxying. Preface a domain with `.` to match subdomains only. For example, `.y.com` matches `x.y.com`, but not `y.com`. Use `*` to bypass the proxy for all destinations. If you have added the Amazon `EC2`, `Elastic Load Balancing`, and `S3` VPC endpoints to your VPC, you must add these endpoints to the `noProxy` field.

   `additionalTrustBundle`
   : If you specify this value, the installation program generates a config map named `user-ca-bundle` in the `openshift-config` namespace to hold the additional CA certificates. If you specify `additionalTrustBundle` and at least one proxy setting, the `Proxy` object references the `user-ca-bundle` config map in the `trustedCA` field. The Cluster Network Operator then creates a `trusted-ca-bundle` config map that merges the contents specified for the `trustedCA` parameter with the RHCOS trust bundle. You must set the `additionalTrustBundle` field unless an authority from the RHCOS trust bundle signs the proxy's identity certificate.

   `additionalTrustBundlePolicy`
   : Specifies the policy that determines the configuration of the `Proxy` object to reference the `user-ca-bundle` config map in the `trustedCA` field. The allowed values are `Proxyonly` and `Always`. Use `Proxyonly` to reference the `user-ca-bundle` config map only when you configure an `http/https` proxy. Use `Always` to always reference the `user-ca-bundle` config map. The default value is `Proxyonly`. Optional parameter.

   The installation program does not support the proxy `readinessEndpoints` field.

   If the installation program times out, restart and then complete the deployment by using the `wait-for` command of the installation program. For example:

```
$ ./openshift-install wait-for install-complete --log-level debug
```

2. Save the file and reference it when installing OpenShift Container Platform.

   The installation program creates a cluster-wide proxy named `cluster` that uses the proxy settings in the `install-config.yaml` file. If you do not give proxy settings, the installation program still creates a `cluster` `Proxy` object, but it has a nil `spec`.

   Only the `Proxy` object named `cluster` is supported, and you cannot create additional proxies.

3.5.3.7. Applying existing Amazon Web Services (AWS) security groups to the cluster

Applying existing AWS security groups to your control plane and compute machines can help you meet the security needs of your organization, in such cases where you need to control the incoming or outgoing traffic of these machines.

Prerequisites

- You have created the security groups in AWS. For more information, see the AWS documentation about working with security groups.
- The security groups must be associated with the existing VPC that you are deploying the cluster to. The security groups cannot be associated with another VPC.
- You have an existing `install-config.yaml` file.

Procedure

1. In the `install-config.yaml` file, edit the `compute.platform.aws.additionalSecurityGroupIDs` parameter to specify one or more custom security groups for your compute machines.
2. Edit the `controlPlane.platform.aws.additionalSecurityGroupIDs` parameter to specify one or more custom security groups for your control plane machines.

3. Save the file and reference it when deploying the cluster.

Sample `install-config.yaml` file that specifies custom security groups

```
compute:
- hyperthreading: Enabled
  name: worker
  platform:
    aws:
      additionalSecurityGroupIDs:
        - sg-1
        - sg-2
  replicas: 3
controlPlane:
  hyperthreading: Enabled
  name: master
  platform:
    aws:
      additionalSecurityGroupIDs:
        - sg-3
        - sg-4
  replicas: 3
platform:
  aws:
    region: us-east-1
    subnets:
      - subnet-1
      - subnet-2
      - subnet-3
```

where:

`compute.platform.aws.additionalSecurityGroupIDs`
: Specifies the name of the security group as it appears in the Amazon EC2 console, including the `sg` prefix.

`platform.aws.subnets`
: Specifies subnets for each availability zone that your cluster uses.

3.5.3.8. Allocating load balancers to specific subnets

You can manage application traffic efficiently by allocating load balancers. Network administrators can allocate load balancers to customize deployments which can ensure optimal traffic distribution, high availability of applications, uninterrupted service, and network segmentation.

You can also limit subnet selection for a load balancer by using subnet roles. Subnet roles prevent security risks by ensuring Load Balancers are automatically placed only in subnets explicitly designated for public or private traffic. This eliminates manual configuration errors and guarantees architectural compliance across multiple availability zones.

3.5.3.8.1. Allocating API and Ingress load balancers to specific subnets on AWS

You can control the network placement of OpenShift Container Platform load balancers on AWS by defining subnets from the virtual private cloud (VPC) and assigning them roles in the `platform.aws.vpc.subnets` section of `install-config.yaml`.

3.5.3.8.2. Specifying AWS subnets for OpenShift API and ingress load balancers at installation

You can allocate API and ingress load balancers to specific subnets for the purposes of aligning security and networking policies with your organization requirements.

When defining entries for control plane load balancers in the `subnets` list, ensure that you adhere to the following pattern:

```
- id: subnet-0fcf8e0392f0910d6
  roles:
  - type: ControlPlaneExternalLB
- id: subnet-0fcf8e0392f0910d7
  roles:
  - type: ControlPlaneInternalLB
```

For the default public Ingress Controller, any subnet assigned the `IngressControllerLB` role in your `install-config.yaml` file must be a public subnet. For example, the subnet must have a route table entry in AWS that directs outbound traffic to an internet gateway (IGW). Ensure you list all necessary subnets, public and private across the AZs, and assign them appropriate roles according to your cluster architecture.

Subnet IDs define the subnets in an existing VPC and can optionally specify their intended roles. If no roles are specified on any subnet, the subnet roles are decided automatically. In this case, the VPC must not contain any other non-cluster subnets without the `kubernetes.io/cluster/<cluster-id>` tag.

If roles are specified for subnets, each subnet must have at least one assigned role, and the `ClusterNode`, `BootstrapNode`, `IngressControllerLB`, `ControlPlaneExternalLB`, and `ControlPlaneInternalLB` roles must be assigned to at least one subnet. However, if the cluster scope is internal, `ControlPlaneExternalLB` is not required.

Prerequisites

- An existing AWS virtual private cloud (VPC).
- Pre-configured AWS subnets intended for use by the OpenShift Container Platform cluster, with the following considerations:
    - You have a list of their subnet IDs (for example, `subnet-0123456789abcdef0`). These IDs will be used in the `install-config.yaml` file.
    - Use subnets spanning at least two availability zones (AZs) for high availability of load balancers and other critical components, like control planes.
    - You have sufficient available IP addresses within these subnets for all assigned roles.
    - The AWS configuration for these subnets, including network ACLs and security groups, must permit necessary traffic for all roles assigned to them. For subnets hosting an ingress controller, this typically includes TCP ports 80 and 443 from required sources.
- You have the OpenShift Container Platform installation program binary for your target OpenShift Container Platform version.
- You have an `install-config.yaml` file.

Procedure

1. Generate the installation configuration file by using the OpenShift Container Platform installation program by entering the following command:

   ```
   $ openshift-install create install-config --dir=<your_installation_directory>
   ```

2. Use a text editor to open the `install-config.yaml` file.

3. Define subnets and assign roles. You must define your VPC subnets and their designated roles under the `platform.aws.vpc.subnets` parameter. For each AWS subnet, create an entry by specifying an `id` and a list of `roles`. Each role is an object with a `type` key. To designate a subnet for the default Ingress Controller, assign a role with `type: IngressControllerLB` to the subnet.

```
apiVersion: v1
baseDomain: example.com
metadata:
  name: my-cluster
platform:
  aws:
    region: us-east-1
    vpc:
      subnets:
      - id: subnet-0fcf8e0392f0910d5
        roles:
        - type: IngressControllerLB
        - type: BootstrapNode
      - id: subnet-0xxxxxxxxxxxxxxza
        roles:
        - type: IngressControllerLB
      - id: subnet-0fcf8e0392f0910d4
        roles:
        - type: ClusterNode
      - id: subnet-0yyyyyyyyyyyyyyzb
        roles:
        - type: ClusterNode

pullSecret: '...'
sshKey: '...'
```

where:

`baseDomain`
: Specifies the base domain.

`region`
: Specifies the AWS region.

`vpc`
: Specifies the VPC object under `platform.aws` contains the subnets list.

`subnets`
: Specifies a list of all subnet objects that OpenShift will use. Each object defines a subnet id and its roles.

`id`
: Specifies the AWS Subnet ID.

`type.IngressControllerLB`
: Specifies the `type: IngressControllerLB` role specifically designates this subnet for the default Ingress Controller's LoadBalancer. In private/internal cluster, the subnet with `IngressControllerLB` role must be private.

`type.ClusterNode`
: Specifies the `type: ClusterNode` role designates this subnet for control plane and compute nodes. These are typically private subnets.

`pullSecret`
: Specifies the pull secret.

`sshKey`

Specifies the SSH key.

4. Save you changes to the `install-config.yaml` file.
5. Install the cluster by running the following command:

```
$ openshift-install create cluster --dir=<your_installation_directory>
```

The installation program uses the subnet definitions and explicit role assignments from the `platform.aws.vpc.subnets` section of your `install-config.yaml` file to provision cluster resources. This includes placing the LoadBalancer of the Ingress Controller in the subnets you designated with the `IngressControllerLB` role.

The role assignment mechanism within `platform.aws.vpc.subnets`, such as specifying types like `IngressControllerLB`, `ClusterNode`, `ControlPlaneExternalLB`, `ControlPlaneInternalLB`, `BootstrapNode` is the comprehensive way the installation program identifies suitable subnets for various cluster services and components.

3.5.4. Alternatives to storing administrator-level secrets in the kube-system project

By default, administrator secrets are stored in the `kube-system` project.

If you configured the `credentialsMode` parameter in the `install-config.yaml` file to `Manual`, you must use one of the following alternatives:

- To manage long-term cloud credentials manually, follow the procedure in "Manually creating long-term credentials".
- To implement short-term credentials that are managed outside the cluster for individual components, follow the procedures in "Configuring an AWS cluster to use short-term credentials".

3.5.4.1. Manually creating long-term credentials

You can put the Cloud Credential Operator (CCO) into manual mode before OpenShift Container Platform installation if the cloud identity and access management (IAM) APIs are not reachable, or if you prefer not to store an administrator-level credential secret in the cluster `kube-system` namespace.

Procedure

1. If you did not set the `credentialsMode` parameter in the `install-config.yaml` configuration file to `Manual`, modify the value as shown:

   Sample configuration file snippet

   ```
   apiVersion: v1
   baseDomain: example.com
   credentialsMode: Manual
   ```

2. If you have not previously created installation manifest files, do so by running the following command:

   ```
   $ openshift-install create manifests --dir <installation_directory>
   ```

   where `<installation_directory>` is the directory in which the installation program creates files.

3. Set a `$RELEASE_IMAGE` variable with the release image from your installation file by running the following command:

   ```
   $ RELEASE_IMAGE=$(./openshift-install version | awk '/release image/ {print $3}')
   ```

4. Extract the list of `CredentialsRequest` custom resources (CRs) from the OpenShift Container Platform release image by running the following command:

```
$ oc adm release extract \
  --from=$RELEASE_IMAGE \
  --credentials-requests \
  --included \
  --install-config=<path_to_directory_with_installation_configuration>/install-config.yaml \
  --to=<path_to_directory_for_credentials_requests>
```

where:

`--included`
: Specifies only the manifests that your specific cluster configuration requires.

`<path_to_directory_with_installation_configuration>`
: Specifies the location of the `install-config.yaml` file.

`<path_to_directory_for_credentials_requests>`
: Specifies the path to the directory where you want to store the `CredentialsRequest` objects. If the specified directory does not exist, this command creates it.

  This command creates a YAML file for each `CredentialsRequest` object.

  Sample `CredentialsRequest` object

```
apiVersion: cloudcredential.openshift.io/v1
kind: CredentialsRequest
metadata:
  name: <component_credentials_request>
  namespace: openshift-cloud-credential-operator
  ...
spec:
  providerSpec:
    apiVersion: cloudcredential.openshift.io/v1
    kind: AWSProviderSpec
    statementEntries:
    - effect: Allow
      action:
      - iam:GetUser
      - iam:GetUserPolicy
      - iam:ListAccessKeys
      resource: "*"
  ...
```

5. Create YAML files for secrets in the `openshift-install` manifests directory that you generated previously. The secrets must be stored using the namespace and secret name defined in the `spec.secretRef` for each `CredentialsRequest` object.

   Sample `CredentialsRequest` object with secrets

   ```
   apiVersion: cloudcredential.openshift.io/v1
   kind: CredentialsRequest
   metadata:
     name: <component_credentials_request>
     namespace: openshift-cloud-credential-operator
     ...
   spec:
     providerSpec:
       apiVersion: cloudcredential.openshift.io/v1
       kind: AWSProviderSpec
       statementEntries:
       - effect: Allow
         action:
         - s3:CreateBucket
         - s3:DeleteBucket
         resource: "*"
         ...
     secretRef:
       name: <component_secret>
       namespace: <component_namespace>
     ...
   ```

   Sample `Secret` object

   ```
   apiVersion: v1
   kind: Secret
   metadata:
     name: <component_secret>
     namespace: <component_namespace>
   data:
     aws_access_key_id: <base64_encoded_aws_access_key_id>
     aws_secret_access_key: <base64_encoded_aws_secret_access_key>
   ```

   Before upgrading a cluster that uses manually maintained credentials, you must ensure that the CCO is in an upgradeable state.

3.5.4.2. Configuring an AWS cluster to use short-term credentials

To install a cluster that is configured to use the AWS Security Token Service (STS), you must configure the CCO utility and create the required AWS resources for your cluster.

3.5.4.2.1. Configuring the Cloud Credential Operator utility

To create and manage cloud credentials from outside of the cluster when the Cloud Credential Operator (CCO) is operating in manual mode, extract and prepare the CCO utility (`ccoctl`) binary.

The `ccoctl` utility is a Linux binary that must run in a Linux environment.

Prerequisites

- You have access to an OpenShift Container Platform account with cluster administrator access.
- You have installed the OpenShift CLI (`oc`).

- You have created an AWS account for the `ccoctl` utility to use with the following permissions:

  Required `iam` permissions

  - `iam:CreateOpenIDConnectProvider`
  - `iam:CreateRole`
  - `iam:DeleteOpenIDConnectProvider`
  - `iam:DeleteRole`
  - `iam:DeleteRolePolicy`
  - `iam:GetOpenIDConnectProvider`
  - `iam:GetRole`
  - `iam:GetUser`
  - `iam:ListOpenIDConnectProviders`
  - `iam:ListRolePolicies`
  - `iam:ListRoles`
  - `iam:PutRolePolicy`
  - `iam:TagOpenIDConnectProvider`
  - `iam:TagRole`

  Required `s3` permissions

  - `s3:CreateBucket`
  - `s3:DeleteBucket`
  - `s3:DeleteObject`
  - `s3:GetBucketAcl`
  - `s3:GetBucketTagging`
  - `s3:GetObject`
  - `s3:GetObjectAcl`
  - `s3:GetObjectTagging`
  - `s3:ListBucket`
  - `s3:PutBucketAcl`
  - `s3:PutBucketPolicy`
  - `s3:PutBucketPublicAccessBlock`
  - `s3:PutBucketTagging`
  - `s3:PutObject`
  - `s3:PutObjectAcl`
  - `s3:PutObjectTagging`

  Required `cloudfront` permissions

  - `cloudfront:ListCloudFrontOriginAccessIdentities`
  - `cloudfront:ListDistributions`
  - `cloudfront:ListTagsForResource`

- If you plan to store the OIDC configuration in a private S3 bucket that is accessed by the IAM identity provider through a public CloudFront distribution URL, the AWS account that runs the `ccoctl` utility requires the following additional permissions:
  - `cloudfront:CreateCloudFrontOriginAccessIdentity`
  - `cloudfront:CreateDistribution`
  - `cloudfront:DeleteCloudFrontOriginAccessIdentity`
  - `cloudfront:DeleteDistribution`
  - `cloudfront:GetCloudFrontOriginAccessIdentity`
  - `cloudfront:GetCloudFrontOriginAccessIdentityConfig`
  - `cloudfront:GetDistribution`
  - `cloudfront:TagResource`
  - `cloudfront:UpdateDistribution`

  These additional permissions support the use of the `--create-private-s3-bucket` option when processing credentials requests with the `ccoctl aws create-all` command.

Procedure

1. Set a variable for the OpenShift Container Platform release image by running the following command:

   ```
   $ RELEASE_IMAGE=$(./openshift-install version | awk '/release image/ {print $3}')
   ```

2. Obtain the CCO container image from the OpenShift Container Platform release image by running the following command:

   ```
   $ CCO_IMAGE=$(oc adm release info --image-for='cloud-credential-operator' $RELEASE_IMAGE -a
   ~/.pull-secret)
   ```

   Ensure that the architecture of the `$RELEASE_IMAGE` matches the architecture of the environment in which you will use the `ccoctl` tool.

3. Extract the `ccoctl` binary from the CCO container image within the OpenShift Container Platform release image by running the following command:

   ```
   $ oc image extract $CCO_IMAGE \
     --file="/usr/bin/ccoctl.<rhel_version>" \
     -a ~/.pull-secret
   ```

   For `<rhel_version>`, specify the value that corresponds to the version of Red Hat Enterprise Linux (RHEL) that the host uses. If no value is specified, `ccoctl.rhel8` is used by default. The following values are valid:

   - `rhel8`: Specify this value for hosts that use RHEL 8.
   - `rhel9`: Specify this value for hosts that use RHEL 9.

   The `ccoctl` binary is created in the directory from where you executed the command and not in `/usr/bin/`. You must rename the directory or move the `ccoctl.<rhel_version>` binary to `ccoctl`.

4. Change the permissions to make `ccoctl` executable by running the following command:

   ```
   $ chmod 775 ccoctl
   ```

Verification

To verify that `ccoctl` is ready to use, display the help file. Use a relative file name when you run the command, for example:

```
$ ./ccoctl
```

Example output

```
OpenShift credentials provisioning tool

Usage:
  ccoctl [command]

Available Commands:
  aws          Manage credentials objects for AWS cloud
  azure        Manage credentials objects for Azure
  gcp          Manage credentials objects for Google cloud
  help         Help about any command
  ibmcloud     Manage credentials objects for IBM Cloud
  nutanix      Manage credentials objects for Nutanix

Flags:
  -h, --help   help for ccoctl

Use "ccoctl [command] --help" for more information about a command.
```

3.5.4.2.2. Creating AWS resources with a single command

If the process the `ccoctl` tool uses to create AWS resources automatically meets the requirements of your organization, you can use the `ccoctl aws create-all` command to automate the creation of AWS resources.

Otherwise, you can create the AWS resources individually. For more information, see "Creating AWS resources individually".

By default, `ccoctl` creates objects in the directory in which the commands are run. To create the objects in a different directory, use the `--output-dir` flag. This procedure uses `<path_to_ccoctl_output_dir>` to refer to this directory.

Prerequisites

You must have:

- Extracted and prepared the `ccoctl` binary.

Procedure

1. Set a `$RELEASE_IMAGE` variable with the release image from your installation file by running the following command:

```
$ RELEASE_IMAGE=$(./openshift-install version | awk '/release image/ {print $3}')
```

2. Extract the list of `CredentialsRequest` objects from the OpenShift Container Platform release image by running the following command:

```
$ oc adm release extract \
  --from=$RELEASE_IMAGE \
  --credentials-requests \
  --included \
  --install-config=<path_to_directory_with_installation_configuration>/install-config.yaml \
  --to=<path_to_directory_for_credentials_requests>
```

where:

`--included`
: Specifies to include only the manifests that your specific cluster configuration requires.

`<path_to_directory_with_installation_configuration>`
: Specifies the location of the `install-config.yaml` file.

`<path_to_directory_for_credentials_requests>`
: Specifies the path to the directory where you want to store the `CredentialsRequest` objects. If the specified directory does not exist, this command creates it.

  This command might take a few moments to run.

3. Use the `ccoctl` tool to process all `CredentialsRequest` objects by running the following command:

```
$ ccoctl aws create-all \
  --name=<name> \
  --region=<aws_region> \
  --credentials-requests-dir=<path_to_credentials_requests_directory> \
  --output-dir=<path_to_ccoctl_output_dir> \
  --create-private-s3-bucket \
  --permissions-boundary-arn=<policy_arn>
```

where:

`<name>`
: Specifies the name used to tag any cloud resources that are created for tracking.

`<aws_region>`
: Specifies the AWS region in which cloud resources will be created.

`<path_to_credentials_requests_directory>`
: Specifies the directory containing the files for the component `CredentialsRequest` objects.

`<path_to_ccoctl_output_dir>`
: Specifies the directory in which you want the `ccoctl` utility to create objects. By default, the utility creates objects in the directory in which the commands are run. This parameter is optional.

`--create-private-s3-bucket`
: Specifies that the OpenID Connect (OIDC) configuration files should be stored in a private S3 bucket that is accessed by the IAM identity provider through a public CloudFront distribution URL. Note that by default, the `ccoctl` utility stores the OIDC configuration files in a public S3 bucket and uses the S3 URL as the public OIDC endpoint. This parameter is optional.

`<policy_arn>`
: Specifies the Amazon Resource Name (ARN) of the AWS IAM policy to use as the permissions boundary for the IAM roles created by the `ccoctl` utility. This parameter is optional.

  If your cluster uses Technology Preview features that are enabled by the `TechPreviewNoUpgrade` feature set, you must include the `--enable-tech-preview` parameter.

Verification

To verify that the OpenShift Container Platform secrets are created, list the files in the `<path_to_ccoctl_output_dir>/manifests` directory:

```
$ ls <path_to_ccoctl_output_dir>/manifests
```

Example output

```
cluster-authentication-02-config.yaml
openshift-cloud-credential-operator-cloud-credential-operator-iam-ro-creds-credentials.yaml
openshift-cloud-network-config-controller-cloud-credentials-credentials.yaml
openshift-cluster-api-capa-manager-bootstrap-credentials-credentials.yaml
openshift-cluster-csi-drivers-ebs-cloud-credentials-credentials.yaml
openshift-image-registry-installer-cloud-credentials-credentials.yaml
openshift-ingress-operator-cloud-credentials-credentials.yaml
openshift-machine-api-aws-cloud-credentials-credentials.yaml
```

You can verify that the IAM roles are created by querying AWS. For more information, refer to AWS documentation on listing IAM roles.

3.5.4.2.3. Creating AWS resources individually

If you need to review the JSON files that the `ccoctl` tool creates before modifying AWS resources, or if the process the `ccoctl` tool uses to create AWS resources automatically does not meet the requirements of your organization, you can create the AWS resources individually.

This option might be useful for an organization that shares the responsibility for creating these resources among different users or departments.

Otherwise, you can use the `ccoctl aws create-all` command to create the AWS resources automatically. For more information, see "Creating AWS resources with a single command".

By default, `ccoctl` creates objects in the directory in which the commands are run. To create the objects in a different directory, use the `--output-dir` flag. This procedure uses `<path_to_ccoctl_output_dir>` to refer to this directory.

Some `ccoctl` commands make AWS API calls to create or modify AWS resources. You can use the `--dry-run` flag to avoid making API calls. Using this flag creates JSON files on the local file system instead. You can review and modify the JSON files and then apply them with the AWS CLI tool using the `--cli-input-json` parameters.

Prerequisites

Extract and prepare the `ccoctl` binary.

Procedure

1. Generate the public and private RSA key files that are used to set up the OpenID Connect provider for the cluster by running the following command:

```
$ ccoctl aws create-key-pair
```

Example output

```
2021/04/13 11:01:02 Generating RSA keypair
2021/04/13 11:01:03 Writing private key to /<path_to_ccoctl_output_dir>/serviceaccount-
signer.private
2021/04/13 11:01:03 Writing public key to /<path_to_ccoctl_output_dir>/serviceaccount-
signer.public
2021/04/13 11:01:03 Copying signing key for use by installer
```

where `serviceaccount-signer.private` and `serviceaccount-signer.public` are the generated key files.

This command also creates a private key that the cluster requires during installation in `/<path_to_ccoctl_output_dir>/tls/bound-service-account-signing-key.key`.

2. Create an OpenID Connect identity provider and S3 bucket on AWS by running the following command:

```
$ ccoctl aws create-identity-provider \
  --name=<name> \
  --region=<aws_region> \
  --public-key-file=<path_to_ccoctl_output_dir>/serviceaccount-signer.public
```

where:

`<name>`
: Specifies the name used to tag any cloud resources that are created for tracking.

`<aws_region>`
: Specifies the AWS region in which cloud resources will be created.

`<path_to_ccoctl_output_dir>`
: Specifies the path to the public key file that the `ccoctl aws create-key-pair` command generated.

Example output

```
2021/04/13 11:16:09 Bucket <name>-oidc created
2021/04/13 11:16:10 OpenID Connect discovery document in the S3 bucket <name>-oidc at .well-
known/openid-configuration updated
2021/04/13 11:16:10 Reading public key
2021/04/13 11:16:10 JSON web key set (JWKS) in the S3 bucket <name>-oidc at keys.json updated
2021/04/13 11:16:18 Identity Provider created with ARN: arn:aws:iam::<aws_account_id>:oidc-
provider/<name>-oidc.s3.<aws_region>.amazonaws.com
```

where `openid-configuration` is a discovery document and `keys.json` is a JSON web key set file.

This command also creates a YAML configuration file in `/<path_to_ccoctl_output_dir>/manifests/cluster-authentication-02-config.yaml`. This file sets the issuer URL field for the service account tokens that the cluster generates, so that the AWS IAM identity provider trusts the tokens.

3. Create IAM roles for each component in the cluster:

   a. Set a `$RELEASE_IMAGE` variable with the release image from your installation file by running the following command:

   ```
   $ RELEASE_IMAGE=$(./openshift-install version | awk '/release image/ {print $3}')
   ```

   b. Extract the list of `CredentialsRequest` objects from the OpenShift Container Platform release image:

   ```
   $ oc adm release extract \
     --from=$RELEASE_IMAGE \
     --credentials-requests \
     --included \
     --install-config=<path_to_directory_with_installation_configuration>/install-config.yaml
   \
     --to=<path_to_directory_for_credentials_requests>
   ```

   where:

   `--included`
   Specifies the `--included` parameter, which includes only the manifests that your specific cluster configuration requires.

   `--install-config`
   Specifies the location of the `install-config.yaml` file.

   `-to`
   Specifies the path to the directory where you want to store the `CredentialsRequest` objects. If the specified directory does not exist, this command creates it.

   c. Use the `ccoctl` tool to process all `CredentialsRequest` objects by running the following command:

   ```
   $ ccoctl aws create-iam-roles \
     --name=<name> \
     --region=<aws_region> \
     --credentials-requests-dir=<path_to_credentials_requests_directory> \
     --identity-provider-arn=arn:aws:iam::<aws_account_id>:oidc-provider/<name>-oidc.s3.
   <aws_region>.amazonaws.com
   ```

   For AWS environments that use alternative IAM API endpoints, such as GovCloud, you must also specify your region with the `--region` parameter.

   If your cluster uses Technology Preview features that are enabled by the `TechPreviewNoUpgrade` feature set, you must include the `--enable-tech-preview` parameter.

   For each `CredentialsRequest` object, `ccoctl` creates an IAM role with a trust policy that is tied to the specified OIDC identity provider, and a permissions policy as defined in each `CredentialsRequest` object from the OpenShift Container Platform release image.

Verification

To verify that the OpenShift Container Platform secrets are created, list the files in the `<path_to_ccoctl_output_dir>/manifests` directory:

```
$ ls <path_to_ccoctl_output_dir>/manifests
```

Example output

```
cluster-authentication-02-config.yaml
openshift-cloud-credential-operator-cloud-credential-operator-iam-ro-creds-credentials.yaml
openshift-cloud-network-config-controller-cloud-credentials-credentials.yaml
openshift-cluster-api-capa-manager-bootstrap-credentials-credentials.yaml
openshift-cluster-csi-drivers-ebs-cloud-credentials-credentials.yaml
openshift-image-registry-installer-cloud-credentials-credentials.yaml
openshift-ingress-operator-cloud-credentials-credentials.yaml
openshift-machine-api-aws-cloud-credentials-credentials.yaml
```

You can verify that the IAM roles are created by querying AWS. For more information, refer to AWS documentation on listing IAM roles.

3.5.4.2.4. Incorporating the Cloud Credential Operator utility manifests

To implement short-term security credentials managed outside the cluster for individual OpenShift Container Platform components, you must move the manifest files that the Cloud Credential Operator utility (`ccoctl`) created to the correct directories for the installation program.

Prerequisites

- You have configured an account with the cloud platform that hosts your cluster.
- You have configured the Cloud Credential Operator utility (`ccoctl`).
- You have created the cloud provider resources that are required for your cluster with the `ccoctl` utility.

Procedure

1. If you did not set the `credentialsMode` parameter in the `install-config.yaml` configuration file to `Manual`, modify the value as shown:

   Sample configuration file snippet

   ```
   apiVersion: v1
   baseDomain: example.com
   credentialsMode: Manual
   ```

2. If you have not previously created installation manifest files, do so by running the following command:

   ```
   $ openshift-install create manifests --dir <installation_directory>
   ```

   where `<installation_directory>` is the directory in which the installation program creates files.

3. Copy the manifests that the `ccoctl` utility generated to the `manifests` directory that the installation program created by running the following command:

   ```
   $ cp /<path_to_ccoctl_output_dir>/manifests/* ./manifests/
   ```

4. Copy the `tls` directory that contains the private key to the installation directory:

   ```
   $ cp -a /<path_to_ccoctl_output_dir>/tls .
   ```

3.5.5. Deploying the cluster

To deploy your OpenShift Container Platform cluster, you initialize installation by running the `openshift-install create cluster` command from the directory that contains the installation program. The

installation program provisions the required infrastructure and completes the cluster setup.

You can run the `create cluster` command of the installation program only once, during initial installation.

Prerequisites

- You have configured an account with the cloud platform that hosts your cluster.
- You have the OpenShift Container Platform installation program and the pull secret for your cluster.
- You have verified that the cloud provider account on your host has the correct permissions to deploy the cluster. An account with incorrect permissions causes the installation process to fail with an error message that displays the missing permissions.

Procedure

1. In the directory that contains the installation program, initialize the cluster deployment by running the following command:

   ```
   $ ./openshift-install create cluster --dir <installation_directory> \
       --log-level=info
   ```

   where:

   - `<installation_directory>`: Specifies the location of your customized `./install-config.yaml` file.
   - `--log-level`: Specifies the log level. To view different installation details, specify `warn`, `debug`, or `error` instead of `info`.
2. Optional: Remove or disable the `AdministratorAccess` policy from the IAM account that you used to install the cluster.

   The elevated permissions provided by the `AdministratorAccess` policy are required only during installation.

Verification

When the cluster deployment completes successfully:

- The terminal displays directions for accessing your cluster, including a link to the web console and credentials for the `kubeadmin` user.

- Credential information also outputs to `<installation_directory>/.openshift_install.log`.

  Do not delete the installation program or the files that the installation program creates. Both are required to delete the cluster.

  The following example shows the expected output:

```
...
INFO Install complete!
INFO To access the cluster as the system:admin user when using 'oc', run 'export
KUBECONFIG=/home/myuser/install_dir/auth/kubeconfig'
INFO Access the OpenShift web-console here: https://console-openshift-
console.apps.mycluster.example.com
INFO Login to the console with user: "kubeadmin", and password: "password"
INFO Time elapsed: 36m22s
```

  - The Ignition config files that the installation program generates contain certificates that expire after 24 hours, which are then renewed at that time. If the cluster is shut down before renewing the certificates and the cluster is later restarted after the 24 hours have elapsed, the cluster automatically recovers the expired certificates. The exception is that you must manually approve the pending `node-bootstrapper` certificate signing requests (CSRs) to recover kubelet certificates. See the documentation for *Recovering from expired control plane certificates* for more information.
  - It is recommended that you use Ignition config files within 12 hours after they are generated because the 24-hour certificate rotates from 16 to 22 hours after the cluster is installed. By using the Ignition config files within 12 hours, you can avoid installation failure if the certificate update runs during installation.

3.5.6. Provisioning your own DNS records

Use your cluster name and base cluster domain to configure a CNAME record for the API service `api.<cluster_name>.<base_domain>.` with the API load balancer DNS name, and provision a CNAME record for the Ingress hostname `*.apps.<cluster_name>.<base_domain>.` with the Ingress load balancer DNS name.

User-provisioned DNS is a Technology Preview feature only. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see Technology Preview Features Support Scope.

Prerequisites

You have installed the AWS CLI.

Procedure

1. Add the `userProvisionedDNS` parameter to the `install-config.yaml` file and enable the parameter. For more information, see "Enabling a user-managed DNS".
2. Install your cluster.
3. If you are installing a private cluster, set the `api_lb_name` variable by running the following command:

```
$ api_lb_name="${INFRA_ID}-int"
```

4. If you are installing a public cluster, set the `api_lb_name` variable by running the following command:

```
$ api_lb_name="${INFRA_ID}-ext"
```

5. To retrieve the DNS name of the API service, run the following command:

```
$ aws --region ${REGION} elbv2 describe-load-balancers --names ${api_lb_name} --query
'LoadBalancers[*].DNSName' --output text
```

6. Use the DNS name and your cluster name and base cluster domain to configure your own DNS record with the `api.<cluster_name>.<base_domain>.` hostname.
7. To retrieve the DNS name of the Ingress service, run the following command:

```
$ ingress_lb_name=$(aws --region ${REGION} resourcegroupstaggingapi get-resources --resource-
type-filters elasticloadbalancing:loadbalancer --tag-filters
Key=kubernetes.io/cluster/${INFRA_ID},Values=owned Key=kubernetes.io/service-
name,Values=openshift-ingress/router-default --query 'ResourceTagMappingList[*].ResourceARN |
[0]' --output text | awk -F'/' '{print $2}')
```

8. Run the following command, which uses the variable `ingress_lb_name` generated from the previous command:

```
$ aws --region ${REGION} elb describe-load-balancers --load-balancer-names ${ingress_lb_name}
--query 'LoadBalancerDescriptions[].DNSName' --output text
```

9. Use the DNS name and your cluster name and base cluster domain to configure your own DNS record with the `*.apps.<cluster_name>.<base_domain>.` hostname.

3.5.7. Logging in to the cluster by using the CLI

To log in to your cluster as the default system user, export the `kubeconfig` file. This configuration enables the CLI to authenticate and connect to the specific API server created during OpenShift Container Platform installation.

The `kubeconfig` file is specific to a cluster and OpenShift Container Platform generates it during installation.

Prerequisites

- You deployed an OpenShift Container Platform cluster.
- You installed the OpenShift CLI (`oc`).

Procedure

1. Export the `kubeadmin` credentials by running the following command:

```
$ export KUBECONFIG=<installation_directory>/auth/kubeconfig
```

   where:

   `<installation_directory>`
   Specifies the path to the directory that stores the installation files.

2. Verify you can run `oc` commands successfully using the exported configuration by running the following command:

```
$ oc whoami
```

   Example output

```
system:admin
```

Next steps

- "Customize your cluster"
- "Remote health reporting"

3.5.8. Logging in to the cluster by using the web console

To verify that your cluster deployed successfully and access its features, log in to the OpenShift Container Platform web console as the `kubeadmin` user.

Prerequisites

- You have access to the installation host.
- You completed a cluster installation and all cluster Operators are available.

Procedure

1. Obtain the password for the `kubeadmin` user from the `kubeadmin-password` file on the installation host:

   ```
   $ cat <installation_directory>/auth/kubeadmin-password
   ```

   Or, you can obtain the `kubeadmin` password from the `<installation_directory>/.openshift_install.log` log file on the installation host.

2. List the OpenShift Container Platform web console route:

   ```
   $ oc get routes -n openshift-console | grep 'console-openshift'
   ```

   Or, you can obtain the OpenShift Container Platform route from the `<installation_directory>/.openshift_install.log` log file on the installation host.

   Example output

   ```
   console     console-openshift-console.apps.<cluster_name>.<base_domain>            console
   https   reencrypt/Redirect   None
   ```

3. Navigate to the route detailed in the output of the preceding command in a web browser and log in as the `kubeadmin` user.

3.6. Installing a private cluster on AWS

In OpenShift Container Platform version 4.22, you can install a private cluster into an existing Virtual Private Cloud (VPC) on Amazon Web Services (AWS). The installation program provisions the rest of the required infrastructure, which you can further customize. To customize the installation, you modify parameters in the `install-config.yaml` file before you install the cluster.

3.6.1. Prerequisites

Before you install a private cluster into an existing Virtual Private Cloud (VPC) on Amazon Web Services (AWS), you must meet several prerequisites.

The following prerequisites must be met:

- You reviewed details about the OpenShift Container Platform installation and update processes.
- You read the documentation on selecting a cluster installation method and preparing it for users.

- You configured an AWS account to host the cluster.

  If you have an AWS profile stored on your computer, it must not use a temporary session token that you generated while using a multi-factor authentication device. The cluster continues to use your current AWS credentials to create AWS resources for the entire life of the cluster, so you must use long-term credentials. To generate appropriate keys, see Managing Access Keys for IAM Users in the AWS documentation. You can supply the keys when you run the installation program.

- If you use a firewall, you configured it to allow the sites that your cluster requires access to.

3.6.2. Private clusters

You can deploy a private OpenShift Container Platform cluster that does not expose external endpoints. Private clusters are accessible from only an internal network and are not visible to the internet.

By default, OpenShift Container Platform is provisioned to use publicly-accessible DNS and endpoints. A private cluster sets the DNS, Ingress Controller, and API server to private when you deploy your cluster. This means that the cluster resources are only accessible from your internal network and are not visible to the internet.

If the cluster has any public subnets, load balancer services created by administrators might be publicly accessible. To ensure cluster security, verify that these services are explicitly annotated as private.

To deploy a private cluster, you must:

- Use existing networking that meets your requirements. Your cluster resources might be shared between other clusters on the network.
- Deploy from a machine that has access to:
  - The API services for the cloud to which you provision.
  - The hosts on the network that you provision.
  - The internet to obtain installation media.

3.6.2.1. Private clusters in AWS

To create a private cluster on Amazon Web Services (AWS), you must provide an existing private VPC and subnets to host the cluster.

The installation program must also be able to resolve the DNS records that the cluster requires. The installation program configures the Ingress Operator and API server for access from only the private network.

The cluster still requires access to internet to access the AWS APIs.

The following items are not required or created when you install a private cluster:

- Public subnets
- Public load balancers, which support public ingress
- A public Route 53 zone that matches the `baseDomain` for the cluster

The installation program does use the `baseDomain` that you specify to create a private Route 53 zone and the required records for the cluster. The cluster is configured so that the Operators do not create public records for the cluster and all cluster machines are placed in the private subnets that you specify.

3.6.2.1.1. Limitations

The ability to add public functionality to a private cluster is limited.

- You cannot make the Kubernetes API endpoints public after installation without taking additional actions, including creating public subnets in the VPC for each availability zone in use, creating a public load balancer, and configuring the control plane security groups to allow traffic from the internet on 6443 (Kubernetes API port).
- If you use a public Service type load balancer, you must tag a public subnet in each availability zone with `kubernetes.io/cluster/<cluster-infra-id>: shared` so that AWS can use them to create public load balancers.

3.6.3. About using a custom VPC

You can deploy a cluster into existing subnets in an existing Amazon Virtual Private Cloud (VPC) in Amazon Web Services (AWS).

By deploying OpenShift Container Platform into an existing AWS VPC, you might be able to avoid limit constraints in new accounts or more easily abide by the operational constraints that your company's guidelines set. If you cannot obtain the infrastructure creation permissions that are required to create the VPC yourself, use this installation option.

Because the installation program cannot know what other components are also in your existing subnets, it cannot choose subnet CIDRs and so forth on your behalf. You must configure networking for the subnets that you install your cluster to yourself.

The installation program no longer creates the following components:

- Internet gateways
- NAT gateways
- Subnets
- Route tables
- VPCs
- VPC DHCP options
- VPC endpoints

The installation program requires that you use the cloud-provided DNS server. Using a custom DNS server is not supported and causes the installation to fail.

If you use a custom VPC, you must correctly configure it and its subnets for the installation program and the cluster to use. See "Create a VPC" in the Amazon Web Services documentation for more information about AWS VPC console wizard configurations and creating and managing an AWS VPC.

The installation program cannot:

- Subdivide network ranges for the cluster to use.
- Set route tables for the subnets.
- Set VPC options like DHCP.

You must complete these tasks before you install the cluster. See "VPC networking components" and "Route tables for your VPC" for more information on configuring networking in an AWS VPC.

Your VPC must meet the following characteristics:

- The VPC must not use the `kubernetes.io/cluster/.*: owned`, `Name`, and `openshift.io/cluster` tags.

  The installation program modifies your subnets to add the `kubernetes.io/cluster/.*: shared` tag, so your subnets must have at least one free tag slot available for it. See "Tag Restrictions" in the AWS documentation to confirm that the installation program can add a tag to each subnet that you specify. You cannot use a `Name` tag, because it overlaps with the EC2 `Name` field and the installation fails.

- If you want to extend your OpenShift Container Platform cluster into an AWS Outpost and have an existing Outpost subnet, the existing subnet must use the `kubernetes.io/cluster/unmanaged: true` tag. If you do not apply this tag, the installation might fail due to the Cloud Controller Manager creating a service load balancer in the Outpost subnet, which is an unsupported configuration.
- You must enable the `enableDnsSupport` and `enableDnsHostnames` attributes in your VPC, so that the cluster can use the Route 53 zones that are attached to the VPC to resolve cluster's internal DNS records. See "DNS Support in Your VPC" in the AWS documentation.

  If you prefer to use your own Route 53 hosted private zone, you must associate the existing hosted zone with your VPC prior to installing a cluster. You can define your hosted zone using the `platform.aws.hostedZone` and `platform.aws.hostedZoneRole` fields in the `install-config.yaml` file. You can use a private hosted zone from another account by sharing it with the account where you install the cluster. If you use a private hosted zone from another account, you must use the `Passthrough` or `Manual` credentials mode.

If you are working in a disconnected environment, you are unable to reach the public IP addresses for EC2, ELB, and S3 endpoints. Depending on the level to which you want to restrict internet traffic during the installation, the following configuration options are available:

3.6.3.1. Option 1: Create VPC endpoints

Create a VPC endpoint and attach it to the subnets that the clusters are using. Name the endpoints as follows:

- `ec2.<aws_region>.amazonaws.com`
- `elasticloadbalancing.<aws_region>.amazonaws.com`
- `s3.<aws_region>.amazonaws.com`

With this option, network traffic remains private between your VPC and the required AWS services.

3.6.3.2. Option 2: Create a proxy without VPC endpoints

As part of the installation process, you can configure an HTTP or HTTPS proxy. With this option, internet traffic goes through the proxy to reach the required AWS services.

3.6.3.3. Option 3: Create a proxy with VPC endpoints

As part of the installation process, you can configure an HTTP or HTTPS proxy with VPC endpoints. Create a VPC endpoint and attach it to the subnets that the clusters are using. Name the endpoints as follows:

- `ec2.<aws_region>.amazonaws.com`
- `elasticloadbalancing.<aws_region>.amazonaws.com`
- `s3.<aws_region>.amazonaws.com`

When configuring the proxy in the `install-config.yaml` file, add these endpoints to the `noProxy` field. With this option, the proxy prevents the cluster from accessing the internet directly. However, network traffic remains private between your VPC and the required AWS services.

You must provide a suitable VPC and subnets that allow communication to your machines.

Component	AWS type	Description
VPC	• `AWS::EC2::VPC` • `AWS::EC2::VPCEndpoint`	You must provide a public VPC for the cluster to use. The VPC uses an endpoint that references the route tables for each subnet to improve communication with the registry that is hosted in S3.
Public subnets	• `AWS::EC2::Subnet` • `AWS::EC2::SubnetNetworkAclAssociation`	Your VPC must have public subnets for between 1 and 3 availability zones and associate them with appropriate Ingress rules.
Internet gateway	• `AWS::EC2::InternetGateway` • `AWS::EC2::VPCGatewayAttachment` • `AWS::EC2::RouteTable` • `AWS::EC2::Route` • `AWS::EC2::SubnetRouteTableAssociation` • `AWS::EC2::NatGateway` • `AWS::EC2::EIP`	You must have a public internet gateway, with public routes, attached to the VPC. In the provided templates, each public subnet has a NAT gateway with an EIP address. These NAT gateways allow cluster resources, like private subnet instances, to reach the internet and are not required for some restricted network or proxy scenarios.
Network access control	• `AWS::EC2::NetworkAcl` • `AWS::EC2::NetworkAclEntry`	You must allow the VPC to access the following ports:

Port	Reason
`80`	Inbound HTTP traffic
`443`	Inbound HTTPS traffic
`22`	Inbound SSH traffic
`1024 - 65535`	Inbound ephemeral traffic
`0 - 65535`	Outbound ephemeral traffic

Component	AWS type	Description
Private subnets	• `AWS::EC2::Subnet` • `AWS::EC2::RouteTable` • `AWS::EC2::SubnetRouteTableAssociation`	Your VPC can have private subnets. The provided CloudFormation templates can create private subnets for between 1 and 3 availability zones. If you use private subnets, you must provide appropriate routes and tables for them.

Table 3.16. Required VPC components

3.6.3.4. VPC validation

To ensure that the subnets that you provide are suitable, the installation program confirms the following data:

- All the subnets that you specify exist.
- You provide private subnets.
- The subnet CIDRs belong to the machine CIDR that you specified.
- You provide subnets for each availability zone. Each availability zone contains no more than one public and one private subnet. If you use a private cluster, provide only a private subnet for each availability zone. Otherwise, provide exactly one public and private subnet for each availability zone.
- You provide a public subnet for each private subnet availability zone. Machines are not provisioned in availability zones that you do not provide private subnets for.

If you destroy a cluster that uses an existing VPC, the VPC is not deleted. When you remove the OpenShift Container Platform cluster from a VPC, the `kubernetes.io/cluster/.*: shared` tag is removed from the subnets that it used.

3.6.3.5. Division of permissions

Starting with OpenShift Container Platform 4.3, you do not need all of the permissions that are required for an installation program-provisioned infrastructure cluster to deploy a cluster. This change mimics the division of permissions that you might have at your company: some individuals can create different resource in your clouds than others. For example, you might be able to create application-specific items, like instances, buckets, and load balancers, but not networking-related components such as VPCs, subnets, or ingress rules.

The AWS credentials that you use when you create your cluster do not need the networking permissions that are required to make VPCs and core networking components within the VPC, such as subnets, routing tables, internet gateways, NAT, and VPN. You still need permission to make the application resources that the machines within the cluster require, such as ELBs, security groups, S3 buckets, and nodes.

3.6.3.6. Isolation between clusters

If you deploy OpenShift Container Platform to an existing network, the isolation of cluster services is reduced in the following ways:

- You can install multiple OpenShift Container Platform clusters in the same VPC.
- ICMP ingress is allowed from the entire network.
- TCP 22 ingress (SSH) is allowed to the entire network.
- Control plane TCP 6443 ingress (Kubernetes API) is allowed to the entire network.

- Control plane TCP 22623 ingress (MCS) is allowed to the entire network.

3.6.3.7. Optional: AWS security groups

By default, the installation program creates and attaches security groups to control plane and compute machines. The rules associated with the default security groups cannot be modified.

However, you can apply additional existing AWS security groups, which are associated with your existing VPC, to control plane and compute machines. Applying custom security groups can help you meet the security needs of your organization, in such cases where you need to control the incoming or outgoing traffic of these machines.

As part of the installation process, you apply custom security groups by modifying the `install-config.yaml` file before deploying the cluster.

For more information, see "Applying existing AWS security groups to the cluster".

By default, AWS allows 5 security groups per network interface. The installation program creates 2 security groups for compute machines and 3 security groups for control plane machines. If you are installing a cluster into a shared VPC, there are three scenarios in which you must increase this quota:

- You specified 4 or more custom security groups for compute machines using the `compute.platform.aws.additionalSecurityGroupIDs` parameter in the `install-config.yaml` file.
- You specified 3 or more custom security groups for control plane machines using the `controlPlane.platform.aws.additionalSecurityGroupIDs` parameter in the `install-config.yaml` file.
- You specified 3 or more custom security groups for all machines using the `platform.aws.defaultMachinePlatform` parameter in the `install-config.yaml` file.

You must increase the quota of security groups per network interface to a number greater than or equal to `3 + (number of control plane custom security groups OR number of default machine platform custom security groups)`, or `2 + (number of compute custom security groups OR number of default machine platform custom security groups)`, whichever is higher. If you do not specify a sufficient quota, the installation will succeed, but it will generate `SecurityGroupsPerInterfaceLimitExceeded` errors in the installation log, and the additional security groups will not be applied. The maximum allowed quota is 16 and the maximum number of user-specified security groups is 10.

3.6.4. Manually creating the installation configuration file

Installing the cluster requires that you manually create the installation configuration file.

Prerequisites

- You have an SSH public key on your local machine for use with the installation program. You can use the key for SSH authentication onto your cluster nodes for debugging and disaster recovery.
- You have obtained the OpenShift Container Platform installation program and the pull secret for your cluster.

Procedure

1. Create an installation directory to store your required installation assets in:

   ```
   $ mkdir <installation_directory>
   ```

   You must create a directory. Some installation assets, such as bootstrap X.509 certificates have short expiration intervals, so you must not reuse an installation directory. If you want to reuse individual files from another cluster installation, you can copy them into your directory. However, the file names for the installation assets might change between releases. Use caution when copying installation files from an earlier OpenShift Container Platform version.

2. Customize the provided sample `install-config.yaml` file template and save the file in the `<installation_directory>`.

   You must name this configuration file `install-config.yaml`.

3. Back up the `install-config.yaml` file so that you can use it to install many clusters.

   Back up the `install-config.yaml` file now, because the installation process consumes the file in the next step.

3.6.4.1. Minimum resource requirements for cluster installation

To ensure that your OpenShift Container Platform cluster runs as expected, each cluster machine must meet minimum CPU, memory, and storage requirements.

Machine	Operating system	vCPU	Virtual RAM	Storage	Input/Output Per Second (IOPS)
Bootstrap	RHCOS	4	16 GB	100 GB	300
Control plane	RHCOS	4	16 GB	100 GB	300
Compute	RHCOS	2	8 GB	100 GB	300

Table 3.17. Minimum resource requirements

- One vCPU is equal to one physical core when simultaneous multithreading (SMT), or Hyper-Threading, is not enabled. When enabled, use the following formula to calculate the corresponding ratio: (threads per core × cores) × sockets = vCPUs.
- OpenShift Container Platform and Kubernetes are sensitive to disk performance, and Red Hat recommends faster storage, particularly for etcd on the control plane nodes which require a 10 ms p99 fsync duration. On many cloud platforms, storage size and IOPS scale together, so you might need to provision more storage to get enough performance.
- As with all user-provisioned installations, if you choose to use RHEL compute machines in your cluster, you take responsibility for all operating system life cycle management and maintenance, including performing system updates, applying patches, and completing all other required tasks. OpenShift Container Platform 4.10 and later do not support RHEL 7 compute machines.

In OpenShift Container Platform version 4.22, RHCOS uses RHEL version 9.8, which updates the micro-architecture requirements. Each architecture requires the following minimum instruction set architectures (ISA):

- x86-64 architecture requires x86-64-v2 ISA

- ARM64 architecture requires ARMv8.0-A ISA
- ppc64le architecture requires IBM® Power9 ISA
- s390x architecture requires IBM® z14 ISA

For more information, see Architectures in the RHEL documentation.

If an instance type for your platform meets the minimum requirements for cluster machines, it is supported to use in OpenShift Container Platform.

3.6.4.2. Tested instance types for AWS

To ensure cluster stability and performance, use one of the tested Amazon Web Services (AWS) instance types for your OpenShift Container Platform machines.

The following AWS instance types have been tested with OpenShift Container Platform.

Use the machine types included in the following charts for your AWS instances. If you use an instance type that is not listed in the chart, ensure that the instance size you use matches the minimum resource requirements in "Minimum resource requirements for cluster installation".

See the following machine types based on 64-bit x86 architecture:

- `c4.*`
- `c5.*`
- `c5a.*`
- `i3.*`
- `m4.*`
- `m5.*`
- `m5a.*`
- `m6a.*`
- `m6i.*`
- `m7a.*`
- `m7i.*`
- `m7i-flex.*`
- `r4.*`
- `r5.*`
- `r5a.*`
- `r6i.*`
- `t3.*`
- `t3a.*`

3.6.4.3. Tested instance types for AWS on 64-bit ARM infrastructures

To ensure cluster stability and performance, use one of the tested Amazon Web Services (AWS) 64-bit ARM instance types for your OpenShift Container Platform machines.

The following AWS 64-bit ARM instance types have been tested with OpenShift Container Platform.

Use the machine types included in the following charts for your AWS ARM instances. If you use an instance type that is not listed in the chart, ensure that the instance size you use matches the minimum resource requirements listed in "Minimum resource requirements for cluster installation".

See the following machine types based on 64-bit ARM architecture:

- `c6g.*`
- `c7g.*`
- `c8g.*`
- `m6g.*`
- `m7g.*`
- `m7gd.*`
- `r8g.*`

3.6.4.4. Enabling a user-managed DNS

You can install a cluster with a domain name server (DNS) solution that you manage instead of the default cluster-provisioned DNS solution that uses the Route 53 service for Amazon Web Services (AWS).

For example, your organization's security policies might not allow the use of public DNS services such as Amazon Web Services DNS. In such scenarios, you can use your own DNS service to bypass the public DNS service and manage your own DNS for the IP addresses of the API and Ingress services.

If you enable user-managed DNS during installation, the installation program provisions DNS records for the API and Ingress services only within the cluster. To ensure access from outside the cluster, you must provision the DNS records in an external DNS service of your choice for the API and Ingress services after installation.

User-provisioned DNS is a Technology Preview feature only. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see Technology Preview Features Support Scope.

Procedure

Before you deploy your cluster, use a text editor to open the `install-config.yaml` file and add the following stanza:

To enable user-managed DNS:

```
featureSet: CustomNoUpgrade
featureGates: ["AWSClusterHostedDNSInstall=true"]
```

```
platform:
  aws:
    userProvisionedDNS: Enabled
```

where:

`userProvisionedDNS`
Enables user-provisioned DNS management.

Next steps

For information about provisioning your DNS records for the API server and the Ingress services, see "Provisioning your own DNS records".

3.6.4.5. Sample customized install-config.yaml file for AWS

You can customize the installation configuration file (`install-config.yaml`) to specify more details about your OpenShift Container Platform cluster's platform or modify the values of the required parameters.

This sample YAML file is provided for reference only. You must obtain your `install-config.yaml` file by using the installation program and modify it. For a full list and description of all installation configuration parameters, see *Installation configuration parameters for AWS*.

Sample `install-config.yaml` file for AWS

```
apiVersion: v1
baseDomain: example.com
sshKey: ssh-ed25519 AAAA...
pullSecret: '{"auths": ...}'
metadata:
  name: example-cluster
controlPlane:
  name: master
  platform:
    aws:
      type: m6i.xlarge
  replicas: 3
compute:
-  name: worker
  platform:
    aws:
      type: c5.4xlarge
  replicas: 3
networking:
  clusterNetwork:
  - cidr: 10.128.0.0/14
    hostPrefix: 23
platform:
  aws:
    region: us-west-2
```

- Parameters at the first level of indentation apply to the cluster globally.
- The `controlPlane` stanza applies to control plane machines.
- The `compute` stanza applies to compute machines.
- The `networking` stanza applies to the cluster networking configuration. If you do not provide networking values, the installation program provides default values.
- The `platform` stanza applies to the infrastructure platform that hosts the cluster.

3.6.4.6. Configuring the cluster-wide proxy during installation

Production environments can deny direct access to the internet and instead have an HTTP or HTTPS proxy available. You can configure a new OpenShift Container Platform cluster to use a proxy by configuring the proxy settings in the `install-config.yaml` file.

Prerequisites

- You have an existing `install-config.yaml` file.

- You have reviewed the sites that your cluster requires access to and determined whether any of them need to bypass the proxy. By default, the proxy handles all cluster egress traffic, including calls to hosting cloud provider APIs. You added sites to the `Proxy` object's `spec.noProxy` field to bypass the proxy if necessary.

  The `Proxy` object `status.noProxy` field includes the values of the `networking.machineNetwork[].cidr`, `networking.clusterNetwork[].cidr`, and `networking.serviceNetwork[]` fields from your installation configuration.

  For installations on Amazon Web Services (AWS), Google Cloud, Microsoft Azure, and Red Hat OpenStack Platform (RHOSP), the `Proxy` object `status.noProxy` field also includes the instance metadata endpoint (`169.254.169.254`).

Procedure

1. Edit your `install-config.yaml` file and add the proxy settings. For example:

```
apiVersion: v1
baseDomain: my.domain.com
proxy:
  httpProxy: http://<username>:<pswd>@<ip>:<port>
  httpsProxy: https://<username>:<pswd>@<ip>:<port>
  noProxy: ec2.<aws_region>.amazonaws.com,elasticloadbalancing.<aws_region>.amazonaws.com,s3.
<aws_region>.amazonaws.com
additionalTrustBundle: |
    -----BEGIN CERTIFICATE-----
    <MY_TRUSTED_CA_CERT>
    -----END CERTIFICATE-----
additionalTrustBundlePolicy: <policy_to_add_additionalTrustBundle>
```

   where:

   `proxy.httpProxy`
   : Specifies a proxy URL to use for creating HTTP connections outside the cluster. The URL scheme must be `http`.

   `proxy.httpsProxy`
   : Specifies a proxy URL to use for creating HTTPS connections outside the cluster.

   `proxy.noProxy`
   : Specifies a comma-separated list of destination domain names, IP addresses, or other network CIDRs to exclude from proxying. Preface a domain with `.` to match subdomains only. For example, `.y.com` matches `x.y.com`, but not `y.com`. Use `*` to bypass the proxy for all destinations. If you have added the Amazon `EC2`, `Elastic Load Balancing`, and `S3` VPC endpoints to your VPC, you must add these endpoints to the `noProxy` field.

   `additionalTrustBundle`
   : If you specify this value, the installation program generates a config map named `user-ca-bundle` in the `openshift-config` namespace to hold the additional CA certificates. If you specify `additionalTrustBundle` and at least one proxy setting, the `Proxy` object references the `user-ca-bundle` config map in the `trustedCA` field. The Cluster Network Operator then creates a `trusted-ca-bundle` config map that merges the contents specified for the `trustedCA` parameter with the RHCOS trust bundle. You must set the `additionalTrustBundle` field unless an authority from the RHCOS trust bundle signs the proxy's identity certificate.

   `additionalTrustBundlePolicy`
   : Specifies the policy that determines the configuration of the `Proxy` object to reference the `user-ca-bundle` config map in the `trustedCA` field. The allowed values are `Proxyonly` and `Always`. Use `Proxyonly` to reference the `user-ca-bundle` config map only when you configure an `http/https` proxy. Use `Always` to always reference the `user-ca-bundle` config map. The default value is `Proxyonly`. Optional parameter.

   The installation program does not support the proxy `readinessEndpoints` field.

   If the installation program times out, restart and then complete the deployment by using the `wait-for` command of the installation program. For example:

   ```
   $ ./openshift-install wait-for install-complete --log-level debug
   ```

2. Save the file and reference it when installing OpenShift Container Platform.

   The installation program creates a cluster-wide proxy named `cluster` that uses the proxy settings in the `install-config.yaml` file. If you do not give proxy settings, the installation program still creates a `cluster` `Proxy` object, but it has a nil `spec`.

   Only the `Proxy` object named `cluster` is supported, and you cannot create additional proxies.

3.6.4.7. Applying existing Amazon Web Services (AWS) security groups to the cluster

Applying existing AWS security groups to your control plane and compute machines can help you meet the security needs of your organization, in such cases where you need to control the incoming or outgoing traffic of these machines.

Prerequisites

- You have created the security groups in AWS. For more information, see the AWS documentation about working with security groups.
- The security groups must be associated with the existing VPC that you are deploying the cluster to. The security groups cannot be associated with another VPC.
- You have an existing `install-config.yaml` file.

Procedure

1. In the `install-config.yaml` file, edit the `compute.platform.aws.additionalSecurityGroupIDs` parameter to specify one or more custom security groups for your compute machines.
2. Edit the `controlPlane.platform.aws.additionalSecurityGroupIDs` parameter to specify one or more custom security groups for your control plane machines.

3. Save the file and reference it when deploying the cluster.

Sample `install-config.yaml` file that specifies custom security groups

```
compute:
- hyperthreading: Enabled
  name: worker
  platform:
    aws:
      additionalSecurityGroupIDs:
        - sg-1
        - sg-2
  replicas: 3
controlPlane:
  hyperthreading: Enabled
  name: master
  platform:
    aws:
      additionalSecurityGroupIDs:
        - sg-3
        - sg-4
  replicas: 3
platform:
  aws:
    region: us-east-1
    subnets:
      - subnet-1
      - subnet-2
      - subnet-3
```

where:

`compute.platform.aws.additionalSecurityGroupIDs`
: Specifies the name of the security group as it appears in the Amazon EC2 console, including the `sg` prefix.

`platform.aws.subnets`
: Specifies subnets for each availability zone that your cluster uses.

3.6.5. Alternatives to storing administrator-level secrets in the kube-system project

By default, administrator secrets are stored in the `kube-system` project.

If you configured the `credentialsMode` parameter in the `install-config.yaml` file to `Manual`, you must use one of the following alternatives:

- To manage long-term cloud credentials manually, follow the procedure in "Manually creating long-term credentials".
- To implement short-term credentials that are managed outside the cluster for individual components, follow the procedures in "Configuring an AWS cluster to use short-term credentials".

3.6.5.1. Manually creating long-term credentials

You can put the Cloud Credential Operator (CCO) into manual mode before OpenShift Container Platform installation if the cloud identity and access management (IAM) APIs are not reachable, or if you prefer not to store an administrator-level credential secret in the cluster `kube-system` namespace.

Procedure

1. If you did not set the `credentialsMode` parameter in the `install-config.yaml` configuration file to `Manual`, modify the value as shown:

   Sample configuration file snippet

   ```
   apiVersion: v1
   baseDomain: example.com
   credentialsMode: Manual
   ```

2. If you have not previously created installation manifest files, do so by running the following command:

   ```
   $ openshift-install create manifests --dir <installation_directory>
   ```

   where `<installation_directory>` is the directory in which the installation program creates files.

3. Set a `$RELEASE_IMAGE` variable with the release image from your installation file by running the following command:

   ```
   $ RELEASE_IMAGE=$(./openshift-install version | awk '/release image/ {print $3}')
   ```

4. Extract the list of `CredentialsRequest` custom resources (CRs) from the OpenShift Container Platform release image by running the following command:

```
$ oc adm release extract \
  --from=$RELEASE_IMAGE \
  --credentials-requests \
  --included \
  --install-config=<path_to_directory_with_installation_configuration>/install-config.yaml \
  --to=<path_to_directory_for_credentials_requests>
```

where:

`--included`
: Specifies only the manifests that your specific cluster configuration requires.

`<path_to_directory_with_installation_configuration>`
: Specifies the location of the `install-config.yaml` file.

`<path_to_directory_for_credentials_requests>`
: Specifies the path to the directory where you want to store the `CredentialsRequest` objects. If the specified directory does not exist, this command creates it.

  This command creates a YAML file for each `CredentialsRequest` object.

  Sample `CredentialsRequest` object

```
apiVersion: cloudcredential.openshift.io/v1
kind: CredentialsRequest
metadata:
  name: <component_credentials_request>
  namespace: openshift-cloud-credential-operator
  ...
spec:
  providerSpec:
    apiVersion: cloudcredential.openshift.io/v1
    kind: AWSProviderSpec
    statementEntries:
    - effect: Allow
      action:
      - iam:GetUser
      - iam:GetUserPolicy
      - iam:ListAccessKeys
      resource: "*"
  ...
```

5. Create YAML files for secrets in the `openshift-install` manifests directory that you generated previously. The secrets must be stored using the namespace and secret name defined in the `spec.secretRef` for each `CredentialsRequest` object.

   Sample `CredentialsRequest` object with secrets

   ```
   apiVersion: cloudcredential.openshift.io/v1
   kind: CredentialsRequest
   metadata:
     name: <component_credentials_request>
     namespace: openshift-cloud-credential-operator
     ...
   spec:
     providerSpec:
       apiVersion: cloudcredential.openshift.io/v1
       kind: AWSProviderSpec
       statementEntries:
       - effect: Allow
         action:
         - s3:CreateBucket
         - s3:DeleteBucket
         resource: "*"
         ...
     secretRef:
       name: <component_secret>
       namespace: <component_namespace>
     ...
   ```

   Sample `Secret` object

   ```
   apiVersion: v1
   kind: Secret
   metadata:
     name: <component_secret>
     namespace: <component_namespace>
   data:
     aws_access_key_id: <base64_encoded_aws_access_key_id>
     aws_secret_access_key: <base64_encoded_aws_secret_access_key>
   ```

   Before upgrading a cluster that uses manually maintained credentials, you must ensure that the CCO is in an upgradeable state.

3.6.5.2. Configuring an AWS cluster to use short-term credentials

To install a cluster that is configured to use the AWS Security Token Service (STS), you must configure the CCO utility and create the required AWS resources for your cluster.

3.6.5.2.1. Configuring the Cloud Credential Operator utility

To create and manage cloud credentials from outside of the cluster when the Cloud Credential Operator (CCO) is operating in manual mode, extract and prepare the CCO utility (`ccoctl`) binary.

The `ccoctl` utility is a Linux binary that must run in a Linux environment.

Prerequisites

- You have access to an OpenShift Container Platform account with cluster administrator access.
- You have installed the OpenShift CLI (`oc`).

- You have created an AWS account for the `ccoctl` utility to use with the following permissions:

  Required `iam` permissions

  - `iam:CreateOpenIDConnectProvider`
  - `iam:CreateRole`
  - `iam:DeleteOpenIDConnectProvider`
  - `iam:DeleteRole`
  - `iam:DeleteRolePolicy`
  - `iam:GetOpenIDConnectProvider`
  - `iam:GetRole`
  - `iam:GetUser`
  - `iam:ListOpenIDConnectProviders`
  - `iam:ListRolePolicies`
  - `iam:ListRoles`
  - `iam:PutRolePolicy`
  - `iam:TagOpenIDConnectProvider`
  - `iam:TagRole`

  Required `s3` permissions

  - `s3:CreateBucket`
  - `s3:DeleteBucket`
  - `s3:DeleteObject`
  - `s3:GetBucketAcl`
  - `s3:GetBucketTagging`
  - `s3:GetObject`
  - `s3:GetObjectAcl`
  - `s3:GetObjectTagging`
  - `s3:ListBucket`
  - `s3:PutBucketAcl`
  - `s3:PutBucketPolicy`
  - `s3:PutBucketPublicAccessBlock`
  - `s3:PutBucketTagging`
  - `s3:PutObject`
  - `s3:PutObjectAcl`
  - `s3:PutObjectTagging`

  Required `cloudfront` permissions

  - `cloudfront:ListCloudFrontOriginAccessIdentities`
  - `cloudfront:ListDistributions`
  - `cloudfront:ListTagsForResource`

- If you plan to store the OIDC configuration in a private S3 bucket that is accessed by the IAM identity provider through a public CloudFront distribution URL, the AWS account that runs the `ccoctl` utility requires the following additional permissions:
  - `cloudfront:CreateCloudFrontOriginAccessIdentity`
  - `cloudfront:CreateDistribution`
  - `cloudfront:DeleteCloudFrontOriginAccessIdentity`
  - `cloudfront:DeleteDistribution`
  - `cloudfront:GetCloudFrontOriginAccessIdentity`
  - `cloudfront:GetCloudFrontOriginAccessIdentityConfig`
  - `cloudfront:GetDistribution`
  - `cloudfront:TagResource`
  - `cloudfront:UpdateDistribution`

  These additional permissions support the use of the `--create-private-s3-bucket` option when processing credentials requests with the `ccoctl aws create-all` command.

Procedure

1. Set a variable for the OpenShift Container Platform release image by running the following command:

   ```
   $ RELEASE_IMAGE=$(./openshift-install version | awk '/release image/ {print $3}')
   ```

2. Obtain the CCO container image from the OpenShift Container Platform release image by running the following command:

   ```
   $ CCO_IMAGE=$(oc adm release info --image-for='cloud-credential-operator' $RELEASE_IMAGE -a
   ~/.pull-secret)
   ```

   Ensure that the architecture of the `$RELEASE_IMAGE` matches the architecture of the environment in which you will use the `ccoctl` tool.

3. Extract the `ccoctl` binary from the CCO container image within the OpenShift Container Platform release image by running the following command:

   ```
   $ oc image extract $CCO_IMAGE \
     --file="/usr/bin/ccoctl.<rhel_version>" \
     -a ~/.pull-secret
   ```

   For `<rhel_version>`, specify the value that corresponds to the version of Red Hat Enterprise Linux (RHEL) that the host uses. If no value is specified, `ccoctl.rhel8` is used by default. The following values are valid:

   - `rhel8`: Specify this value for hosts that use RHEL 8.
   - `rhel9`: Specify this value for hosts that use RHEL 9.

   The `ccoctl` binary is created in the directory from where you executed the command and not in `/usr/bin/`. You must rename the directory or move the `ccoctl.<rhel_version>` binary to `ccoctl`.

4. Change the permissions to make `ccoctl` executable by running the following command:

   ```
   $ chmod 775 ccoctl
   ```

Verification

To verify that `ccoctl` is ready to use, display the help file. Use a relative file name when you run the command, for example:

```
$ ./ccoctl
```

Example output

```
OpenShift credentials provisioning tool

Usage:
  ccoctl [command]

Available Commands:
  aws          Manage credentials objects for AWS cloud
  azure        Manage credentials objects for Azure
  gcp          Manage credentials objects for Google cloud
  help         Help about any command
  ibmcloud     Manage credentials objects for IBM Cloud
  nutanix      Manage credentials objects for Nutanix

Flags:
  -h, --help   help for ccoctl

Use "ccoctl [command] --help" for more information about a command.
```

3.6.5.2.2. Creating AWS resources with a single command

If the process the `ccoctl` tool uses to create AWS resources automatically meets the requirements of your organization, you can use the `ccoctl aws create-all` command to automate the creation of AWS resources.

Otherwise, you can create the AWS resources individually. For more information, see "Creating AWS resources individually".

By default, `ccoctl` creates objects in the directory in which the commands are run. To create the objects in a different directory, use the `--output-dir` flag. This procedure uses `<path_to_ccoctl_output_dir>` to refer to this directory.

Prerequisites

You must have:

- Extracted and prepared the `ccoctl` binary.

Procedure

1. Set a `$RELEASE_IMAGE` variable with the release image from your installation file by running the following command:

```
$ RELEASE_IMAGE=$(./openshift-install version | awk '/release image/ {print $3}')
```

2. Extract the list of `CredentialsRequest` objects from the OpenShift Container Platform release image by running the following command:

```
$ oc adm release extract \
  --from=$RELEASE_IMAGE \
  --credentials-requests \
  --included \
  --install-config=<path_to_directory_with_installation_configuration>/install-config.yaml \
  --to=<path_to_directory_for_credentials_requests>
```

where:

`--included`
: Specifies to include only the manifests that your specific cluster configuration requires.

`<path_to_directory_with_installation_configuration>`
: Specifies the location of the `install-config.yaml` file.

`<path_to_directory_for_credentials_requests>`
: Specifies the path to the directory where you want to store the `CredentialsRequest` objects. If the specified directory does not exist, this command creates it.

  This command might take a few moments to run.

3. Use the `ccoctl` tool to process all `CredentialsRequest` objects by running the following command:

```
$ ccoctl aws create-all \
  --name=<name> \
  --region=<aws_region> \
  --credentials-requests-dir=<path_to_credentials_requests_directory> \
  --output-dir=<path_to_ccoctl_output_dir> \
  --create-private-s3-bucket \
  --permissions-boundary-arn=<policy_arn>
```

where:

`<name>`
: Specifies the name used to tag any cloud resources that are created for tracking.

`<aws_region>`
: Specifies the AWS region in which cloud resources will be created.

`<path_to_credentials_requests_directory>`
: Specifies the directory containing the files for the component `CredentialsRequest` objects.

`<path_to_ccoctl_output_dir>`
: Specifies the directory in which you want the `ccoctl` utility to create objects. By default, the utility creates objects in the directory in which the commands are run. This parameter is optional.

`--create-private-s3-bucket`
: Specifies that the OpenID Connect (OIDC) configuration files should be stored in a private S3 bucket that is accessed by the IAM identity provider through a public CloudFront distribution URL. Note that by default, the `ccoctl` utility stores the OIDC configuration files in a public S3 bucket and uses the S3 URL as the public OIDC endpoint. This parameter is optional.

`<policy_arn>`
: Specifies the Amazon Resource Name (ARN) of the AWS IAM policy to use as the permissions boundary for the IAM roles created by the `ccoctl` utility. This parameter is optional.

  If your cluster uses Technology Preview features that are enabled by the `TechPreviewNoUpgrade` feature set, you must include the `--enable-tech-preview` parameter.

Verification

To verify that the OpenShift Container Platform secrets are created, list the files in the `<path_to_ccoctl_output_dir>/manifests` directory:

```
$ ls <path_to_ccoctl_output_dir>/manifests
```

Example output

```
cluster-authentication-02-config.yaml
openshift-cloud-credential-operator-cloud-credential-operator-iam-ro-creds-credentials.yaml
openshift-cloud-network-config-controller-cloud-credentials-credentials.yaml
openshift-cluster-api-capa-manager-bootstrap-credentials-credentials.yaml
openshift-cluster-csi-drivers-ebs-cloud-credentials-credentials.yaml
openshift-image-registry-installer-cloud-credentials-credentials.yaml
openshift-ingress-operator-cloud-credentials-credentials.yaml
openshift-machine-api-aws-cloud-credentials-credentials.yaml
```

You can verify that the IAM roles are created by querying AWS. For more information, refer to AWS documentation on listing IAM roles.

3.6.5.2.3. Creating AWS resources individually

If you need to review the JSON files that the `ccoctl` tool creates before modifying AWS resources, or if the process the `ccoctl` tool uses to create AWS resources automatically does not meet the requirements of your organization, you can create the AWS resources individually.

This option might be useful for an organization that shares the responsibility for creating these resources among different users or departments.

Otherwise, you can use the `ccoctl aws create-all` command to create the AWS resources automatically. For more information, see "Creating AWS resources with a single command".

By default, `ccoctl` creates objects in the directory in which the commands are run. To create the objects in a different directory, use the `--output-dir` flag. This procedure uses `<path_to_ccoctl_output_dir>` to refer to this directory.

Some `ccoctl` commands make AWS API calls to create or modify AWS resources. You can use the `--dry-run` flag to avoid making API calls. Using this flag creates JSON files on the local file system instead. You can review and modify the JSON files and then apply them with the AWS CLI tool using the `--cli-input-json` parameters.

Prerequisites

Extract and prepare the `ccoctl` binary.

Procedure

1. Generate the public and private RSA key files that are used to set up the OpenID Connect provider for the cluster by running the following command:

```
$ ccoctl aws create-key-pair
```

Example output

```
2021/04/13 11:01:02 Generating RSA keypair
2021/04/13 11:01:03 Writing private key to /<path_to_ccoctl_output_dir>/serviceaccount-
signer.private
2021/04/13 11:01:03 Writing public key to /<path_to_ccoctl_output_dir>/serviceaccount-
signer.public
2021/04/13 11:01:03 Copying signing key for use by installer
```

where `serviceaccount-signer.private` and `serviceaccount-signer.public` are the generated key files.

This command also creates a private key that the cluster requires during installation in `/<path_to_ccoctl_output_dir>/tls/bound-service-account-signing-key.key`.

2. Create an OpenID Connect identity provider and S3 bucket on AWS by running the following command:

```
$ ccoctl aws create-identity-provider \
  --name=<name> \
  --region=<aws_region> \
  --public-key-file=<path_to_ccoctl_output_dir>/serviceaccount-signer.public
```

where:

`<name>`
: Specifies the name used to tag any cloud resources that are created for tracking.

`<aws_region>`
: Specifies the AWS region in which cloud resources will be created.

`<path_to_ccoctl_output_dir>`
: Specifies the path to the public key file that the `ccoctl aws create-key-pair` command generated.

Example output

```
2021/04/13 11:16:09 Bucket <name>-oidc created
2021/04/13 11:16:10 OpenID Connect discovery document in the S3 bucket <name>-oidc at .well-
known/openid-configuration updated
2021/04/13 11:16:10 Reading public key
2021/04/13 11:16:10 JSON web key set (JWKS) in the S3 bucket <name>-oidc at keys.json updated
2021/04/13 11:16:18 Identity Provider created with ARN: arn:aws:iam::<aws_account_id>:oidc-
provider/<name>-oidc.s3.<aws_region>.amazonaws.com
```

where `openid-configuration` is a discovery document and `keys.json` is a JSON web key set file.

This command also creates a YAML configuration file in `/<path_to_ccoctl_output_dir>/manifests/cluster-authentication-02-config.yaml`. This file sets the issuer URL field for the service account tokens that the cluster generates, so that the AWS IAM identity provider trusts the tokens.

3. Create IAM roles for each component in the cluster:

   a. Set a `$RELEASE_IMAGE` variable with the release image from your installation file by running the following command:

   ```
   $ RELEASE_IMAGE=$(./openshift-install version | awk '/release image/ {print $3}')
   ```

   b. Extract the list of `CredentialsRequest` objects from the OpenShift Container Platform release image:

   ```
   $ oc adm release extract \
     --from=$RELEASE_IMAGE \
     --credentials-requests \
     --included \
     --install-config=<path_to_directory_with_installation_configuration>/install-config.yaml
   \
     --to=<path_to_directory_for_credentials_requests>
   ```

   where:

   `--included`
   Specifies the `--included` parameter, which includes only the manifests that your specific cluster configuration requires.

   `--install-config`
   Specifies the location of the `install-config.yaml` file.

   `-to`
   Specifies the path to the directory where you want to store the `CredentialsRequest` objects. If the specified directory does not exist, this command creates it.

   c. Use the `ccoctl` tool to process all `CredentialsRequest` objects by running the following command:

   ```
   $ ccoctl aws create-iam-roles \
     --name=<name> \
     --region=<aws_region> \
     --credentials-requests-dir=<path_to_credentials_requests_directory> \
     --identity-provider-arn=arn:aws:iam::<aws_account_id>:oidc-provider/<name>-oidc.s3.
   <aws_region>.amazonaws.com
   ```

   For AWS environments that use alternative IAM API endpoints, such as GovCloud, you must also specify your region with the `--region` parameter.

   If your cluster uses Technology Preview features that are enabled by the `TechPreviewNoUpgrade` feature set, you must include the `--enable-tech-preview` parameter.

   For each `CredentialsRequest` object, `ccoctl` creates an IAM role with a trust policy that is tied to the specified OIDC identity provider, and a permissions policy as defined in each `CredentialsRequest` object from the OpenShift Container Platform release image.

Verification

To verify that the OpenShift Container Platform secrets are created, list the files in the `<path_to_ccoctl_output_dir>/manifests` directory:

```
$ ls <path_to_ccoctl_output_dir>/manifests
```

Example output

```
cluster-authentication-02-config.yaml
openshift-cloud-credential-operator-cloud-credential-operator-iam-ro-creds-credentials.yaml
openshift-cloud-network-config-controller-cloud-credentials-credentials.yaml
openshift-cluster-api-capa-manager-bootstrap-credentials-credentials.yaml
openshift-cluster-csi-drivers-ebs-cloud-credentials-credentials.yaml
openshift-image-registry-installer-cloud-credentials-credentials.yaml
openshift-ingress-operator-cloud-credentials-credentials.yaml
openshift-machine-api-aws-cloud-credentials-credentials.yaml
```

You can verify that the IAM roles are created by querying AWS. For more information, refer to AWS documentation on listing IAM roles.

3.6.5.2.4. Incorporating the Cloud Credential Operator utility manifests

To implement short-term security credentials managed outside the cluster for individual OpenShift Container Platform components, you must move the manifest files that the Cloud Credential Operator utility (`ccoctl`) created to the correct directories for the installation program.

Prerequisites

- You have configured an account with the cloud platform that hosts your cluster.
- You have configured the Cloud Credential Operator utility (`ccoctl`).
- You have created the cloud provider resources that are required for your cluster with the `ccoctl` utility.

Procedure

1. If you did not set the `credentialsMode` parameter in the `install-config.yaml` configuration file to `Manual`, modify the value as shown:

   Sample configuration file snippet

   ```
   apiVersion: v1
   baseDomain: example.com
   credentialsMode: Manual
   ```

2. If you have not previously created installation manifest files, do so by running the following command:

   ```
   $ openshift-install create manifests --dir <installation_directory>
   ```

   where `<installation_directory>` is the directory in which the installation program creates files.

3. Copy the manifests that the `ccoctl` utility generated to the `manifests` directory that the installation program created by running the following command:

   ```
   $ cp /<path_to_ccoctl_output_dir>/manifests/* ./manifests/
   ```

4. Copy the `tls` directory that contains the private key to the installation directory:

   ```
   $ cp -a /<path_to_ccoctl_output_dir>/tls .
   ```

3.6.6. Deploying the cluster

To deploy your OpenShift Container Platform cluster, you initialize installation by running the `openshift-install create cluster` command from the directory that contains the installation program. The

installation program provisions the required infrastructure and completes the cluster setup.

You can run the `create cluster` command of the installation program only once, during initial installation.

Prerequisites

- You have configured an account with the cloud platform that hosts your cluster.
- You have the OpenShift Container Platform installation program and the pull secret for your cluster.
- You have verified that the cloud provider account on your host has the correct permissions to deploy the cluster. An account with incorrect permissions causes the installation process to fail with an error message that displays the missing permissions.

Procedure

1. In the directory that contains the installation program, initialize the cluster deployment by running the following command:

   ```
   $ ./openshift-install create cluster --dir <installation_directory> \
       --log-level=info
   ```

   where:

   - `<installation_directory>`: Specifies the location of your customized `./install-config.yaml` file.
   - `--log-level`: Specifies the log level. To view different installation details, specify `warn`, `debug`, or `error` instead of `info`.
2. Optional: Remove or disable the `AdministratorAccess` policy from the IAM account that you used to install the cluster.

   The elevated permissions provided by the `AdministratorAccess` policy are required only during installation.

Verification

When the cluster deployment completes successfully:

- The terminal displays directions for accessing your cluster, including a link to the web console and credentials for the `kubeadmin` user.

- Credential information also outputs to `<installation_directory>/.openshift_install.log`.

  Do not delete the installation program or the files that the installation program creates. Both are required to delete the cluster.

  The following example shows the expected output:

```
...
INFO Install complete!
INFO To access the cluster as the system:admin user when using 'oc', run 'export
KUBECONFIG=/home/myuser/install_dir/auth/kubeconfig'
INFO Access the OpenShift web-console here: https://console-openshift-
console.apps.mycluster.example.com
INFO Login to the console with user: "kubeadmin", and password: "password"
INFO Time elapsed: 36m22s
```

  - The Ignition config files that the installation program generates contain certificates that expire after 24 hours, which are then renewed at that time. If the cluster is shut down before renewing the certificates and the cluster is later restarted after the 24 hours have elapsed, the cluster automatically recovers the expired certificates. The exception is that you must manually approve the pending `node-bootstrapper` certificate signing requests (CSRs) to recover kubelet certificates. See the documentation for *Recovering from expired control plane certificates* for more information.
  - It is recommended that you use Ignition config files within 12 hours after they are generated because the 24-hour certificate rotates from 16 to 22 hours after the cluster is installed. By using the Ignition config files within 12 hours, you can avoid installation failure if the certificate update runs during installation.

3.6.7. Provisioning your own DNS records

Use your cluster name and base cluster domain to configure a CNAME record for the API service `api.<cluster_name>.<base_domain>.` with the API load balancer DNS name, and provision a CNAME record for the Ingress hostname `*.apps.<cluster_name>.<base_domain>.` with the Ingress load balancer DNS name.

User-provisioned DNS is a Technology Preview feature only. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see Technology Preview Features Support Scope.

Prerequisites

You have installed the AWS CLI.

Procedure

1. Add the `userProvisionedDNS` parameter to the `install-config.yaml` file and enable the parameter. For more information, see "Enabling a user-managed DNS".
2. Install your cluster.
3. If you are installing a private cluster, set the `api_lb_name` variable by running the following command:

```
$ api_lb_name="${INFRA_ID}-int"
```

4. If you are installing a public cluster, set the `api_lb_name` variable by running the following command:

```
$ api_lb_name="${INFRA_ID}-ext"
```

5. To retrieve the DNS name of the API service, run the following command:

```
$ aws --region ${REGION} elbv2 describe-load-balancers --names ${api_lb_name} --query
'LoadBalancers[*].DNSName' --output text
```

6. Use the DNS name and your cluster name and base cluster domain to configure your own DNS record with the `api.<cluster_name>.<base_domain>.` hostname.
7. To retrieve the DNS name of the Ingress service, run the following command:

```
$ ingress_lb_name=$(aws --region ${REGION} resourcegroupstaggingapi get-resources --resource-
type-filters elasticloadbalancing:loadbalancer --tag-filters
Key=kubernetes.io/cluster/${INFRA_ID},Values=owned Key=kubernetes.io/service-
name,Values=openshift-ingress/router-default --query 'ResourceTagMappingList[*].ResourceARN |
[0]' --output text | awk -F'/' '{print $2}')
```

8. Run the following command, which uses the variable `ingress_lb_name` generated from the previous command:

```
$ aws --region ${REGION} elb describe-load-balancers --load-balancer-names ${ingress_lb_name}
--query 'LoadBalancerDescriptions[].DNSName' --output text
```

9. Use the DNS name and your cluster name and base cluster domain to configure your own DNS record with the `*.apps.<cluster_name>.<base_domain>.` hostname.

3.6.8. Logging in to the cluster by using the CLI

To log in to your cluster as the default system user, export the `kubeconfig` file. This configuration enables the CLI to authenticate and connect to the specific API server created during OpenShift Container Platform installation.

The `kubeconfig` file is specific to a cluster and OpenShift Container Platform generates it during installation.

Prerequisites

- You deployed an OpenShift Container Platform cluster.
- You installed the OpenShift CLI (`oc`).

Procedure

1. Export the `kubeadmin` credentials by running the following command:

```
$ export KUBECONFIG=<installation_directory>/auth/kubeconfig
```

   where:

   `<installation_directory>`
   Specifies the path to the directory that stores the installation files.

2. Verify you can run `oc` commands successfully using the exported configuration by running the following command:

```
$ oc whoami
```

   Example output

```
system:admin
```

Next steps

- "Customize your cluster"
- "Remote health reporting"

3.6.9. Logging in to the cluster by using the web console

To verify that your cluster deployed successfully and access its features, log in to the OpenShift Container Platform web console as the `kubeadmin` user.

Prerequisites

- You have access to the installation host.
- You completed a cluster installation and all cluster Operators are available.

Procedure

1. Obtain the password for the `kubeadmin` user from the `kubeadmin-password` file on the installation host:

   ```
   $ cat <installation_directory>/auth/kubeadmin-password
   ```

   Or, you can obtain the `kubeadmin` password from the `<installation_directory>/.openshift_install.log` log file on the installation host.

2. List the OpenShift Container Platform web console route:

   ```
   $ oc get routes -n openshift-console | grep 'console-openshift'
   ```

   Or, you can obtain the OpenShift Container Platform route from the `<installation_directory>/.openshift_install.log` log file on the installation host.

   Example output

   ```
   console     console-openshift-console.apps.<cluster_name>.<base_domain>            console
   https   reencrypt/Redirect   None
   ```

3. Navigate to the route detailed in the output of the preceding command in a web browser and log in as the `kubeadmin` user.

3.7. Installing a cluster on AWS into a specialized region

You can install a cluster on Amazon Web Services (AWS) into specialized regions, including secret and top secret regions, government regions, and China regions. To configure the region, modify parameters in the `install-config.yaml` file before you install the cluster.

The following specialized regions are supported:

Region type	Supported regions
China	• `cn-north-1` (Beijing) • `cn-northwest-1` (Ningxia)
Secret and Top Secret	• `us-isob-east-1` (SC2S) • `us-iso-east-1` (C2S)

Region type	Supported regions
Government	• `us-gov-east-1` • `us-gov-west-1`

Table 3.18. Specialized regions

In OpenShift Container Platform 4.22, the installation program uses Cluster API instead of Terraform to provision cluster infrastructure during installations on AWS. Installing a cluster on AWS into a secret or top-secret region by using the Cluster API implementation has not been tested as of the release of OpenShift Container Platform 4.22. This document will be updated when installation into a secret region has been tested.

There is a known issue with Network Load Balancers' support for security groups in secret or top secret regions that causes installations in these regions to fail. For more information, see "OCPBUGS-33311".

The maximum supported MTU in the AWS SC2S and C2S regions is not the same as the public regions. For more information about configuring MTU during installation, see the *Cluster Network Operator configuration object* section in *Installing a cluster on AWS with network customizations*

3.7.1. Prerequisites

Before you install a cluster on Amazon Web Services (AWS) into specialized regions, you must meet several prerequisites.

The following prerequisites must be met:

- If you install a cluster into a China region, you have an Internet Content Provider (ICP) license.
- You reviewed details about the OpenShift Container Platform installation and update processes.
- You read the documentation on selecting a cluster installation method and preparing it for users.
- You configured an AWS account to host the cluster.
- If you use a firewall, you configured it to allow the sites that your cluster requires access to.

If you have an AWS profile stored on your computer, it must not use a temporary session token that you generated while using a multi-factor authentication device. The cluster continues to use your current AWS credentials to create AWS resources for the entire life of the cluster, so you must use long-term credentials. To generate appropriate keys, see Managing Access Keys for IAM Users in the AWS documentation. You can supply the keys when you run the installation program.

3.7.2. Installation requirements for government regions

If you are deploying an OpenShift Container Platform cluster using an AWS Marketplace image in a government region, you must first subscribe through AWS. Subscribing to the offer provides you with the AMI ID that the installation program uses to deploy compute nodes.

You should only modify the RHCOS image for compute machines to use an AWS Marketplace image. Control plane machines and infrastructure nodes do not require an OpenShift Container Platform subscription and use the public RHCOS default image by default, which does not incur subscription costs on your AWS bill. Therefore, you should not modify the cluster default boot image or the control plane boot images. Applying the AWS Marketplace image to them will incur additional licensing costs that cannot be recovered.

Prerequisites

You have an AWS account to purchase the offer. This account does not have to be the same account that is used to install the cluster.

Procedure

1. Complete the OpenShift Container Platform subscription from the AWS Marketplace.
2. Record the AMI ID for your specific AWS Region. As part of the installation process, you must update the `install-config.yaml` file with this value before deploying the cluster.

3.7.3. Installation requirements for China, Secret, and Top Secret regions

Red Hat does not publish a Red Hat Enterprise Linux CoreOS (RHCOS) Amazon Machine Image (AMI) for the Amazon Web Services (AWS) China, Secret, or Top Secret regions.

Before you can install a cluster into one of these regions, you must:

- Upload a custom RHCOS AMI.
- Manually create the installation configuration file (`install-config.yaml`).
- Specify the AWS region, and the accompanying custom AMI, in the installation configuration file.

You cannot use the OpenShift Container Platform installation program to create the installation configuration file. The installer does not list an AWS region without native support for an RHCOS AMI.

If you install a cluster into a Secret or Top Secret region, you must also define a custom CA certificate in the `additionalTrustBundle` field of the `install-config.yaml` file because the AWS API requires a custom CA trust bundle. To allow the installation program to access the AWS API, the CA certificates must also be defined on the machine that runs the installation program. You must add the CA bundle to the trust store on the machine, use the `AWS_CA_BUNDLE` environment variable, or define the CA bundle in the `ca_bundle` field of the AWS config file.

3.7.4. Private clusters

You can deploy a private OpenShift Container Platform cluster that does not expose external endpoints. Private clusters are accessible from only an internal network and are not visible to the internet.

Public zones are not supported in Route 53 in AWS Government, Secret, and Top Secret regions. Therefore, clusters must be private if they are deployed to one of these regions.

By default, OpenShift Container Platform is provisioned to use publicly-accessible DNS and endpoints. A private cluster sets the DNS, Ingress Controller, and API server to private when you deploy your cluster. This means that the cluster resources are only accessible from your internal network and are not visible to the internet.

If the cluster has any public subnets, load balancer services created by administrators might be publicly accessible. To ensure cluster security, verify that these services are explicitly annotated as private.

To deploy a private cluster, you must:

- Use existing networking that meets your requirements. Your cluster resources might be shared between other clusters on the network.

- Deploy from a machine that has access to:
    - The API services for the cloud to which you provision.
    - The hosts on the network that you provision.
    - The internet to obtain installation media.

You can use any machine that meets these access requirements and follows your company's guidelines. For example, this machine can be a bastion host on your cloud network.

AWS China does not support a VPN connection between the VPC and your network.

3.7.4.1. Private clusters in AWS

To create a private cluster on Amazon Web Services (AWS), you must provide an existing private VPC and subnets to host the cluster.

The installation program must also be able to resolve the DNS records that the cluster requires. The installation program configures the Ingress Operator and API server for access from only the private network.

The cluster still requires access to internet to access the AWS APIs.

The following items are not required or created when you install a private cluster:

- Public subnets
- Public load balancers, which support public ingress
- A public Route 53 zone that matches the `baseDomain` for the cluster

The installation program does use the `baseDomain` that you specify to create a private Route 53 zone and the required records for the cluster. The cluster is configured so that the Operators do not create public records for the cluster and all cluster machines are placed in the private subnets that you specify.

3.7.4.1.1. Limitations

The ability to add public functionality to a private cluster is limited.

- You cannot make the Kubernetes API endpoints public after installation without taking additional actions, including creating public subnets in the VPC for each availability zone in use, creating a public load balancer, and configuring the control plane security groups to allow traffic from the internet on 6443 (Kubernetes API port).
- If you use a public Service type load balancer, you must tag a public subnet in each availability zone with `kubernetes.io/cluster/<cluster-infra-id>: shared` so that AWS can use them to create public load balancers.

3.7.5. About using a custom VPC

You can deploy a cluster into existing subnets in an existing Amazon Virtual Private Cloud (VPC) in Amazon Web Services (AWS).

By deploying OpenShift Container Platform into an existing AWS VPC, you might be able to avoid limit constraints in new accounts or more easily abide by the operational constraints that your company's guidelines set. If you cannot obtain the infrastructure creation permissions that are required to create the VPC yourself, use this installation option.

Because the installation program cannot know what other components are also in your existing subnets, it cannot choose subnet CIDRs and so forth on your behalf. You must configure networking for the subnets

that you install your cluster to yourself.

The installation program no longer creates the following components:

- Internet gateways
- NAT gateways
- Subnets
- Route tables
- VPCs
- VPC DHCP options
- VPC endpoints

The installation program requires that you use the cloud-provided DNS server. Using a custom DNS server is not supported and causes the installation to fail.

If you use a custom VPC, you must correctly configure it and its subnets for the installation program and the cluster to use. See "Create a VPC" in the Amazon Web Services documentation for more information about AWS VPC console wizard configurations and creating and managing an AWS VPC.

The installation program cannot:

- Subdivide network ranges for the cluster to use.
- Set route tables for the subnets.
- Set VPC options like DHCP.

You must complete these tasks before you install the cluster. See "VPC networking components" and "Route tables for your VPC" for more information on configuring networking in an AWS VPC.

Your VPC must meet the following characteristics:

- The VPC must not use the `kubernetes.io/cluster/.*: owned`, `Name`, and `openshift.io/cluster` tags.

  The installation program modifies your subnets to add the `kubernetes.io/cluster/.*: shared` tag, so your subnets must have at least one free tag slot available for it. See "Tag Restrictions" in the AWS documentation to confirm that the installation program can add a tag to each subnet that you specify. You cannot use a `Name` tag, because it overlaps with the EC2 `Name` field and the installation fails.

- If you want to extend your OpenShift Container Platform cluster into an AWS Outpost and have an existing Outpost subnet, the existing subnet must use the `kubernetes.io/cluster/unmanaged: true` tag. If you do not apply this tag, the installation might fail due to the Cloud Controller Manager creating a service load balancer in the Outpost subnet, which is an unsupported configuration.
- You must enable the `enableDnsSupport` and `enableDnsHostnames` attributes in your VPC, so that the cluster can use the Route 53 zones that are attached to the VPC to resolve cluster's internal DNS records. See "DNS Support in Your VPC" in the AWS documentation.

  If you prefer to use your own Route 53 hosted private zone, you must associate the existing hosted zone with your VPC prior to installing a cluster. You can define your hosted zone using the `platform.aws.hostedZone` and `platform.aws.hostedZoneRole` fields in the `install-config.yaml` file. You can use a private hosted zone from another account by sharing it with the account where you install the cluster. If you use a private hosted zone from another account, you must use the `Passthrough` or `Manual` credentials mode.

A cluster in an SC2S or C2S Region is unable to reach the public IP addresses for the EC2, ELB, and S3 endpoints. Depending on the level to which you want to restrict internet traffic during the installation, the

following configuration options are available:

3.7.5.1. Option 1: Create VPC endpoints

Create a VPC endpoint and attach it to the subnets that the clusters are using. Name the endpoints as follows:

China

- `ec2.<aws_region>.amazonaws.com.cn`
- `elasticloadbalancing.<aws_region>.amazonaws.com`
- `s3.<aws_region>.amazonaws.com`

SC2S

- `elasticloadbalancing.<aws_region>.sc2s.sgov.gov`
- `ec2.<aws_region>.sc2s.sgov.gov`
- `s3.<aws_region>.sc2s.sgov.gov`

C2S

- `elasticloadbalancing.<aws_region>.c2s.ic.gov`
- `ec2.<aws_region>.c2s.ic.gov`
- `s3.<aws_region>.c2s.ic.gov`

With this option, network traffic remains private between your VPC and the required AWS services.

3.7.5.2. Option 2: Create a proxy without VPC endpoints

As part of the installation process, you can configure an HTTP or HTTPS proxy. With this option, internet traffic goes through the proxy to reach the required AWS services.

3.7.5.3. Option 3: Create a proxy with VPC endpoints

As part of the installation process, you can configure an HTTP or HTTPS proxy with VPC endpoints. Create a VPC endpoint and attach it to the subnets that the clusters are using. Name the endpoints as follows:

China

- `ec2.<aws_region>.amazonaws.com.cn`
- `elasticloadbalancing.<aws_region>.amazonaws.com`
- `s3.<aws_region>.amazonaws.com`

SC2S

- `elasticloadbalancing.<aws_region>.sc2s.sgov.gov`
- `ec2.<aws_region>.sc2s.sgov.gov`
- `s3.<aws_region>.sc2s.sgov.gov`

C2S

- `elasticloadbalancing.<aws_region>.c2s.ic.gov`
- `ec2.<aws_region>.c2s.ic.gov`
- `s3.<aws_region>.c2s.ic.gov`

When configuring the proxy in the `install-config.yaml` file, add these endpoints to the `noProxy` field. With this option, the proxy prevents the cluster from accessing the internet directly. However, network traffic remains private between your VPC and the required AWS services.

You must provide a suitable VPC and subnets that allow communication to your machines.

Component	AWS type	Description
VPC	• `AWS::EC2::VPC` • `AWS::EC2::VPCEndpoint`	You must provide a public VPC for the cluster to use. The VPC uses an endpoint that references the route tables for each subnet to improve communication with the registry that is hosted in S3.
Public subnets	• `AWS::EC2::Subnet` • `AWS::EC2::SubnetNetworkAclAssociation`	Your VPC must have public subnets for between 1 and 3 availability zones and associate them with appropriate Ingress rules.
Internet gateway	• `AWS::EC2::InternetGateway` • `AWS::EC2::VPCGatewayAttachment` • `AWS::EC2::RouteTable` • `AWS::EC2::Route` • `AWS::EC2::SubnetRouteTableAssociation` • `AWS::EC2::NatGateway` • `AWS::EC2::EIP`	You must have a public internet gateway, with public routes, attached to the VPC. In the provided templates, each public subnet has a NAT gateway with an EIP address. These NAT gateways allow cluster resources, like private subnet instances, to reach the internet and are not required for some restricted network or proxy scenarios.
Network access control	• `AWS::EC2::NetworkAcl` • `AWS::EC2::NetworkAclEntry`	You must allow the VPC to access the following ports:

Port	Reason
`80`	Inbound HTTP traffic
`443`	Inbound HTTPS traffic
`22`	Inbound SSH traffic
`1024 - 65535`	Inbound ephemeral traffic
`0 - 65535`	Outbound ephemeral traffic

Component	AWS type	Description
Private subnets	• `AWS::EC2::Subnet` • `AWS::EC2::RouteTable` • `AWS::EC2::SubnetRouteTableAssociation`	Your VPC can have private subnets. The provided CloudFormation templates can create private subnets for between 1 and 3 availability zones. If you use private subnets, you must provide appropriate routes and tables for them.

Table 3.19. Required VPC components

3.7.5.4. VPC validation

To ensure that the subnets that you provide are suitable, the installation program confirms the following data:

- All the subnets that you specify exist.
- You provide private subnets.
- The subnet CIDRs belong to the machine CIDR that you specified.
- You provide subnets for each availability zone. Each availability zone contains no more than one public and one private subnet. If you use a private cluster, provide only a private subnet for each availability zone. Otherwise, provide exactly one public and private subnet for each availability zone.
- You provide a public subnet for each private subnet availability zone. Machines are not provisioned in availability zones that you do not provide private subnets for.

If you destroy a cluster that uses an existing VPC, the VPC is not deleted. When you remove the OpenShift Container Platform cluster from a VPC, the `kubernetes.io/cluster/.*: shared` tag is removed from the subnets that it used.

3.7.5.5. Division of permissions

Starting with OpenShift Container Platform 4.3, you do not need all of the permissions that are required for an installation program-provisioned infrastructure cluster to deploy a cluster. This change mimics the division of permissions that you might have at your company: some individuals can create different resource in your clouds than others. For example, you might be able to create application-specific items, like instances, buckets, and load balancers, but not networking-related components such as VPCs, subnets, or ingress rules.

The AWS credentials that you use when you create your cluster do not need the networking permissions that are required to make VPCs and core networking components within the VPC, such as subnets, routing tables, internet gateways, NAT, and VPN. You still need permission to make the application resources that the machines within the cluster require, such as ELBs, security groups, S3 buckets, and nodes.

3.7.5.6. Isolation between clusters

If you deploy OpenShift Container Platform to an existing network, the isolation of cluster services is reduced in the following ways:

- You can install multiple OpenShift Container Platform clusters in the same VPC.
- ICMP ingress is allowed from the entire network.
- TCP 22 ingress (SSH) is allowed to the entire network.
- Control plane TCP 6443 ingress (Kubernetes API) is allowed to the entire network.

- Control plane TCP 22623 ingress (MCS) is allowed to the entire network.

3.7.5.7. Optional: AWS security groups

By default, the installation program creates and attaches security groups to control plane and compute machines. The rules associated with the default security groups cannot be modified.

However, you can apply additional existing AWS security groups, which are associated with your existing VPC, to control plane and compute machines. Applying custom security groups can help you meet the security needs of your organization, in such cases where you need to control the incoming or outgoing traffic of these machines.

As part of the installation process, you apply custom security groups by modifying the `install-config.yaml` file before deploying the cluster.

For more information, see "Applying existing AWS security groups to the cluster".

By default, AWS allows 5 security groups per network interface. The installation program creates 2 security groups for compute machines and 3 security groups for control plane machines. If you are installing a cluster into a shared VPC, there are three scenarios in which you must increase this quota:

- You specified 4 or more custom security groups for compute machines using the `compute.platform.aws.additionalSecurityGroupIDs` parameter in the `install-config.yaml` file.
- You specified 3 or more custom security groups for control plane machines using the `controlPlane.platform.aws.additionalSecurityGroupIDs` parameter in the `install-config.yaml` file.
- You specified 3 or more custom security groups for all machines using the `platform.aws.defaultMachinePlatform` parameter in the `install-config.yaml` file.

You must increase the quota of security groups per network interface to a number greater than or equal to `3 + (number of control plane custom security groups OR number of default machine platform custom security groups)`, or `2 + (number of compute custom security groups OR number of default machine platform custom security groups)`, whichever is higher. If you do not specify a sufficient quota, the installation will succeed, but it will generate `SecurityGroupsPerInterfaceLimitExceeded` errors in the installation log, and the additional security groups will not be applied. The maximum allowed quota is 16 and the maximum number of user-specified security groups is 10.

3.7.6. Uploading a custom RHCOS AMI in AWS

If you are deploying to a custom AWS region, you must upload a custom Red Hat Enterprise Linux CoreOS (RHCOS) Amazon Machine Image (AMI) that belongs to that region.

Prerequisites

- You configured an AWS account.
- You created an Amazon S3 bucket with the required IAM service role.
- You uploaded your RHCOS VMDK file to Amazon S3. The RHCOS VMDK file must be the highest version that is less than or equal to the OpenShift Container Platform version you are installing.
- You downloaded the AWS CLI and installed it on your computer. See Install the AWS CLI Using the Bundled Installer.

Procedure

1. Export your AWS profile as an environment variable by running the following command:

   ```
   $ export AWS_PROFILE=<aws_profile>
   ```

   Replace `<aws_profile>` with the AWS profile name that holds your AWS credentials, such as `govcloud` or `beijingadmin`.

2. Export the region to associate with your custom AMI as an environment variable by running the following command:

   ```
   $ export AWS_DEFAULT_REGION=<aws_region>
   ```

   Replace `<aws_region>` with the AWS region, such as `us-gov-east-1` or `cn-north-1`.

3. Export the version of RHCOS you uploaded to Amazon S3 as an environment variable by running the following command:

   ```
   $ export RHCOS_VERSION=<version>
   ```

   Replace `<version>` with the RHCOS VMDK version, such as `4.22.0`.

4. Export the Amazon S3 bucket name as an environment variable by running the following command:

   ```
   $ export VMIMPORT_BUCKET_NAME=<s3_bucket_name>
   ```

5. Create the `containers.json` file and define your RHCOS VMDK file by running the following command:

   ```
   $ cat <<EOF > containers.json
   {
      "Description": "rhcos-${RHCOS_VERSION}-x86_64-aws.x86_64",
      "Format": "vmdk",
      "UserBucket": {
         "S3Bucket": "${VMIMPORT_BUCKET_NAME}",
         "S3Key": "rhcos-${RHCOS_VERSION}-x86_64-aws.x86_64.vmdk"
      }
   }
   EOF
   ```

6. Import the RHCOS disk as an Amazon EBS snapshot by running the following command:

   ```
   $ aws ec2 import-snapshot --region ${AWS_DEFAULT_REGION} \
        --description "<description>" \
        --disk-container "file://<file_path>/containers.json"
   ```

   where:

   `--description`
   : Specifies the description of your RHCOS disk being imported, like `rhcos-${RHCOS_VERSION}-x86_64-aws.x86_64`.

   `--disk-container`
   : Specifies the file path to the JSON file describing your RHCOS disk. The JSON file should contain your Amazon S3 bucket name and key.

7. Check the status of the image import by running the following command::

```
$ watch -n 5 aws ec2 describe-import-snapshot-tasks --region ${AWS_DEFAULT_REGION}
```

Example output

```
{
    "ImportSnapshotTasks": [
        {
            "Description": "rhcos-4.7.0-x86_64-aws.x86_64",
            "ImportTaskId": "import-snap-fh6i8uil",
            "SnapshotTaskDetail": {
                "Description": "rhcos-4.7.0-x86_64-aws.x86_64",
                "DiskImageSize": 819056640.0,
                "Format": "VMDK",
                "SnapshotId": "snap-06331325870076318",
                "Status": "completed",
                "UserBucket": {
                    "S3Bucket": "external-images",
                    "S3Key": "rhcos-4.7.0-x86_64-aws.x86_64.vmdk"
                }
            }
        }
    ]
}
```

Copy the `SnapshotId` to register the image.

8. Create a custom RHCOS AMI from the RHCOS snapshot by running the following command:

```
$ aws ec2 register-image \
   --region ${AWS_DEFAULT_REGION} \
   --architecture x86_64 \
   --description "rhcos-${RHCOS_VERSION}-x86_64-aws.x86_64" \
   --ena-support \
   --name "rhcos-${RHCOS_VERSION}-x86_64-aws.x86_64" \
   --virtualization-type hvm \
   --root-device-name '/dev/xvda' \
   --block-device-mappings 'DeviceName=/dev/xvda,Ebs={DeleteOnTermination=true,SnapshotId=
<snapshot_ID>}'
```

where:

`--architecture`
: Specifies the RHCOS VMDK architecture type, such as `x86_64`, `aarch64`, `s390x`, or `ppc64le`.

`--description`
: Specifies the `Description` from the imported snapshot.

`--name`
: Specifies the name of the RHCOS AMI.

`--block-device-mappings`
: Specifies the `SnapshotID` from the imported snapshot.

3.7.7. Manually creating the installation configuration file

Installing the cluster requires that you manually create the installation configuration file.

Prerequisites

- You have an SSH public key on your local machine for use with the installation program. You can use the key for SSH authentication onto your cluster nodes for debugging and disaster recovery.
- You have obtained the OpenShift Container Platform installation program and the pull secret for your cluster.

Procedure

1. Create an installation directory to store your required installation assets in:

   ```
   $ mkdir <installation_directory>
   ```

   You must create a directory. Some installation assets, such as bootstrap X.509 certificates have short expiration intervals, so you must not reuse an installation directory. If you want to reuse individual files from another cluster installation, you can copy them into your directory. However, the file names for the installation assets might change between releases. Use caution when copying installation files from an earlier OpenShift Container Platform version.

2. Customize the provided sample `install-config.yaml` file template and save the file in the `<installation_directory>`.

   You must name this configuration file `install-config.yaml`.

3. Back up the `install-config.yaml` file so that you can use it to install many clusters.

   Back up the `install-config.yaml` file now, because the installation process consumes the file in the next step.

3.7.7.1. Sample customized install-config.yaml file for AWS

You can customize the installation configuration file (`install-config.yaml`) to specify more details about your OpenShift Container Platform cluster's platform or modify the values of the required parameters.

This sample YAML file is provided for reference only. You must obtain your `install-config.yaml` file by using the installation program and modify it. For a full list and description of all installation configuration parameters, see *Installation configuration parameters for AWS*.

Sample `install-config.yaml` file for AWS

```
apiVersion: v1
baseDomain: example.com
sshKey: ssh-ed25519 AAAA...
pullSecret: '{"auths": ...}'
metadata:
  name: example-cluster
controlPlane:
  name: master
  platform:
    aws:
      type: m6i.xlarge
  replicas: 3
compute:
-  name: worker
  platform:
    aws:
      type: c5.4xlarge
  replicas: 3
networking:
  clusterNetwork:
  - cidr: 10.128.0.0/14
```

```
    hostPrefix: 23
platform:
  aws:
    region: us-west-2
```

- Parameters at the first level of indentation apply to the cluster globally.
- The `controlPlane` stanza applies to control plane machines.
- The `compute` stanza applies to compute machines.
- The `networking` stanza applies to the cluster networking configuration. If you do not provide networking values, the installation program provides default values.
- The `platform` stanza applies to the infrastructure platform that hosts the cluster.

3.7.7.2. Minimum resource requirements for cluster installation

To ensure that your OpenShift Container Platform cluster runs as expected, each cluster machine must meet minimum CPU, memory, and storage requirements.

Machine	Operating system	vCPU	Virtual RAM	Storage	Input/Output Per Second (IOPS)
Bootstrap	RHCOS	4	16 GB	100 GB	300
Control plane	RHCOS	4	16 GB	100 GB	300
Compute	RHCOS	2	8 GB	100 GB	300

Table 3.20. Minimum resource requirements

- One vCPU is equal to one physical core when simultaneous multithreading (SMT), or Hyper-Threading, is not enabled. When enabled, use the following formula to calculate the corresponding ratio: (threads per core × cores) × sockets = vCPUs.
- OpenShift Container Platform and Kubernetes are sensitive to disk performance, and Red Hat recommends faster storage, particularly for etcd on the control plane nodes which require a 10 ms p99 fsync duration. On many cloud platforms, storage size and IOPS scale together, so you might need to provision more storage to get enough performance.
- As with all user-provisioned installations, if you choose to use RHEL compute machines in your cluster, you take responsibility for all operating system life cycle management and maintenance, including performing system updates, applying patches, and completing all other required tasks. OpenShift Container Platform 4.10 and later do not support RHEL 7 compute machines.

In OpenShift Container Platform version 4.22, RHCOS uses RHEL version 9.8, which updates the micro-architecture requirements. Each architecture requires the following minimum instruction set architectures (ISA):

- x86-64 architecture requires x86-64-v2 ISA
- ARM64 architecture requires ARMv8.0-A ISA
- ppc64le architecture requires IBM® Power9 ISA
- s390x architecture requires IBM® z14 ISA

For more information, see Architectures in the RHEL documentation.

If an instance type for your platform meets the minimum requirements for cluster machines, it is supported to use in OpenShift Container Platform.

3.7.7.3. Tested instance types for AWS

To ensure cluster stability and performance, use one of the tested Amazon Web Services (AWS) instance types for your OpenShift Container Platform machines.

The following AWS instance types have been tested with OpenShift Container Platform.

Use the machine types included in the following charts for your AWS instances. If you use an instance type that is not listed in the chart, ensure that the instance size you use matches the minimum resource requirements in "Minimum resource requirements for cluster installation".

See the following machine types based on 64-bit x86 architecture:

- `c4.*`
- `c5.*`
- `c5a.*`
- `i3.*`
- `m4.*`
- `m5.*`
- `m5a.*`
- `m6a.*`
- `m6i.*`
- `m7a.*`
- `m7i.*`
- `m7i-flex.*`
- `r4.*`
- `r5.*`
- `r5a.*`
- `r6i.*`
- `t3.*`
- `t3a.*`

3.7.7.4. Tested instance types for AWS on 64-bit ARM infrastructures

To ensure cluster stability and performance, use one of the tested Amazon Web Services (AWS) 64-bit ARM instance types for your OpenShift Container Platform machines.

The following AWS 64-bit ARM instance types have been tested with OpenShift Container Platform.

Use the machine types included in the following charts for your AWS ARM instances. If you use an instance type that is not listed in the chart, ensure that the instance size you use matches the minimum resource requirements listed in "Minimum resource requirements for cluster installation".

See the following machine types based on 64-bit ARM architecture:

- `c6g.*`
- `c7g.*`
- `c8g.*`
- `m6g.*`
- `m7g.*`

- `m7gd.*`
- `r8g.*`

3.7.7.5. Configuring the cluster-wide proxy during installation

Production environments can deny direct access to the internet and instead have an HTTP or HTTPS proxy available. You can configure a new OpenShift Container Platform cluster to use a proxy by configuring the proxy settings in the `install-config.yaml` file.

Prerequisites

- You have an existing `install-config.yaml` file.
- You have reviewed the sites that your cluster requires access to and determined whether any of them need to bypass the proxy. By default, the proxy handles all cluster egress traffic, including calls to hosting cloud provider APIs. You added sites to the `Proxy` object's `spec.noProxy` field to bypass the proxy if necessary.

  The `Proxy` object `status.noProxy` field includes the values of the `networking.machineNetwork[].cidr`, `networking.clusterNetwork[].cidr`, and `networking.serviceNetwork[]` fields from your installation configuration.

  For installations on Amazon Web Services (AWS), Google Cloud, Microsoft Azure, and Red Hat OpenStack Platform (RHOSP), the `Proxy` object `status.noProxy` field also includes the instance metadata endpoint (`169.254.169.254`).

Procedure

1. Edit your `install-config.yaml` file and add the proxy settings. For example:

```
apiVersion: v1
baseDomain: my.domain.com
proxy:
  httpProxy: http://<username>:<pswd>@<ip>:<port>
  httpsProxy: https://<username>:<pswd>@<ip>:<port>
  noProxy: ec2.<aws_region>.amazonaws.com,elasticloadbalancing.<aws_region>.amazonaws.com,s3.
<aws_region>.amazonaws.com
additionalTrustBundle: |
    -----BEGIN CERTIFICATE-----
    <MY_TRUSTED_CA_CERT>
    -----END CERTIFICATE-----
additionalTrustBundlePolicy: <policy_to_add_additionalTrustBundle>
```

   where:

   `proxy.httpProxy`
   : Specifies a proxy URL to use for creating HTTP connections outside the cluster. The URL scheme must be `http`.

   `proxy.httpsProxy`
   : Specifies a proxy URL to use for creating HTTPS connections outside the cluster.

   `proxy.noProxy`
   : Specifies a comma-separated list of destination domain names, IP addresses, or other network CIDRs to exclude from proxying. Preface a domain with `.` to match subdomains only. For example, `.y.com` matches `x.y.com`, but not `y.com`. Use `*` to bypass the proxy for all destinations. If you have added the Amazon `EC2`, `Elastic Load Balancing`, and `S3` VPC endpoints to your VPC, you must add these endpoints to the `noProxy` field.

   `additionalTrustBundle`
   : If you specify this value, the installation program generates a config map named `user-ca-bundle` in the `openshift-config` namespace to hold the additional CA certificates. If you specify `additionalTrustBundle` and at least one proxy setting, the `Proxy` object references the `user-ca-bundle` config map in the `trustedCA` field. The Cluster Network Operator then creates a `trusted-ca-bundle` config map that merges the contents specified for the `trustedCA` parameter with the RHCOS trust bundle. You must set the `additionalTrustBundle` field unless an authority from the RHCOS trust bundle signs the proxy's identity certificate.

   `additionalTrustBundlePolicy`
   : Specifies the policy that determines the configuration of the `Proxy` object to reference the `user-ca-bundle` config map in the `trustedCA` field. The allowed values are `Proxyonly` and `Always`. Use `Proxyonly` to reference the `user-ca-bundle` config map only when you configure an `http/https` proxy. Use `Always` to always reference the `user-ca-bundle` config map. The default value is `Proxyonly`. Optional parameter.

   The installation program does not support the proxy `readinessEndpoints` field.

   If the installation program times out, restart and then complete the deployment by using the `wait-for` command of the installation program. For example:

```
$ ./openshift-install wait-for install-complete --log-level debug
```

2. Save the file and reference it when installing OpenShift Container Platform.

   The installation program creates a cluster-wide proxy named `cluster` that uses the proxy settings in the `install-config.yaml` file. If you do not give proxy settings, the installation program still creates a `cluster` `Proxy` object, but it has a nil `spec`.

   Only the `Proxy` object named `cluster` is supported, and you cannot create additional proxies.

3.7.7.6. Applying existing Amazon Web Services (AWS) security groups to the cluster

Applying existing AWS security groups to your control plane and compute machines can help you meet the security needs of your organization, in such cases where you need to control the incoming or outgoing traffic of these machines.

Prerequisites

- You have created the security groups in AWS. For more information, see the AWS documentation about working with security groups.
- The security groups must be associated with the existing VPC that you are deploying the cluster to. The security groups cannot be associated with another VPC.
- You have an existing `install-config.yaml` file.

Procedure

1. In the `install-config.yaml` file, edit the `compute.platform.aws.additionalSecurityGroupIDs` parameter to specify one or more custom security groups for your compute machines.
2. Edit the `controlPlane.platform.aws.additionalSecurityGroupIDs` parameter to specify one or more custom security groups for your control plane machines.

3. Save the file and reference it when deploying the cluster.

Sample `install-config.yaml` file that specifies custom security groups

```
compute:
- hyperthreading: Enabled
  name: worker
  platform:
    aws:
      additionalSecurityGroupIDs:
        - sg-1
        - sg-2
  replicas: 3
controlPlane:
  hyperthreading: Enabled
  name: master
  platform:
    aws:
      additionalSecurityGroupIDs:
        - sg-3
        - sg-4
  replicas: 3
platform:
  aws:
    region: us-east-1
    subnets:
      - subnet-1
      - subnet-2
      - subnet-3
```

where:

`compute.platform.aws.additionalSecurityGroupIDs`
: Specifies the name of the security group as it appears in the Amazon EC2 console, including the `sg` prefix.

`platform.aws.subnets`
: Specifies subnets for each availability zone that your cluster uses.

3.7.8. Alternatives to storing administrator-level secrets in the kube-system project

By default, administrator secrets are stored in the `kube-system` project.

If you configured the `credentialsMode` parameter in the `install-config.yaml` file to `Manual`, you must use one of the following alternatives:

- To manage long-term cloud credentials manually, follow the procedure in "Manually creating long-term credentials".
- To implement short-term credentials that are managed outside the cluster for individual components, follow the procedures in "Configuring an AWS cluster to use short-term credentials".

3.7.8.1. Manually creating long-term credentials

You can put the Cloud Credential Operator (CCO) into manual mode before OpenShift Container Platform installation if the cloud identity and access management (IAM) APIs are not reachable, or if you prefer not to store an administrator-level credential secret in the cluster `kube-system` namespace.

Procedure

1. If you did not set the `credentialsMode` parameter in the `install-config.yaml` configuration file to `Manual`, modify the value as shown:

   Sample configuration file snippet

   ```
   apiVersion: v1
   baseDomain: example.com
   credentialsMode: Manual
   ```

2. If you have not previously created installation manifest files, do so by running the following command:

   ```
   $ openshift-install create manifests --dir <installation_directory>
   ```

   where `<installation_directory>` is the directory in which the installation program creates files.

3. Set a `$RELEASE_IMAGE` variable with the release image from your installation file by running the following command:

   ```
   $ RELEASE_IMAGE=$(./openshift-install version | awk '/release image/ {print $3}')
   ```

4. Extract the list of `CredentialsRequest` custom resources (CRs) from the OpenShift Container Platform release image by running the following command:

```
$ oc adm release extract \
  --from=$RELEASE_IMAGE \
  --credentials-requests \
  --included \
  --install-config=<path_to_directory_with_installation_configuration>/install-config.yaml \
  --to=<path_to_directory_for_credentials_requests>
```

where:

`--included`
: Specifies only the manifests that your specific cluster configuration requires.

`<path_to_directory_with_installation_configuration>`
: Specifies the location of the `install-config.yaml` file.

`<path_to_directory_for_credentials_requests>`
: Specifies the path to the directory where you want to store the `CredentialsRequest` objects. If the specified directory does not exist, this command creates it.

  This command creates a YAML file for each `CredentialsRequest` object.

  Sample `CredentialsRequest` object

```
apiVersion: cloudcredential.openshift.io/v1
kind: CredentialsRequest
metadata:
  name: <component_credentials_request>
  namespace: openshift-cloud-credential-operator
  ...
spec:
  providerSpec:
    apiVersion: cloudcredential.openshift.io/v1
    kind: AWSProviderSpec
    statementEntries:
    - effect: Allow
      action:
      - iam:GetUser
      - iam:GetUserPolicy
      - iam:ListAccessKeys
      resource: "*"
  ...
```

5. Create YAML files for secrets in the `openshift-install` manifests directory that you generated previously. The secrets must be stored using the namespace and secret name defined in the `spec.secretRef` for each `CredentialsRequest` object.

   Sample `CredentialsRequest` object with secrets

   ```
   apiVersion: cloudcredential.openshift.io/v1
   kind: CredentialsRequest
   metadata:
     name: <component_credentials_request>
     namespace: openshift-cloud-credential-operator
     ...
   spec:
     providerSpec:
       apiVersion: cloudcredential.openshift.io/v1
       kind: AWSProviderSpec
       statementEntries:
       - effect: Allow
         action:
         - s3:CreateBucket
         - s3:DeleteBucket
         resource: "*"
         ...
     secretRef:
       name: <component_secret>
       namespace: <component_namespace>
     ...
   ```

   Sample `Secret` object

   ```
   apiVersion: v1
   kind: Secret
   metadata:
     name: <component_secret>
     namespace: <component_namespace>
   data:
     aws_access_key_id: <base64_encoded_aws_access_key_id>
     aws_secret_access_key: <base64_encoded_aws_secret_access_key>
   ```

   Before upgrading a cluster that uses manually maintained credentials, you must ensure that the CCO is in an upgradeable state.

3.7.8.2. Configuring an AWS cluster to use short-term credentials

To install a cluster that is configured to use the AWS Security Token Service (STS), you must configure the CCO utility and create the required AWS resources for your cluster.

3.7.8.2.1. Configuring the Cloud Credential Operator utility

To create and manage cloud credentials from outside of the cluster when the Cloud Credential Operator (CCO) is operating in manual mode, extract and prepare the CCO utility (`ccoctl`) binary.

The `ccoctl` utility is a Linux binary that must run in a Linux environment.

Prerequisites

- You have access to an OpenShift Container Platform account with cluster administrator access.
- You have installed the OpenShift CLI (`oc`).

- You have created an AWS account for the `ccoctl` utility to use with the following permissions:

  Required `iam` permissions

  - `iam:CreateOpenIDConnectProvider`
  - `iam:CreateRole`
  - `iam:DeleteOpenIDConnectProvider`
  - `iam:DeleteRole`
  - `iam:DeleteRolePolicy`
  - `iam:GetOpenIDConnectProvider`
  - `iam:GetRole`
  - `iam:GetUser`
  - `iam:ListOpenIDConnectProviders`
  - `iam:ListRolePolicies`
  - `iam:ListRoles`
  - `iam:PutRolePolicy`
  - `iam:TagOpenIDConnectProvider`
  - `iam:TagRole`

  Required `s3` permissions

  - `s3:CreateBucket`
  - `s3:DeleteBucket`
  - `s3:DeleteObject`
  - `s3:GetBucketAcl`
  - `s3:GetBucketTagging`
  - `s3:GetObject`
  - `s3:GetObjectAcl`
  - `s3:GetObjectTagging`
  - `s3:ListBucket`
  - `s3:PutBucketAcl`
  - `s3:PutBucketPolicy`
  - `s3:PutBucketPublicAccessBlock`
  - `s3:PutBucketTagging`
  - `s3:PutObject`
  - `s3:PutObjectAcl`
  - `s3:PutObjectTagging`

  Required `cloudfront` permissions

  - `cloudfront:ListCloudFrontOriginAccessIdentities`
  - `cloudfront:ListDistributions`
  - `cloudfront:ListTagsForResource`

- If you plan to store the OIDC configuration in a private S3 bucket that is accessed by the IAM identity provider through a public CloudFront distribution URL, the AWS account that runs the `ccoctl` utility requires the following additional permissions:
  - `cloudfront:CreateCloudFrontOriginAccessIdentity`
  - `cloudfront:CreateDistribution`
  - `cloudfront:DeleteCloudFrontOriginAccessIdentity`
  - `cloudfront:DeleteDistribution`
  - `cloudfront:GetCloudFrontOriginAccessIdentity`
  - `cloudfront:GetCloudFrontOriginAccessIdentityConfig`
  - `cloudfront:GetDistribution`
  - `cloudfront:TagResource`
  - `cloudfront:UpdateDistribution`

  These additional permissions support the use of the `--create-private-s3-bucket` option when processing credentials requests with the `ccoctl aws create-all` command.

Procedure

1. Set a variable for the OpenShift Container Platform release image by running the following command:

   ```
   $ RELEASE_IMAGE=$(./openshift-install version | awk '/release image/ {print $3}')
   ```

2. Obtain the CCO container image from the OpenShift Container Platform release image by running the following command:

   ```
   $ CCO_IMAGE=$(oc adm release info --image-for='cloud-credential-operator' $RELEASE_IMAGE -a
   ~/.pull-secret)
   ```

   Ensure that the architecture of the `$RELEASE_IMAGE` matches the architecture of the environment in which you will use the `ccoctl` tool.

3. Extract the `ccoctl` binary from the CCO container image within the OpenShift Container Platform release image by running the following command:

   ```
   $ oc image extract $CCO_IMAGE \
     --file="/usr/bin/ccoctl.<rhel_version>" \
     -a ~/.pull-secret
   ```

   For `<rhel_version>`, specify the value that corresponds to the version of Red Hat Enterprise Linux (RHEL) that the host uses. If no value is specified, `ccoctl.rhel8` is used by default. The following values are valid:

   - `rhel8`: Specify this value for hosts that use RHEL 8.
   - `rhel9`: Specify this value for hosts that use RHEL 9.

   The `ccoctl` binary is created in the directory from where you executed the command and not in `/usr/bin/`. You must rename the directory or move the `ccoctl.<rhel_version>` binary to `ccoctl`.

4. Change the permissions to make `ccoctl` executable by running the following command:

   ```
   $ chmod 775 ccoctl
   ```

Verification

To verify that `ccoctl` is ready to use, display the help file. Use a relative file name when you run the command, for example:

```
$ ./ccoctl
```

Example output

```
OpenShift credentials provisioning tool

Usage:
  ccoctl [command]

Available Commands:
  aws          Manage credentials objects for AWS cloud
  azure        Manage credentials objects for Azure
  gcp          Manage credentials objects for Google cloud
  help         Help about any command
  ibmcloud     Manage credentials objects for IBM Cloud
  nutanix      Manage credentials objects for Nutanix

Flags:
  -h, --help   help for ccoctl

Use "ccoctl [command] --help" for more information about a command.
```

3.7.8.2.2. Creating AWS resources with a single command

If the process the `ccoctl` tool uses to create AWS resources automatically meets the requirements of your organization, you can use the `ccoctl aws create-all` command to automate the creation of AWS resources.

Otherwise, you can create the AWS resources individually. For more information, see "Creating AWS resources individually".

By default, `ccoctl` creates objects in the directory in which the commands are run. To create the objects in a different directory, use the `--output-dir` flag. This procedure uses `<path_to_ccoctl_output_dir>` to refer to this directory.

Prerequisites

You must have:

- Extracted and prepared the `ccoctl` binary.

Procedure

1. Set a `$RELEASE_IMAGE` variable with the release image from your installation file by running the following command:

```
$ RELEASE_IMAGE=$(./openshift-install version | awk '/release image/ {print $3}')
```

2. Extract the list of `CredentialsRequest` objects from the OpenShift Container Platform release image by running the following command:

```
$ oc adm release extract \
  --from=$RELEASE_IMAGE \
  --credentials-requests \
  --included \
  --install-config=<path_to_directory_with_installation_configuration>/install-config.yaml \
  --to=<path_to_directory_for_credentials_requests>
```

where:

`--included`
: Specifies to include only the manifests that your specific cluster configuration requires.

`<path_to_directory_with_installation_configuration>`
: Specifies the location of the `install-config.yaml` file.

`<path_to_directory_for_credentials_requests>`
: Specifies the path to the directory where you want to store the `CredentialsRequest` objects. If the specified directory does not exist, this command creates it.

  This command might take a few moments to run.

3. Use the `ccoctl` tool to process all `CredentialsRequest` objects by running the following command:

```
$ ccoctl aws create-all \
  --name=<name> \
  --region=<aws_region> \
  --credentials-requests-dir=<path_to_credentials_requests_directory> \
  --output-dir=<path_to_ccoctl_output_dir> \
  --create-private-s3-bucket \
  --permissions-boundary-arn=<policy_arn>
```

where:

`<name>`
: Specifies the name used to tag any cloud resources that are created for tracking.

`<aws_region>`
: Specifies the AWS region in which cloud resources will be created.

`<path_to_credentials_requests_directory>`
: Specifies the directory containing the files for the component `CredentialsRequest` objects.

`<path_to_ccoctl_output_dir>`
: Specifies the directory in which you want the `ccoctl` utility to create objects. By default, the utility creates objects in the directory in which the commands are run. This parameter is optional.

`--create-private-s3-bucket`
: Specifies that the OpenID Connect (OIDC) configuration files should be stored in a private S3 bucket that is accessed by the IAM identity provider through a public CloudFront distribution URL. Note that by default, the `ccoctl` utility stores the OIDC configuration files in a public S3 bucket and uses the S3 URL as the public OIDC endpoint. This parameter is optional.

`<policy_arn>`
: Specifies the Amazon Resource Name (ARN) of the AWS IAM policy to use as the permissions boundary for the IAM roles created by the `ccoctl` utility. This parameter is optional.

  If your cluster uses Technology Preview features that are enabled by the `TechPreviewNoUpgrade` feature set, you must include the `--enable-tech-preview` parameter.

Verification

To verify that the OpenShift Container Platform secrets are created, list the files in the `<path_to_ccoctl_output_dir>/manifests` directory:

```
$ ls <path_to_ccoctl_output_dir>/manifests
```

Example output

```
cluster-authentication-02-config.yaml
openshift-cloud-credential-operator-cloud-credential-operator-iam-ro-creds-credentials.yaml
openshift-cloud-network-config-controller-cloud-credentials-credentials.yaml
openshift-cluster-api-capa-manager-bootstrap-credentials-credentials.yaml
openshift-cluster-csi-drivers-ebs-cloud-credentials-credentials.yaml
openshift-image-registry-installer-cloud-credentials-credentials.yaml
openshift-ingress-operator-cloud-credentials-credentials.yaml
openshift-machine-api-aws-cloud-credentials-credentials.yaml
```

You can verify that the IAM roles are created by querying AWS. For more information, refer to AWS documentation on listing IAM roles.

3.7.8.2.3. Creating AWS resources individually

If you need to review the JSON files that the `ccoctl` tool creates before modifying AWS resources, or if the process the `ccoctl` tool uses to create AWS resources automatically does not meet the requirements of your organization, you can create the AWS resources individually.

This option might be useful for an organization that shares the responsibility for creating these resources among different users or departments.

Otherwise, you can use the `ccoctl aws create-all` command to create the AWS resources automatically. For more information, see "Creating AWS resources with a single command".

By default, `ccoctl` creates objects in the directory in which the commands are run. To create the objects in a different directory, use the `--output-dir` flag. This procedure uses `<path_to_ccoctl_output_dir>` to refer to this directory.

Some `ccoctl` commands make AWS API calls to create or modify AWS resources. You can use the `--dry-run` flag to avoid making API calls. Using this flag creates JSON files on the local file system instead. You can review and modify the JSON files and then apply them with the AWS CLI tool using the `--cli-input-json` parameters.

Prerequisites

Extract and prepare the `ccoctl` binary.

Procedure

1. Generate the public and private RSA key files that are used to set up the OpenID Connect provider for the cluster by running the following command:

```
$ ccoctl aws create-key-pair
```

Example output

```
2021/04/13 11:01:02 Generating RSA keypair
2021/04/13 11:01:03 Writing private key to /<path_to_ccoctl_output_dir>/serviceaccount-
signer.private
2021/04/13 11:01:03 Writing public key to /<path_to_ccoctl_output_dir>/serviceaccount-
signer.public
2021/04/13 11:01:03 Copying signing key for use by installer
```

where `serviceaccount-signer.private` and `serviceaccount-signer.public` are the generated key files.

This command also creates a private key that the cluster requires during installation in `/<path_to_ccoctl_output_dir>/tls/bound-service-account-signing-key.key`.

2. Create an OpenID Connect identity provider and S3 bucket on AWS by running the following command:

```
$ ccoctl aws create-identity-provider \
  --name=<name> \
  --region=<aws_region> \
  --public-key-file=<path_to_ccoctl_output_dir>/serviceaccount-signer.public
```

where:

`<name>`
: Specifies the name used to tag any cloud resources that are created for tracking.

`<aws_region>`
: Specifies the AWS region in which cloud resources will be created.

`<path_to_ccoctl_output_dir>`
: Specifies the path to the public key file that the `ccoctl aws create-key-pair` command generated.

Example output

```
2021/04/13 11:16:09 Bucket <name>-oidc created
2021/04/13 11:16:10 OpenID Connect discovery document in the S3 bucket <name>-oidc at .well-
known/openid-configuration updated
2021/04/13 11:16:10 Reading public key
2021/04/13 11:16:10 JSON web key set (JWKS) in the S3 bucket <name>-oidc at keys.json updated
2021/04/13 11:16:18 Identity Provider created with ARN: arn:aws:iam::<aws_account_id>:oidc-
provider/<name>-oidc.s3.<aws_region>.amazonaws.com
```

where `openid-configuration` is a discovery document and `keys.json` is a JSON web key set file.

This command also creates a YAML configuration file in `/<path_to_ccoctl_output_dir>/manifests/cluster-authentication-02-config.yaml`. This file sets the issuer URL field for the service account tokens that the cluster generates, so that the AWS IAM identity provider trusts the tokens.

3. Create IAM roles for each component in the cluster:

   a. Set a `$RELEASE_IMAGE` variable with the release image from your installation file by running the following command:

   ```
   $ RELEASE_IMAGE=$(./openshift-install version | awk '/release image/ {print $3}')
   ```

   b. Extract the list of `CredentialsRequest` objects from the OpenShift Container Platform release image:

   ```
   $ oc adm release extract \
     --from=$RELEASE_IMAGE \
     --credentials-requests \
     --included \
     --install-config=<path_to_directory_with_installation_configuration>/install-config.yaml
   \
     --to=<path_to_directory_for_credentials_requests>
   ```

   where:

   `--included`
   Specifies the `--included` parameter, which includes only the manifests that your specific cluster configuration requires.

   `--install-config`
   Specifies the location of the `install-config.yaml` file.

   `-to`
   Specifies the path to the directory where you want to store the `CredentialsRequest` objects. If the specified directory does not exist, this command creates it.

   c. Use the `ccoctl` tool to process all `CredentialsRequest` objects by running the following command:

   ```
   $ ccoctl aws create-iam-roles \
     --name=<name> \
     --region=<aws_region> \
     --credentials-requests-dir=<path_to_credentials_requests_directory> \
     --identity-provider-arn=arn:aws:iam::<aws_account_id>:oidc-provider/<name>-oidc.s3.
   <aws_region>.amazonaws.com
   ```

   For AWS environments that use alternative IAM API endpoints, such as GovCloud, you must also specify your region with the `--region` parameter.

   If your cluster uses Technology Preview features that are enabled by the `TechPreviewNoUpgrade` feature set, you must include the `--enable-tech-preview` parameter.

   For each `CredentialsRequest` object, `ccoctl` creates an IAM role with a trust policy that is tied to the specified OIDC identity provider, and a permissions policy as defined in each `CredentialsRequest` object from the OpenShift Container Platform release image.

Verification

To verify that the OpenShift Container Platform secrets are created, list the files in the `<path_to_ccoctl_output_dir>/manifests` directory:

```
$ ls <path_to_ccoctl_output_dir>/manifests
```

Example output

```
cluster-authentication-02-config.yaml
openshift-cloud-credential-operator-cloud-credential-operator-iam-ro-creds-credentials.yaml
openshift-cloud-network-config-controller-cloud-credentials-credentials.yaml
openshift-cluster-api-capa-manager-bootstrap-credentials-credentials.yaml
openshift-cluster-csi-drivers-ebs-cloud-credentials-credentials.yaml
openshift-image-registry-installer-cloud-credentials-credentials.yaml
openshift-ingress-operator-cloud-credentials-credentials.yaml
openshift-machine-api-aws-cloud-credentials-credentials.yaml
```

You can verify that the IAM roles are created by querying AWS. For more information, refer to AWS documentation on listing IAM roles.

3.7.8.2.4. Incorporating the Cloud Credential Operator utility manifests

To implement short-term security credentials managed outside the cluster for individual OpenShift Container Platform components, you must move the manifest files that the Cloud Credential Operator utility (`ccoctl`) created to the correct directories for the installation program.

Prerequisites

- You have configured an account with the cloud platform that hosts your cluster.
- You have configured the Cloud Credential Operator utility (`ccoctl`).
- You have created the cloud provider resources that are required for your cluster with the `ccoctl` utility.

Procedure

1. If you did not set the `credentialsMode` parameter in the `install-config.yaml` configuration file to `Manual`, modify the value as shown:

   Sample configuration file snippet

   ```
   apiVersion: v1
   baseDomain: example.com
   credentialsMode: Manual
   ```

2. If you have not previously created installation manifest files, do so by running the following command:

   ```
   $ openshift-install create manifests --dir <installation_directory>
   ```

   where `<installation_directory>` is the directory in which the installation program creates files.

3. Copy the manifests that the `ccoctl` utility generated to the `manifests` directory that the installation program created by running the following command:

   ```
   $ cp /<path_to_ccoctl_output_dir>/manifests/* ./manifests/
   ```

4. Copy the `tls` directory that contains the private key to the installation directory:

   ```
   $ cp -a /<path_to_ccoctl_output_dir>/tls .
   ```

3.7.9. Deploying the cluster

To deploy your OpenShift Container Platform cluster, you initialize installation by running the `openshift-install create cluster` command from the directory that contains the installation program. The

installation program provisions the required infrastructure and completes the cluster setup.

You can run the `create cluster` command of the installation program only once, during initial installation.

Prerequisites

- You have configured an account with the cloud platform that hosts your cluster.
- You have the OpenShift Container Platform installation program and the pull secret for your cluster.
- You have verified that the cloud provider account on your host has the correct permissions to deploy the cluster. An account with incorrect permissions causes the installation process to fail with an error message that displays the missing permissions.

Procedure

1. In the directory that contains the installation program, initialize the cluster deployment by running the following command:

   ```
   $ ./openshift-install create cluster --dir <installation_directory> \
       --log-level=info
   ```

   where:

   - `<installation_directory>`: Specifies the location of your customized `./install-config.yaml` file.
   - `--log-level`: Specifies the log level. To view different installation details, specify `warn`, `debug`, or `error` instead of `info`.
2. Optional: Remove or disable the `AdministratorAccess` policy from the IAM account that you used to install the cluster.

   The elevated permissions provided by the `AdministratorAccess` policy are required only during installation.

Verification

When the cluster deployment completes successfully:

- The terminal displays directions for accessing your cluster, including a link to the web console and credentials for the `kubeadmin` user.

- Credential information also outputs to `<installation_directory>/.openshift_install.log`.

  Do not delete the installation program or the files that the installation program creates. Both are required to delete the cluster.

  The following example shows the expected output:

```
...
INFO Install complete!
INFO To access the cluster as the system:admin user when using 'oc', run 'export
KUBECONFIG=/home/myuser/install_dir/auth/kubeconfig'
INFO Access the OpenShift web-console here: https://console-openshift-
console.apps.mycluster.example.com
INFO Login to the console with user: "kubeadmin", and password: "password"
INFO Time elapsed: 36m22s
```

  - The Ignition config files that the installation program generates contain certificates that expire after 24 hours, which are then renewed at that time. If the cluster is shut down before renewing the certificates and the cluster is later restarted after the 24 hours have elapsed, the cluster automatically recovers the expired certificates. The exception is that you must manually approve the pending `node-bootstrapper` certificate signing requests (CSRs) to recover kubelet certificates. See the documentation for *Recovering from expired control plane certificates* for more information.
  - It is recommended that you use Ignition config files within 12 hours after they are generated because the 24-hour certificate rotates from 16 to 22 hours after the cluster is installed. By using the Ignition config files within 12 hours, you can avoid installation failure if the certificate update runs during installation.

3.7.10. Logging in to the cluster by using the CLI

To log in to your cluster as the default system user, export the `kubeconfig` file. This configuration enables the CLI to authenticate and connect to the specific API server created during OpenShift Container Platform installation.

The `kubeconfig` file is specific to a cluster and OpenShift Container Platform generates it during installation.

Prerequisites

- You deployed an OpenShift Container Platform cluster.
- You installed the OpenShift CLI (`oc`).

Procedure

1. Export the `kubeadmin` credentials by running the following command:

   ```
   $ export KUBECONFIG=<installation_directory>/auth/kubeconfig
   ```

   where:

   `<installation_directory>`
   Specifies the path to the directory that stores the installation files.

2. Verify you can run `oc` commands successfully using the exported configuration by running the following command:

```
$ oc whoami
```

Example output

```
system:admin
```

Next steps

- "Customize your cluster"
- "Remote health reporting"

3.7.11. Logging in to the cluster by using the web console

To verify that your cluster deployed successfully and access its features, log in to the OpenShift Container Platform web console as the `kubeadmin` user.

Prerequisites

- You have access to the installation host.
- You completed a cluster installation and all cluster Operators are available.

Procedure

1. Obtain the password for the `kubeadmin` user from the `kubeadmin-password` file on the installation host:

```
$ cat <installation_directory>/auth/kubeadmin-password
```

Or, you can obtain the `kubeadmin` password from the `<installation_directory>/.openshift_install.log` log file on the installation host.

2. List the OpenShift Container Platform web console route:

```
$ oc get routes -n openshift-console | grep 'console-openshift'
```

Or, you can obtain the OpenShift Container Platform route from the `<installation_directory>/.openshift_install.log` log file on the installation host.

Example output

```
console     console-openshift-console.apps.<cluster_name>.<base_domain>            console
https   reencrypt/Redirect   None
```

3. Navigate to the route detailed in the output of the preceding command in a web browser and log in as the `kubeadmin` user.

3.8. Installing a cluster with compute nodes on AWS Local Zones

You can quickly install an OpenShift Container Platform cluster on Amazon Web Services (AWS) Local Zones by setting the zone names in the edge compute pool of the `install-config.yaml` file, or install a cluster in an existing Amazon Virtual Private Cloud (VPC) with Local Zone subnets.

AWS Local Zones is an infrastructure that place Cloud Resources close to metropolitan regions. For more information, see "AWS Local Zones Documentation".

3.8.1. Infrastructure prerequisites

Before you install an OpenShift Container Platform cluster on Amazon Web Services (AWS) Local Zones, you must meet several prerequisites.

The following prerequisites must be met:

- You reviewed details about OpenShift Container Platform installation and update processes.
- You are familiar with Selecting a cluster installation method and preparing it for users.
- You configured an AWS account to host the cluster.

  If you have an AWS profile stored on your computer, it must not use a temporary session token that you generated while using a multifactor authentication device. The cluster continues to use your current AWS credentials to create AWS resources for the entire life of the cluster, so you must use key-based, long-term credentials. To generate appropriate keys, see Managing Access Keys for IAM Users in the AWS documentation. You can supply the keys when you run the installation program.

- You downloaded the AWS CLI and installed it on your computer. See Install the AWS CLI Using the Bundled Installer (Linux, macOS, or UNIX) in the AWS documentation.
- If you use a firewall, you configured it to allow the sites that your cluster must access.
- You noted the region and supported AWS Local Zones locations to create the network resources in.
- You read the AWS Local Zones features in the AWS documentation.
- You added permissions for creating network resources that support AWS Local Zones to the Identity and Access Management (IAM) user or role. The following example enables a zone group that can give a user or role access for creating network resources that support AWS Local Zones.

  Example of an additional IAM policy with the `ec2:ModifyAvailabilityZoneGroup` permission attached to an IAM user or role.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Action": [
        "ec2:ModifyAvailabilityZoneGroup"
      ],
      "Effect": "Allow",
      "Resource": "*"
    }
  ]
}
```

3.8.2. Cluster limitations in AWS Local Zones

Some limitations exist when you try to deploy a cluster with a default installation configuration in an Amazon Web Services (AWS) Local Zone.

The following list details limitations when deploying a cluster in a pre-configured AWS zone:

- The maximum transmission unit (MTU) between an Amazon EC2 instance in a zone and an Amazon EC2 instance in the Region is `1300`. This causes the cluster-wide network MTU to change according to the network plugin that is used with the deployment.
- Network resources such as Network Load Balancer (NLB), Classic Load Balancer, and Network Address Translation (NAT) Gateways are not globally supported.

- For an OpenShift Container Platform cluster on AWS, the AWS Elastic Block Storage (EBS) `gp3` type volume is the default for node volumes and the default for the storage class. This volume type is not globally available on zone locations. By default, the nodes running in zones are deployed with the `gp2` EBS volume. The `gp2-csi` `StorageClass` parameter must be set when creating workloads on zone nodes.

If you want the installation program to automatically create Local Zone subnets for your OpenShift Container Platform cluster, specific configuration limitations apply with this method.

The following configuration limitation applies when you set the installation program to automatically create subnets for your OpenShift Container Platform cluster:

- When the installation program creates private subnets in AWS Local Zones, the program associates each subnet with the route table of its parent zone. This operation ensures that each private subnet can route egress traffic to the internet by way of NAT Gateways in an AWS Region.
- If the parent-zone route table does not exist during cluster installation, the installation program associates any private subnet with the first available private route table in the Amazon Virtual Private Cloud (VPC). This approach is valid only for AWS Local Zones subnets in an OpenShift Container Platform cluster.

3.8.3. About edge compute pools

The edge compute pool configuration is common between Amazon Web Services (AWS) Local Zones locations.

You can use the edge compute pool to create new labels to deploy applications onto Amazon Web Services (AWS) Local Zones nodes. Edge compute nodes are tainted compute nodes that run in AWS Local Zones locations.

When deploying a cluster that uses Local Zones, consider the following points:

- Amazon EC2 instances in the Local Zones are more expensive than Amazon EC2 instances in the Availability Zones.
- The latency is lower between the applications running in AWS Local Zones and the user. A latency impact exists for some workloads if, for example, ingress traffic is mixed between Local Zones and Availability Zones.

Generally, the maximum transmission unit (MTU) between an Amazon EC2 instance in a Local Zones and an Amazon EC2 instance in the Region is 1300. The cluster network MTU must be always less than the EC2 MTU to account for the overhead. The specific overhead is determined by the network plugin. For example: OVN-Kubernetes has an overhead of `100 bytes`.

The network plugin can provide additional features, such as IPsec, that also affect the MTU sizing.

OpenShift Container Platform 4.12 introduced a new compute pool, *edge*, that is designed for use in remote zones. The edge compute pool configuration is common between Amazon Web Services (AWS) Local Zones locations. Because of the type and size limitations of resources like EC2 and EBS on Local Zones resources, the default instance type can vary from the traditional compute pool.

The default Elastic Block Store (EBS) for Local Zones locations is `gp2`, which differs from the non-edge compute pool. The instance type used for each Local Zones on an edge compute pool also might differ from other compute pools, depending on the instance offerings on the zone.

The edge compute pool creates new labels that developers can use to deploy applications onto AWS Local Zones nodes. The new labels are:

- `node-role.kubernetes.io/edge=''`
- `machine.openshift.io/zone-type=local-zone`
- `machine.openshift.io/zone-group=$ZONE_GROUP_NAME`

By default, the machine sets for the edge compute pool define the taint of `NoSchedule` to prevent other workloads from spreading on Local Zones instances. Users can only run user workloads if they define tolerations in the pod specification.

3.8.4. Opting in to an AWS Local Zones

Create a subnet in an Amazon Web Services (AWS) Local Zones when you need workloads to run physically closer to users or data sources than a standard AWS Local Zones. If you plan to create subnets in AWS Local Zones, you must opt in to each zone group separately.

Prerequisites

- You have installed the AWS CLI.
- You have determined an AWS Region for where you want to deploy your OpenShift Container Platform cluster.
- You have attached a permissive IAM policy to a user or role account that opts in to the zone group.

Procedure

1. List the zones that are available in your AWS Region by running the following command:

   Example command for listing available AWS Local Zones in an AWS Region

   ```
   $ aws --region "<value_of_AWS_Region>" ec2 describe-availability-zones \
       --query 'AvailabilityZones[].[{ZoneName: ZoneName, GroupName: GroupName, Status:
   OptInStatus}]' \
       --filters Name=zone-type,Values=local-zone \
       --all-availability-zones
   ```

   Depending on the AWS Region, the list of available zones might be long. The command returns the following fields:

   `ZoneName`
   : The name of the Local Zones.

   `GroupName`
   : The group that comprises the zone. To opt in to the Region, save the name.

   `Status`
   : The status of the Local Zones group. If the status is `not-opted-in`, you must opt in the `GroupName` as described in the next step.

2. Opt in to the zone group on your AWS account by running the following command:

   ```
   $ aws ec2 modify-availability-zone-group \
       --group-name "<value_of_GroupName>" \
       --opt-in-status opted-in
   ```

   `<value_of_GroupName>`
   : Replace with the name of the group of the Local Zones where you want to create subnets. For example, specify `us-east-1-nyc-1` to use the zone `us-east-1-nyc-1a` (US East New York).

3.8.5. Obtaining an AWS Marketplace image

If you are deploying an OpenShift Container Platform cluster by using an Amazon Web Services (AWS) Marketplace image, you must first subscribe through AWS. Subscribing to the offer provides you with the Amazon Machine Image (AMI) ID that the installation program uses to deploy compute nodes.

You should only modify the RHCOS image for compute machines to use an AWS Marketplace image. Control plane machines and infrastructure nodes do not require an OpenShift Container Platform subscription and use the public RHCOS default image by default, which does not incur subscription costs on your AWS bill. Therefore, you should not modify the cluster default boot image or the control plane boot images. Applying the AWS Marketplace image to them will incur additional licensing costs that cannot be recovered.

Prerequisites

You have an AWS account to buy the offer. This account does not have to be the same account that you use to install the cluster.

Procedure

1. Complete the OpenShift Container Platform subscription from the AWS Marketplace.
2. Record the AMI ID for your specific AWS region. As part of the installation process, you must update the `install-config.yaml` file with this value before deploying the cluster.

Sample `install-config.yaml` file with AWS Marketplace compute nodes

```
apiVersion: v1
baseDomain: example.com
compute:
- hyperthreading: Enabled
  name: worker
  platform:
    aws:
      amiID: ami-06c4d345f7c207239
      type: m5.4xlarge
  replicas: 3
metadata:
  name: test-cluster
platform:
  aws:
    region: us-east-2
sshKey: ssh-ed25519 AAAA...
pullSecret: '{"auths": ...}'
```

where:

`compute.platform.aws.amiID`
Specifies the AMI ID from your AWS Marketplace subscription.

`platform.aws.region`
Specifies the `platform.aws.region` parameter. Your AMI ID is associated with a specific AWS region. When creating the installation configuration file, ensure that you select the same AWS region that you specified when configuring your subscription.

3.8.6. Minimum resource requirements for cluster installation

To ensure that your OpenShift Container Platform cluster runs as expected, each cluster machine must meet minimum CPU, memory, and storage requirements.

Machine	Operating system	vCPU	Virtual RAM	Storage	Input/Output Per Second (IOPS)
Bootstrap	RHCOS	4	16 GB	100 GB	300
Control plane	RHCOS	4	16 GB	100 GB	300
Compute	RHCOS	2	8 GB	100 GB	300

Table 3.21. Minimum resource requirements

- One vCPU is equal to one physical core when simultaneous multithreading (SMT), or Hyper-Threading, is not enabled. When enabled, use the following formula to calculate the corresponding ratio: (threads per core × cores) × sockets = vCPUs.
- OpenShift Container Platform and Kubernetes are sensitive to disk performance, and Red Hat recommends faster storage, particularly for etcd on the control plane nodes which require a 10 ms p99 fsync duration. On many cloud platforms, storage size and IOPS scale together, so you might need to provision more storage to get enough performance.
- As with all user-provisioned installations, if you choose to use RHEL compute machines in your cluster, you take responsibility for all operating system life cycle management and maintenance, including performing system updates, applying patches, and completing all other required tasks. OpenShift Container Platform 4.10 and later do not support RHEL 7 compute machines.

In OpenShift Container Platform version 4.22, RHCOS uses RHEL version 9.8, which updates the micro-architecture requirements. Each architecture requires the following minimum instruction set architectures (ISA):

- x86-64 architecture requires x86-64-v2 ISA
- ARM64 architecture requires ARMv8.0-A ISA
- ppc64le architecture requires IBM® Power9 ISA
- s390x architecture requires IBM® z14 ISA

For more information, see Architectures in the RHEL documentation.

If an instance type for your platform meets the minimum requirements for cluster machines, it is supported to use in OpenShift Container Platform.

3.8.7. Tested instance types for AWS

To ensure cluster stability and performance, use one of the tested Amazon Web Services (AWS) instance types for your OpenShift Container Platform machines.

The following AWS instance types have been tested with OpenShift Container Platform for use with AWS Local Zones.

Use the machine types included in the following charts for your AWS instances. If you use an instance type that is not listed in the chart, ensure that the instance size you use matches the minimum resource requirements in "Minimum resource requirements for cluster installation".

See the following machine types based on 64-bit x86 architecture for AWS Local Zones:

- `c5.*`

- `c5d.*`
- `m6i.*`
- `m5.*`
- `r5.*`
- `t3.*`

3.8.8. Creating the installation configuration file

Generate and customize the installation configuration file that the installation program needs to deploy your cluster.

Prerequisites

- You obtained the OpenShift Container Platform installation program for user-provisioned infrastructure and the pull secret for your cluster.
- You checked that you are deploying your cluster to an Amazon Web Services (AWS) Region with an accompanying Red Hat Enterprise Linux CoreOS (RHCOS) AMI published by Red Hat. If you are deploying to an AWS Region that requires a custom AMI, such as an AWS GovCloud Region, you must create the `install-config.yaml` file manually.

Procedure

1. Create the `install-config.yaml` file.

    a. Change to the directory that contains the installation program and run the following command:

    ```
    $ ./openshift-install create install-config --dir <installation_directory>
    ```

    For `<installation_directory>`, specify the directory name to store the files that the installation program creates.

    Specify an empty directory. Some installation assets, such as bootstrap X.509 certificates have short expiration intervals, so you must not reuse an installation directory. If you want to reuse individual files from another cluster installation, you can copy them into your directory. However, the file names for the installation assets might change between releases. Use caution when copying installation files from an earlier OpenShift Container Platform version.

    b. At the prompts, provide the configuration details for your cloud:

        i. Optional: Select an SSH key to use to access your cluster machines.

        For production OpenShift Container Platform clusters on which you want to perform installation debugging or disaster recovery, specify an SSH key that your `ssh-agent` process uses.

        ii. Select **aws** as the platform to target.
        iii. If you do not have an AWS profile stored on your computer, enter the AWS access key ID and secret access key for the user that you configured to run the installation program.

        The AWS access key ID and secret access key are stored in `~/.aws/credentials` in the home directory of the current user on the installation host. You are prompted for the credentials by the installation program if the credentials for the exported profile are not present in the file. Any credentials that you provide to the installation program are stored in the file.

        iv. Select the AWS Region to deploy the cluster to.
        v. Select the base domain for the Route 53 service that you configured for your cluster.
        vi. Enter a descriptive name for your cluster.
        vii. Paste the pull secret from Red Hat OpenShift Cluster Manager.

2. Optional: Back up the `install-config.yaml` file.

    The `install-config.yaml` file is consumed during the installation process. If you want to reuse the file, you must back it up now.

3.8.9. Examples of installation configuration files with edge compute pools

See an example `install-config.yaml` file to better understand configurations for edge compute pools.

The following examples show `install-config.yaml` files that contain an edge machine pool configuration.

Configuration that uses an edge pool with a custom instance type

```
apiVersion: v1
baseDomain: devcluster.openshift.com
metadata:
  name: ipi-edgezone
compute:
- name: edge
```

```
  platform:
    aws:
      type: r5.2xlarge
platform:
  aws:
    region: us-west-2
pullSecret: '{"auths": ...}'
sshKey: ssh-ed25519 AAAA...
```

Instance types differ between locations. To verify availability in the Local Zones in which the cluster runs, see the AWS documentation.

Configuration that uses an edge pool with a custom Amazon Elastic Block Store (EBS) type

```
apiVersion: v1
baseDomain: devcluster.openshift.com
metadata:
  name: ipi-edgezone
compute:
- name: edge
  platform:
    aws:
      zones:
      - us-west-2-lax-1a
      - us-west-2-lax-1b
      - us-west-2-phx-2a
      rootVolume:
        type: gp3
        size: 120
platform:
  aws:
    region: us-west-2
pullSecret: '{"auths": ...}'
sshKey: ssh-ed25519 AAAA...
```

Elastic Block Storage (EBS) types differ between locations. Check the AWS documentation to verify availability in the Local Zones in which the cluster runs.

Configuration that uses an edge pool with custom security groups

```
apiVersion: v1
baseDomain: devcluster.openshift.com
metadata:
  name: ipi-edgezone
compute:
- name: edge
  platform:
    aws:
      additionalSecurityGroupIDs:
        - sg-1
        - sg-2
platform:
  aws:
    region: us-west-2
pullSecret: '{"auths": ...}'
sshKey: ssh-ed25519 AAAA...
```

For the `additionalSecurityGroupIDs` parameter, specify the name of the security group as it is displayed on the Amazon EC2 console. Ensure that you include the `sg` prefix.

3.8.10. Customizing the cluster network MTU

Before you deploy a cluster on AWS, you can customize the cluster network maximum transmission unit (MTU) for your cluster network to meet the needs of your infrastructure.

By default, when you install a cluster with supported Local Zones capabilities, the MTU value for the cluster network is automatically adjusted to the lowest value that the network plugin accepts.

Setting an unsupported MTU value for EC2 instances that operate in the Local Zones infrastructure can cause issues for your OpenShift Container Platform cluster.

If the Local Zone supports higher MTU values in between EC2 instances in the Local Zone and the AWS Region, you can manually configure the higher value to increase the network performance of the cluster network.

You can customize the MTU for a cluster by specifying the `networking.clusterNetworkMTU` parameter in the `install-config.yaml` configuration file.

All subnets in Local Zones must support the higher MTU value, so that each node in that zone can successfully communicate with services in the AWS Region and deploy your workloads.

Example of overwriting the default MTU value

```
apiVersion: v1
baseDomain: devcluster.openshift.com
metadata:
  name: edge-zone
networking:
  clusterNetworkMTU: 8901
compute:
- name: edge
  platform:
    aws:
      zones:
      - us-west-2-lax-1a
      - us-west-2-lax-1b
platform:
  aws:
    region: us-west-2
pullSecret: '{"auths": ...}'
sshKey: ssh-ed25519 AAAA...
```

3.8.11. Cluster installation options for an AWS Local Zones environment

You can install an OpenShift Container Platform cluster on AWS with edge compute nodes defined in Local Zones.

Choose one of the following installation options:

- Fully automated option: Installing a cluster to quickly extend compute nodes to edge compute pools, where the installation program automatically creates infrastructure resources for the OpenShift Container Platform cluster.
- Existing VPC option: Installing a cluster on AWS into an existing VPC, where you supply Local Zones subnets to the `install-config.yaml` file.

3.8.12. Install a cluster quickly in AWS Local Zones

You can quickly install a cluster on Amazon Web Services (AWS) to extend compute nodes to Local Zones locations.

By using this installation route, the installation program automatically creates network resources and Local Zones subnets for each zone that you defined in your configuration file. To customize the installation, you must modify parameters in the `install-config.yaml` file before you deploy the cluster.

3.8.12.1. Modifying an installation configuration file to use Amazon Web Services (AWS) Local Zones

Modify an `install-config.yaml` file to include AWS Local Zones.

Prerequisites

- You have configured an AWS account.
- You added your AWS keys and AWS Region to your local AWS profile by running `aws configure`.
- You are familiar with the configuration limitations that apply when you specify the installation program to automatically create subnets for your OpenShift Container Platform cluster.
- You opted in to the Local Zones group for each zone.
- You created an `install-config.yaml` file by using the procedure "Creating the installation configuration file".

Procedure

1. Modify the `install-config.yaml` file by specifying Local Zones names in the `platform.aws.zones` property of the edge compute pool.

```
platform:
  aws:
    region: <region_name>
compute:
- name: edge
  platform:
    aws:
      zones:
      - <local_zone_name>
```

where:

`platform.aws.region`
Specifies the AWS Region name.

`compute.platform.aws.zones`
Specifies the list of Local Zones names to use. The zones must exist in the same AWS Region specified in the `platform.aws.region` field.

The following example shows a configuration for installing a cluster in the `us-west-2` AWS Region that extends edge nodes to Local Zones in `Los Angeles` and `Las Vegas` locations:

```
apiVersion: v1
baseDomain: example.com
metadata:
  name: cluster-name
platform:
  aws:
    region: us-west-2
compute:
- name: edge
  platform:
    aws:
      zones:
      - us-west-2-lax-1a
      - us-west-2-lax-1b
      - us-west-2-las-1a
pullSecret: '{"auths": ...}'
sshKey: 'ssh-ed25519 AAAA...'
```

2. Deploy your cluster.

3.8.13. Installing a cluster in an existing VPC that has Local Zone subnets

You can install a cluster into an existing Amazon Virtual Private Cloud (VPC) on Amazon Web Services (AWS). The installation program provisions the rest of the required infrastructure, which you can further customize.

To customize the installation, change parameters in the `install-config.yaml` file before you install the cluster.

Installing a cluster on AWS into an existing VPC requires extending compute nodes to the edge of the Cloud Infrastructure by using AWS Local Zones.

Local Zone subnets extend regular compute nodes to edge networks. Each edge compute nodes runs a user workload. After you create an Amazon Web Service (AWS) Local Zone environment, and you deploy your cluster, you can use edge compute nodes to create user workloads in Local Zone subnets.

If you want to create private subnets, you must either change the provided CloudFormation template or create your own template.

You can use a provided CloudFormation template to create network resources. Additionally, you can change a template to customize your infrastructure or use the information that they contain to create AWS resources according to your company's policies.

The documentation provides the steps for performing an installer-provisioned infrastructure installation for example purposes only. Installing a cluster in an existing VPC requires that you have knowledge of the cloud provider and the installation process of OpenShift Container Platform. You can use a CloudFormation template to assist you with completing these steps or to help model your own cluster installation. Instead of using the CloudFormation template to create resources, you can decide to use other methods for generating these resources.

3.8.13.1. Creating a VPC in Amazon Web Services (AWS)

You can create a Virtual Private Cloud (VPC), and subnets for all Local Zones locations, in AWS for your OpenShift Container Platform cluster to extend compute nodes to edge locations.

You can further customize your VPC to meet your requirements, including a VPN and route tables. You can also add new Local Zones subnets not included at initial deployment.

You can use the provided CloudFormation template and a custom parameter file to create a stack of AWS resources that represent the VPC.

If you do not use the provided CloudFormation template to create your AWS infrastructure, you must review the provided information and manually create the infrastructure. If your cluster does not initialize correctly, you might have to contact Red Hat support with your installation logs.

Prerequisites

- You configured an AWS account.
- You added your AWS keys and AWS Region to your local AWS profile by running `aws configure`.
- You opted in to the AWS Local Zones on your AWS account.

Procedure

1. Create a JSON file that contains the parameter values that the CloudFormation template requires:

```
[
  {
    "ParameterKey": "VpcCidr",
    "ParameterValue": "10.0.0.0/16"
  },
  {
    "ParameterKey": "AvailabilityZoneCount",
    "ParameterValue": "3"
  },
  {
    "ParameterKey": "SubnetBits",
    "ParameterValue": "12"
  }
]
```

   where:

   `VpcCidr`
   Specifies the CIDR block for the VPC. Use a CIDR block in the format `x.x.x.x/16-24`.

   `AvailabilityZoneCount`
   Specifies the number of availability zones to deploy the VPC in. Use an integer between `1` and `3`.

   `SubnetBits`
   Specifies the size of each subnet in each availability zone. Use an integer between `5` and `13`, where `5` is `/27` and `13` is `/19`.

2. Go to the section of the documentation named "CloudFormation template for the VPC", and then copy the syntax from the provided template. Save the copied template syntax as a YAML file on your local system. This template describes the VPC that your cluster requires.
3. Launch the CloudFormation template to create a stack of AWS resources that represent the VPC by running the following command:

   You must enter the command on a single line.

```
$ aws cloudformation create-stack --stack-name <name> \
     --template-body file://<template>.yaml \
     --parameters file://<parameters>.json
```

   where:

   `--stack-name <name>`
   Specifies the name for the CloudFormation stack, such as `cluster-vpc`. You need the name of this stack if you remove the cluster.

   `--template-body file://<template>.yaml`
   Specifies the relative path to and name of the CloudFormation template YAML file that you saved.

   `--parameters file://<parameters>.json`
   Specifies the relative path to and name of the CloudFormation parameters JSON file.

   Example output

```
arn:aws:cloudformation:us-east-1:123456789012:stack/cluster-vpc/dbedae40-2fd3-11eb-820e-12a48460849f
```

4. Confirm that the template components exist by running the following command:

```
$ aws cloudformation describe-stacks --stack-name <name>
```

After the `StackStatus` displays `CREATE_COMPLETE`, the output displays values for the following parameters. You must provide these parameter values to the other CloudFormation templates that you run to create your cluster.

`VpcId`	The ID of your VPC.
`PublicSubnetIds`	The IDs of the new public subnets.
`PrivateSubnetIds`	The IDs of the new private subnets.
`PublicRouteTableId`	The ID of the new public route table ID.

3.8.13.2. CloudFormation template for the VPC

You can use a CloudFormation template to deploy the VPC that you need for your OpenShift Container Platform cluster.

See the following template:

Example 3.2. CloudFormation template for the VPC

```
AWSTemplateFormatVersion: 2010-09-09
Description: Template for Best Practice VPC with 1-3 AZs

Parameters:
  VpcCidr:
    AllowedPattern: ^(([0-9]|[1-9][0-9]|1[0-9]{2}|2[0-4][0-9]|25[0-5])\.){3}([0-9]|[1-9][0-9]|1[0-
9]{2}|2[0-4][0-9]|25[0-5])(\/(1[6-9]|2[0-4]))$
    ConstraintDescription: CIDR block parameter must be in the form x.x.x.x/16-24.
    Default: 10.0.0.0/16
    Description: CIDR block for VPC.
    Type: String
  AvailabilityZoneCount:
    ConstraintDescription: "The number of availability zones. (Min: 1, Max: 3)"
    MinValue: 1
    MaxValue: 3
    Default: 1
    Description: "How many AZs to create VPC subnets for. (Min: 1, Max: 3)"
    Type: Number
  SubnetBits:
    ConstraintDescription: CIDR block parameter must be in the form x.x.x.x/19-27.
    MinValue: 5
    MaxValue: 13
    Default: 12
    Description: "Size of each subnet to create within the availability zones. (Min: 5 = /27, Max:
13 = /19)"
    Type: Number

Metadata:
  AWS::CloudFormation::Interface:
    ParameterGroups:
    - Label:
        default: "Network Configuration"
      Parameters:
```

```
      - VpcCidr
      - SubnetBits
    - Label:
        default: "Availability Zones"
      Parameters:
      - AvailabilityZoneCount
    ParameterLabels:
      AvailabilityZoneCount:
        default: "Availability Zone Count"
      VpcCidr:
        default: "VPC CIDR"
      SubnetBits:
        default: "Bits Per Subnet"

Conditions:
  DoAz3: !Equals [3, !Ref AvailabilityZoneCount]
  DoAz2: !Or [!Equals [2, !Ref AvailabilityZoneCount], Condition: DoAz3]

Resources:
  VPC:
    Type: "AWS::EC2::VPC"
    Properties:
      EnableDnsSupport: "true"
      EnableDnsHostnames: "true"
      CidrBlock: !Ref VpcCidr
  PublicSubnet:
    Type: "AWS::EC2::Subnet"
    Properties:
      VpcId: !Ref VPC
      CidrBlock: !Select [0, !Cidr [!Ref VpcCidr, 6, !Ref SubnetBits]]
      AvailabilityZone: !Select
      - 0
      - Fn::GetAZs: !Ref "AWS::Region"
  PublicSubnet2:
    Type: "AWS::EC2::Subnet"
    Condition: DoAz2
    Properties:
      VpcId: !Ref VPC
      CidrBlock: !Select [1, !Cidr [!Ref VpcCidr, 6, !Ref SubnetBits]]
      AvailabilityZone: !Select
      - 1
      - Fn::GetAZs: !Ref "AWS::Region"
  PublicSubnet3:
    Type: "AWS::EC2::Subnet"
    Condition: DoAz3
    Properties:
      VpcId: !Ref VPC
      CidrBlock: !Select [2, !Cidr [!Ref VpcCidr, 6, !Ref SubnetBits]]
      AvailabilityZone: !Select
      - 2
      - Fn::GetAZs: !Ref "AWS::Region"
  InternetGateway:
    Type: "AWS::EC2::InternetGateway"
  GatewayToInternet:
    Type: "AWS::EC2::VPCGatewayAttachment"
    Properties:
      VpcId: !Ref VPC
      InternetGatewayId: !Ref InternetGateway
  PublicRouteTable:
    Type: "AWS::EC2::RouteTable"
    Properties:
      VpcId: !Ref VPC
  PublicRoute:
    Type: "AWS::EC2::Route"
```

```
  DependsOn: GatewayToInternet
  Properties:
    RouteTableId: !Ref PublicRouteTable
    DestinationCidrBlock: 0.0.0.0/0
    GatewayId: !Ref InternetGateway
PublicSubnetRouteTableAssociation:
  Type: "AWS::EC2::SubnetRouteTableAssociation"
  Properties:
    SubnetId: !Ref PublicSubnet
    RouteTableId: !Ref PublicRouteTable
PublicSubnetRouteTableAssociation2:
  Type: "AWS::EC2::SubnetRouteTableAssociation"
  Condition: DoAz2
  Properties:
    SubnetId: !Ref PublicSubnet2
    RouteTableId: !Ref PublicRouteTable
PublicSubnetRouteTableAssociation3:
  Condition: DoAz3
  Type: "AWS::EC2::SubnetRouteTableAssociation"
  Properties:
    SubnetId: !Ref PublicSubnet3
    RouteTableId: !Ref PublicRouteTable
PrivateSubnet:
  Type: "AWS::EC2::Subnet"
  Properties:
    VpcId: !Ref VPC
    CidrBlock: !Select [3, !Cidr [!Ref VpcCidr, 6, !Ref SubnetBits]]
    AvailabilityZone: !Select
    - 0
    - Fn::GetAZs: !Ref "AWS::Region"
PrivateRouteTable:
  Type: "AWS::EC2::RouteTable"
  Properties:
    VpcId: !Ref VPC
PrivateSubnetRouteTableAssociation:
  Type: "AWS::EC2::SubnetRouteTableAssociation"
  Properties:
    SubnetId: !Ref PrivateSubnet
    RouteTableId: !Ref PrivateRouteTable
NAT:
  DependsOn:
  - GatewayToInternet
  Type: "AWS::EC2::NatGateway"
  Properties:
    AllocationId:
      "Fn::GetAtt":
      - EIP
      - AllocationId
    SubnetId: !Ref PublicSubnet
EIP:
  Type: "AWS::EC2::EIP"
  Properties:
    Domain: vpc
Route:
  Type: "AWS::EC2::Route"
  Properties:
    RouteTableId:
      Ref: PrivateRouteTable
    DestinationCidrBlock: 0.0.0.0/0
    NatGatewayId:
      Ref: NAT
PrivateSubnet2:
  Type: "AWS::EC2::Subnet"
  Condition: DoAz2
```

```
  Properties:
    VpcId: !Ref VPC
    CidrBlock: !Select [4, !Cidr [!Ref VpcCidr, 6, !Ref SubnetBits]]
    AvailabilityZone: !Select
    - 1
    - Fn::GetAZs: !Ref "AWS::Region"
PrivateRouteTable2:
  Type: "AWS::EC2::RouteTable"
  Condition: DoAz2
  Properties:
    VpcId: !Ref VPC
PrivateSubnetRouteTableAssociation2:
  Type: "AWS::EC2::SubnetRouteTableAssociation"
  Condition: DoAz2
  Properties:
    SubnetId: !Ref PrivateSubnet2
    RouteTableId: !Ref PrivateRouteTable2
NAT2:
  DependsOn:
  - GatewayToInternet
  Type: "AWS::EC2::NatGateway"
  Condition: DoAz2
  Properties:
    AllocationId:
      "Fn::GetAtt":
      - EIP2
      - AllocationId
    SubnetId: !Ref PublicSubnet2
EIP2:
  Type: "AWS::EC2::EIP"
  Condition: DoAz2
  Properties:
    Domain: vpc
Route2:
  Type: "AWS::EC2::Route"
  Condition: DoAz2
  Properties:
    RouteTableId:
      Ref: PrivateRouteTable2
    DestinationCidrBlock: 0.0.0.0/0
    NatGatewayId:
      Ref: NAT2
PrivateSubnet3:
  Type: "AWS::EC2::Subnet"
  Condition: DoAz3
  Properties:
    VpcId: !Ref VPC
    CidrBlock: !Select [5, !Cidr [!Ref VpcCidr, 6, !Ref SubnetBits]]
    AvailabilityZone: !Select
    - 2
    - Fn::GetAZs: !Ref "AWS::Region"
PrivateRouteTable3:
  Type: "AWS::EC2::RouteTable"
  Condition: DoAz3
  Properties:
    VpcId: !Ref VPC
PrivateSubnetRouteTableAssociation3:
  Type: "AWS::EC2::SubnetRouteTableAssociation"
  Condition: DoAz3
  Properties:
    SubnetId: !Ref PrivateSubnet3
    RouteTableId: !Ref PrivateRouteTable3
NAT3:
  DependsOn:
```

```
    - GatewayToInternet
    Type: "AWS::EC2::NatGateway"
    Condition: DoAz3
    Properties:
      AllocationId:
        "Fn::GetAtt":
        - EIP3
        - AllocationId
      SubnetId: !Ref PublicSubnet3
  EIP3:
    Type: "AWS::EC2::EIP"
    Condition: DoAz3
    Properties:
      Domain: vpc
  Route3:
    Type: "AWS::EC2::Route"
    Condition: DoAz3
    Properties:
      RouteTableId:
        Ref: PrivateRouteTable3
      DestinationCidrBlock: 0.0.0.0/0
      NatGatewayId:
        Ref: NAT3
  S3Endpoint:
    Type: AWS::EC2::VPCEndpoint
    Properties:
      PolicyDocument:
        Version: 2012-10-17
        Statement:
        - Effect: Allow
          Principal: '*'
          Action:
          - '*'
          Resource:
          - '*'
      RouteTableIds:
      - !Ref PublicRouteTable
      - !Ref PrivateRouteTable
      - !If [DoAz2, !Ref PrivateRouteTable2, !Ref "AWS::NoValue"]
      - !If [DoAz3, !Ref PrivateRouteTable3, !Ref "AWS::NoValue"]
      ServiceName: !Join
      - ''
      - - com.amazonaws.
        - !Ref 'AWS::Region'
        - .s3
      VpcId: !Ref VPC

Outputs:
  VpcId:
    Description: ID of the new VPC.
    Value: !Ref VPC
  PublicSubnetIds:
    Description: Subnet IDs of the public subnets.
    Value:
      !Join [
        ",",
        [!Ref PublicSubnet, !If [DoAz2, !Ref PublicSubnet2, !Ref "AWS::NoValue"], !If [DoAz3, !Ref
PublicSubnet3, !Ref "AWS::NoValue"]]
      ]
  PrivateSubnetIds:
    Description: Subnet IDs of the private subnets.
    Value:
      !Join [
        ",",
```

```
        [!Ref PrivateSubnet, !If [DoAz2, !Ref PrivateSubnet2, !Ref "AWS::NoValue"], !If [DoAz3,
!Ref PrivateSubnet3, !Ref "AWS::NoValue"]]
      ]
  PublicRouteTableId:
    Description: Public Route table ID
    Value: !Ref PublicRouteTable
  PrivateRouteTableIds:
    Description: Private Route table IDs
    Value:
      !Join [
        ",",
        [
          !Join ["=", [
            !Select [0, "Fn::GetAZs": !Ref "AWS::Region"],
            !Ref PrivateRouteTable
          ]],
          !If [DoAz2,
               !Join ["=", [!Select [1, "Fn::GetAZs": !Ref "AWS::Region"], !Ref
PrivateRouteTable2]],
               !Ref "AWS::NoValue"
          ],
          !If [DoAz3,
               !Join ["=", [!Select [2, "Fn::GetAZs": !Ref "AWS::Region"], !Ref
PrivateRouteTable3]],
               !Ref "AWS::NoValue"
          ]
        ]
      ]
```

3.8.13.3. Creating subnets in Local Zones

Before you configure a machine set for edge compute nodes in your OpenShift Container Platform cluster, you must create the subnets in Local Zones.

Complete the following procedure for each Local Zone that you want to deploy compute nodes to.

You can use the provided CloudFormation template and create a CloudFormation stack. You can then use this stack to custom provision a subnet.

If you do not use the provided CloudFormation template to create your Amazon Web Services (AWS) infrastructure, you must review the provided information and manually create the infrastructure. If your cluster does not initialize correctly, you might have to contact Red Hat support with your installation logs.

Prerequisites

- You configured an AWS account.
- You added your AWS keys and region to your local AWS profile by running `aws configure`.
- You opted in to the Local Zones group.

Procedure

1. Go to the section of the documentation named "CloudFormation template for the VPC subnet", and copy the syntax from the template. Save the copied template syntax as a YAML file on your local system. This template describes the VPC that your cluster requires.

2. Run the following command to deploy the CloudFormation template, which creates a stack of AWS resources that represent the VPC:

```
$ aws cloudformation create-stack --stack-name <stack_name> \
  --region ${CLUSTER_REGION} \
  --template-body file://<template>.yaml \
  --parameters \
    ParameterKey=VpcId,ParameterValue="${VPC_ID}" \
    ParameterKey=ClusterName,ParameterValue="${CLUSTER_NAME}" \
    ParameterKey=ZoneName,ParameterValue="${ZONE_NAME}" \
    ParameterKey=PublicRouteTableId,ParameterValue="${ROUTE_TABLE_PUB}" \
    ParameterKey=PublicSubnetCidr,ParameterValue="${SUBNET_CIDR_PUB}" \
    ParameterKey=PrivateRouteTableId,ParameterValue="${ROUTE_TABLE_PVT}" \
    ParameterKey=PrivateSubnetCidr,ParameterValue="${SUBNET_CIDR_PVT}"
```

where:

<stack_name>
: Specifies the name for the CloudFormation stack, such as `cluster-wl-<local_zone_shortname>`. You need the name of this stack if you remove the cluster.

<template>
: Specifies the relative path and the name of the CloudFormation template YAML file that you saved.

${VPC_ID}
: Specifies the VPC ID, which is the value `VpcID` in the output of the CloudFormation template for the VPC.

${CLUSTER_NAME}
: Specifies the value of `ClusterName` to be used as a prefix of the new AWS resource names.

${ZONE_NAME}
: Specifies the value of the Local Zones name to create the subnets.

${ROUTE_TABLE_PUB}
: Specifies the `PublicRouteTableId` extracted from the output of the VPC's CloudFormation stack.

${SUBNET_CIDR_PUB}
: Specifies a valid CIDR block that is used to create the public subnet. This block must be part of the VPC CIDR block `VpcCidr`.

${ROUTE_TABLE_PVT}
: Specifies the `PrivateRouteTableId` extracted from the output of the VPC's CloudFormation stack.

${SUBNET_CIDR_PVT}
: Specifies a valid CIDR block that is used to create the private subnet. This block must be part of the VPC CIDR block `VpcCidr`.

Example output

```
arn:aws:cloudformation:us-east-1:123456789012:stack/<stack_name>/dbedae40-820e-11eb-2fd3-12a48460849f
```

Verification

Confirm that the template components exist by running the following command:

```
$ aws cloudformation describe-stacks --stack-name <stack_name>
```

After the `StackStatus` displays `CREATE_COMPLETE`, the output displays values for the following parameters. Ensure that you provide these parameter values to the other CloudFormation templates that you run to create for your cluster.

`PublicSubnetId`	The IDs of the public subnet created by the CloudFormation stack.
`PrivateSubnetId`	The IDs of the private subnet created by the CloudFormation stack.

3.8.13.4. CloudFormation template for the VPC subnet

Use the CloudFormation template to deploy the private and public subnets in a zone on Local Zones infrastructure. The template provisions an `AWS::EC2::Subnet` and associates it with a specific Local Zones and VPC route table to reduce latency.

CloudFormation template for VPC subnets

```
AWSTemplateFormatVersion: 2010-09-09
Description: Template for Best Practice Subnets (Public and Private)

Parameters:
  VpcId:
    Description: VPC ID that comprises all the target subnets.
    Type: String
    AllowedPattern: ^(?:(?:vpc)(?:-[a-zA-Z0-9]+)?\b|(?:[0-9]{1,3}\.){3}[0-9]{1,3})$
    ConstraintDescription: VPC ID must be with valid name, starting with vpc-.*.
  ClusterName:
    Description: Cluster name or prefix name to prepend the Name tag for each subnet.
    Type: String
    AllowedPattern: ".+"
    ConstraintDescription: ClusterName parameter must be specified.
  ZoneName:
    Description: Zone Name to create the subnets, such as us-west-2-lax-1a.
    Type: String
    AllowedPattern: ".+"
    ConstraintDescription: ZoneName parameter must be specified.
  PublicRouteTableId:
    Description: Public Route Table ID to associate the public subnet.
    Type: String
    AllowedPattern: ".+"
    ConstraintDescription: PublicRouteTableId parameter must be specified.
  PublicSubnetCidr:
    AllowedPattern: ^(([0-9]|[1-9][0-9]|1[0-9]{2}|2[0-4][0-9]|25[0-5])\.){3}([0-9]|[1-9][0-9]|1[0-
9]{2}|2[0-4][0-9]|25[0-5])(\/(1[6-9]|2[0-4]))$
    ConstraintDescription: CIDR block parameter must be in the form x.x.x.x/16-24.
    Default: 10.0.128.0/20
    Description: CIDR block for public subnet.
    Type: String
  PrivateRouteTableId:
    Description: Private Route Table ID to associate the private subnet.
    Type: String
    AllowedPattern: ".+"
    ConstraintDescription: PrivateRouteTableId parameter must be specified.
  PrivateSubnetCidr:
    AllowedPattern: ^(([0-9]|[1-9][0-9]|1[0-9]{2}|2[0-4][0-9]|25[0-5])\.){3}([0-9]|[1-9][0-9]|1[0-
9]{2}|2[0-4][0-9]|25[0-5])(\/(1[6-9]|2[0-4]))$
    ConstraintDescription: CIDR block parameter must be in the form x.x.x.x/16-24.
```

```
    Default: 10.0.128.0/20
    Description: CIDR block for private subnet.
    Type: String


Resources:
  PublicSubnet:
    Type: "AWS::EC2::Subnet"
    Properties:
      VpcId: !Ref VpcId
      CidrBlock: !Ref PublicSubnetCidr
      AvailabilityZone: !Ref ZoneName
      Tags:
      - Key: Name
        Value: !Join ['-', [!Ref ClusterName, "public", !Ref ZoneName]]

  PublicSubnetRouteTableAssociation:
    Type: "AWS::EC2::SubnetRouteTableAssociation"
    Properties:
      SubnetId: !Ref PublicSubnet
      RouteTableId: !Ref PublicRouteTableId

  PrivateSubnet:
    Type: "AWS::EC2::Subnet"
    Properties:
      VpcId: !Ref VpcId
      CidrBlock: !Ref PrivateSubnetCidr
      AvailabilityZone: !Ref ZoneName
      Tags:
      - Key: Name
        Value: !Join ['-', [!Ref ClusterName, "private", !Ref ZoneName]]

  PrivateSubnetRouteTableAssociation:
    Type: "AWS::EC2::SubnetRouteTableAssociation"
    Properties:
      SubnetId: !Ref PrivateSubnet
      RouteTableId: !Ref PrivateRouteTableId

Outputs:
  PublicSubnetId:
    Description: Subnet ID of the public subnets.
    Value:
      !Join ["", [!Ref PublicSubnet]]

  PrivateSubnetId:
    Description: Subnet ID of the private subnets.
    Value:
      !Join ["", [!Ref PrivateSubnet]]
```

3.8.13.5. Modifying an installation configuration file to use Amazon Web Services (AWS) Local Zones subnets

Modify your `install-config.yaml` file to include Local Zones subnets.

Prerequisites

- You created subnets by using the procedure "Creating subnets in Local Zones".
- You created an `install-config.yaml` file by using the procedure "Creating the installation configuration file".

Procedure

Modify the `install-config.yaml` configuration file by specifying Local Zones subnets in the `platform.aws.subnets` parameter.

Example installation configuration file with Local Zones subnets

```
platform:
  aws:
    region: us-west-2
    subnets:
    - publicSubnetId-1
    - publicSubnetId-2
    - publicSubnetId-3
    - privateSubnetId-1
    - privateSubnetId-2
    - privateSubnetId-3
    - publicSubnetId-LocalZone-1
```

`platform.aws.subnets` specifies the list of subnet IDs created in the zones: Availability and Local Zones.

3.8.14. Optional: AWS security groups

By default, the installation program creates and attaches security groups to control plane and compute machines. The rules associated with the default security groups cannot be modified.

However, you can apply additional existing AWS security groups, which are associated with your existing VPC, to control plane and compute machines. Applying custom security groups can help you meet the security needs of your organization, in such cases where you need to control the incoming or outgoing traffic of these machines.

As part of the installation process, you apply custom security groups by modifying the `install-config.yaml` file before deploying the cluster.

For more information, see "Edge compute pools and AWS Local Zones".

By default, AWS allows 5 security groups per network interface. The installation program creates 2 security groups for compute machines and 3 security groups for control plane machines. If you are installing a cluster into a shared VPC, there are three scenarios in which you must increase this quota:

- You specified 4 or more custom security groups for compute machines using the `compute.platform.aws.additionalSecurityGroupIDs` parameter in the `install-config.yaml` file.
- You specified 3 or more custom security groups for control plane machines using the `controlPlane.platform.aws.additionalSecurityGroupIDs` parameter in the `install-config.yaml` file.
- You specified 3 or more custom security groups for all machines using the `platform.aws.defaultMachinePlatform` parameter in the `install-config.yaml` file.

You must increase the quota of security groups per network interface to a number greater than or equal to `3 + (number of control plane custom security groups OR number of default machine platform custom security groups)`, or `2 + (number of compute custom security groups OR number of default machine platform custom security groups)`, whichever is higher. If you do not specify a sufficient quota, the installation will succeed, but it will generate `SecurityGroupsPerInterfaceLimitExceeded` errors in the installation log, and the additional security groups will not be applied. The maximum allowed quota is 16 and the maximum number of user-specified security groups is 10.

3.8.15. Optional: Assign public IP addresses to edge compute nodes

If your workload requires deploying the edge compute nodes in public subnets on Local Zones infrastructure, you can configure the machine set manifests when installing a cluster.

AWS Local Zones infrastructure accesses the network traffic in a specified zone, so applications can take advantage of lower latency when serving end users that are closer to that zone.

The default setting that deploys compute nodes in private subnets might not meet your needs, so consider creating edge compute nodes in public subnets when you want to apply more customization to your infrastructure.

By default, OpenShift Container Platform deploy the compute nodes in private subnets. For best performance, consider placing compute nodes in subnets that have their Public IP addresses attached to the subnets.

You must create additional security groups, but ensure that you only open the groups' rules over the internet when you really need to.

Procedure

1. Change to the directory that contains the installation program and generate the manifest files. Ensure that the installation manifests get created at the `openshift` and `manifests` directory level.

   ```
   $ ./openshift-install create manifests --dir <installation_directory>
   ```

2. Edit the machine set manifest that the installation program generates for the Local Zones, so that the manifest gets deployed in public subnets. Specify `true` for the `spec.template.spec.providerSpec.value.publicIP` parameter.

   Example machine set manifest configuration for installing a cluster quickly in Local Zones

   ```
   spec:
     template:
       spec:
         providerSpec:
           value:
             publicIp: true
             subnet:
               filters:
                 - name: tag:Name
                   values:
                     - ${INFRA_ID}-public-${ZONE_NAME}
   ```

   Example machine set manifest configuration for installing a cluster in an existing VPC that has Local Zones subnets

   ```
   apiVersion: machine.openshift.io/v1beta1
   kind: MachineSet
   metadata:
     name: <infrastructure_id>-edge-<zone>
     namespace: openshift-machine-api
   spec:
     template:
       spec:
         providerSpec:
           value:
             publicIp: true
   ```

3.8.16. Deploying the cluster

To deploy your OpenShift Container Platform cluster, you initialize installation by running the `openshift-install create cluster` command from the directory that contains the installation program. The installation program provisions the required infrastructure and completes the cluster setup.

You can run the `create cluster` command of the installation program only once, during initial installation.

Prerequisites

- You have configured an account with the cloud platform that hosts your cluster.
- You have the OpenShift Container Platform installation program and the pull secret for your cluster.
- You have verified that the cloud provider account on your host has the correct permissions to deploy the cluster. An account with incorrect permissions causes the installation process to fail with an error message that displays the missing permissions.

Procedure

1. In the directory that contains the installation program, initialize the cluster deployment by running the following command:

   ```
   $ ./openshift-install create cluster --dir <installation_directory> \
       --log-level=info
   ```

   where:

   - `<installation_directory>`: Specifies the location of your customized `./install-config.yaml` file.
   - `--log-level`: Specifies the log level. To view different installation details, specify `warn`, `debug`, or `error` instead of `info`.
2. Optional: Remove or disable the `AdministratorAccess` policy from the IAM account that you used to install the cluster.

   The elevated permissions provided by the `AdministratorAccess` policy are required only during installation.

Verification

When the cluster deployment completes successfully:

- The terminal displays directions for accessing your cluster, including a link to the web console and credentials for the `kubeadmin` user.

- Credential information also outputs to `<installation_directory>/.openshift_install.log`.

  Do not delete the installation program or the files that the installation program creates. Both are required to delete the cluster.

  The following example shows the expected output:

```
...
INFO Install complete!
INFO To access the cluster as the system:admin user when using 'oc', run 'export
KUBECONFIG=/home/myuser/install_dir/auth/kubeconfig'
INFO Access the OpenShift web-console here: https://console-openshift-
console.apps.mycluster.example.com
INFO Login to the console with user: "kubeadmin", and password: "password"
INFO Time elapsed: 36m22s
```

  - The Ignition config files that the installation program generates contain certificates that expire after 24 hours, which are then renewed at that time. If the cluster is shut down before renewing the certificates and the cluster is later restarted after the 24 hours have elapsed, the cluster automatically recovers the expired certificates. The exception is that you must manually approve the pending `node-bootstrapper` certificate signing requests (CSRs) to recover kubelet certificates. See the documentation for *Recovering from expired control plane certificates* for more information.
  - It is recommended that you use Ignition config files within 12 hours after they are generated because the 24-hour certificate rotates from 16 to 22 hours after the cluster is installed. By using the Ignition config files within 12 hours, you can avoid installation failure if the certificate update runs during installation.

3.8.17. Logging in to the cluster by using the CLI

To log in to your cluster as the default system user, export the `kubeconfig` file. This configuration enables the CLI to authenticate and connect to the specific API server created during OpenShift Container Platform installation.

The `kubeconfig` file is specific to a cluster and OpenShift Container Platform generates it during installation.

Prerequisites

- You deployed an OpenShift Container Platform cluster.
- You installed the OpenShift CLI (`oc`).

Procedure

1. Export the `kubeadmin` credentials by running the following command:

   ```
   $ export KUBECONFIG=<installation_directory>/auth/kubeconfig
   ```

   where:

   `<installation_directory>`
   Specifies the path to the directory that stores the installation files.

2. Verify you can run `oc` commands successfully using the exported configuration by running the following command:

```
$ oc whoami
```

Example output

```
system:admin
```

Next steps

- "Customize your cluster"
- "Remote health reporting"

3.8.18. Logging in to the cluster by using the web console

To verify that your cluster deployed successfully and access its features, log in to the OpenShift Container Platform web console as the `kubeadmin` user.

Prerequisites

- You have access to the installation host.
- You completed a cluster installation and all cluster Operators are available.

Procedure

1. Obtain the password for the `kubeadmin` user from the `kubeadmin-password` file on the installation host:

```
$ cat <installation_directory>/auth/kubeadmin-password
```

Or, you can obtain the `kubeadmin` password from the `<installation_directory>/.openshift_install.log` log file on the installation host.

2. List the OpenShift Container Platform web console route:

```
$ oc get routes -n openshift-console | grep 'console-openshift'
```

Or, you can obtain the OpenShift Container Platform route from the `<installation_directory>/.openshift_install.log` log file on the installation host.

Example output

```
console     console-openshift-console.apps.<cluster_name>.<base_domain>            console
https   reencrypt/Redirect   None
```

3. Navigate to the route detailed in the output of the preceding command in a web browser and log in as the `kubeadmin` user.

3.8.19. Verifying nodes that were created with edge compute pool

After you install a cluster that uses AWS Local Zones infrastructure, check the status of the machine that was created by the machine set manifests created during installation.

Procedure

1. To check the machine sets created from the subnet you added to the `install-config.yaml` file, run the following command:

   ```
   $ oc get machineset -n openshift-machine-api
   ```

   Example output

   ```
   NAME                                   DESIRED   CURRENT   READY   AVAILABLE   AGE
   cluster-7xw5g-edge-us-east-1-nyc-1a   1         1         1       1           3h4m
   cluster-7xw5g-worker-us-east-1a       1         1         1       1           3h4m
   cluster-7xw5g-worker-us-east-1b       1         1         1       1           3h4m
   cluster-7xw5g-worker-us-east-1c       1         1         1       1           3h4m
   ```

2. To check the machines that were created from the machine sets, run the following command:

   ```
   $ oc get machines -n openshift-machine-api
   ```

   Example output

   ```
   NAME                                         PHASE     TYPE          REGION      ZONE
   AGE
   cluster-7xw5g-edge-us-east-1-nyc-1a-wbclh   Running   c5d.2xlarge   us-east-1   us-east-1-nyc-
   1a   3h
   cluster-7xw5g-master-0                       Running   m6i.xlarge    us-east-1   us-east-1a
   3h4m
   cluster-7xw5g-master-1                       Running   m6i.xlarge    us-east-1   us-east-1b
   3h4m
   cluster-7xw5g-master-2                       Running   m6i.xlarge    us-east-1   us-east-1c
   3h4m
   cluster-7xw5g-worker-us-east-1a-rtp45        Running   m6i.xlarge    us-east-1   us-east-1a
   3h
   cluster-7xw5g-worker-us-east-1b-glm7c        Running   m6i.xlarge    us-east-1   us-east-1b
   3h
   cluster-7xw5g-worker-us-east-1c-qfvz4        Running   m6i.xlarge    us-east-1   us-east-1c
   3h
   ```

3. To check nodes with edge roles, run the following command:

   ```
   $ oc get nodes -l node-role.kubernetes.io/edge
   ```

   Example output

   ```
   NAME                           STATUS   ROLES         AGE    VERSION
   ip-10-0-207-188.ec2.internal   Ready    edge,worker   172m   v1.25.2+d2e245f
   ```

3.9. Installing a cluster with compute nodes on AWS Wavelength Zones

You can quickly install an OpenShift Container Platform cluster on Amazon Web Services (AWS) Wavelength Zones by setting the zone names in the edge compute pool of the `install-config.yaml` file, or install a cluster in an existing Amazon Virtual Private Cloud (VPC) with Wavelength Zone subnets.

AWS Wavelength Zones is an infrastructure that AWS configured for mobile edge computing (MEC) applications.

A Wavelength Zone embeds AWS compute and storage services within the 5G network of a communication service provider (CSP). By placing application servers in a Wavelength Zone, the application traffic from your 5G devices can stay in the 5G network. The application traffic of the device reaches the target server directly, making latency a non-issue.

3.9.1. Infrastructure prerequisites

Before you install an OpenShift Container Platform cluster on Amazon Web Services (AWS) Wavelength Zones, you must meet several prerequisites.

The following prerequisites must be met:

- You reviewed details about OpenShift Container Platform installation and update processes.
- You are familiar with Selecting a cluster installation method and preparing it for users.
- You configured an AWS account to host the cluster.

  If you have an AWS profile stored on your computer, it must not use a temporary session token that you generated while using a multi-factor authentication device. The cluster continues to use your current AWS credentials to create AWS resources for the entire life of the cluster, so you must use key-based, long-term credentials. To generate appropriate keys, see Managing Access Keys for IAM Users in the AWS documentation. You can supply the keys when you run the installation program.

- You downloaded the AWS CLI and installed it on your computer. See Install the AWS CLI Using the Bundled Installer (Linux, macOS, or UNIX) in the AWS documentation.
- If you use a firewall, you configured it to allow the sites that your cluster must access.
- You noted the region and supported AWS Wavelength Zone locations to create the network resources in.
- You read AWS Wavelength features in the AWS documentation.
- You read the Quotas and considerations for Wavelength Zones in the AWS documentation.
- You added permissions for creating network resources that support AWS Wavelength Zones to the Identity and Access Management (IAM) user or role. For example:

  Example of an additional IAM policy that attached `ec2:ModifyAvailabilityZoneGroup`, `ec2:CreateCarrierGateway`, and `ec2:DeleteCarrierGateway` permissions to a user or role

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "ec2:DeleteCarrierGateway",
        "ec2:CreateCarrierGateway"
      ],
      "Resource": "*"
    },
    {
      "Action": [
        "ec2:ModifyAvailabilityZoneGroup"
      ],
      "Effect": "Allow",
      "Resource": "*"
    }
  ]
}
```

3.9.2. Cluster limitations in AWS Wavelength Zones

Some limitations exist when you try to deploy a cluster with a default installation configuration in an Amazon Web Services (AWS) Wavelength Zone.

The following list details limitations when deploying a cluster in a pre-configured AWS zone:

- The maximum transmission unit (MTU) between an Amazon EC2 instance in a zone and an Amazon EC2 instance in the Region is `1300`. This causes the cluster-wide network MTU to change according to the network plugin that is used with the deployment.
- Network resources such as Network Load Balancer (NLB), Classic Load Balancer, and Network Address Translation (NAT) Gateways are not globally supported.
- For an OpenShift Container Platform cluster on AWS, the AWS Elastic Block Storage (EBS) `gp3` type volume is the default for node volumes and the default for the storage class. This volume type is not globally available on zone locations. By default, the nodes running in zones are deployed with the `gp2` EBS volume. The `gp2-csi` `StorageClass` parameter must be set when creating workloads on zone nodes.

If you want the installation program to automatically create Wavelength Zone subnets for your OpenShift Container Platform cluster, specific configuration limitations apply with this method. The following note details some of these limitations. For other limitations, ensure that you read the "Quotas and considerations for Wavelength Zones" document that Red Hat provides in the "Infrastructure prerequisites" section.

The following configuration limitation applies when you set the installation program to automatically create subnets for your OpenShift Container Platform cluster:

- When the installation program creates private subnets in AWS Wavelength Zones, the program associates each subnet with the route table of its parent zone. This operation ensures that each private subnet can route egress traffic to the internet by way of NAT Gateways in an AWS Region.
- If the parent-zone route table does not exist during cluster installation, the installation program associates any private subnet with the first available private route table in the Amazon Virtual Private Cloud (VPC). This approach is valid only for AWS Wavelength Zones subnets in an OpenShift Container Platform cluster.

3.9.3. About edge compute pools

The edge compute pool configuration is common between Amazon Web Services (AWS) Wavelength Zones locations.

You can use the edge compute pool to create new labels to deploy applications onto Amazon Web Services (AWS) Wavelength Zones nodes. Edge compute nodes are tainted compute nodes that run in AWS Wavelength Zones locations.

When deploying a cluster that uses Wavelength Zones, consider the following points:

- Amazon EC2 instances in the Wavelength Zones are more expensive than Amazon EC2 instances in the Availability Zones.
- The latency is lower between the applications running in AWS Wavelength Zones and the user. A latency impact exists for some workloads if, for example, ingress traffic is mixed between Wavelength Zones and Availability Zones.

Generally, the maximum transmission unit (MTU) between an Amazon EC2 instance in a Wavelength Zones and an Amazon EC2 instance in the Region is 1300. The cluster network MTU must be always less than the EC2 MTU to account for the overhead. The specific overhead is determined by the network plugin. For example: OVN-Kubernetes has an overhead of `100 bytes`.

The network plugin can provide additional features, such as IPsec, that also affect the MTU sizing.

OpenShift Container Platform 4.12 introduced a new compute pool, *edge*, that is designed for use in remote zones. The edge compute pool configuration is common between Amazon Web Services (AWS)

Wavelength Zones locations. Because of the type and size limitations of resources like EC2 and EBS on Wavelength Zones resources, the default instance type can vary from the traditional compute pool.

The default Elastic Block Store (EBS) for Wavelength Zones locations is `gp2`, which differs from the non-edge compute pool. The instance type used for each Wavelength Zones on an edge compute pool also might differ from other compute pools, depending on the instance offerings on the zone.

The edge compute pool creates new labels that developers can use to deploy applications onto AWS Wavelength Zones nodes. The new labels are:

- `node-role.kubernetes.io/edge=''`
- `machine.openshift.io/zone-type=wavelength-zone`
- `machine.openshift.io/zone-group=$ZONE_GROUP_NAME`

By default, the machine sets for the edge compute pool define the taint of `NoSchedule` to prevent other workloads from spreading on Wavelength Zones instances. Users can only run user workloads if they define tolerations in the pod specification.

3.9.4. Opting in to an AWS Wavelength Zones

Create a subnet in an Amazon Web Services (AWS) Wavelength Zones when you need workloads to run physically closer to users or data sources than a standard AWS Wavelength Zones. If you plan to create subnets in AWS Wavelength Zones, you must opt in to each zone group separately.

Prerequisites

- You have installed the AWS CLI.
- You have determined an AWS Region for where you want to deploy your OpenShift Container Platform cluster.
- You have attached a permissive IAM policy to a user or role account that opts in to the zone group.

Procedure

1. List the zones that are available in your AWS Region by running the following command:

   Example command for listing available AWS Wavelength Zones in an AWS Region

   ```
   $ aws --region "<value_of_AWS_Region>" ec2 describe-availability-zones \
       --query 'AvailabilityZones[].[{ZoneName: ZoneName, GroupName: GroupName, Status:
   OptInStatus}]' \
       --filters Name=zone-type,Values=wavelength-zone \
       --all-availability-zones
   ```

   Depending on the AWS Region, the list of available zones might be long. The command returns the following fields:

   `ZoneName`
   The name of the Wavelength Zones.

   `GroupName`
   The group that comprises the zone. To opt in to the Region, save the name.

   `Status`
   The status of the Wavelength Zones group. If the status is `not-opted-in`, you must opt in the `GroupName` as described in the next step.

2. Opt in to the zone group on your AWS account by running the following command:

```
$ aws ec2 modify-availability-zone-group \
    --group-name "<value_of_GroupName>" \
    --opt-in-status opted-in
```

<value_of_GroupName>
: Replace with the name of the group of the Wavelength Zones where you want to create subnets. As an example for Wavelength Zones, specify `us-east-1-wl1` to use the zone `us-east-1-wl1-nyc-wlz-1` (US East New York).

3.9.5. Obtaining an AWS Marketplace image

If you are deploying an OpenShift Container Platform cluster by using an Amazon Web Services (AWS) Marketplace image, you must first subscribe through AWS. Subscribing to the offer provides you with the Amazon Machine Image (AMI) ID that the installation program uses to deploy compute nodes.

You should only modify the RHCOS image for compute machines to use an AWS Marketplace image. Control plane machines and infrastructure nodes do not require an OpenShift Container Platform subscription and use the public RHCOS default image by default, which does not incur subscription costs on your AWS bill. Therefore, you should not modify the cluster default boot image or the control plane boot images. Applying the AWS Marketplace image to them will incur additional licensing costs that cannot be recovered.

Prerequisites

You have an AWS account to buy the offer. This account does not have to be the same account that you use to install the cluster.

Procedure

1. Complete the OpenShift Container Platform subscription from the AWS Marketplace.

2. Record the AMI ID for your specific AWS region. As part of the installation process, you must update the `install-config.yaml` file with this value before deploying the cluster.

   Sample `install-config.yaml` file with AWS Marketplace compute nodes

```
apiVersion: v1
baseDomain: example.com
compute:
- hyperthreading: Enabled
  name: worker
  platform:
    aws:
      amiID: ami-06c4d345f7c207239
      type: m5.4xlarge
  replicas: 3
metadata:
  name: test-cluster
platform:
  aws:
    region: us-east-2
sshKey: ssh-ed25519 AAAA...
pullSecret: '{"auths": ...}'
```

   where:

   `compute.platform.aws.amiID`
   Specifies the AMI ID from your AWS Marketplace subscription.

   `platform.aws.region`
   Specifies the `platform.aws.region` parameter. Your AMI ID is associated with a specific AWS region. When creating the installation configuration file, ensure that you select the same AWS region that you specified when configuring your subscription.

3.9.6. Minimum resource requirements for cluster installation

To ensure that your OpenShift Container Platform cluster runs as expected, each cluster machine must meet minimum CPU, memory, and storage requirements.

Machine	Operating system	vCPU	Virtual RAM	Storage	Input/Output Per Second (IOPS)
Bootstrap	RHCOS	4	16 GB	100 GB	300
Control plane	RHCOS	4	16 GB	100 GB	300
Compute	RHCOS	2	8 GB	100 GB	300

Table 3.22. Minimum resource requirements

- One vCPU is equal to one physical core when simultaneous multithreading (SMT), or Hyper-Threading, is not enabled. When enabled, use the following formula to calculate the corresponding ratio: (threads per core × cores) × sockets = vCPUs.

- OpenShift Container Platform and Kubernetes are sensitive to disk performance, and Red Hat recommends faster storage, particularly for etcd on the control plane nodes which require a 10 ms p99 fsync duration. On many cloud platforms, storage size and IOPS scale together, so you might need to provision more storage to get enough performance.
- As with all user-provisioned installations, if you choose to use RHEL compute machines in your cluster, you take responsibility for all operating system life cycle management and maintenance, including performing system updates, applying patches, and completing all other required tasks. OpenShift Container Platform 4.10 and later do not support RHEL 7 compute machines.

In OpenShift Container Platform version 4.22, RHCOS uses RHEL version 9.8, which updates the micro-architecture requirements. Each architecture requires the following minimum instruction set architectures (ISA):

- x86-64 architecture requires x86-64-v2 ISA
- ARM64 architecture requires ARMv8.0-A ISA
- ppc64le architecture requires IBM® Power9 ISA
- s390x architecture requires IBM® z14 ISA

For more information, see Architectures in the RHEL documentation.

If an instance type for your platform meets the minimum requirements for cluster machines, it is supported to use in OpenShift Container Platform.

3.9.7. Tested instance types for AWS

To ensure cluster stability and performance, use one of the tested Amazon Web Services (AWS) instance types for your OpenShift Container Platform machines.

The following AWS instance types have been tested with OpenShift Container Platform for use with AWS Wavelength Zones.

Use the machine types included in the following charts for your AWS instances. If you use an instance type that is not listed in the chart, ensure that the instance size you use matches the minimum resource requirements in "Minimum resource requirements for cluster installation".

See the following machine types based on 64-bit x86 architecture for AWS Wavelength Zones:

- `r5.*`
- `t3.*`

3.9.8. Creating the installation configuration file

Generate and customize the installation configuration file that the installation program needs to deploy your cluster.

Prerequisites

- You obtained the OpenShift Container Platform installation program and the pull secret for your cluster.
- You checked that you are deploying your cluster to an Amazon Web Services (AWS) Region with an accompanying Red Hat Enterprise Linux CoreOS (RHCOS) AMI published by Red Hat. If you are deploying to an AWS Region that requires a custom AMI, such as an AWS GovCloud Region, you must create the `install-config.yaml` file manually.

Procedure

1. Create the `install-config.yaml` file.

    a. Change to the directory that contains the installation program and run the following command:

    ```
    $ ./openshift-install create install-config --dir <installation_directory>
    ```

    For `<installation_directory>`, specify the directory name to store the files that the installation program creates.

    Specify an empty directory. Some installation assets, such as bootstrap X.509 certificates have short expiration intervals, so you must not reuse an installation directory. If you want to reuse individual files from another cluster installation, you can copy them into your directory. However, the file names for the installation assets might change between releases. Use caution when copying installation files from an earlier OpenShift Container Platform version.

    b. At the prompts, provide the configuration details for your cloud:

        i. Optional: Select an SSH key to use to access your cluster machines.

        For production OpenShift Container Platform clusters on which you want to perform installation debugging or disaster recovery, specify an SSH key that your `ssh-agent` process uses.

        ii. Select **aws** as the platform to target.
        iii. If you do not have an AWS profile stored on your computer, enter the AWS access key ID and secret access key for the user that you configured to run the installation program.

        The AWS access key ID and secret access key are stored in `~/.aws/credentials` in the home directory of the current user on the installation host. You are prompted for the credentials by the installation program if the credentials for the exported profile are not present in the file. Any credentials that you provide to the installation program are stored in the file.

        iv. Select the AWS Region to deploy the cluster to.
        v. Select the base domain for the Route 53 service that you configured for your cluster.
        vi. Enter a descriptive name for your cluster.
        vii. Paste the pull secret from Red Hat OpenShift Cluster Manager.

2. Optional: Back up the `install-config.yaml` file.

    The `install-config.yaml` file is consumed during the installation process. If you want to reuse the file, you must back it up now.

3.9.9. Examples of installation configuration files with edge compute pools

See an example `install-config.yaml` file to better understand configurations for edge compute pools.

The following examples show `install-config.yaml` files that contain an edge machine pool configuration.

Configuration that uses an edge pool with a custom instance type

```
apiVersion: v1
baseDomain: devcluster.openshift.com
metadata:
  name: ipi-edgezone
compute:
- name: edge
```

```
  platform:
    aws:
      type: r5.2xlarge
platform:
  aws:
    region: us-west-2
pullSecret: '{"auths": ...}'
sshKey: ssh-ed25519 AAAA...
```

Instance types differ between locations. To verify availability in the Wavelength Zones in which the cluster runs, see the AWS documentation.

Configuration that uses an edge pool with custom security groups

```
apiVersion: v1
baseDomain: devcluster.openshift.com
metadata:
  name: ipi-edgezone
compute:
- name: edge
  platform:
    aws:
      additionalSecurityGroupIDs:
        - sg-1
        - sg-2
platform:
  aws:
    region: us-west-2
pullSecret: '{"auths": ...}'
sshKey: ssh-ed25519 AAAA...
```

For the `additionalSecurityGroupIDs` parameter, specify the name of the security group as it is displayed on the Amazon EC2 console. Ensure that you include the `sg` prefix.

3.9.10. Cluster installation options for an AWS Wavelength Zones environment

You can install an OpenShift Container Platform cluster on AWS with edge compute nodes defined in Wavelength Zones.

Choose one of the following installation options:

- Fully automated option: Installing a cluster to quickly extend compute nodes to edge compute pools, where the installation program automatically creates infrastructure resources for the OpenShift Container Platform cluster.
- Existing VPC option: Installing a cluster on AWS into an existing VPC, where you supply Wavelength Zones subnets to the `install-config.yaml` file.

3.9.11. Install a cluster quickly in AWS Wavelength Zones

You can quickly install a cluster on Amazon Web Services (AWS) to extend compute nodes to Wavelength Zones locations.

By using this installation route, the installation program automatically creates network resources and Wavelength Zones subnets for each zone that you defined in your configuration file. To customize the installation, you must modify parameters in the `install-config.yaml` file before you deploy the cluster.

3.9.11.1. Modifying an installation configuration file to use Amazon Web Services (AWS) Wavelength Zones

Modify an `install-config.yaml` file to include AWS Wavelength Zones.

Prerequisites

- You have configured an AWS account.
- You added your AWS keys and AWS Region to your local AWS profile by running `aws configure`.
- You are familiar with the configuration limitations that apply when you specify the installation program to automatically create subnets for your OpenShift Container Platform cluster.
- You opted in to the Wavelength Zones group for each zone.
- You created an `install-config.yaml` file by using the procedure "Creating the installation configuration file".

Procedure

1. Modify the `install-config.yaml` file by specifying Wavelength Zones names in the `platform.aws.zones` property of the edge compute pool.

```
platform:
  aws:
    region: <region_name>
compute:
- name: edge
  platform:
    aws:
      zones:
      - <wavelength_zone_name>
```

where:

`platform.aws.region`
: Specifies the AWS Region name.

`compute.platform.aws.zones`
: Specifies the list of Wavelength Zones names to use. The zones must exist in the same AWS Region specified in the `platform.aws.region` field.

  The following example shows a configuration for installing a cluster in the `us-west-2` AWS Region that extends edge nodes to Wavelength Zones in `Los Angeles` and `Las Vegas` locations:

```
apiVersion: v1
baseDomain: example.com
metadata:
  name: cluster-name
platform:
  aws:
    region: us-west-2
compute:
- name: edge
  platform:
    aws:
      zones:
      - us-west-2-wl1-lax-wlz-1
      - us-west-2-wl1-las-wlz-1
pullSecret: '{"auths": ...}'
sshKey: 'ssh-ed25519 AAAA...'
```

2. Deploy your cluster.

3.9.12. Installing a cluster in an existing VPC that has Wavelength Zone subnets

You can install a cluster into an existing Amazon Virtual Private Cloud (VPC) on Amazon Web Services (AWS). The installation program provisions the rest of the required infrastructure, which you can further customize.

To customize the installation, modify parameters in the `install-config.yaml` file before you install the cluster.

Installing a cluster on AWS into an existing VPC requires extending compute nodes to the edge of the Cloud Infrastructure by using AWS Wavelength Zones.

You can use a provided CloudFormation template to create network resources. Additionally, you can modify a template to customize your infrastructure or use the information that they contain to create AWS resources according to your company's policies.

The steps for performing an installer-provisioned infrastructure installation are provided for example purposes only. Installing a cluster in an existing VPC requires that you have knowledge of the cloud provider and the installation process of OpenShift Container Platform. You can use a CloudFormation template to assist you with completing these steps or to help model your own cluster installation. Instead of using the CloudFormation template to create resources, you can decide to use other methods for generating these resources.

3.9.12.1. Creating a VPC in Amazon Web Services (AWS)

You can create a Virtual Private Cloud (VPC), and subnets for all Wavelength Zones locations, in AWS for your OpenShift Container Platform cluster to extend compute nodes to edge locations.

You can further customize your VPC to meet your requirements, including a VPN and route tables. You can also add new Wavelength Zones subnets not included at initial deployment.

You can use the provided CloudFormation template and a custom parameter file to create a stack of AWS resources that represent the VPC.

If you do not use the provided CloudFormation template to create your AWS infrastructure, you must review the provided information and manually create the infrastructure. If your cluster does not initialize correctly, you might have to contact Red Hat support with your installation logs.

Prerequisites

- You configured an AWS account.
- You added your AWS keys and AWS Region to your local AWS profile by running `aws configure`.
- You opted in to the AWS Wavelength Zones on your AWS account.

Procedure

1. Create a JSON file that contains the parameter values that the CloudFormation template requires:

```
[
  {
    "ParameterKey": "VpcCidr",
    "ParameterValue": "10.0.0.0/16"
  },
  {
    "ParameterKey": "AvailabilityZoneCount",
    "ParameterValue": "3"
  },
  {
    "ParameterKey": "SubnetBits",
    "ParameterValue": "12"
  }
]
```

   where:

   `VpcCidr`
   Specifies the CIDR block for the VPC. Use a CIDR block in the format `x.x.x.x/16-24`.

   `AvailabilityZoneCount`
   Specifies the number of availability zones to deploy the VPC in. Use an integer between `1` and `3`.

   `SubnetBits`
   Specifies the size of each subnet in each availability zone. Use an integer between `5` and `13`, where `5` is `/27` and `13` is `/19`.

2. Go to the section of the documentation named "CloudFormation template for the VPC", and then copy the syntax from the provided template. Save the copied template syntax as a YAML file on your local system. This template describes the VPC that your cluster requires.
3. Launch the CloudFormation template to create a stack of AWS resources that represent the VPC by running the following command:

   You must enter the command on a single line.

```
$ aws cloudformation create-stack --stack-name <name> \
     --template-body file://<template>.yaml \
     --parameters file://<parameters>.json
```

   where:

   `--stack-name <name>`
   Specifies the name for the CloudFormation stack, such as `cluster-vpc`. You need the name of this stack if you remove the cluster.

   `--template-body file://<template>.yaml`
   Specifies the relative path to and name of the CloudFormation template YAML file that you saved.

   `--parameters file://<parameters>.json`
   Specifies the relative path to and name of the CloudFormation parameters JSON file.

   Example output

```
arn:aws:cloudformation:us-east-1:123456789012:stack/cluster-vpc/dbedae40-2fd3-11eb-820e-12a48460849f
```

4. Confirm that the template components exist by running the following command:

```
$ aws cloudformation describe-stacks --stack-name <name>
```

After the `StackStatus` displays `CREATE_COMPLETE`, the output displays values for the following parameters. You must provide these parameter values to the other CloudFormation templates that you run to create your cluster.

`VpcId`	The ID of your VPC.
`PublicSubnetIds`	The IDs of the new public subnets.
`PrivateSubnetIds`	The IDs of the new private subnets.
`PublicRouteTableId`	The ID of the new public route table ID.

3.9.12.2. CloudFormation template for the VPC

You can use a CloudFormation template to deploy the VPC that you need for your OpenShift Container Platform cluster.

See the following template:

Example 3.3. CloudFormation template for the VPC

```
AWSTemplateFormatVersion: 2010-09-09
Description: Template for Best Practice VPC with 1-3 AZs

Parameters:
  VpcCidr:
    AllowedPattern: ^(([0-9]|[1-9][0-9]|1[0-9]{2}|2[0-4][0-9]|25[0-5])\.){3}([0-9]|[1-9][0-9]|1[0-
9]{2}|2[0-4][0-9]|25[0-5])(\/(1[6-9]|2[0-4]))$
    ConstraintDescription: CIDR block parameter must be in the form x.x.x.x/16-24.
    Default: 10.0.0.0/16
    Description: CIDR block for VPC.
    Type: String
  AvailabilityZoneCount:
    ConstraintDescription: "The number of availability zones. (Min: 1, Max: 3)"
    MinValue: 1
    MaxValue: 3
    Default: 1
    Description: "How many AZs to create VPC subnets for. (Min: 1, Max: 3)"
    Type: Number
  SubnetBits:
    ConstraintDescription: CIDR block parameter must be in the form x.x.x.x/19-27.
    MinValue: 5
    MaxValue: 13
    Default: 12
    Description: "Size of each subnet to create within the availability zones. (Min: 5 = /27, Max:
13 = /19)"
    Type: Number

Metadata:
  AWS::CloudFormation::Interface:
    ParameterGroups:
    - Label:
        default: "Network Configuration"
      Parameters:
```

```
      - VpcCidr
      - SubnetBits
    - Label:
        default: "Availability Zones"
      Parameters:
      - AvailabilityZoneCount
    ParameterLabels:
      AvailabilityZoneCount:
        default: "Availability Zone Count"
      VpcCidr:
        default: "VPC CIDR"
      SubnetBits:
        default: "Bits Per Subnet"

Conditions:
  DoAz3: !Equals [3, !Ref AvailabilityZoneCount]
  DoAz2: !Or [!Equals [2, !Ref AvailabilityZoneCount], Condition: DoAz3]

Resources:
  VPC:
    Type: "AWS::EC2::VPC"
    Properties:
      EnableDnsSupport: "true"
      EnableDnsHostnames: "true"
      CidrBlock: !Ref VpcCidr
  PublicSubnet:
    Type: "AWS::EC2::Subnet"
    Properties:
      VpcId: !Ref VPC
      CidrBlock: !Select [0, !Cidr [!Ref VpcCidr, 6, !Ref SubnetBits]]
      AvailabilityZone: !Select
      - 0
      - Fn::GetAZs: !Ref "AWS::Region"
  PublicSubnet2:
    Type: "AWS::EC2::Subnet"
    Condition: DoAz2
    Properties:
      VpcId: !Ref VPC
      CidrBlock: !Select [1, !Cidr [!Ref VpcCidr, 6, !Ref SubnetBits]]
      AvailabilityZone: !Select
      - 1
      - Fn::GetAZs: !Ref "AWS::Region"
  PublicSubnet3:
    Type: "AWS::EC2::Subnet"
    Condition: DoAz3
    Properties:
      VpcId: !Ref VPC
      CidrBlock: !Select [2, !Cidr [!Ref VpcCidr, 6, !Ref SubnetBits]]
      AvailabilityZone: !Select
      - 2
      - Fn::GetAZs: !Ref "AWS::Region"
  InternetGateway:
    Type: "AWS::EC2::InternetGateway"
  GatewayToInternet:
    Type: "AWS::EC2::VPCGatewayAttachment"
    Properties:
      VpcId: !Ref VPC
      InternetGatewayId: !Ref InternetGateway
  PublicRouteTable:
    Type: "AWS::EC2::RouteTable"
    Properties:
      VpcId: !Ref VPC
  PublicRoute:
    Type: "AWS::EC2::Route"
```

```
  DependsOn: GatewayToInternet
  Properties:
    RouteTableId: !Ref PublicRouteTable
    DestinationCidrBlock: 0.0.0.0/0
    GatewayId: !Ref InternetGateway
PublicSubnetRouteTableAssociation:
  Type: "AWS::EC2::SubnetRouteTableAssociation"
  Properties:
    SubnetId: !Ref PublicSubnet
    RouteTableId: !Ref PublicRouteTable
PublicSubnetRouteTableAssociation2:
  Type: "AWS::EC2::SubnetRouteTableAssociation"
  Condition: DoAz2
  Properties:
    SubnetId: !Ref PublicSubnet2
    RouteTableId: !Ref PublicRouteTable
PublicSubnetRouteTableAssociation3:
  Condition: DoAz3
  Type: "AWS::EC2::SubnetRouteTableAssociation"
  Properties:
    SubnetId: !Ref PublicSubnet3
    RouteTableId: !Ref PublicRouteTable
PrivateSubnet:
  Type: "AWS::EC2::Subnet"
  Properties:
    VpcId: !Ref VPC
    CidrBlock: !Select [3, !Cidr [!Ref VpcCidr, 6, !Ref SubnetBits]]
    AvailabilityZone: !Select
    - 0
    - Fn::GetAZs: !Ref "AWS::Region"
PrivateRouteTable:
  Type: "AWS::EC2::RouteTable"
  Properties:
    VpcId: !Ref VPC
PrivateSubnetRouteTableAssociation:
  Type: "AWS::EC2::SubnetRouteTableAssociation"
  Properties:
    SubnetId: !Ref PrivateSubnet
    RouteTableId: !Ref PrivateRouteTable
NAT:
  DependsOn:
  - GatewayToInternet
  Type: "AWS::EC2::NatGateway"
  Properties:
    AllocationId:
      "Fn::GetAtt":
      - EIP
      - AllocationId
    SubnetId: !Ref PublicSubnet
EIP:
  Type: "AWS::EC2::EIP"
  Properties:
    Domain: vpc
Route:
  Type: "AWS::EC2::Route"
  Properties:
    RouteTableId:
      Ref: PrivateRouteTable
    DestinationCidrBlock: 0.0.0.0/0
    NatGatewayId:
      Ref: NAT
PrivateSubnet2:
  Type: "AWS::EC2::Subnet"
  Condition: DoAz2
```

```
  Properties:
    VpcId: !Ref VPC
    CidrBlock: !Select [4, !Cidr [!Ref VpcCidr, 6, !Ref SubnetBits]]
    AvailabilityZone: !Select
    - 1
    - Fn::GetAZs: !Ref "AWS::Region"
PrivateRouteTable2:
  Type: "AWS::EC2::RouteTable"
  Condition: DoAz2
  Properties:
    VpcId: !Ref VPC
PrivateSubnetRouteTableAssociation2:
  Type: "AWS::EC2::SubnetRouteTableAssociation"
  Condition: DoAz2
  Properties:
    SubnetId: !Ref PrivateSubnet2
    RouteTableId: !Ref PrivateRouteTable2
NAT2:
  DependsOn:
  - GatewayToInternet
  Type: "AWS::EC2::NatGateway"
  Condition: DoAz2
  Properties:
    AllocationId:
      "Fn::GetAtt":
      - EIP2
      - AllocationId
    SubnetId: !Ref PublicSubnet2
EIP2:
  Type: "AWS::EC2::EIP"
  Condition: DoAz2
  Properties:
    Domain: vpc
Route2:
  Type: "AWS::EC2::Route"
  Condition: DoAz2
  Properties:
    RouteTableId:
      Ref: PrivateRouteTable2
    DestinationCidrBlock: 0.0.0.0/0
    NatGatewayId:
      Ref: NAT2
PrivateSubnet3:
  Type: "AWS::EC2::Subnet"
  Condition: DoAz3
  Properties:
    VpcId: !Ref VPC
    CidrBlock: !Select [5, !Cidr [!Ref VpcCidr, 6, !Ref SubnetBits]]
    AvailabilityZone: !Select
    - 2
    - Fn::GetAZs: !Ref "AWS::Region"
PrivateRouteTable3:
  Type: "AWS::EC2::RouteTable"
  Condition: DoAz3
  Properties:
    VpcId: !Ref VPC
PrivateSubnetRouteTableAssociation3:
  Type: "AWS::EC2::SubnetRouteTableAssociation"
  Condition: DoAz3
  Properties:
    SubnetId: !Ref PrivateSubnet3
    RouteTableId: !Ref PrivateRouteTable3
NAT3:
  DependsOn:
```

```
    - GatewayToInternet
    Type: "AWS::EC2::NatGateway"
    Condition: DoAz3
    Properties:
      AllocationId:
        "Fn::GetAtt":
        - EIP3
        - AllocationId
      SubnetId: !Ref PublicSubnet3
  EIP3:
    Type: "AWS::EC2::EIP"
    Condition: DoAz3
    Properties:
      Domain: vpc
  Route3:
    Type: "AWS::EC2::Route"
    Condition: DoAz3
    Properties:
      RouteTableId:
        Ref: PrivateRouteTable3
      DestinationCidrBlock: 0.0.0.0/0
      NatGatewayId:
        Ref: NAT3
  S3Endpoint:
    Type: AWS::EC2::VPCEndpoint
    Properties:
      PolicyDocument:
        Version: 2012-10-17
        Statement:
        - Effect: Allow
          Principal: '*'
          Action:
          - '*'
          Resource:
          - '*'
      RouteTableIds:
      - !Ref PublicRouteTable
      - !Ref PrivateRouteTable
      - !If [DoAz2, !Ref PrivateRouteTable2, !Ref "AWS::NoValue"]
      - !If [DoAz3, !Ref PrivateRouteTable3, !Ref "AWS::NoValue"]
      ServiceName: !Join
      - ''
      - - com.amazonaws.
        - !Ref 'AWS::Region'
        - .s3
      VpcId: !Ref VPC

Outputs:
  VpcId:
    Description: ID of the new VPC.
    Value: !Ref VPC
  PublicSubnetIds:
    Description: Subnet IDs of the public subnets.
    Value:
      !Join [
        ",",
        [!Ref PublicSubnet, !If [DoAz2, !Ref PublicSubnet2, !Ref "AWS::NoValue"], !If [DoAz3, !Ref
PublicSubnet3, !Ref "AWS::NoValue"]]
      ]
  PrivateSubnetIds:
    Description: Subnet IDs of the private subnets.
    Value:
      !Join [
        ",",
```

```
        [!Ref PrivateSubnet, !If [DoAz2, !Ref PrivateSubnet2, !Ref "AWS::NoValue"], !If [DoAz3,
!Ref PrivateSubnet3, !Ref "AWS::NoValue"]]
      ]
  PublicRouteTableId:
    Description: Public Route table ID
    Value: !Ref PublicRouteTable
  PrivateRouteTableIds:
    Description: Private Route table IDs
    Value:
      !Join [
        ",",
        [
          !Join ["=", [
            !Select [0, "Fn::GetAZs": !Ref "AWS::Region"],
            !Ref PrivateRouteTable
          ]],
          !If [DoAz2,
               !Join ["=", [!Select [1, "Fn::GetAZs": !Ref "AWS::Region"], !Ref
PrivateRouteTable2]],
               !Ref "AWS::NoValue"
          ],
          !If [DoAz3,
               !Join ["=", [!Select [2, "Fn::GetAZs": !Ref "AWS::Region"], !Ref
PrivateRouteTable3]],
               !Ref "AWS::NoValue"
          ]
        ]
      ]
```

3.9.12.3. Creating a VPC carrier gateway

To use public subnets in your OpenShift Container Platform cluster that runs on Wavelength Zones, you must create the carrier gateway and associate the carrier gateway to the VPC. Subnets are useful for deploying load balancers or edge compute nodes.

To create edge nodes or internet-facing load balancers in Wavelength Zones locations for your OpenShift Container Platform cluster, you must create the following required network components:

- A carrier gateway that associates to the existing VPC.
- A carrier route table that lists route entries.
- A subnet that associates to the carrier route table.

Carrier gateways exist for VPCs that only contain subnets in a Wavelength Zone.

The following list explains the functions of a carrier gateway in the context of an AWS Wavelength Zones location:

- Provides connectivity between your Wavelength Zone and the carrier network, which includes any available devices from the carrier network.
- Performs Network Address Translation (NAT) functions, such as translating IP addresses that are public IP addresses stored in a network border group, from Wavelength Zones to carrier IP addresses. These translation functions apply to inbound and outbound traffic.
- Authorizes inbound traffic from a carrier network that is located in a specific location.
- Authorizes outbound traffic to a carrier network and the internet.

No inbound connection configuration exists from the internet to a Wavelength Zone through the carrier gateway.

You can use the provided CloudFormation template to create a stack of the following AWS resources:

- One carrier gateway that associates to the VPC ID in the template.
- One public route table for the Wavelength Zone named as `<ClusterName>-public-carrier`.
- Default IPv4 route entry in the new route table that targets the carrier gateway.
- VPC gateway endpoint for an AWS Simple Storage Service (S3).

If you do not use the provided CloudFormation template to create your AWS infrastructure, you must review the provided information and manually create the infrastructure. If your cluster does not initialize correctly, you might have to contact Red Hat support with your installation logs.

Prerequisites

- You configured an AWS account.
- You added your AWS keys and region to your local AWS profile by running `aws configure`.

Procedure

1. Go to the next section of the documentation named "CloudFormation template for the VPC Carrier Gateway", and then copy the syntax from the **CloudFormation template for VPC Carrier Gateway** template. Save the copied template syntax as a YAML file on your local system. This template describes the VPC that your cluster requires.
2. Run the following command to deploy the CloudFormation template, which creates a stack of AWS resources that represent the VPC:

   ```
   $ aws cloudformation create-stack --stack-name <stack_name> \
     --region ${CLUSTER_REGION} \
     --template-body file://<template>.yaml \
     --parameters \//
       ParameterKey=VpcId,ParameterValue="${VpcId}" \
       ParameterKey=ClusterName,ParameterValue="${ClusterName}"
   ```

   where

   `<stack_name>`
   The name for the CloudFormation stack, such as `clusterName-vpc-carrier-gw`. You need the name of this stack if you remove the cluster.

   `<template>`
   The relative path and the name of the CloudFormation template YAML file that you saved.

   `<VpcId>`
   The VPC ID extracted from the CloudFormation stack output created in the section named "Creating a VPC in AWS".

   `<ClusterName>`
   A custom value that prefixes to resources that the CloudFormation stack creates. You can use the same name that is defined in the `metadata.name` section of the `install-config.yaml` configuration file.

   Example output

   ```
   arn:aws:cloudformation:us-east-1:123456789012:stack/<stack_name>/dbedae40-2fd3-11eb-820e-12a48460849f
   ```

Verification

Confirm that the CloudFormation template components exist by running the following command:

```
$ aws cloudformation describe-stacks --stack-name <stack_name>
```

After the `StackStatus` displays `CREATE_COMPLETE`, the output displays values for the following parameter. Ensure that you provide the parameter value to the other CloudFormation templates that you run to create for your cluster.

`PublicRouteTableId`	The ID of the Route Table in the Carrier infrastructure.

3.9.12.4. CloudFormation template for the VPC Carrier Gateway

Use the CloudFormation template to deploy the Carrier Gateway on Amazon Web Services (AWS) Wavelength infrastructure. The template automates the creation of an AWS Carrier Gateway for OpenShift Container Platform.

The template provisions an `AWS::EC2::CarrierGateway` and associates it with the cluster VPC to enable traffic routing to provide direct internet connectivity for resources in Wavelength Zones that use carrier networks.

CloudFormation template for VPC Carrier Gateway

```
AWSTemplateFormatVersion: 2010-09-09
Description: Template for Creating Wavelength Zone Gateway (Carrier Gateway).

Parameters:
  VpcId:
    Description: VPC ID to associate the Carrier Gateway.
    Type: String
    AllowedPattern: ^(?:(?:vpc)(?:-[a-zA-Z0-9]+)?\b|(?:[0-9]{1,3}\.){3}[0-9]{1,3})$
    ConstraintDescription: VPC ID must be with valid name, starting with vpc-.*.
  ClusterName:
    Description: Cluster Name or Prefix name to prepend the tag Name for each subnet.
    Type: String
    AllowedPattern: ".+"
    ConstraintDescription: ClusterName parameter must be specified.

Resources:
  CarrierGateway:
    Type: "AWS::EC2::CarrierGateway"
    Properties:
      VpcId: !Ref VpcId
      Tags:
      - Key: Name
        Value: !Join ['-', [!Ref ClusterName, "cagw"]]

  PublicRouteTable:
    Type: "AWS::EC2::RouteTable"
    Properties:
      VpcId: !Ref VpcId
      Tags:
      - Key: Name
        Value: !Join ['-', [!Ref ClusterName, "public-carrier"]]

  PublicRoute:
    Type: "AWS::EC2::Route"
    DependsOn: CarrierGateway
    Properties:
      RouteTableId: !Ref PublicRouteTable
      DestinationCidrBlock: 0.0.0.0/0
```

```
      CarrierGatewayId: !Ref CarrierGateway

  S3Endpoint:
    Type: AWS::EC2::VPCEndpoint
    Properties:
      PolicyDocument:
        Version: 2012-10-17
        Statement:
        - Effect: Allow
          Principal: '*'
          Action:
          - '*'
          Resource:
          - '*'
      RouteTableIds:
      - !Ref PublicRouteTable
      ServiceName: !Join
      - ''
      - - com.amazonaws.
        - !Ref 'AWS::Region'
        - .s3
      VpcId: !Ref VpcId

Outputs:
  PublicRouteTableId:
    Description: Public Route table ID
    Value: !Ref PublicRouteTable
```

3.9.12.5. Creating subnets in Wavelength Zones

Before you configure a machine set for edge compute nodes in your OpenShift Container Platform cluster, you must create the subnets in Wavelength Zones. Use the provided CloudFormation template to provision public and private subnets for each Wavelength Zone where you want to deploy compute nodes.

You can use the provided CloudFormation template and create a CloudFormation stack. You can then use this stack to custom provision a subnet.

If you do not use the provided CloudFormation template to create your Amazon Web Services (AWS) infrastructure, you must review the provided information and manually create the infrastructure. If your cluster does not initialize correctly, you might have to contact Red Hat support with your installation logs.

Prerequisites

- You configured an AWS account.
- You added your AWS keys and region to your local AAWS profile by running `aws configure`.
- You opted in to the Wavelength Zones group.

Procedure

1. Go to the section of the documentation named "CloudFormation template for the VPC subnet", and copy the syntax from the template. Save the copied template syntax as a YAML file on your local system. This template describes the VPC that your cluster requires.

2. Run the following command to deploy the CloudFormation template, which creates a stack of AWS resources that represent the VPC:

```
$ aws cloudformation create-stack --stack-name <stack_name> \
  --region ${CLUSTER_REGION} \
  --template-body file://<template>.yaml \
  --parameters \
    ParameterKey=VpcId,ParameterValue="${VPC_ID}" \
    ParameterKey=ClusterName,ParameterValue="${CLUSTER_NAME}" \
    ParameterKey=ZoneName,ParameterValue="${ZONE_NAME}" \
    ParameterKey=PublicRouteTableId,ParameterValue="${ROUTE_TABLE_PUB}" \
    ParameterKey=PublicSubnetCidr,ParameterValue="${SUBNET_CIDR_PUB}" \
    ParameterKey=PrivateRouteTableId,ParameterValue="${ROUTE_TABLE_PVT}" \
    ParameterKey=PrivateSubnetCidr,ParameterValue="${SUBNET_CIDR_PVT}"
```

where:

<stack_name>
: Specifies the name for the CloudFormation stack, such as `cluster-wl-<wavelength_zone_shortname>`. You need the name of this stack if you remove the cluster.

<template>
: Specifies the relative path and the name of the CloudFormation template YAML file that you saved.

${VPC_ID}
: Specifies the VPC ID, which is the value `VpcID` in the output of the CloudFormation template for the VPC.

${CLUSTER_NAME}
: Specifies the value of `ClusterName` to be used as a prefix of the new AWS resource names.

${ZONE_NAME}
: Specifies the value of the Wavelength Zones name to create the subnets.

${ROUTE_TABLE_PUB}
: Specifies the `PublicRouteTableId` extracted from the output of the VPC's carrier gateway CloudFormation stack.

${SUBNET_CIDR_PUB}
: Specifies a valid CIDR block that is used to create the public subnet. This block must be part of the VPC CIDR block `VpcCidr`.

${ROUTE_TABLE_PVT}
: Specifies the `PrivateRouteTableId` extracted from the output of the VPC's CloudFormation stack.

${SUBNET_CIDR_PVT}
: Specifies a valid CIDR block that is used to create the private subnet. This block must be part of the VPC CIDR block `VpcCidr`.

Example output

```
arn:aws:cloudformation:us-east-1:123456789012:stack/<stack_name>/dbedae40-820e-11eb-2fd3-12a48460849f
```

Verification

Confirm that the template components exist by running the following command:

```
$ aws cloudformation describe-stacks --stack-name <stack_name>
```

After the `StackStatus` displays `CREATE_COMPLETE`, the output displays values for the following parameters. Ensure that you provide these parameter values to the other CloudFormation templates that you run to create for your cluster.

`PublicSubnetId`	The IDs of the public subnet created by the CloudFormation stack.
`PrivateSubnetId`	The IDs of the private subnet created by the CloudFormation stack.

3.9.12.6. CloudFormation template for the VPC subnet

Use the CloudFormation template to deploy the private and public subnets in a zone on Wavelength Zones infrastructure. The template provisions an `AWS::EC2::Subnet` and associates it with a specific Wavelength Zones and VPC route table to reduce latency.

CloudFormation template for VPC subnets

```
AWSTemplateFormatVersion: 2010-09-09
Description: Template for Best Practice Subnets (Public and Private)

Parameters:
  VpcId:
    Description: VPC ID that comprises all the target subnets.
    Type: String
    AllowedPattern: ^(?:(?:vpc)(?:-[a-zA-Z0-9]+)?\b|(?:[0-9]{1,3}\.){3}[0-9]{1,3})$
    ConstraintDescription: VPC ID must be with valid name, starting with vpc-.*.
  ClusterName:
    Description: Cluster name or prefix name to prepend the Name tag for each subnet.
    Type: String
    AllowedPattern: ".+"
    ConstraintDescription: ClusterName parameter must be specified.
  ZoneName:
    Description: Zone Name to create the subnets, such as us-west-2-lax-1a.
    Type: String
    AllowedPattern: ".+"
    ConstraintDescription: ZoneName parameter must be specified.
  PublicRouteTableId:
    Description: Public Route Table ID to associate the public subnet.
    Type: String
    AllowedPattern: ".+"
    ConstraintDescription: PublicRouteTableId parameter must be specified.
  PublicSubnetCidr:
    AllowedPattern: ^(([0-9]|[1-9][0-9]|1[0-9]{2}|2[0-4][0-9]|25[0-5])\.){3}([0-9]|[1-9][0-9]|1[0-
9]{2}|2[0-4][0-9]|25[0-5])(\/(1[6-9]|2[0-4]))$
    ConstraintDescription: CIDR block parameter must be in the form x.x.x.x/16-24.
    Default: 10.0.128.0/20
    Description: CIDR block for public subnet.
    Type: String
  PrivateRouteTableId:
    Description: Private Route Table ID to associate the private subnet.
    Type: String
    AllowedPattern: ".+"
    ConstraintDescription: PrivateRouteTableId parameter must be specified.
  PrivateSubnetCidr:
    AllowedPattern: ^(([0-9]|[1-9][0-9]|1[0-9]{2}|2[0-4][0-9]|25[0-5])\.){3}([0-9]|[1-9][0-9]|1[0-
9]{2}|2[0-4][0-9]|25[0-5])(\/(1[6-9]|2[0-4]))$
    ConstraintDescription: CIDR block parameter must be in the form x.x.x.x/16-24.
```

```
    Default: 10.0.128.0/20
    Description: CIDR block for private subnet.
    Type: String


Resources:
  PublicSubnet:
    Type: "AWS::EC2::Subnet"
    Properties:
      VpcId: !Ref VpcId
      CidrBlock: !Ref PublicSubnetCidr
      AvailabilityZone: !Ref ZoneName
      Tags:
      - Key: Name
        Value: !Join ['-', [!Ref ClusterName, "public", !Ref ZoneName]]

  PublicSubnetRouteTableAssociation:
    Type: "AWS::EC2::SubnetRouteTableAssociation"
    Properties:
      SubnetId: !Ref PublicSubnet
      RouteTableId: !Ref PublicRouteTableId

  PrivateSubnet:
    Type: "AWS::EC2::Subnet"
    Properties:
      VpcId: !Ref VpcId
      CidrBlock: !Ref PrivateSubnetCidr
      AvailabilityZone: !Ref ZoneName
      Tags:
      - Key: Name
        Value: !Join ['-', [!Ref ClusterName, "private", !Ref ZoneName]]

  PrivateSubnetRouteTableAssociation:
    Type: "AWS::EC2::SubnetRouteTableAssociation"
    Properties:
      SubnetId: !Ref PrivateSubnet
      RouteTableId: !Ref PrivateRouteTableId

Outputs:
  PublicSubnetId:
    Description: Subnet ID of the public subnets.
    Value:
      !Join ["", [!Ref PublicSubnet]]

  PrivateSubnetId:
    Description: Subnet ID of the private subnets.
    Value:
      !Join ["", [!Ref PrivateSubnet]]
```

3.9.12.7. Modifying an installation configuration file to use Amazon Web Services (AWS) Wavelength Zones subnets

Modify your `install-config.yaml` file to include Wavelength Zones subnets.

Prerequisites

- You created subnets by using the procedure "Creating subnets in Wavelength Zones".
- You created an `install-config.yaml` file by using the procedure "Creating the installation configuration file".

Procedure

Modify the `install-config.yaml` configuration file by specifying Wavelength Zones subnets in the `platform.aws.subnets` parameter.

Example installation configuration file with Wavelength Zones subnets

```
platform:
  aws:
    region: us-west-2
    subnets:
    - publicSubnetId-1
    - publicSubnetId-2
    - publicSubnetId-3
    - privateSubnetId-1
    - privateSubnetId-2
    - privateSubnetId-3
    - publicOrPrivateSubnetID-Wavelength-1
```

`platform.aws.subnets` specifies the list of subnet IDs created in the zones: Availability and Wavelength Zones.

3.9.13. Optional: Assign public IP addresses to edge compute nodes

If your workload requires deploying the edge compute nodes in public subnets on Wavelength Zones infrastructure, you can configure the machine set manifests when installing a cluster.

AWS Wavelength Zones infrastructure accesses the network traffic in a specified zone, so applications can take advantage of lower latency when serving end users that are closer to that zone.

The default setting that deploys compute nodes in private subnets might not meet your needs, so consider creating edge compute nodes in public subnets when you want to apply more customization to your infrastructure.

By default, OpenShift Container Platform deploy the compute nodes in private subnets. For best performance, consider placing compute nodes in subnets that have their Public IP addresses attached to the subnets.

You must create additional security groups, but ensure that you only open the groups' rules over the internet when you really need to.

Procedure

1. Change to the directory that contains the installation program and generate the manifest files. Ensure that the installation manifests get created at the `openshift` and `manifests` directory level.

```
$ ./openshift-install create manifests --dir <installation_directory>
```

2. Edit the machine set manifest that the installation program generates for the Wavelength Zones, so that the manifest gets deployed in public subnets. Specify `true` for the `spec.template.spec.providerSpec.value.publicIP` parameter.

   Example machine set manifest configuration for installing a cluster quickly in Wavelength Zones

```
spec:
  template:
    spec:
      providerSpec:
        value:
          publicIp: true
          subnet:
            filters:
              - name: tag:Name
                values:
                  - ${INFRA_ID}-public-${ZONE_NAME}
```

   Example machine set manifest configuration for installing a cluster in an existing VPC that has Wavelength Zones subnets

```
apiVersion: machine.openshift.io/v1beta1
kind: MachineSet
metadata:
  name: <infrastructure_id>-edge-<zone>
  namespace: openshift-machine-api
spec:
  template:
    spec:
      providerSpec:
        value:
          publicIp: true
```

3.9.14. Deploying the cluster

To deploy your OpenShift Container Platform cluster, you initialize installation by running the `openshift-install create cluster` command from the directory that contains the installation program. The installation program provisions the required infrastructure and completes the cluster setup.

You can run the `create cluster` command of the installation program only once, during initial installation.

Prerequisites

- You have configured an account with the cloud platform that hosts your cluster.
- You have the OpenShift Container Platform installation program and the pull secret for your cluster.
- You have verified that the cloud provider account on your host has the correct permissions to deploy the cluster. An account with incorrect permissions causes the installation process to fail with an error message that displays the missing permissions.

Procedure

1. In the directory that contains the installation program, initialize the cluster deployment by running the following command:

   ```
   $ ./openshift-install create cluster --dir <installation_directory> \
       --log-level=info
   ```

   where:

   - `<installation_directory>`: Specifies the location of your customized `./install-config.yaml` file.
   - `--log-level`: Specifies the log level. To view different installation details, specify `warn`, `debug`, or `error` instead of `info`.
2. Optional: Remove or disable the `AdministratorAccess` policy from the IAM account that you used to install the cluster.

   The elevated permissions provided by the `AdministratorAccess` policy are required only during installation.

Verification

When the cluster deployment completes successfully:

- The terminal displays directions for accessing your cluster, including a link to the web console and credentials for the `kubeadmin` user.
- Credential information also outputs to `<installation_directory>/.openshift_install.log`.

  Do not delete the installation program or the files that the installation program creates. Both are required to delete the cluster.

  The following example shows the expected output:

  ```
  ...
  INFO Install complete!
  INFO To access the cluster as the system:admin user when using 'oc', run 'export
  KUBECONFIG=/home/myuser/install_dir/auth/kubeconfig'
  INFO Access the OpenShift web-console here: https://console-openshift-
  console.apps.mycluster.example.com
  INFO Login to the console with user: "kubeadmin", and password: "password"
  INFO Time elapsed: 36m22s
  ```

  - The Ignition config files that the installation program generates contain certificates that expire after 24 hours, which are then renewed at that time. If the cluster is shut down before renewing the certificates and the cluster is later restarted after the 24 hours have elapsed, the cluster automatically recovers the expired certificates. The exception is that you must manually approve the pending `node-bootstrapper` certificate signing requests (CSRs) to recover kubelet certificates. See the documentation for *Recovering from expired control plane certificates* for more information.
  - It is recommended that you use Ignition config files within 12 hours after they are generated because the 24-hour certificate rotates from 16 to 22 hours after the cluster is installed. By using the Ignition config files within 12 hours, you can avoid installation failure if the certificate update runs during installation.

3.9.15. Logging in to the cluster by using the CLI

To log in to your cluster as the default system user, export the `kubeconfig` file. This configuration enables the CLI to authenticate and connect to the specific API server created during OpenShift Container Platform installation.

The `kubeconfig` file is specific to a cluster and OpenShift Container Platform generates it during installation.

Prerequisites

- You deployed an OpenShift Container Platform cluster.
- You installed the OpenShift CLI (`oc`).

Procedure

1. Export the `kubeadmin` credentials by running the following command:

   ```
   $ export KUBECONFIG=<installation_directory>/auth/kubeconfig
   ```

   where:

   `<installation_directory>`
   Specifies the path to the directory that stores the installation files.

2. Verify you can run `oc` commands successfully using the exported configuration by running the following command:

   ```
   $ oc whoami
   ```

   Example output

   ```
   system:admin
   ```

Next steps

- "Customize your cluster"
- "Remote health reporting"

3.9.16. Logging in to the cluster by using the web console

To verify that your cluster deployed successfully and access its features, log in to the OpenShift Container Platform web console as the `kubeadmin` user.

Prerequisites

- You have access to the installation host.
- You completed a cluster installation and all cluster Operators are available.

Procedure

1. Obtain the password for the `kubeadmin` user from the `kubeadmin-password` file on the installation host:

   ```
   $ cat <installation_directory>/auth/kubeadmin-password
   ```

   Or, you can obtain the `kubeadmin` password from the `<installation_directory>/.openshift_install.log` log file on the installation host.

2. List the OpenShift Container Platform web console route:

```
$ oc get routes -n openshift-console | grep 'console-openshift'
```

Or, you can obtain the OpenShift Container Platform route from the `<installation_directory>/.openshift_install.log` log file on the installation host.

Example output

```
console     console-openshift-console.apps.<cluster_name>.<base_domain>            console
https   reencrypt/Redirect   None
```

3. Navigate to the route detailed in the output of the preceding command in a web browser and log in as the `kubeadmin` user.

3.9.17. Verifying nodes that were created with edge compute pool

After you install a cluster that uses AWS Wavelength Zones infrastructure, check the status of the machine that was created by the machine set manifests created during installation.

Procedure

1. To check the machine sets created from the subnet you added to the `install-config.yaml` file, run the following command:

```
$ oc get machineset -n openshift-machine-api
```

Example output

```
NAME                                        DESIRED   CURRENT   READY   AVAILABLE   AGE
cluster-7xw5g-edge-us-east-1-wl1-nyc-wlz-1   1         1         1       1           3h4m
cluster-7xw5g-worker-us-east-1a              1         1         1       1           3h4m
cluster-7xw5g-worker-us-east-1b              1         1         1       1           3h4m
cluster-7xw5g-worker-us-east-1c              1         1         1       1           3h4m
```

2. To check the machines that were created from the machine sets, run the following command:

```
$ oc get machines -n openshift-machine-api
```

Example output

```
NAME                                         PHASE     TYPE          REGION      ZONE
AGE
cluster-7xw5g-edge-us-east-1-wl1-nyc-wlz-1-wbclh  Running   c5d.2xlarge   us-east-1   us-east-
1-wl1-nyc-wlz-1  3h
cluster-7xw5g-master-0                           Running   m6i.xlarge    us-east-1   us-east-
1a               3h4m
cluster-7xw5g-master-1                           Running   m6i.xlarge    us-east-1   us-east-
1b               3h4m
cluster-7xw5g-master-2                           Running   m6i.xlarge    us-east-1   us-east-
1c               3h4m
cluster-7xw5g-worker-us-east-1a-rtp45            Running   m6i.xlarge    us-east-1   us-east-
1a               3h
cluster-7xw5g-worker-us-east-1b-glm7c            Running   m6i.xlarge    us-east-1   us-east-
1b               3h
cluster-7xw5g-worker-us-east-1c-qfvz4            Running   m6i.xlarge    us-east-1   us-east-
1c               3h
```

3. To check nodes with edge roles, run the following command:

```
$ oc get nodes -l node-role.kubernetes.io/edge
```

Example output

```
NAME                           STATUS   ROLES         AGE    VERSION
ip-10-0-207-188.ec2.internal   Ready    edge,worker   172m   v1.25.2+d2e245f
```

3.10. Extending an AWS VPC cluster into an AWS Outpost

In OpenShift Container Platform version 4.14, you could install a cluster on Amazon Web Services (AWS) with compute nodes running in AWS Outposts as a Technology Preview. As of OpenShift Container Platform version 4.15, this installation method is no longer supported.

Instead, you can install a cluster on AWS into an existing VPC, and provision compute nodes on AWS Outposts as a postinstallation configuration task.

After following the instructions in "Installing a cluster on Amazon Web Services (AWS) into an existing Amazon Virtual Private Cloud (VPC)", you can create a compute machine set that deploys compute machines in AWS Outposts. AWS Outposts is an AWS edge compute service that enables using many features of a cloud-based AWS deployment with the reduced latency of an on-premise environment. For more information, see the "AWS Outposts documentation".

3.10.1. AWS Outposts on OpenShift Container Platform requirements and limitations

You can manage the resources on your AWS Outpost similarly to those on a cloud-based AWS cluster if you configure your OpenShift Container Platform cluster to accommodate several requirements and limitations.

You must accommodate the following requirements and limitations:

- To extend an OpenShift Container Platform cluster on AWS into an Outpost, you must have installed the cluster into an existing Amazon Virtual Private Cloud (VPC).
- The infrastructure of an Outpost is tied to an availability zone in an AWS region and uses a dedicated subnet. Edge compute machines deployed into an Outpost must use the Outpost subnet and the availability zone that the Outpost is tied to.
- When the AWS Kubernetes cloud controller manager discovers an Outpost subnet, it attempts to create service load balancers in the Outpost subnet. AWS Outposts do not support running service load balancers. To prevent the cloud controller manager from creating unsupported services in the Outpost subnet, you must include the `kubernetes.io/cluster/unmanaged` tag in the Outpost subnet configuration. This requirement is a workaround in OpenShift Container Platform version 4.22. For more information, see OCPBUGS-30041.
- OpenShift Container Platform clusters on AWS include the `gp3-csi` and `gp2-csi` storage classes. These classes correspond to Amazon Elastic Block Store (EBS) gp3 and gp2 volumes. OpenShift Container Platform clusters use the `gp3-csi` storage class by default, but AWS Outposts does not support EBS gp3 volumes.
- This implementation uses the `node-role.kubernetes.io/outposts` taint to prevent spreading regular cluster workloads to the Outpost nodes. To schedule user workloads in the Outpost, you must specify a corresponding toleration in the `Deployment` resource for your application. Reserving the AWS Outpost infrastructure for user workloads avoids additional configuration requirements, such as updating the default CSI to `gp2-csi` so that it is compatible.

- To create a volume in the Outpost, the CSI driver requires the Outpost Amazon Resource Name (ARN). The driver uses the topology keys stored on the `CSINode` objects to determine the Outpost ARN. To ensure that the driver uses the correct topology values, you must set the volume binding mode to `WaitForConsumer` and avoid setting allowed topologies on any new storage classes that you create.
- When you extend an AWS VPC cluster into an Outpost, you have two types of compute resources. The Outpost has edge compute nodes, while the VPC has cloud-based compute nodes. The cloud-based AWS Elastic Block volume cannot attach to Outpost edge compute nodes, and the Outpost volumes cannot attach to cloud-based compute nodes.

  As a result, you cannot use CSI snapshots to migrate applications that use persistent storage from cloud-based compute nodes to edge compute nodes or directly use the original persistent volume. To migrate persistent storage data for applications, you must perform a manual backup and restore operation.

- AWS Outposts does not support AWS Network Load Balancers or AWS Classic Load Balancers. You must use AWS Application Load Balancers to enable load balancing for edge compute resources in the AWS Outposts environment.

  To provision an Application Load Balancer, you must use an Ingress resource and install the AWS Load Balancer Operator. If your cluster contains both edge and cloud-based compute instances that share workloads, additional configuration is required.

  For more information, see "Using the AWS Load Balancer Operator in an AWS VPC cluster extended into an Outpost".

3.10.2. Obtaining information about your environment

To extend an AWS VPC cluster to your Outpost, you must provide information about your OpenShift Container Platform cluster and your Outpost environment. You use this information to complete network configuration tasks and configure a compute machine set that creates compute machines in your Outpost.

You can use command-line tools to gather the required details.

3.10.2.1. Obtaining information from your OpenShift Container Platform cluster

You can use the OpenShift CLI (`oc`) to obtain information from your OpenShift Container Platform cluster.

You might find it convenient to store some or all of these values as environment variables by using the `export` command.

Prerequisites

- You have installed an OpenShift Container Platform cluster into a custom VPC on AWS.
- You have access to the cluster using an account with `cluster-admin` permissions.
- You have installed the OpenShift CLI (`oc`).

Procedure

1. List the infrastructure ID for the cluster by running the following command. Retain this value.

```
$ oc get -o jsonpath='{.status.infrastructureName}{"\n"}' infrastructures.config.openshift.io
cluster
```

2. Obtain details about the compute machine sets that the installation program created by running the following commands:

   a. List the compute machine sets on your cluster:

   ```
   $ oc get machinesets.machine.openshift.io -n openshift-machine-api
   ```

   Example output

   ```
   NAME                           DESIRED   CURRENT   READY   AVAILABLE   AGE
   <compute_machine_set_name_1>   1         1         1       1           55m
   <compute_machine_set_name_2>   1         1         1       1           55m
   ```

   b. Display the Amazon Machine Image (AMI) ID for one of the listed compute machine sets. Retain this value.

   ```
   $ oc get machinesets.machine.openshift.io <compute_machine_set_name_1> \
     -n openshift-machine-api \
     -o jsonpath='{.spec.template.spec.providerSpec.value.ami.id}'
   ```

   c. Display the subnet ID for the AWS VPC cluster. Retain this value.

   ```
   $ oc get machinesets.machine.openshift.io <compute_machine_set_name_1> \
     -n openshift-machine-api \
     -o jsonpath='{.spec.template.spec.providerSpec.value.subnet.id}'
   ```

3.10.2.2. Obtaining information from your AWS account

You can use the AWS CLI (`aws`) to obtain information from your AWS account.

You might find it convenient to store some or all of these values as environment variables by using the `export` command.

Prerequisites

- You have an AWS Outposts site with the required hardware setup complete.
- Your Outpost is connected to your AWS account.
- You have access to your AWS account by using the AWS CLI (`aws`) as a user with permissions to perform the required tasks.

Procedure

1. List the Outposts that are connected to your AWS account by running the following command:

   ```
   $ aws outposts list-outposts
   ```

2. Retain the following values from the output of the `aws outposts list-outposts` command:

   - The Outpost ID.
   - The Amazon Resource Name (ARN) for the Outpost.
   - The Outpost availability zone.

     The output of the `aws outposts list-outposts` command includes two values related to the availability zone: `AvailabilityZone` and `AvailabilityZoneId`. You use the `AvailablilityZone` value to configure a compute machine set that creates compute machines in your Outpost.

3. Using the value of the Outpost ID, show the instance types that are available in your Outpost by running the following command. Retain the values of the available instance types.

   ```
   $ aws outposts get-outpost-instance-types \
     --outpost-id <outpost_id_value>
   ```

4. Using the value of the Outpost ARN, show the subnet ID for the Outpost by running the following command. Retain this value.

```
$ aws ec2 describe-subnets \
  --filters Name=outpost-arn,Values=<outpost_arn_value>
```

3.10.3. Configuring your network for your Outpost

To extend your VPC cluster into an Outpost, you must complete two network configuration tasks.

The following network configuration tasks must be completed:

- Change the Cluster Network MTU.
- Create a subnet in your Outpost.

3.10.3.1. Changing the cluster network MTU to support AWS Outposts

You might need to decrease the maximum transmission unit (MTU) value for the cluster network to support an AWS Outposts subnet. During installation, the MTU for the cluster network is detected automatically based on the MTU of the primary network interface of nodes in the cluster.

You cannot roll back an MTU value for nodes during the MTU migration process, but you can roll back the value after the MTU migration process completes.

The migration is disruptive and nodes in your cluster might be temporarily unavailable as the MTU update takes effect.

For more details about the migration process, including important service interruption considerations, see "Changing the MTU for the cluster network".

3.10.3.2. Prerequisites for changing the cluster network MTU

Before you change the cluster network maximum transmission unit (MTU), verify that you have the required access, tools, and network infrastructure to support the new MTU value.

Ensure that the following conditions are met before you begin:

- You have installed the OpenShift CLI (`oc`).
- You have access to the cluster using an account with `cluster-admin` permissions.
- You have identified the target MTU for your cluster. The MTU for the OVN-Kubernetes network plugin must be set to `100` less than the lowest hardware MTU value in your cluster.
- If your nodes are physical machines, ensure that the cluster network and the connected network switches support jumbo frames.
- If your nodes are virtual machines (VMs), ensure that the hypervisor and the connected network switches support jumbo frames.

3.10.3.2.1. Check the current cluster MTU value

To ensure network stability and performance in a hybrid environment where part of your cluster is in the cloud and part is an on-premise environment, you can obtain the current maximum transmission unit (MTU) for the cluster network.

Procedure

To obtain the current MTU for the cluster network, enter the following command:

```
$ oc describe network.config cluster
```

Example output

```
...
Status:
  Cluster Network:
    Cidr:               10.217.0.0/22
    Host Prefix:        23
  Cluster Network MTU:  1400
  Network Type:         OVNKubernetes
  Service Network:
    10.217.4.0/23
...
```

3.10.3.2.2. Begin the MTU migration

Start the maximum transmission unit (MTU) migration by specifying the migration configuration for the cluster network and machine interfaces. The Machine Config Operator performs a rolling reboot of the nodes to prepare the cluster for the MTU change.

Procedure

1. To begin the MTU migration, specify the migration configuration by entering the following command. The Machine Config Operator performs a rolling reboot of the nodes in the cluster in preparation for the MTU change.

```
$ oc patch Network.operator.openshift.io cluster --type=merge --patch \
  '{"spec": { "migration": { "mtu": { "network": { "from": <overlay_from>, "to": <overlay_to>
} , "machine": { "to" : <machine_to> } } } } }'
```

   where:

   <overlay_from>
   Specifies the current cluster network MTU value.

   <overlay_to>
   Specifies the target MTU for the cluster network. This value is set relative to the value of <machine_to>. For OVN-Kubernetes, this value must be 100 less than the value of <machine_to>.

   <machine_to>
   Specifies the MTU for the primary network interface on the underlying host network.

```
$ oc patch Network.operator.openshift.io cluster --type=merge --patch \
  '{"spec": { "migration": { "mtu": { "network": { "from": 1400, "to": 1000 } , "machine": {
"to" : 1100} } } } }'
```

2. As the Machine Config Operator updates machines in each machine config pool, the Operator reboots each node one by one. You must wait until all the nodes are updated. Check the machine config pool status by entering the following command:

```
$ oc get machineconfigpools
```

   A successfully updated node has the following status: UPDATED=true, UPDATING=false, DEGRADED=false.

   By default, the Machine Config Operator updates one machine per pool at a time, causing the total time the migration takes to increase with the size of the cluster.

3.10.3.2.3. Verify the machine configuration

Verify the machine configuration on your hosts to confirm that the maximum transmission unit (MTU) migration applied successfully. Checking the configuration state and system settings help ensures that the nodes use the correct migration script.

Procedure

Confirm the status of the new machine configuration on the hosts:

a. To list the machine configuration state and the name of the applied machine configuration, enter the following command:

```
$ oc describe node | egrep "hostname|machineconfig"
```

Example output

```
kubernetes.io/hostname=master-0
machineconfiguration.openshift.io/currentConfig: rendered-master-
c53e221d9d24e1c8bb6ee89dd3d8ad7b
machineconfiguration.openshift.io/desiredConfig: rendered-master-
c53e221d9d24e1c8bb6ee89dd3d8ad7b
machineconfiguration.openshift.io/reason:
machineconfiguration.openshift.io/state: Done
```

b. Verify that the following statements are true:

- The value of `machineconfiguration.openshift.io/state` field is `Done`.
- The value of the `machineconfiguration.openshift.io/currentConfig` field is equal to the value of the `machineconfiguration.openshift.io/desiredConfig` field.

c. To confirm that the machine config is correct, enter the following command:

```
$ oc get machineconfig <config_name> -o yaml | grep ExecStart
```

where:

`<config_name>`
Specifies the name of the machine config from the `machineconfiguration.openshift.io/currentConfig` field.

The machine config must include the following update to the systemd configuration:

```
ExecStart=/usr/local/bin/mtu-migration.sh
```

3.10.3.2.4. Finalize the MTU migration

Finalize the MTU migration to apply the new maximum transmission unit (MTU) settings to the OVN-Kubernetes network plugin. This updates the cluster configuration and triggers a rolling reboot of the nodes to complete the process.

Procedure

1. To finalize the MTU migration, enter the following command for the OVN-Kubernetes network plugin:

```
$ oc patch Network.operator.openshift.io cluster --type=merge --patch \
  '{"spec": { "migration": null, "defaultNetwork":{ "ovnKubernetesConfig": { "mtu": <mtu>
}}}}'
```

where:

`<mtu>`
Specifies the new cluster network MTU that you specified with `<overlay_to>`.

2. After finalizing the MTU migration, each machine config pool node is rebooted one by one. You must wait until all the nodes are updated. Check the machine config pool status by entering the following command:

   ```
   $ oc get machineconfigpools
   ```

   A successfully updated node has the following status: `UPDATED=true`, `UPDATING=false`, `DEGRADED=false`.

Verification

Verify that the node in your cluster uses the MTU that you specified by entering the following command:

```
$ oc describe network.config cluster
```

3.10.3.3. Creating subnets for AWS edge compute services

You can automate creating subnets, route tables, and Carrier Gateways in AWS AWS Outposts to extend clusters into ultra-low-latency edge locations for edge workloads. Before you configure a machine set for edge compute nodes in your OpenShift Container Platform cluster, you must create a subnet in AWS Outposts.

You can use the provided CloudFormation template and create a CloudFormation stack. You can then use this stack to custom provision a subnet.

If you do not use the provided CloudFormation template to create your AWS infrastructure, you must review the provided information and manually create the infrastructure. If your cluster does not initialize correctly, you might have to contact Red Hat support with your installation logs.

Prerequisites

- You configured an AWS account.
- You added your AWS keys and region to your local AWS profile by running `aws configure`.
- You have obtained the required information about your environment from your OpenShift Container Platform cluster, Outpost, and AWS account.

Procedure

1. Go to the section of the documentation named "CloudFormation template for the VPC subnet", and copy the syntax from the template. Save the copied template syntax as a YAML file on your local system. This template describes the VPC that your cluster requires.

2. Run the following command to deploy the CloudFormation template, which creates a stack of AWS resources that represent the VPC:

```
$ aws cloudformation create-stack --stack-name <stack_name> \
  --region ${CLUSTER_REGION} \
  --template-body file://<template>.yaml \
  --parameters \
    ParameterKey=VpcId,ParameterValue="${VPC_ID}" \
    ParameterKey=ClusterName,ParameterValue="${CLUSTER_NAME}" \
    ParameterKey=ZoneName,ParameterValue="${ZONE_NAME}" \
    ParameterKey=PublicRouteTableId,ParameterValue="${ROUTE_TABLE_PUB}" \
    ParameterKey=PublicSubnetCidr,ParameterValue="${SUBNET_CIDR_PUB}" \
    ParameterKey=PrivateRouteTableId,ParameterValue="${ROUTE_TABLE_PVT}" \
    ParameterKey=PrivateSubnetCidr,ParameterValue="${SUBNET_CIDR_PVT}" \
    ParameterKey=PrivateSubnetLabel,ParameterValue="private-outpost" \
    ParameterKey=PublicSubnetLabel,ParameterValue="public-outpost" \
    ParameterKey=OutpostArn,ParameterValue="${OUTPOST_ARN}"
```

where

<stack_name>
: Specifies the name for the CloudFormation stack, such as `cluster-<outpost_name>`.

<template>
: Specifies the relative path and the name of the CloudFormation template YAML file that you saved.

${VPC_ID}
: Specifies the VPC ID, which is the value `VpcID` in the output of the CloudFormation template for the VPC.

${CLUSTER_NAME}
: Specifies the value of **ClusterName** to be used as a prefix of the new AWS resource names.

${ZONE_NAME}
: Specifies the value of AWS Outposts name to create the subnets.

${ROUTE_TABLE_PUB}
: Specifies the Public Route Table ID created in the `${VPC_ID}` used to associate the public subnets on Outposts. Specify the public route table to associate the Outpost subnet created by this stack.

${SUBNET_CIDR_PUB}
: Specifies a valid CIDR block that is used to create the public subnet. This block must be part of the VPC CIDR block `VpcCidr`.

${OUTPOST_ARN}
: Specifies the Amazon Resource Name (ARN) for the Outpost.

${SUBNET_CIDR_PVT}
: Specifies a valid CIDR block that is used to create the private subnet. This block must be part of the VPC CIDR block `VpcCidr`.

${ROUTE_TABLE_PVT}
: Specifies the Private Route Table ID created in the `${VPC_ID}` used to associate the private subnets on Outposts. Specify the private route table to associate the Outpost subnet created by this stack.

Example output

```
arn:aws:cloudformation:us-east-1:123456789012:stack/<stack_name>/dbedae40-820e-11eb-2fd3-12a48460849f
```

Verification

Confirm that the template components exist by running the following command:

```
$ aws cloudformation describe-stacks --stack-name <stack_name>
```

After the `StackStatus` displays `CREATE_COMPLETE`, the output displays values for the following parameters:

`PublicSubnetId`	The IDs of the public subnet created by the CloudFormation stack.
`PrivateSubnetId`	The IDs of the private subnet created by the CloudFormation stack.

Ensure that you provide these parameter values to the other CloudFormation templates that you run to create for your cluster.

3.10.3.4. CloudFormation template for the VPC subnet

Use the CloudFormation template to deploy the private and public subnets in a zone on AWS Outposts infrastructure. The template provisions an `AWS::EC2::Subnet` and associates it with a specific AWS Outposts and VPC route table to reduce latency.

CloudFormation template for VPC subnets

```
AWSTemplateFormatVersion: 2010-09-09
Description: Template for Best Practice Subnets (Public and Private)

Parameters:
  VpcId:
    Description: VPC ID that comprises all the target subnets.
    Type: String
    AllowedPattern: ^(?:(?:vpc)(?:-[a-zA-Z0-9]+)?\b|(?:[0-9]{1,3}\.){3}[0-9]{1,3})$
    ConstraintDescription: VPC ID must be with valid name, starting with vpc-.*.
  ClusterName:
    Description: Cluster name or prefix name to prepend the Name tag for each subnet.
    Type: String
    AllowedPattern: ".+"
    ConstraintDescription: ClusterName parameter must be specified.
  ZoneName:
    Description: Zone Name to create the subnets, such as us-west-2-lax-1a.
    Type: String
    AllowedPattern: ".+"
    ConstraintDescription: ZoneName parameter must be specified.
  PublicRouteTableId:
    Description: Public Route Table ID to associate the public subnet.
    Type: String
    AllowedPattern: ".+"
    ConstraintDescription: PublicRouteTableId parameter must be specified.
  PublicSubnetCidr:
    AllowedPattern: ^(([0-9]|[1-9][0-9]|1[0-9]{2}|2[0-4][0-9]|25[0-5])\.){3}([0-9]|[1-9][0-9]|1[0-
9]{2}|2[0-4][0-9]|25[0-5])(\/(1[6-9]|2[0-4]))$
    ConstraintDescription: CIDR block parameter must be in the form x.x.x.x/16-24.
    Default: 10.0.128.0/20
    Description: CIDR block for public subnet.
    Type: String
  PrivateRouteTableId:
    Description: Private Route Table ID to associate the private subnet.
    Type: String
    AllowedPattern: ".+"
```

```
    ConstraintDescription: PrivateRouteTableId parameter must be specified.
  PrivateSubnetCidr:
    AllowedPattern: ^(([0-9]|[1-9][0-9]|1[0-9]{2}|2[0-4][0-9]|25[0-5])\.){3}([0-9]|[1-9][0-9]|1[0-
9]{2}|2[0-4][0-9]|25[0-5])(\/(1[6-9]|2[0-4]))$
    ConstraintDescription: CIDR block parameter must be in the form x.x.x.x/16-24.
    Default: 10.0.128.0/20
    Description: CIDR block for private subnet.
    Type: String
  PrivateSubnetLabel:
    Default: "private"
    Description: Subnet label to be added when building the subnet name.
    Type: String
  PublicSubnetLabel:
    Default: "public"
    Description: Subnet label to be added when building the subnet name.
    Type: String
  OutpostArn:
    Default: ""
    Description: OutpostArn when creating subnets on AWS Outpost.
    Type: String

Conditions:
  OutpostEnabled: !Not [!Equals [!Ref "OutpostArn", ""]]

Resources:
  PublicSubnet:
    Type: "AWS::EC2::Subnet"
    Properties:
      VpcId: !Ref VpcId
      CidrBlock: !Ref PublicSubnetCidr
      AvailabilityZone: !Ref ZoneName
      OutpostArn: !If [ OutpostEnabled, !Ref OutpostArn, !Ref "AWS::NoValue"]
      Tags:
      - Key: Name
        Value: !Join ['-', [ !Ref ClusterName, !Ref PublicSubnetLabel, !Ref ZoneName]]
      - Key: kubernetes.io/cluster/unmanaged
        Value: true

  PublicSubnetRouteTableAssociation:
    Type: "AWS::EC2::SubnetRouteTableAssociation"
    Properties:
      SubnetId: !Ref PublicSubnet
      RouteTableId: !Ref PublicRouteTableId

  PrivateSubnet:
    Type: "AWS::EC2::Subnet"
    Properties:
      VpcId: !Ref VpcId
      CidrBlock: !Ref PrivateSubnetCidr
      AvailabilityZone: !Ref ZoneName
      OutpostArn: !If [ OutpostEnabled, !Ref OutpostArn, !Ref "AWS::NoValue"]
      Tags:
      - Key: Name
        Value: !Join ['-', [!Ref ClusterName, !Ref PrivateSubnetLabel, !Ref ZoneName]]
      - Key: kubernetes.io/cluster/unmanaged
        Value: true

  PrivateSubnetRouteTableAssociation:
    Type: "AWS::EC2::SubnetRouteTableAssociation"
    Properties:
      SubnetId: !Ref PrivateSubnet
      RouteTableId: !Ref PrivateRouteTableId

Outputs:
```

```
  PublicSubnetId:
    Description: Subnet ID of the public subnets.
    Value:
      !Join ["", [!Ref PublicSubnet]]

  PrivateSubnetId:
    Description: Subnet ID of the private subnets.
    Value:
      !Join ["", [!Ref PrivateSubnet]]
```

where:

`kubernetes.io/cluster/unmanaged`
: You must include the `kubernetes.io/cluster/unmanaged` tag in the public subnet configuration for AWS Outposts.

`kubernetes.io/cluster/unmanaged`
: You must include the `kubernetes.io/cluster/unmanaged` tag in the private subnet configuration for AWS Outposts.

3.10.4. Creating a compute machine set that deploys edge compute machines on an Outpost

To deploy edge compute machines on Amazon Web Services (AWS) Outposts, you must create a compute machine set with a configuration compatible with the Outpost environment.

Prerequisites

- You have an AWS Outposts site.
- You have installed an OpenShift Container Platform cluster into a custom VPC on AWS.
- You have access to the cluster using an account with `cluster-admin` permissions.
- You have installed the OpenShift CLI (`oc`).

Procedure

1. List the compute machine sets in your cluster by running the following command:

   ```
   $ oc get machinesets.machine.openshift.io -n openshift-machine-api
   ```

   Example output

   ```
   NAME                            DESIRED   CURRENT   READY   AVAILABLE   AGE
   <original_machine_set_name_1>   1         1         1       1           55m
   <original_machine_set_name_2>   1         1         1       1           55m
   ```

2. Record the names of the existing compute machine sets.

3. Create a YAML file that contains the values for a new compute machine set custom resource (CR) by using one of the following methods:

   - Copy an existing compute machine set configuration into a new file by running the following command:

     ```
     $ oc get machinesets.machine.openshift.io <original_machine_set_name_1> \
       -n openshift-machine-api -o yaml > <new_machine_set_name_1>.yaml
     ```

     You can edit this YAML file with your preferred text editor.

   - Create an empty YAML file named `<new_machine_set_name_1>.yaml` with your preferred text editor and include the required values for your new compute machine set.

     If you are not sure which value to set for a specific field, you can view values of an existing compute machine set CR by running the following command:

     ```
     $ oc get machinesets.machine.openshift.io <original_machine_set_name_1> \
       -n openshift-machine-api -o yaml
     ```

     Example output

     ```
     apiVersion: machine.openshift.io/v1beta1
     kind: MachineSet
     metadata:
       labels:
         machine.openshift.io/cluster-api-cluster: <infrastructure_id>
       name: <infrastructure_id>-<role>-<availability_zone>
       namespace: openshift-machine-api
     spec:
       replicas: 1
       selector:
         matchLabels:
           machine.openshift.io/cluster-api-cluster: <infrastructure_id>
           machine.openshift.io/cluster-api-machineset: <infrastructure_id>-<role>-
     <availability_zone>
       template:
         metadata:
           labels:
             machine.openshift.io/cluster-api-cluster: <infrastructure_id>
             machine.openshift.io/cluster-api-machine-role: <role>
             machine.openshift.io/cluster-api-machine-type: <role>
             machine.openshift.io/cluster-api-machineset: <infrastructure_id>-<role>-
     <availability_zone>
         spec:
           providerSpec:
     ```

     where:

     `machine.openshift.io/cluster-api-cluster`
     Specifies the cluster infrastructure ID.

     `metadata.name`
     Specifies a default node label. For AWS Outposts, you use the `outposts` role.

     `spec.providerSpec`
     Specifies the `providerSpec` section, which is omitted here but includes values that must be configured for your Outpost.

4. Configure the new compute machine set to create edge compute machines in the Outpost by editing the `<new_machine_set_name_1>.yaml` file:

Example compute machine set for AWS Outposts

```
apiVersion: machine.openshift.io/v1beta1
kind: MachineSet
metadata:
  labels:
    machine.openshift.io/cluster-api-cluster: <infrastructure_id>
  name: <infrastructure_id>-outposts-<availability_zone>
  namespace: openshift-machine-api
spec:
  replicas: 1
  selector:
    matchLabels:
      machine.openshift.io/cluster-api-cluster: <infrastructure_id>
      machine.openshift.io/cluster-api-machineset: <infrastructure_id>-outposts-
<availability_zone>
  template:
    metadata:
      labels:
        machine.openshift.io/cluster-api-cluster: <infrastructure_id>
        machine.openshift.io/cluster-api-machine-role: outposts
        machine.openshift.io/cluster-api-machine-type: outposts
        machine.openshift.io/cluster-api-machineset: <infrastructure_id>-outposts-
<availability_zone>
    spec:
      metadata:
        labels:
          node-role.kubernetes.io/outposts: ""
          location: outposts
      providerSpec:
        value:
          ami:
            id: <ami_id>
          apiVersion: machine.openshift.io/v1beta1
          blockDevices:
            - ebs:
                volumeSize: 120
                volumeType: gp2
          credentialsSecret:
            name: aws-cloud-credentials
          deviceIndex: 0
          iamInstanceProfile:
            id: <infrastructure_id>-worker-profile
          instanceType: m5.xlarge
          kind: AWSMachineProviderConfig
          placement:
            availabilityZone: <availability_zone>
            region: <region>
          securityGroups:
            - filters:
              - name: tag:Name
                values:
                  - <infrastructure_id>-worker-sg
          subnet:
            id: <subnet_id>
          tags:
            - name: kubernetes.io/cluster/<infrastructure_id>
              value: owned
          userDataSecret:
            name: worker-user-data
      taints:
```

```
        - key: node-role.kubernetes.io/outposts
          effect: NoSchedule
```

where:

machine.openshift.io/cluster-api-cluster
: Specifies the cluster infrastructure ID.

metadata.name
: Specifies the name of the compute machine set. The name is composed of the cluster infrastructure ID, the `outposts` role name, and the Outpost availability zone.

spec.providerSpec.value.ami.id
: Specifies the Amazon Machine Image (AMI) ID.

spec.providerSpec.value.blockDevices.ebs.volumeType
: Specifies the EBS volume type. AWS Outposts requires gp2 volumes.

spec.providerSpec.value.instanceType
: Specifies the AWS instance type. You must use an instance type that is configured in your Outpost.

spec.providerSpec.value.placement.region
: Specifies the AWS region in which the Outpost availability zone exists.

spec.providerSpec.value.subnet.id
: Specifies the dedicated subnet for your Outpost.

spec.template.spec.taints
: Specifies a taint to prevent workloads from being scheduled on nodes that have the `node-role.kubernetes.io/outposts` label. To schedule user workloads in the Outpost, you must specify a corresponding toleration in the `Deployment` resource for your application.

5. Save your changes.
6. Create a compute machine set CR by running the following command:

```
$ oc create -f <new_machine_set_name_1>.yaml
```

Verification

- To verify that the compute machine set is created, list the compute machine sets in your cluster by running the following command:

```
$ oc get machinesets.machine.openshift.io -n openshift-machine-api
```

Example output

```
NAME                            DESIRED   CURRENT   READY   AVAILABLE   AGE
<new_machine_set_name_1>        1         1         1       1           4m12s
<original_machine_set_name_1>   1         1         1       1           55m
<original_machine_set_name_2>   1         1         1       1           55m
```

- To list the machines that are managed by the new compute machine set, run the following command:

```
$ oc get -n openshift-machine-api machines.machine.openshift.io \
  -l machine.openshift.io/cluster-api-machineset=<new_machine_set_name_1>
```

Example output

```
NAME                            PHASE          TYPE        REGION      ZONE         AGE
<machine_from_new_1>            Provisioned    m5.xlarge   us-east-1   us-east-1a   25s
<machine_from_new_2>            Provisioning   m5.xlarge   us-east-1   us-east-1a   25s
```

- To verify that a machine created by the new compute machine set has the correct configuration, examine the relevant fields in the CR for one of the new machines by running the following command:

```
$ oc describe machine <machine_from_new_1> -n openshift-machine-api
```

3.10.5. Creating user workloads in an Outpost

After you extend a OpenShift Container Platform VPC cluster into an Amazon Web Services (AWS) Outpost, you can deploy user workloads to edge compute nodes that carry the `node-role.kubernetes.io/outposts` label.

Prerequisites

- You have extended an AWS VPC cluster into an Outpost.
- You have access to the cluster using an account with `cluster-admin` permissions.
- You have installed the OpenShift CLI (`oc`).
- You have created a compute machine set that deploys edge compute machines compatible with the Outpost environment.

Procedure

1. Configure a `Deployment` resource file for an application that you want to deploy to the edge compute node in the edge subnet.

   Example `Deployment` manifest

```
kind: Namespace
apiVersion: v1
metadata:
  name: <application_name>
---
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: <application_name>
  namespace: <application_namespace>
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 10Gi
  storageClassName: gp2-csi
  volumeMode: Filesystem
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: <application_name>
  namespace: <application_namespace>
spec:
  selector:
    matchLabels:
      app: <application_name>
  replicas: 1
  template:
    metadata:
      labels:
        app: <application_name>
        location: outposts
    spec:
      securityContext:
        seccompProfile:
          type: RuntimeDefault
      nodeSelector:
        node-role.kubernetes.io/outpost: ''
      tolerations:
      - key: "node-role.kubernetes.io/outposts"
        operator: "Equal"
        value: ""
        effect: "NoSchedule"
      containers:
        - image: openshift/origin-node
          command:
           - "/bin/socat"
          args:
            - TCP4-LISTEN:8080,reuseaddr,fork
            - EXEC:'/bin/bash -c \"printf \\\"HTTP/1.0 200 OK\r\n\r\n\\\"; sed -e
\\\"/^\r/q\\\"\"'
          imagePullPolicy: Always
          name: <application_name>
          ports:
            - containerPort: 8080
          volumeMounts:
            - mountPath: "/mnt/storage"
```

```
            name: data
      volumes:
      - name: data
        persistentVolumeClaim:
          claimName: <application_name>
```

where:

`metadata.name`
: Specifies a name for your application.

`metadata.namespace`
: Specifies a namespace for your application. The application namespace can be the same as the application name.

`spec.storageClassName`
: Specifies the storage class name. For an edge compute configuration, you must use the `gp2-csi` storage class.

`spec.template.metadata.labels.location`
: Specifies a label to identify workloads deployed in the Outpost.

`spec.template.spec.nodeSelector`
: Specifies the node selector label that targets edge compute nodes.

`spec.template.spec.tolerations`
: Specifies tolerations that match the `key` and `effects` taints in the compute machine set for your edge compute machines. Set the `value` and `operator` tolerations as shown.

2. Create the `Deployment` resource by running the following command:

```
$ oc create -f <application_deployment>.yaml
```

3. Configure a `Service` object that exposes a pod from a targeted edge compute node to services that run inside your edge network.

Example `Service` manifest

```
apiVersion: v1
kind: Service
metadata:
  name:  <application_name>
  namespace: <application_namespace>
spec:
  ports:
    - port: 80
      targetPort: 8080
      protocol: TCP
  type: NodePort
  selector:
    app: <application_name>
```

where:

`kind`
: Specifies the `Service` resource type.

`spec.selector`
: Specifies the label type to apply to managed pods.

4. Create the `Service` CR by running the following command:

```
$ oc create -f <application_service>.yaml
```

3.10.6. Scheduling workloads on edge and cloud-based AWS compute resources

When you extend an AWS VPC cluster into an Outpost, the Outpost uses edge compute nodes and the VPC uses cloud-based compute nodes.

The following load balancer considerations apply to an AWS VPC cluster extended into an Outpost:

- Outposts cannot run AWS Network Load Balancers or AWS Classic Load Balancers, but a Classic Load Balancer for a VPC cluster extended into an Outpost can attach to the Outpost edge compute nodes. For more information, see "Using AWS Classic Load Balancers in an AWS VPC cluster extended into an Outpost".
- To run a load balancer on an Outpost instance, you must use an AWS Application Load Balancer. You can use the AWS Load Balancer Operator to deploy an instance of the AWS Load Balancer Controller. The controller provisions AWS Application Load Balancers for Kubernetes Ingress resources. For more information, see "Using the AWS Load Balancer Operator in an AWS VPC cluster extended into an Outpost".

3.10.6.1. Using Amazon Web Services (AWS) Classic Load Balancers in an AWS VPC cluster extended into an Outpost

To prevent Classic Load Balancers in your AWS VPC cluster from scheduling pods on Outpost edge compute nodes, you can label cloud-based compute nodes and configure the load balancer to target only those labeled nodes.

AWS Outposts infrastructure cannot run AWS Classic Load Balancers, but Classic Load Balancers in the AWS VPC cluster can target edge compute nodes in the Outpost if edge and cloud-based subnets are in the same availability zone. As a result, Classic Load Balancers on the VPC cluster might schedule pods on either of these node types.

Scheduling the workloads on edge compute nodes and cloud-based compute nodes can introduce latency. If you want to prevent a Classic Load Balancer in the VPC cluster from targeting Outpost edge compute nodes, you can apply labels to the cloud-based compute nodes and configure the Classic Load Balancer to only schedule on nodes with the applied labels.

If you do not need to prevent a Classic Load Balancer in the VPC cluster from targeting Outpost edge compute nodes, you do not need to complete these steps.

Prerequisites

- You have extended an AWS VPC cluster into an Outpost.
- You have access to the cluster using an account with `cluster-admin` permissions.
- You have installed the OpenShift CLI (`oc`).
- You have created a user workload in the Outpost with tolerations that match the taints for your edge compute machines.

Procedure

1. Optional: Verify that the edge compute nodes have the `location=outposts` label by running the following command and verifying that the output includes only the edge compute nodes in your Outpost:

   ```
   $ oc get nodes -l location=outposts
   ```

2. Label the cloud-based compute nodes in the VPC cluster with a key-value pair by running the following command:

```
$ for NODE in $(oc get node -l node-role.kubernetes.io/worker --no-headers | grep -v outposts
| awk '{print$1}'); do oc label node $NODE <key_name>=<value>; done
```

   where `<key_name>=<value>` is the label you want to use to distinguish cloud-based compute nodes.

   Example output

```
node1.example.com labeled
node2.example.com labeled
node3.example.com labeled
```

3. Optional: Verify that the cloud-based compute nodes have the specified label by running the following command and confirming that the output includes all cloud-based compute nodes in your VPC cluster:

```
$ oc get nodes -l <key_name>=<value>
```

   Example output

```
NAME                   STATUS    ROLES     AGE       VERSION
node1.example.com      Ready     worker    7h        v1.35.4
node2.example.com      Ready     worker    7h        v1.35.4
node3.example.com      Ready     worker    7h        v1.35.4
```

4. Configure the Classic Load Balancer service by adding the cloud-based subnet information to the `annotations` field of the `Service` manifest:

   Example service configuration

```
apiVersion: v1
kind: Service
metadata:
  labels:
    app: <application_name>
  name: <application_name>
  namespace: <application_namespace>
  annotations:
    service.beta.kubernetes.io/aws-load-balancer-subnets: <aws_subnet>
    service.beta.kubernetes.io/aws-load-balancer-target-node-labels: <key_name>=<value>
spec:
  ports:
  - name: http
    port: 80
    protocol: TCP
    targetPort: 8080
  selector:
    app: <application_name>
  type: LoadBalancer
```

   where:

   `service.beta.kubernetes.io/aws-load-balancer-subnets`
   : Specifies the subnet ID for the AWS VPC cluster.

   `service.beta.kubernetes.io/aws-load-balancer-target-node-labels`
   : Specifies the key-value pair that matches the pair in the node label.

5. Create the `Service` CR by running the following command:

```
$ oc create -f <file_name>.yaml
```

Verification

1. Verify the status of the `service` resource to show the host of the provisioned Classic Load Balancer by running the following command:

```
$ HOST=$(oc get service <application_name> -n <application_namespace> --template='{{(index
.status.loadBalancer.ingress 0).hostname}}')
```

2. Verify the status of the provisioned Classic Load Balancer host by running the following command:

```
$ curl $HOST
```

3. In the AWS console, verify that only the labeled instances appear as the targeted instances for the load balancer.

3.10.6.2. Use the AWS Load Balancer Operator in an AWS VPC cluster extended into an Outpost

You can configure the AWS Load Balancer Operator to provision an AWS Application Load Balancer in an AWS VPC cluster extended into an Outpost. AWS Outposts does not support AWS Network Load Balancers. As a result, the AWS Load Balancer Operator cannot provision Network Load Balancers in an Outpost.

You can create an AWS Application Load Balancer either in the cloud subnet or in the Outpost subnet.

An Application Load Balancer in the cloud can attach to cloud-based compute nodes. An Application Load Balancer in the Outpost can attach to edge compute nodes.

You must annotate Ingress resources with the Outpost subnet or the VPC subnet, but not both.

Prerequisites

- You have extended an AWS VPC cluster into an Outpost.
- You have installed the OpenShift CLI (`oc`).
- You have installed the AWS Load Balancer Operator and created the AWS Load Balancer Controller.

Procedure

Configure the `Ingress` resource to use a specified subnet:

Example `Ingress` resource configuration

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: <application_name>
  annotations:
    alb.ingress.kubernetes.io/subnets: <subnet_id>
spec:
  ingressClassName: alb
  rules:
    - http:
        paths:
          - path: /
            pathType: Exact
            backend:
              service:
                name: <application_name>
                port:
                  number: 80
```

where:

`<subnet_id>`
Specifies the subnet to use. To use the Application Load Balancer in an Outpost, specify the Outpost subnet ID. To use the Application Load Balancer in the cloud, you must specify at least two subnets in different availability zones.

3.11. Installing a cluster with the support for configuring multi-architecture compute machines

You can install an OpenShift Container Platform cluster on Amazon Web Services (AWS) with support for configuring multi-architecture compute machines.

When you have nodes with multiple architectures in your cluster, the architecture of your image must be consistent with the architecture of the node. You must ensure that the pod is assigned to the node with the appropriate architecture and that it matches the image architecture. For more information on assigning pods to nodes, see "Scheduling workloads on clusters with multi-architecture compute machines".

After installing the cluster, you can add multi-architecture compute machines to the cluster in the following ways:

- Adding 64-bit x86 compute machines to a cluster that uses 64-bit ARM control plane machines and already includes 64-bit ARM compute machines. In this case, 64-bit x86 is considered the secondary architecture.
- Adding 64-bit ARM compute machines to a cluster that uses 64-bit x86 control plane machines and already includes 64-bit x86 compute machines. In this case, 64-bit ARM is considered the secondary architecture.

Before adding a secondary architecture node to your cluster, it is recommended to install the Multiarch Tuning Operator, and deploy a `ClusterPodPlacementConfig` custom resource. For more information, see "Managing workloads on multi-architecture clusters by using the Multiarch Tuning Operator".

3.11.1. Install a cluster with multi-architecture support

You can install a cluster with multi-architecture support to use compute machines with different Central Processing Unit (CPU) architectures. Modifying your configuration file helps ensure your control plane and worker nodes deploy with the correct architecture.

Prerequisites

- You installed the OpenShift CLI (`oc`).
- You have the OpenShift Container Platform installation program.
- You downloaded the pull secret for your cluster.

Procedure

1. Check that the `openshift-install` binary is using the `multi` payload by running the following command:

   ```
   $ ./openshift-install version
   ```

   Example output

   ```
   ./openshift-install 4.22.0
   built from commit abc123etc
   release image quay.io/openshift-release-dev/ocp-release@sha256:abc123wxyzetc
   release architecture multi
   default architecture amd64
   ```

   The output must contain `release architecture multi` to indicate that the `openshift-install` binary is using the `multi` payload.

2. Update the `install-config.yaml` file to configure the architecture for the nodes.

   Sample `install-config.yaml` file with multi-architecture configuration

   ```
   apiVersion: v1
   baseDomain: example.openshift.com
   compute:
   - architecture: amd64
     hyperthreading: Enabled
     name: worker
     platform: {}
     replicas: 3
   controlPlane:
     architecture: arm64
     name: master
     platform: {}
     replicas: 3
   ```

   where:

   `compute.architecture`
   : Specifies the architecture of the worker node. You can set this field to either `arm64` or `amd64`.

   `controlPlane.architecture`
   : Specifies the control plane node architecture. You can set this field to either `arm64` or `amd64`.